ExpressQuantumGrid SUITE

PROFESSIONAL

ExpressQuantumGrid,
ExpressTreeList & DBTreeList

Copyright © 2000 Developer Express Inc.

Table of Contents

ExpressQuantumGrid Suite Overview 4

TCustomdxContainer 6

- **TCustomdxContainer Properties** 7
- **TCustomdxContainer Methods** 8

TCustomdxTreeList 13

- **TCustomdxTreeList Properties** 14
- **TCustomdxTreeList Methods** 51
- **TCustomdxTreeList Events** 76

TCustomdxTreeListControl 88

- **TCustomdxTreeListControl Properties** 89
- **TCustomdxTreeListControl Methods** 96
- **TCustomdxTreeListControl Events** 107

TdxTreeList 130

- **TdxTreeList Properties** 131
- **TdxTreeList Methods** 133
- **TdxTreeList Events** 137

TCustomdxDBTreeListControl 142

- **TCustomdxDBTreeListControl Properties** 143
- **TCustomdxDBTreeListControl Methods** 149
- **TCustomdxDBTreeListControl Events** 156

TCustomdxDBTreeList 161

- **TCustomdxDBTreeList Properties** 162
- **TCustomdxDBTreeList Methods** 172
- **TCustomdxDBTreeList Events** 175

Copyright © 2000 Developer Express Inc.

TdxDBTreeList **177**

TdxDBTreeList Properties **178**

TdxDBTreeList Methods **180**

TdxDBTreeList Events **182**

TCustomdxDBGrid **183**

TCustomdxDBGrid Properties **184**

TCustomdxDBGrid Methods **204**

TCustomdxDBGrid Events **213**

TdxDBGrid **221**

TdxDBGrid Properties **222**

TdxDBGrid Methods **224**

TdxDBGrid Events **228**

TdxDBGridFilter **229**

TdxDBGridFilter Properties **231**

TdxDBGridFilter Methods **234**

Index **236**

ExpressQuantumGrid Suite Professional Overview

The ExpressQuantumGrid Suite Professional contains three powerful controls: TdxTreeList (**ExpressTreeList**), TdxDBTreeList (**ExpressDBTreeList**) and TdxDBGrid (**ExpressQuantumGrid**) as well as the **ExpressMemData** in-memory table component.

TdxTreeList represents data to end users using a hybrid tree/list display metaphor. TdxDBGrid offers familiar grid features, as well as a huge set of properties, options and methods, to provide you and your users incredible control, flexibility, and runtime customization capabilities. TdxDBTreeList is a synthesis of a tree/list with data-aware features. Using the elements built into ExpressTreeList, ExpressDBTreeList and ExpressQuantumGrid, you can easily enable:

? Runtime Grouping

? Automatic Sorting

? Automatic Search For a Value

? Automatic Column Customization

? Summary Calculations including: COUNT, MAX, MIN, AVG, SUM

? 18 inplace editors for convenient data displaying and editing

? And much more...

TdxDBGrid, TdxTreeList and TdxDBTreeList include a number of unique columns that provide easy data manipulation facilities. The columns allow you to display and edit dates, memo text, currency values, hyperlinks and even pictures, directly within individual cells.

Copyright © 2000 Developer Express Inc.

Getting Support

The goal of our support team is to answer and alleviate any problems you might encounter when using our components. Feel free to contact us at:

E-mail:

support@devexpress.com

You can address any suggestions, comments, complaints or good thoughts to:

suggest@devexpress.com

Please, visit the ExpressSupportForum at http://www.devexpress.com/dxforum to find articles on our components, tips and tricks, and a myriad of working samples with full source code.

TCustomdxContainer

TCustomdxContainer is the base class for components contained in the ExpressQuantumGrid Suite Professional such as the ExpressTreeList, ExpressDBTreeList and ExpressQuantumGrid.

Unit

dxCntner

```
TCustomdxContainer = class(TCustomControl);
```

Description

TCustomdxContainer is a container that contains basic methods and properties, all of which are protected. Do not use this control directly. It is used for the creation of more comprehensive controls.

Hierarchy

TObject
- TPersistent
  - TComponent
    - TControl
      - TWinControl
        - TCustomControl

Copyright © 2000 Developer Express Inc.

TCustomdxContainer Properties

IsInitEdit	**protected read only**
ReadOnly	**public read only**

IsInitEdit property — read only

```
property IsInitEdit: Boolean;
```

Description

Indicates whether the properties of an in-place editor are initialized (this property is used to prevent excessive generation of the OnChange event).

ReadOnly property — read only

Indicates whether a control is used for display purposes only or whether users can edit data within it.

```
property ReadOnly: Boolean;
```

Description

Prevents users from changing data within a control. By default, ReadOnly for TCustomdxContainer is set to False. This property is protected and TCustomdxContainer descendants may publish it and thus enable data edits.

TCustomdxContainer Methods

BeginInitEdit	**protected**	GetReadOnly	**protected**
CanEditAcceptKey	**protected**	HideDrawFocusRect	**protected**
CanEditModify	**protected**	InplaceEditKeyDown	**protected**
CheckEditorWidth	**protected**	IsEditorMoved	**protected**
CreateEditStyle	**protected**	IsImmediateEditor	**protected**
EndInitEdit	**protected**	IsShowButtonAlways	**protected**
GetDefaultPopupBorderStyle	**public**	IsTabs	**protected**
GetEditColor	**protected**	MoveCol	**protected**
GetEditFont	**protected**	RedrawSelection	**protected**

BeginInitEdit method

Used to begin initialization of the properties for an in-place editor.

```
procedure BeginInitEdit; virtual;
```

Description

The BeginInitEdit method sets the value of the IsInitEdit property to **True**.

CanEditAcceptKey method

Indicates whether a specified character can be entered into the current cell (in-place editor).

```
function CanEditAcceptKey(Key: Char): Boolean;  virtual;
```

Description

The CanEditAcceptKey method provides a simple scanning of keyboard input while editing the contents of a cell (in-place editor). The CanEditAcceptKey method always returns False for TCustomdxContainer. TCustomdxContainer descendants can override this method to to ignore illegal character input. The Key parameter specifies the pressed key.

Note: *More refined validation can be obtained by using an edit mask.*

CanEditModify method

Indicates whether the current tree list (grid) cell can be edited.

```
function CanEditModify: Boolean;
```

Description

Descendant controls use the CanEditModify method internally to determine whether to allow users to edit the contents of the current cell. The CanEditModify method returns **False**, when an editor is disabled for editing.

Copyright © 2000 Developer Express Inc.

CheckEditorWidth method

Used by in-place editors to determine whether the width of an Editor is verified.

```
function CheckEditorWidth: Boolean; virtual;
```

Description

The CheckEditorWidth is a virtual protected method that is called whenever an editor checks its width. It determines whether the width must be equal or greater than the minimum width allowed for in-place editors. By default, returns True.

CreateEditStyle method

```
function CreateEditStyle(AEdit: TdxInplaceEdit): TdxEditStyle; virtual;
```

Description

The CreateEditStyle method is used to create the TdxEditStyle class that determines the look & feel of an editor. The AEdit parameter represents the editor whose style is created.

EndInitEdit method

Used to complete initialization of the properties for an in-place editor.

```
procedure EndInitEdit; virtual;
```

Description

The EndInitEdit method sets the value of the IsInitEdit property to **False**.

GetDefaultPopupBorderStyle method

Specifies the default style of the popup border.

```
type
  TdxEditBorderStyle = (xbsNone, xbsSingle, xbsThick, xbsFlat, xbs3D);
  TdxPopupBorderStyle = pbSingle .. pbFlat;
function GetDefaultPopupBorderStyle(AEditBorderStyle: TdxEditBorderStyle):
TdxPopupBorderStyle; virtual;
```

Description

The GetDefaultPopupBorderStyle method always returns **pbFrame3D** for TCustomdxContainer.

GetEditColor method

Provides an interface that returns the background color of an edited cell (in-place editor).

```
function GetEditColor: TColor; virtual;
```

Description

GetEditColor is a virtual protected method that provides an interface which returns the background color of in-place editors. TCustomdxContainer descendants can override this method to provide a proper return value.

GetEditFont method

Determines the text font of an active in-place editor.

```
function GetEditFont: TFont; virtual;
```

Description

The GetEditFont method returns the font used within an active in-place editor. It does not affect other grid/tree list cells.

GetReadOnly method

Returns a value that determines whether a control provides read-only access.

```
function GetReadOnly: Boolean; virtual;
```

Description

The GetReadOnly method is used internally to determine whether a control provides read-only access. If access is read-only, users cannot modify the contents of any cell (in-place editor). TCustomdxContainer descendants can override this method to provide a proper return value. This method always returns False for TCustomdxContainer.

HideDrawFocusRect method

Indicates whether a descendant control draws a FocusRect.

```
function HideDrawFocusRect: Boolean; virtual;
```

Description

HideDrawFocusRect is a virtual protected method used to determine whether a focus rectangle is displayed. By default, returns False. Descendants must override it to return a proper value.

InplaceEditKeyDown method

Processes the OnKeyDown event.

```
type
  TShiftState = set of(ssShift, ssAlt, ssCtrl, ssLeft, ssRight, ssMiddle,
ssDouble);
procedure InplaceEditKeyDown(var Key: Word; Shift: TShiftState); virtual;
```

Copyright © 2000 Developer Express Inc.

Description

InplaceEditKeyDown is a virtual protected method that verifies whether the OnKeyDown event is assigned and calls the event handler. The Key parameter specifies the key pressed. The Shift parameter indicates the state of the Alt, Ctrl, and Shift keys and mouse buttons.

IsEditorMoved method

Indicates whether an in-place editor has moved.

```
function IsEditorMoved: Boolean; virtual;
```

Description

The IsEditorMoved is a virtual protected method used by in-place editors. This method determines whether the screen position of an editor has changed during drawing. By default, the IsEditorMoved method returns **False**. Descendants must override it to return a proper value.

IsImmediateEditor method

Returns a value that determines whether a control calls an in-place editor immediately when a user clicks a cell.

```
function IsImmediateEditor: Boolean; virtual;
```

Description

The IsImmediateEditor method is used internally to determine whether a control has to call an in-place editor immediately when a user clicks a cell. TCustomdxContainer descendants can override this method to provide a proper return value. The IsImmediateEditor method always returns False for TCustomdxContainer.

IsShowButtonAlways method

Determines whether grid/tree list columns always display buttons.

```
function IsShowButtonAlways: Boolean; virtual;
```

Description

Some grid/tree list columns (certain container editors) can include command buttons to modify data. By default, a button is displayed within the active in-place editor only. The ExpressQuantumGrid/TreeList can display buttons regardless of whether an in-place editor is active/inactive. If the IsShowButtonAlways method returns **True**, command buttons are always displayed.

By default, IsShowButtonAlways returns False.

IsTabs method

Returns a value that determines whether a user can navigate through tree list (grid) cells (in-place editors) using the TAB key.

```
function IsTabs: Boolean; virtual;
```

Description

The IsTabs method is used internally to determine whether a user can navigate through cells (in-place editors) using the TAB key or whether the container control loses focus when a user presses the TAB key. TCustomdxContainer descendants can override this method to provide a proper return value. This method always returns False for TCustomdxContainer.

MoveCol method

Provides an interface to the method that is used to change the currently focused container column.

```
procedure MoveCol(ALeft: Boolean); virtual;
```

Description

By default, the MoveCol method performs no actions. Descendants can override this method to perform specific processing when changing the edited columns.

RedrawSelection method

Provides an interface to the method that redraws the current selection in a control.

```
procedure RedrawSelection; virtual;
```

Description

Descendants use the RedrawSelection method internally to call a method that redraws the current selection.

Copyright © 2000 Developer Express Inc.

TCustomdxTreeList

TCustomdxTreeList is an abstract base class for TdxTreeList and TdxDBGrid controls.

Unit

dxTL

```
TCustomdxTreeList = class(TCustomdxContainer);
```

Description

The TCustomdxTreeList control does not store data and/or information on individual columns. Consider TCustomdxTreeList as a virtual tree list control. It implements the interface for any tree list control and performs basic operations such as painting and customization.

IMPORTANT: You must not create TCustomdxTreeList directly, but instead use TdxDBGrid and TdxTreeList.

Hierarchy

TObject
- TPersistent
  - TComponent
    - TControl
      - TWinControl
        - TCustomControl
          - TCustomdxContainer

TCustomdxTreeList Properties

TCustomdxTreeList

ArrowsColor	**protected / read / write**	HideFocusRect	**protected / read / write**
AutoExpandOnSearch	**protected / read / write**	HideSelection	**protected / read / write**
AutoSearchColor	**protected / read / write**	HideSelectionColor	**protected / read / write**
AutoSearchTextColor	**protected / read / write**	HideSelectionTextColor	**protected / read / write**
BandButtonPushed	**public / read only**	HighlightColor	**protected / read / write**
BandColor	**protected / read / write**	HighlightTextColor	**protected / read / write**
BandFont	**protected / read / write**	Images	**protected / read / write**
BandPanelHeight	**public / read only**	Indent	**protected / read only**
BorderStyle	**protected / read / write**	IndentDesc	**protected / read / write**
Count	**public / read only**	InplaceColumnIndex	**public / read only**
CustomizingBandListBox	**public / read only**	InplaceEditor	**public / read only**
CustomizingForm	**public / read only**	IsCustomizing	**public / read only**
CustomizingHeaderListBox	**public / read only**	Items	**public / read only**
CustomizingPos	**public / read / write**	LastNode	**public / read only**
CustomizingRowCount	**public / read / write**	LeftCoord	**protected / read / write**
DblClkExpanding	**protected / read / write**	LockSelection	**public / read only**
DefaultRowHeight	**protected / read / write**	LockUpdate	**protected / read only**
DescTextHeight	**public / read only**	LookAndFeel	**protected / read / write**
DownBandIndex	**public / read only**	MaxRowLineCount	**protected / read / write**
DownBandPushed	**public / read only**	NewItemRowActive	**protected / read / write**
DownColumnIndex	**public / read only**	NewItemRowHeight	**public / read only**
DownColumnPushed	**public / read only**	Options	**public / read / write**
DragAbsoluteBandIndex	**public / read only**	OptionsEx	**public / read / write**
DragAbsoluteHeaderIndex	**public / read only**	PaintStyle	**protected read / write**
DragNode	**protected / read only**	PreviewFont	**protected read / write**
EditingText	**protected / read / write**	PreviewLines	**protected / read / write**
FixedBandLineColor	**protected / read / write**	RowCount	**protected / read only**
FixedBandLineWidth	**protected / read / write**	RowFooterColor	**protected / read / write**
FocusedAbsoluteIndex	**protected / read only**	RowFooterTextColor	**protected / read / write**
FocusedColumn	**protected / read / write**	RowHeight	**public / read only**
FocusedNode	**public / read only**	RowSeparatorLineWidth	**protected / read / write**
FocusedNumber	**protected / read / write**	ScrollBars	**protected / read / write**
FooterPanelHeight	**public / read only**	SelectedCount	**protected / read only**
FooterRowHeight	**public / read only**	SelectedNodes	**protected / read only**
FooterRowNodeHeight	**public / read only**	SelectionAnchor	**protected / read / write**
GridLineColor	**protected / read / write**	ShowBands	**protected / read / write**
GroupNodeColor	**protected / read / write**	ShowButtons	**protected / read / write**
GroupNodeTextColor	**protected / read / write**	ShowFooter	**protected / read / write**
HeaderButtonPushed	**public / read only**	ShowGrid	**protected / read / write**
HeaderColor	**protected / read / write**	ShowHeader	**protected / read / write**
HeaderFont	**protected / read / write**	ShowHiddenInCustomizeBox	**protected / read / write**
HeaderHeight	**public / read only**	ShowIndicator	**protected / read / write**
HeaderPanelHeight	**public / read only**	ShowLines	**protected / read / write**
HeaderRowHeight	**public / read only**	ShowNewItemRow	**protected / read / write**

Copyright © 2000 Developer Express Inc.

ShowPreviewGrid	**protected / read / write**	TopNode	**public / read only**
ShowRoot	**protected / read / write**	TopVisibleNode	**public / read only**
ShowRowFooter	**protected / read / write**	TreeLineColor	**protected / read / write**
SimpleCustomizeBox	**protected / read / write**	TreeLineStyle	**protected / read / write**
State	**public / read only**	VisibleRowCount	**protected / read only**
StateImages	**protected / read / write**	WaitForExpandNodeTime	**protected / read / write**
TabStop	**published / read / write**		
TopIndex	**protected / read / write**		

Derived from TCustomdxContainer

IsInitEdit	**public / read only**
ReadOnly	**public / read only**

ArrowsColor property — read / write

Specifies the color of arrows that appear when moving a column.

```
property ArrowsColor : TColor;
```

Description

When a user moves a column at run time, two arrows indicate available positions wherein to place a column. Use this property to set or access arrow color. The default value for ArrowColor is clLime.

AutoExpandOnSearch property — read / write

Determines the range of nodes wherein a tree list can perform auto-search.

```
property AutoExpandOnSearch: Boolean;
```

Description

If AutoExpandOnSearch is True, a tree list component searches for a specific node throughout all tree list nodes. If searching is successful, the component expands all parent nodes to display the found node. If False (the default value), the tree list searches only through expanded nodes.

See Also

TCustomdxTreeList.OptionsEx; TCustomdxTreeList.AutoSearchColor; TCustomdxTreeList.AutoSearchTextColor; TCustomdxTreeList.StartSearch; TCustomdxTreeList.EndSearch

AutoSearchColor property — read / write

```
property AutoSearchColor: TColor;
```

Description

AutoSearchColor specifies the background color for auto-search text. If this property is clNone (the default value), background color is set automatically.

See Also

TCustomdxTreeList.AutoExpandOnSearch; TCustomdxTreeList.AutoSearchTextColor; TCustomdxTreeList.StartSearch; TCustomdxTreeList.EndSearch

AutoSearchTextColor property — read / write

```
property AutoSearchTextColor: TColor;
```

Description

AutoSearchTextColor specifies the text color for auto-search text. If this property is clNone (the default value), the color is set automatically.

See Also

TCustomdxTreeList.AutoExpandOnSearch; TCustomdxTreeList.AutoSearchColor; TCustomdxTreeList.StartSearch; TCustomdxTreeList.EndSearch

BandButtonPushed property — read only

Determines whether a band button is pressed.

```
property BandButtonPushed: Boolean;
```

Description

If the band button is pressed, this property returns True. Otherwise, it returns False. This property can be useful for custom drawing procedures.

See Also

TCustomdxTreeListControl.OnBandButtonClick; TCustomdxTreeList.State; TCustomdxTreeList.HeaderButtonPushed

BandColor property — read / write

Specifies color of a band.

```
property BandColor : TColor;
```

Description

Use this property to define the background color of a band. By default, it is clBtnFace.

BandFont property — read / write

Determines the font of band's caption.

```
property BandFont : TFont;
```

Description

Use this property to specify a specific font for a band's caption.

BandPanelHeight property — read only

Specifies the height of the band panel.

```
property BandPanelHeight: Integer;
```

Description

Band panel is an area at the top of the grid, which contains bands (if ShowBands is True). Use this property to access the vertical size of the height of the band panel.

See Also

TCustomdxTreeList.ShowBands

BorderStyle property — read / write

Determines whether the client area of a tree list control has a single line border.

```
type
  TBorderStyle = bsNone .. bsSingle;
property BorderStyle : TBorderStyle;
```

Description

BorderStyle determines whether a control includes a border. If BorderStyle is bsSingle, a control has a border around the client area. If BorderStyle is bsNone, it has none.

Count property — read only

Specifies the number of root nodes.

```
property Count : Integer;
```

Description

Use the Count property to access the number of nodes in TCustomdxTreeList. This property returns the amount of nodes only at the zero level.

See Also

TCustomdxTreeList.Items

CustomizingBandListBox property — read only

Represents a band customizing listbox.

```
type
  TdxCustomizingListBox = class(TListBox);
```

```
  TdxBandsListBox = class(TdxCustomizingListBox);
property CustomizingBandListBox: TdxBandsListBox;
```

Description

The band customizing list box is a list box with bands in the customizing form. Use this property if you need to directly work with the band customizing list box.

See Also

TCustomdxTreeList.CustomizingForm; TCustomdxTreeList.CustomizingHeaderListBox

CustomizingForm property — read only

Specifies the column customization form.

```
property CustomizingForm : TForm;
```

Description

Use the column customization form in order to provide users with the ability to customize grid columns. The TCustomdxTreeList ColumnsCustomizing method activates the column customization form.

See Also

sdxGrColumns; sdxGrBandsCaption; sdxGrHeadersCaption; TCustomdxTreeList.ColumnsCustomizing; TCustomdxTreeList.IsCustomizing; TCustomdxTreeList.OnEndColumnsCustomizing; TCustomdxTreeList.PointInCustomizingForm; TCustomdxTreeList.CustomizingPos

CustomizingHeaderListBox property — read only

Represents a column customizing list box.

```
type
  TdxCustomizingListBox = class(TListBox);
  TdxHeaderListBox = class(TdxCustomizingListBox);
property CustomizingHeaderListBox: TdxHeadersListBox;
```

Description

The column customization list box is a list box with column names in the customizing form. Use this property if you must work directly with the column customizing list box.

See Also

TCustomdxTreeList.CustomizingForm; TCustomdxTreeList.CustomizingBandListBox

CustomizingPos property — read / write

Specifies position of the customization form.

```
property CustomizingPos: TPoint;
```

Copyright © 2000 Developer Express Inc.

Description

Use this property to determine the point in screen coordinates where the customization form is activated.

See Also

TCustomdxTreeList.CustomizingForm

CustomizingRowCount property — read only

Specifies the number of rows within a band.

```
property CustomizingRowCount: Integer;
```

Description

Columns can be placed under one another within a band. Use this property to set the maximum number of rows into which columns can be placed under one another.

See Also

TCustomdxTreeList.CustomizingPos

DblClkExpanding property — read / write

Determines whether a node expands/collapses on a double click event.

```
property DblClkExpanding: Boolean;
```

Description

If active, a node expands (collapses) when a user double-clicks it. Otherwise, it expands (collapses) only when a user presses the '+' / '-' buttons to the left of the node or presses the '+' or '-' on the keyboard.

See Also

TCustomdxTreeList.ShowButtons

DefaultRowHeight property — read / write

Determines row height in pixels.

```
property DefaultRowHeight: Integer;
```

Description

Use this property to specify the height of a row within a tree list control. The row height is a maximum of all row heights reported by the columns.

DescTextHeight property — read only

Specifies height of the preview font.

```
property DescTextHeight : integer;
```

Description

DescTextHeight determines the height of the preview text in pixels. This property is used to correctly print the preview section.

See Also

TCustomdxTreeList.PreviewFont

DownBandIndex property **read only**

Determines index of the pressed band.

```
property DownBandIndex:integer;
```

Description

This property returns the index of a visible column currently pressed. By default, DownBandIndex is –1. When a user clicks a band, this property equals the index of the clicked band. It remains equal to the clicked band index until a user releases the mouse button or the mouse is placed outside of a band.

See Also

TCustomdxTreeList.OnCustomDrawBand; TCustomdxTreeList.DownColumnIndex; TCustomdxTreeListControl.OnBandClick

DownBandPushed property **read only**

Determines whether a band is being pushed.

```
property DownBandPushed : Boolean;
```

Description

By default, this property is Flase. It changes to True when a user clicks any band in a tree list control. This property remains True, until a user releases the mouse button or places the mouse outside of a band. This property is useful for band drawing. It allows you to draw ordinary band headers while the left mous e button is being pressed and the mouse is outside of the band header region.

See Also

TCustomdxTreeList.DownColumnPushed; TCustomdxTreeList.DownBandIndex; TCustomdxTreeList.DownColumnIndex; TCustomdxTreeList.State

DownColumnIndex property **read only**

Determines index of a clicked column header.

```
property DownColumnIndex : integer;
```

Description

This property returns the index of a visible column currently pressed. By default, DownBandColumn is –1. Once a user clicks a column header, this property equals the index of the clicked column. It remains equal to the clicked column index until a user releases the mouse button or the mouse is placed outside of the column header region. This property can be used in custom procedures that draw columns. It specifies whether to draw a "pressed' column header.

See Also

TCustomdxTreeList.OnCustomDrawColumnHeader; TCustomdxTreeList.DownBandIndex; TCustomdxTreeListControl.OnColumnClick

DownColumnPushed property — read only

Determines whether a column header is being pressed.

```
property DownColumnPushed : Boolean;
```

Description

By default, the DownColumnPushed property is False. It is set to True when a user clicks a column header in a tree list control. This property remains True, until a user releases the mouse button or places the mouse outside of the column header region. This property is most useful for column header drawing. It allows drawing ordinary column headers while the left mouse button is being pressed and the mouse is outside of the column header.

See Also

TCustomdxTreeList.DownBandPushed; TCustomdxTreeList.DownBandIndex; TCustomdxTreeList.DownColumnIndex; TCustomdxTreeList.State

DragAbsoluteBandIndex property — read only

Specifies the index of a dragged band.

```
property DragAbsoluteBandIndex:integer;
```

Description

This property returns the index of a currently dragged band. By default, this property is –1. It changes from –1 only when a user drags a band. In this instance, it specifies the absolute index of the dragged band. Use this property when processing drag-and-drop events.

See Also

TCustomdxTreeList.OnDragOverBand; TCustomdxTreeList.OnDragEndBand; TCustomdxTreeList.StartDragBand

DragAbsoluteHeaderIndex property — read only

Specifies index of a dragged column.

```
property DragAbsoluteHeaderIndex: integer;
```

Description

This property returns the absolute index of the currently dragged column. By default, this property is –1. It changes from –1 only when dragging a column. In this instance, it specifies the absolute index of the dragged column. Use this property when processing drag-and-drop events.

See Also

TCustomdxTreeList.OnDragOverHeader; TCustomdxTreeList.OnDragEndHeader; TCustomdxTreeList.StartDragHeader

DragNode property — read only

Specifies the currently dragged node.

```
property DragNode : TdxTreeListNode;
```

Description

Users can drag and re-attach nodes to other nodes of a control. Specifies the currently dragged node.

See Also

TCustomdxTreeList.DoBeginDragNode; TCustomdxTreeList.OnBeginDragNode

EditingText property — read / write

Specifies the text being edited.

```
property EditingText : String;
```

Description

If the Options property includes the aoEditing flag, a user can edit data within columns. EditingText specifies the currently edited text in an inplace editor.

See Also

TCustomdxTreeList.Options; TCustomdxTreeList.OnEditing; TCustomdxTreeList.OnEdited

FixedBandLineColor property — read / write

Determines the color of the fixed band separator line.

```
property FixedBandLineColor : Integer;
```

Description

Fixed bands are separated from other bands by a separator line. Use this property to change the color of this line.

See Also

TdxTreeListBand.Fixed

Copyright © 2000 Developer Express Inc.

FixedBandLineWidth property read / write

Determines the width of a fixed band separator in pixels.

```
property FixedBandLineWidth: Integer;
```

Description

Fixed bands are separated from other bands by a separator line. Use this property to define the width of this line. The default width of a separator is 2 pixels.

Note: *FixedBandLineWidth is in effect if one of the bands has the Fixed property equal to True.*

See Also

TCustomdxTreeList.RowSeparatorLineWidth

FocusedAbsoluteIndex property read only

Returns the absolute index of a focused column.

```
property FocusedAbsoluteIndex: Integer;
```

Description

This property is used internally to obtain the absolute position of a focused column among other tree list columns. If the tree list has no focused column, this method returns -1.

See Also

TCustomdxTreeList.GetFocusedAbsoluteIndex

FocusedColumn property read only

Specifies the index of a focused column.

```
property FocusedColumn : Integer;
```

Description

This property specifies the *visible* index of a focused column. To obtain the *absolute* index of a focused column, use the GetAbsoluteColumnIndex method. The index of the first column is 0. If the aoRowSelect option is active, FocusedColumn always returns 0.

See Also

TCustomdxTreeList.FocusedNode; TCustomdxTreeListControl.GetAbsoluteColumnIndex

FocusedNode property read only

Determines a focused node.

```
property FocusedNode : TdxTreeListNode;
```

Description

Determines the focused node in a control. If a node is focused, a rectangle is drawn around it.

See Also

TCustomdxTreeList.FocusedNumber; TCustomdxTreeList.FocusedColumn

FocusedNumber property — read only

Specifies the index of a focused node.

```
property FocusedNumber : Integer;
```

Description

Specifies the position of a focused node among TCustomdxTreeList visible nodes. The index of the first visible node is 0. Use this property to focus a node by its number.

See Also

TCustomdxTreeList.FocusedNode

FooterPanelHeight property — read only

Determines the height of the footer panel.

```
property FooterPanelHeight: Integer;
```

Description

The purpose of the footer panel is to display a footer summary. Footer panel is displayed at the bottom of a tree list control. This property returns the vertical size of the footer panel in pixels. Footer panel height depends on the font used to draw the tree list as well as the LookAndFeel property.

See Also

TCustomdxTreeList.LookAndFeel; TCustomdxDBGrid.ShowSummaryFooter

FooterRowHeight property — read only

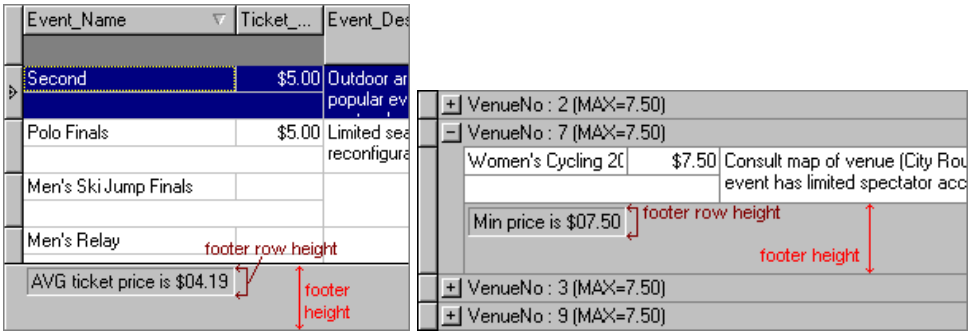
Determines the height of a footer cell.

```
property FooterRowHeight : integer;
```

Description

A Footer represents an area which contains summary cells. The height of the footer area changes according to the height of the column header area. This property returns the height of a cell in a footer area. It can be used by custom procedures that draw a footer.

Copyright © 2000 Developer Express Inc.



See Also

TCustomdxTreeList.FooterPanelHeight; TCustomdxTreeList.OnCustomDrawFooter; TCustomdxTreeList.OnCustomDrawFooterNode; TCustomdxTreeList.HeaderHeight

FooterRowNodeHeight property — read only

```
property FooterRowNodeHeight: Integer;
```

Description

Specifies height of the footer node in pixels. The footer node is a node, drawn under all nodes within a specific group of nodes.

See Also

TCustomdxTreeList.FooterRowHeight

GridLineColor property — read / write

Specifies grid line color.

```
property GridLineColor : TColor;
```

Description

If the ShowGrid property is True, grid lines are drawn. This property specifies the color of these grid lines. The default value for the GridLineColor is clNone.

See Also

TCustomdxTreeList.ShowGrid

GroupNodeColor property — read / write

Specifies the color of a group node.

```
property GroupNodeColor : TColor;
```

Description

GroupNodeColor specifies the background color of a group node. By default, it is clBtnFace. Use the GroupNodeTextColor property to set the text color of a group node.

See Also

TCustomdxTreeList.GroupNodeTextColor

GroupNodeTextColor property — read / write

Determines the text color of a group node.

```
property GroupNodeTextColor : TColor;
```

Description

Use this property to set or access the font color of a group node. By default, it is clBtnText.

See Also

TCustomdxTreeList.GroupNodeColor

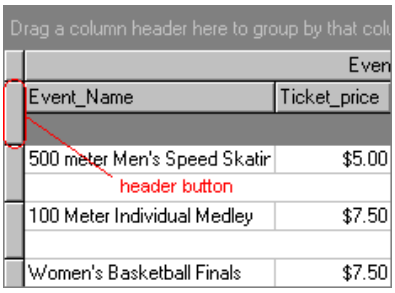
HeaderButtonPushed property — read only

Determines whether the band button is pressed.

```
property HeaderButtonPushed : Boolean;
```

Description

If the header button is pressed, the HeaderButtonPushed property returns True. Otherwise, it returns False. This property can be useful for custom drawing procedures.



See Also

TCustomdxTreeListControl.OnHeaderButtonClick; TCustomdxTreeList.State; TCustomdxTreeList.BandButtonPushed

HeaderColor property — read / write

Determines the color of a control's header (if it exists).

```
property HeaderColor : TColor;
```

Description

The control's header is an area at the top of a control that contains captions of columns. This property determines the color of this header.

HeaderFont property read / write

Determines the font of a control's header.

```
property HeaderFont : TFont;
```

Description

The control's header is an area at the top of a control that contains captions of columns. This property specifies column header font. The HeaderFont property points to a TFont object that determines typographic attributes of a column's caption.

HeaderHeight property read only

Determines the height of a header.

```
property HeaderHeight : Integer;
```

Description

The control's header is an area at the top of a control that contains captions of columns. This property returns the height of the control's header in pixels.

See Also

TCustomdxTreeList.FooterRowHeight

HeaderPanelHeight property read only

Determines the height of a header panel.

```
property HeaderPanelHeight: Integer;
```

Description

Header panel is an area at the top of a control (but under the band panel, if exists) that contains column headers. The HeaderPanelHeight property returns the vertical size of this area in pixels.

See Also

TCustomdxTreeList.HeaderRowHeight; TCustomdxTreeList.BandRowHeight

HeaderRowHeight property read only

Determines the vertical size of a header row in pixels.

```
property HeaderRowHeight: Integer;
```

Description

The height of a header row is equal to the row height of a column header. Use this property to access the height of one column header row in pixels.

See Also

TCustomdxTreeList.HeaderPanelHeight

HideFocusRect property — read / write

Controls the appearance of a rectangle around selected column(s) when a control does not have focus.

```
property HideFocusRect : Boolean;
```

Description

If the HideFocusRect property is True, the bound rectangle is not drawn around selected column(s). Otherwise, selected columns are bound by a rectangle. HideFocusRect specifies rectangle drawing for an unfocused control. In order to hide or show a rectangle around focused columns, use the aoHideFocusRect flag of the TCustomdxTreeList Options property.

See Also

TCustomdxTreeList.Options

HideSelection property — read / write

Determines whether TCustomdxTreeList offers visual indication of selected items when focus shifts to another control.

```
property HideSelection : Boolean;
```

Description

If HideSelection is False, selection is shown even if focus is outside a control. Set HideSelection to True to show the selection only when TCustomdxTreeList has focus. HideSelection does not affect the actual value of the selection, only its visual indication.

See Also

TCustomdxTreeList.HideSelectionColor; TCustomdxTreeList.HideSelectionTextColor

HideSelectionColor property — read / write

Determines the color used to highlight selected nodes when a tree list does not have focus.

```
property HideSelectionColor: TColor;
```

Description

Use this property to determine the color used to draw selected nodes when a tree list control does not have focus.

Note: *HideSelectionColor is not available when the HideSelection property is False.*

See Also

TCustomdxTreeList.HideSelection; TCustomdxTreeList.HideSelectionTextColor

Copyright © 2000 Developer Express Inc.

HideSelectionTextColor property — read / write

Specifies the color of selected node text when a tree list control does not have focus.

```
property HideSelectionTextColor: TColor;
```

Description

Use this property to determine the color used to draw selected node text when a tree list control does not have focus.

Note: *HideSelectionTextColor is effective only when the HideSelection property is False.*

See Also

TCustomdxTreeList.HideSelection; TCustomdxTreeList.HideSelectionColor

HighlightColor property — read only

Determines the color of selected nodes.

```
property HighlightColor: TColor;
```

Description

Specifies the background of selected nodes. Selected nodes are highlighted by this color when the tree list control has focus. By default, selected nodes are highlighted by the standard highlight color (clHighlight) of the current Windows color scheme.

HighlightTextColor property — read / only

Determines the color of a selected node text.

```
property HighlightTextColor: TColor;
```

Description

Use this property to highlight the text of selected nodes with a desired color. The text of selected nodes is drawn with HighlightTextColor when the tree list control has focus. By default, the text color of a selected node is clHighlightText (the color of the current Windows color scheme).

Images property — read / write

Specifies the image list associated with a control.

```
property Images : TImageList;
```

Description

Use Images to provide a customized list of bitmaps that can be displayed to the left of a node.

See Also

TCustomdxTreeList.StateImages; TdxTreeListNode.ImageIndex; TdxTreeListNode.SelectedIndex; TCustomdxTreeListControl.OnGetImageIndex; TCustomdxTreeListControl.OnGetSelectedIndex

Indent property — read only

Determines the indentation in pixels when a list of child nodes is expanded.

```
property Indent: Integer;
```

Description

Use this property to obtain the left indent before a child node relative to a parent node.

IndentDesc property — read only

Specifies the indent before preview section text in pixels.

```
property IndentDesc : Integer;
```

Description

This property specifies the indentation before preview text when drawing the preview section.

See Also

TCutomdxTreeList.LeftCoord

InplaceColumnIndex property — read only

Determines the index of a column edited by an inplace editor.

```
property InplaceColumnIndex : Integer;
```

Description

Returns the index of a column for which an inplace editor has been called.

See Also

TCustomdxTreeList.InplaceEditor

InplaceEditor property — read only

```
property InplaceEditor : TdxInplaceEdit;
```

Description

Returns the active inplace editor.

Copyright © 2000 Developer Express Inc.

See Also

TCustomdxTreeList.InplaceColumnIndex; TCustomdxTreeList.CancelEditor; TCustomdxTreeList.HideEditor; TCustomdxTreeList.ShowEditor; TCustomdxTreeList.CloseEditor; TCustomdxTreeList.OnEditing; TCustomdxTreeList.OnEdited

IsCustomizing property — read only

Determines whether a user is customizing a control.

```
property IsCustomizing : Boolean;
```

Description

If the TCustomdxTreeList object is being customized (the column customization form is open), the IsCustomizing property returns True; otherwise - False.

See Also

TCustomdxTreeList.ColumnsCustomizing; TCustomdxTreeList.CustomizingForm; TCustomdxTreeList.CustomizingHeaderListBox; TCustomdxTreeList.CustomizingBandListBox

Items property — read only

Specifies a TCustomdxTreeList node.

```
property Items[Index:Integer] : TdxTreeListNode;
```

Description

The Items property is an array of TCustomdxTreeList nodes. You can access a given node via its index in the Items array. The number of nodes in the Items array is specified by the Count property.

Note: *The Items property refers only to root nodes. You must use the Items property of a given node to access its children.*

See Also

TCustomdxTreeList.Count; TdxTreeListNode.Items; TCustomdxTreeList.LastNode; TCustomdxTreeList.TopNode; TCustomdxTreeList.ClearNodes

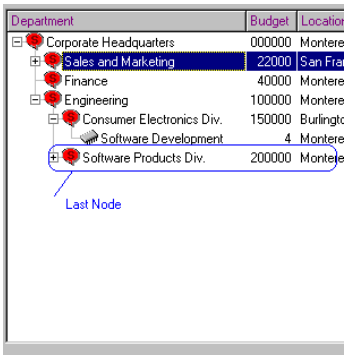
LastNode property — read only

Returns the last visible node of a tree list control.

```
property LastNode : TdxTreeListNode;
```

Description

It returns the last visible node of the TdxTreeList or TdxDBGrid controls. The following figure explains the LastNode property.



See Also

TCustomdxTreeList.TopNode

LeftCoord property — read / write

```
property LeftCoord : Integer;
```

Description

Used internally. Determines the placement of TCustomdxTreeList columns relative to the client window. Generally, the summary width of the TCustomdxTreeList column is greater than the width of the control. If users scroll columns to the right, left columns are hidden. This property determines the distance between the first column and the client window.

Use the LeftCoord property to horizontally scroll the ExpressQuantumGrid/TreeList. For example, the following code moves a grid by 10 pixels to the right.

```
dxDBGrid1.LeftCoord := dxDBGrid1.LeftCoord + 10;
```

See Also

TCustomdxTreeList.OnChangeLeftCoord

LockSelection property — read only

```
property LockSelection: Integer;
```

Description

Used internally. Determines whether a user can select nodes within a tree list control.

See Also

TCustomdxTreeList.BeginSelection; TCustomdxTreeList.EndSelection

LockUpdate property — read only

Determines whether changes in TCustomdxTreeList are visible.

```
property LockUpdate : Integer;
```

Description

Specifies how many times BeginMethod was called. If LockUpdate is greater than 0, all changes in TCustomdxTreeList are invisible. Every call of the BeginUpdate method increases the LockUpdate property by one. Every call of the EndUpdate method decreases the LockUpdate property by one.

See Also

TCustomdxTreeList.BeginUpdate; TCustomdxTreeList.EndUpdate

LookAndFeel property — read / write

Specifies the look of a control.

```
type
  TdxLookAndFeel = (lfStandard, lfFlat);
property LookAndFeel : TdxLookAndFeel;
```

Description

If LookAndFeel is lfFlat, a control has a flat border and flat column captions. If LookAndFeel equals lfStandard, it has a 3D look.

See Also

TCustomdxTreeList.PaintStyle

MaxRowLineCount property — read / write

Determines a column header's height in lines.

```
property MaxRowLineCount : integer;
```

Description

This property is effective only when:

?? The preview section is inactive.

?? The height of column header is one line.

?? The aoRowAutoHeight option is active.

The MaxRowLineCount determines the height of large cells in lines. Use this property to display large data such as memo fields.

Events		
Ticket_price	Event_Date	Event_Description
$5.00	6/16/96	World record for 500 meter speedskating could be set by the hometown favorite.
$7.50	6/21/96	Consult refund policy for cancellations due to weather.
$7.50	6/21/96	Held indoors at Harmon Arena. Last Olympic medal round went to an all-time high scoring game of 127
$7.50	6/19/96	Consult map of venue (City Route). This event has limited spectator access except in beachfront locations.

Note: *Activate the egoRowAutoHeight option for the ExpressQunatumGrid, since aoRowAutoHeight is used for the ExpressTreeList.*

NewItemRowActive property — read / write

Determines whether a new item row has focus.

```
property NewItemRowActive : Boolean;
```

Description

NewItemRowActive is True, when a user modifies data within a new item row.

Note: *NewItemRowActive is in effect only when ShowNewItemRow is set to True.*

See Also

TCustomdxTreeList.ShowNewItemRow

NewItemRowHeight property — read only

Returns the height of a new row area.

```
property NewItemRowHeight: Integer;
```

Description

The new row area is displayed if the ShowNewItemRow property is True. Use NewItemRowHeight to determine the height of a new item row in pixels.

See Also

TCustomdxTreeList.ShowNewItemRow

Options property — read / write

Specifies various display and behavioral properties of TCustomdxTreeList.

```
type
  TdxTreeListOption = (aoColumnSizing, aoColumnMoving, aoEditing, aoTabs,
aoTabThrough, aoRowSelect, aoMultiSelect, aoImmediateEditor, aoPreview,
aoDrawEndEllipsis, aoAutoWidth, aoExtMultiSelect, aoExtCustomizing,
aoHideFocusRect, aoAutoSort, aoCaseInsensitive);
  TdxTreeListOptions = set of TdxTreeListOption;
property Options : TdxTreeListOptions;
```

Description

Set Options to activate the desired properties for TCustomdxTreeList. The following flags are available:

Setting	Value
aoAutoSort	If set, the TCustomdxTreeList nodes are sorted by the column whose Sorted property differs from csNone. Automatic sorting is available only by one column.
aoAutoWidth	When set, ExpressTreeList changes all column widths in order to display all columns on screen thus eliminating the need for a horizontal scroll bar.
aoCaseInsensitive	This option is used for sorting nodes. If set, sorting is case insensitive.
aoColumnMoving	Users can move TCustomdxTreeList columns.
aoColumnSizing	Users can resize TCustomdxTreeList columns.
aoDrawEndEllipsis	The width of data in a column maybe greater than a column's width. When set, ellipsis ('…') are drawn to indicate to users that data did not fit inside the boundaries of a column.
aoEditing	Users can edit the contents of a node's columns.
aoExtCustomizing	Determines column customization style and behavior. By default, aoExtCustomizing is inactive. When a user moves (drags) a given column, this column is either displayed within the grid control or in the column customization window. When active, aoExtCustomizing enables users to remove active treelist columns from the treelist display by simply removing it from the treelist header region. This feature allows users to remove columns without first activating the column customization window.
aoExtMultiSelect	Extends the multi-selection ability. If set, users can include child nodes of different parents in multi-selection. Has no effect, if aoMultiSelect is not set.
aoHideFocusRect	Determines whether to draw a rectangle around a focused column. If set, the bound rectangle around a focused column is hidden.
aoImmediateEditor	If set, the inplace editor activates automatically when entering a column. Otherwise, the inplace editor activates only after a user has pressed a key or mouse button within a column.
aoMultiSelect	Enables Selection of several nodes at a time. By default, multi-selection includes only child nodes of the same parent node. To extend the selection to several parents use the aoExtMultiSelect flag.
aoPreview	Setting the aoPreviewFor flag enables a preview field, specified by the PreviewField property. Has no effect for TdxTreeList.
aoRowSelect	When set, selection of a particular column is unavailable. Entire nodes are selected rather than individual columns. If this flag is set, the aoEditing flag has no effect.
aoTabs	Enables navigation through columns using Tab and Shift+Tab keys. If False, rather than moving focus from one cell to another, TAB key press events move focus away from the treelist and onto the next control on a given form.
aoTabThrough	The Tab key press in the last column of a row (node) moves focus to the first column of the next row (node). This flag is only effective if the egoTabs Option is set.

Note: *[aoEditing, aoColumnSizing, aoColumnMoving, aoRowSelect, aoTabThrough] are the default flags for the Options property.*

OptionsEx property — read / write

Contains extended options of the tree list.

```
type
  TdxTreeListOptionEx = (aoInvertSelect, aoUseBitmap, aoBandHeaderWidth,
aoAutoCalcPreviewLines, aoBandSizing, aoBandMoving, aoHorzThrough,
aoVertThrough, aoCellMultiSelect, aoEnterThrough, aoEnterShowEditor,
aoFullSizing, aoDragScroll, aoDragExpand, aoDragCollapse, aoRowAutoHeight,
aoShowHourGlass, aoBandButtonClicking, aoBandPanelSizing,
aoHeaderButtonClicking, aoHeaderPanelSizing, aoRowSizing, aoMultiSort,
aoAutoSortRefresh, aoShowButtonAlways, aoAutoHeaderPanelHeight,
aoKeepColumnWidth, aoAnsiSort, aoMouseScroll, aoAutoSearch);
  TdxTreeListOptionsEx = set of TdxTreeListOptionEx;

property OptionsEx: TdxTreeListOptionsEx;
```

Description

The OptionsEx property contains options that control tree list behavior and its look and feel. OptionEx is similar to the Options property; its intent is to emphasize options that control new tree list features.

Value	Meaning
aoInvertSelect	If active, tree list highlights only focused columns (MS Outlook™ selection style); else all columns of a focused node except the currently selected column.
aoUseBitmap	If active, a tree list component uses a temporary bitmap for drawing cells within the tree list. It decreases flickers, but draws cells a bit slowly.
aoBandHeaderWidth	If active, band width is equal to the width of columns which belong to this band. This option is effective only when a band is not fixed (the Fixed property of TdxTreeListBand is not bfNone) or until a user re-sizes a band's width.
aoAutoCalcPreviewLines	If active, a tree list component automatically calculates the number of lines in the preview section. Otherwise, the number of the lines occupied by the preview section is defined by the PreviewLines property
aoBandSizing	Activate to allow users to change band size.
aoBandMoving	If active, users can move bands. Otherwise, band moving is disabled.
aoHorzThrough	If active, pressing the Right/Left arrows within the last/first column of a node moves focus to the next/previous node. Otherwise, the last/first column stays focused.
aoVertThrough	Only applicable if nodes are drawn on several rows (multiple lines). If active, pressing the Up/Down arrow moves focus through lines of the same node. Otherwise, it moves focus to the previous/next node.
aoCellMultiSelect	MS Excel™ style for multiple selection. This option is not implemented in ExpressTreeList.
aoEnterThrough	Pressing Enter moves focus to the next column. This option is only effective when aoEnterShowEditor is active.
aoEnterShowEditor	If active, an inplace editor is displayed after pressing Enter (by default it is active).
aoFullSizing	If active, the size of the last column or band changes even if the end user drags cursor out of the tree list window. Otherwise, the end user can resize the last column within the tree list window.

Copyright © 2000 Developer Express Inc.

aoDragScroll	If set, focus moves towards a dragged node during drag & drop operations.
aoDragExpand	If active, a collapsed node is expanded when a user drags a node to it.
aoDragCollapse	If active, an expanded node is collapsed when a user drags a node to it.
aoRowAutoHeight	If active, tree list automatically calculates the number of rows in a cell. Do not activate this option if the tree list includes a huge number of columns (~ 100 or more).
aoBandButtonClicking	If active, the OnBandButtonClick event is generated when a user clicks the band area indicator. See OnBandButtonClick for more detailed information.
aoBandPanelSizing	If active, a user can change the height of the band area.
aoHeaderButtonClicking	If active, the OnHeaderButtonClick event is generated when a user clicks the header area indicator. See OnHeaderButtonClick for more detailed information.
aoHeaderPanelSizing	If set, users can change header area height.
aoRowSizing	If active, users can change row height.
aoMultiSort	If active, multiple sorting is enabled. To sort a tree list by several columns, click these columns while holding the Shift key during the click events.
aoAutoSortRefresh	Turn it on to re-sort elements after adding/editing and deleting nodes.
aoShowHourGlass	If active, the tree list control displays an hourglass cursor when performing long operations. For instance, the hourglass cursor can be displayed during sort operations.
aoShowButtonAlways	If inactive, the tree list displays an edit button only within cells that belong to a focused node.
aoAutoHeaderPanelHeight	Within a tree list control, column headers can be located one under the other. If aoAutoHeaderpanelHeight is active, the tree list sets the height of the header area equal to the number of headers rows. This option is automatically disabled if the HeaderPanelRowChange property is changed or a user manually resizes the header panel at runtime.
aoKeepColumnWidth	If active, a tree list control does not change column widths back to initial values after deactivating the egoAutoWidths option.
aoAnsiSort	If active, the tree list control uses Windows locale settings to sort rows. This option is useful when cells of a tree list contain native language text.
aoMouseScroll	If active, movement of the mouse scrolls the dropdown lists in edited cells.
aoAutoSearch	Enables/disables the auto-search feature. When this option is active, the tree list control automatically locates a node after a user enters a search string within a focused column. It is also possible to search forward: Press Ctrl-Enter, or backward: Press Shift-Ctrl-Enter.

The default values for OptionEx include [aoUseBitmap, aoBandHeaderWidth, aoAutoCalcPreviewLines, aoBandSizing, aoBandMoving, aoDragExpand].

PaintStyle property **read / write**

Specifies the paint style of a control.

```
type
  TdxTreeListPaintStyle = (psStandard, psOutlook);
property PaintStyle : TdxTreeListPaintStyle;
```

Description

Use this property to set the paint style of a control. If the PaintStyle is equal to psOutlook, a control appears in Microsoft Outlook style. Otherwise, a control is in a tree view style.

See Also

TCustomdxTreeList.LookAndFeel

PreviewFont property — read / write

Specifies the preview field font.

```
property PreviewFont : TFont;
```

Description

Determines the font used for drawing the preview section of TdxDBGrid.

PreviewLines property — read / write

Determines the preview section height of a tree list.

```
property PreviewLines : integer;
```

Description

Use this property to specify the number of rows displayed in the preview section. If PreviewLines is 0, it is hidden.

See Also

TdxTreeList.OnGetPreviewLineCount

RowCount property — red only

Determines the number of rows for nodes in a tree list control.

```
property RowCount : Integer;
```

Description

Returns the number of rows for visible nodes in a tree list control. Unlike the VisibleRowCount property, RowCount counts only the rows, which completely fit inside the tree list control window.

See Also

TCustomdxTreeList.VisibleRowCount

Copyright © 2000 Developer Express Inc.

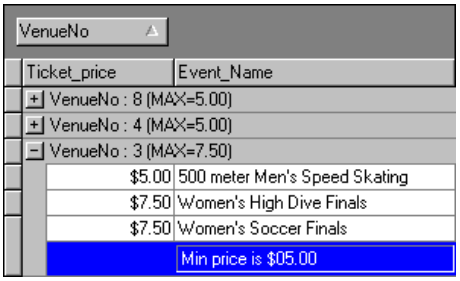
RowFooterColor property read / write

Specifies the color of the row footer.

```
property RowFooterColor : TColor;
```

Description

Row footer is an area that contains a group summary. Use this property to change the background color of the row footer. The picture below displays the row footer when RowFooterColor is set to clBlue:



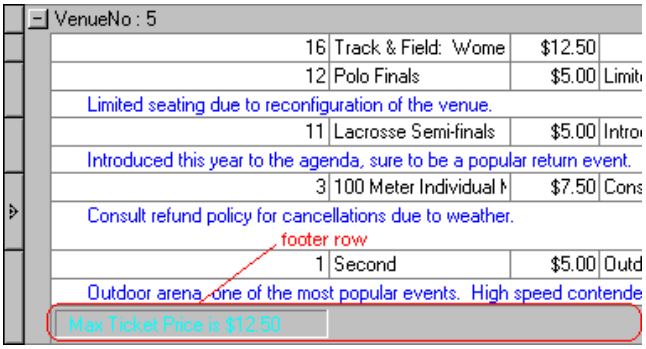
RowFooterTextColor property read / write

Determines the color of the footer row text.

```
property RowFooterTextColor: TColor;
```

Description

The footer row is the region that contains a group summary. Use this property to change footer row text color. The following picture displays the footer row when the RowFooterTextColor property is set to clAqua:



RowHeight property read only

Determines a node's height in TCustomdxTreeList.

```
property RowHeight : Integer;
```

Description

Returns the height of a node in a tree list control in pixels.

RowSeparatorLineWidth property — read / write

Determines the width of separator lines between nodes within a tree list control.

```
property RowSeparatorLineWidth: Integer;
```

Description

Use this property to set or access the width of horizontal separator lines between nodes of a tree list control. This property affects only horizontal lines.

See Also

TCustomdxTreeList.FixedBandLineWidth

ScrollBars property — read / write

Specifies whether a tree list control has horizontal and vertical scroll bars.

```
type
  TScrollStyle = (ssNone, ssHorizontal, ssVertical, ssBoth);
property ScrollBars: TScrollStyle;
```

Description

Use ScrollBars to provide a tree list control with horizontal or vertical scroll bars. ScrollBars can assume one of the following values:

Value	Meaning
ssNone	Tree list control does not include scroll bars.
ssHorizontal	Tree list control has a single scroll bar at the bottom edge.
ssVertical	Tree list control has a single scroll bar at the right edge.
ssBoth	Tree list control has a scroll bar at its bottom and right edges.

SelectedCount property — read only

Determines the number of selected nodes.

```
property SelectedCount : Integer;
```

Description

Returns the number of selected nodes in a tree list control.

Note: *This property is in effect only when the aoMultiSelect option is active (egoMultiSelect for the ExpressQuantumGrid). If inactive, SelectedCount always returns 0.*

The following code enables the Delete button of a TDBNavigator object when a user has selected nodes.

```
// Delphi
procedure TForm1.dxDBGrid1SelectedCountChange(Sender: TObject);
begin
  if dxDBGrid1.SelectedCount > 0 then
    DBNavigator1.VisibleButtons := DBNavigator1.VisibleButtons+[nbDelete]
  else
    DBNavigator1.VisibleButtons := DBNavigator1.VisibleButtons-[nbDelete];
end;

// C++Builder
void __fastcall TForm1::dxDBGrid1SelectedCountChange(TObject *Sender)
{
  if (dxDBGrid1->SelectedCount > 0)
    DBNavigator1->VisibleButtons = DBNavigator1->VisibleButtons <<
nbDelete;
  else
    DBNavigator1->VisibleButtons = DBNavigator1->VisibleButtons >>
nbDelete;
}
```

See Also

TCustomdxTreeList.SelectedNodes; TCustomdxTreeList.OnSelectedCountChange; TdxTreeListNode.Selected; TCustomdxTreeList.SelectionAnchor

SelectedNodes property **read only**

The SelectedNodes property is an array of selected nodes.

```
property SelectedNodes[Index:Integer] : TdxTreeListNode;
```

Description

Use this property to process the current selection within a tree list control. The node order in the SelectedNodes array does not correspond either to the order in which the nodes are displayed, or to the order of their selection. Index of the first selected node in the SelectedNodes array is 0. The total number of selected nodes is determined by the SelectedCount property.

Note: *The ExpressQuantumGrid inherits all methods and properties of TCustomdxTreeList. Use the SelectedNodes property to obtain cell values of the ExpressQuantumGrid when it is not in standard grid mode. In this instance, you can obtain values of selected cells using the Values property of TdxTreeListNode. When in standard grid mode, you should use the SelectedRows property of TCustomdxDBGrid to obtain selected cell values.*

The following code demonstrates how you can obtain the list of nodes in the order they are displayed on screen. (The ExpressQuantumGrid is in LoadAllRecords mode)

```
// Delphi
var
  i : integer;
  L : TList;

begin
  L := TList.Create;
  for i:= 0 to dxDBGrid1.SelectedCount - 1 do
    L.Add(dxDBGrid1.SelectedNodes[i]);
  L.Sort(@CompareByAbsoluteIndex);
  for i:= 0 to dxDBGrid1.SelectedCount - 1 do
    ListBox1.Items.Add(TdxTreeListNode(L.Items[i]).Strings[0]);
  L.Free;
end;

// C++Builder
  int i;
  TList * L;

  L = new TList();
  for(i= 0;i<dxDBGrid1->SelectedCount;i++)
    L->Add(dxDBGrid1->SelectedNodes[i]);
  L->Sort((int (__fastcall *)(void *, void *))CompareByAbsoluteIndex);
  for(i=0;i<dxDBGrid1->SelectedCount;i++)
    ListBox1->Items->Add( ((TdxTreeListNode *)L->Items[i])->Strings[0]);
  delete L;
```

See Also

TCustomdxTreeList.OnSelectedCountChange; TCustomdxDBGrid.SelectedRows; TdxTreeListNode.Selected; TCustomdxTreeList.SelectionA nchor; TCustomdxDBGrid.UseBookmarks; CompareByAbsoluteindex

SelectionAnchor property **read only**

Determines the first selected node.

```
property SelectionAnchor : TdxTreeListNode;
```

Description

If multi-select is available, users can select several nodes at a time. SelectionAnchor determines the first selected node. Other nodes are selected according to their position relative to SelectedAnchor.

See Also

TCustomdxTreeList.SelectedNodes; TCustomdxTreeList.OnSelectedCountChange

Copyright © 2000 Developer Express Inc.

ShowBands property read / write

Determines whether bands are displayed.

```
property ShowBands: Boolean;
```

Description

Use this property to show or hide bands within a tree list control. If ShowBands is True, bands are displayed.

See Also

TCustomdxTreeList.BandPanelHeight; TCustomdxTreeListControl.Bands; TdxTreeListBands; TdxTreeListBand

ShowButtons property read / write

Displays buttons to the left of each node with children.

```
property ShowButtons : Boolean;
```

Description

If True, '+' and '-' buttons are drawn to the left of every node with children. Pressing these buttons expands or collapses a node. You can also expand/collapse a node using the '+' or '-' keys on the keyboard or by a double-click event.

See Also

TCustomdxTreeList.ShowLines; TCustomdxTreeList.ShowRoot

ShowFooter property read / write

Determines whether to display footer at the bottom of a tree list.

```
property ShowFooter : Boolean;
```

Description

Footer is an area at the bottom of a tree list control containing summaries. If the ShowFooter property is True, then footer is displayed.

ShowGrid property read / write

Displays grid lines in a tree list control.

```
property ShowGrid : Boolean;
```

Description

If True, grid lines are drawn in a tree list control. The color of these grid lines is specified by the GridLineColor property.

See Also

TCustomdxTreeList.ShowLines; TCustomdxTreeList.ShowPreviewGrid

ShowHeader property read / write

Displays the header of a tree list control.

```
property ShowHeader : Boolean;
```

Description

The header is a section at the top of a tree list control, which contains column captions. If ShowHeader is True, a header is displayed. Header background color is determined by the HeaderColor property.

See Also

TCustomdxTreeList.ShowFooter

ShowHiddenInCustomizeBox property read / write

Determines whether the customization form contains columns that belong to hidden bands.

```
property ShowHiddenInCustomizeBox: Boolean;
```

Description

By default, ShowHiddenInCustomizeBox is False, so the customization form only displays columns that belong either to visible bands or do not belong to any band. When ShowHiddenInCustomizeBox is True, the customization form displays all columns, including hidden band columns.

See Also

TCustomdxTreeList.CustomizingForm; TCustomdxTreeList.SimpleCustomizeBox; TCustomdxTreeList.ColumnsCustomizing; TCustomdxTreeList.CustomizingBandListBox; TCustomdxTreeList.CustomizingHeadersListBox

ShowIndicator property read / write

Determines whether to show indicators on the left side of tree list nodes.

```
proeprty ShowIndicator : Boolean;
```

Description

If ShowIndicator is True, a row indicator is displayed.

See Also

TCustomdxDBTreeListControl.GrIndicatorWidth

ShowLines property read / write

Displays lines between parent and child nodes.

Copyright © 2000 Developer Express Inc.

```
property ShowLines : Boolean;
```

Description

If True, lines between parent nodes and linked child nodes are drawn. Set ShowLines to False to avoid drawing of these lines.

See Also

TCustomdxTreeList.ShowGrid; TCustomdxTreeList.ShowButtons; TCustomdxTreeList.ShowRoot

ShowNewItemRow property — read / write

Determines whether a new item row is displayed or hidden.

```
property ShowNewItemRow: Boolean;
```

Description

If ShowNewItemRow is True, a tree list control displays an empty node at the top of the control. A new item row allows users to easily add a new record to a tree list control. Users can thus enter data into the new row without having to navigate to the end of the grid to enter data.

Note: *ShowNewItemRow is not available if column headers are located on two or more rows.*

See Also

ShowPreviewGrid property — read / write

Determines whether to display only horizontal lines for a tree list.

```
property ShowPreviewGrid: Boolean;
```

Description

Use this property to customize the look of the grid within a tree list control. If ShowPreviewGrid is False and ShowGrid is True, only horizontal lines are displayed. If ShowPreviewGrid and ShowGrid are True, both vertical and horizontal lines are displayed. If ShowGrid is False, ShowPreviewGrid has no effect and both vertical and horizontal lines are hidden.

See Also

TCustomdxTreeList.ShowGrid

ShowRoot property — read / write

Displays a '+' button to the left of the first node as a root of all nodes.

```
property ShowRoot : Boolean;
```

Description

By default, ShowRoot is True and a '+' button appears to the left of the first node. Set ShowRoot to False to avoid the drawing of this button for the root node.

See Also

TCustomdxTreeList.ShowButtons; TCustomdxTreeList.ShowLines

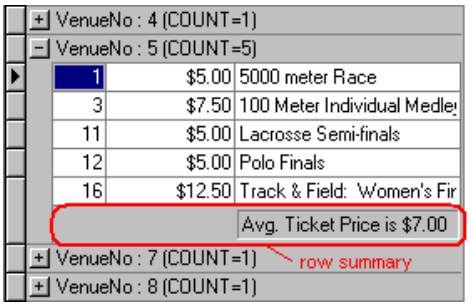
ShowRowFooter property — read / write

Determines whether to show the summary for grouping columns

```
property ShowRowFooter: Boolean;
```

Description

If ShowRowFooter is True, the summary footer for grouping column(s) is displayed.



See Also

TCustomdxTreeList.ShowFooter

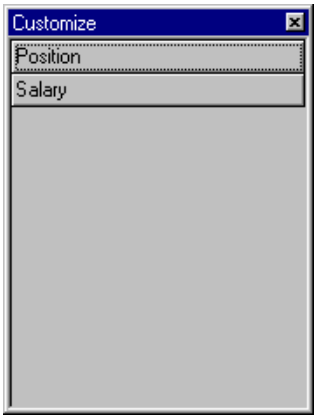
SimpleCustomizeBox property — read / write

Determines the style of the column customization form.

```
property SimpleCustomizeBox: Boolean;
```

Description

If SimpleCustomizingBox is False, the column customization form includes two lists: Bands and Headers. Otherwise, it includes only the header list.



See Also

TCustomdxTreeList.ColumnsCustomizing; TCustomdxTreeList.CustomizingForm; TCustomdxTreeList.CustomizingBandListBox; TCustomdxTreeList.CustomizingHeaderListBox; TCustomdxTreeList.ShowHiddenInCustomizeBox

State property — read / write

Determines a tree list's state.

```
type
  TdxTreeListState = TdxTreeListState = (tsNormal, tsColumnSizing,
tsEditing, tsColumnDragging, tsColumnDown, tsNodeDragging, tsNodeDown,
tsHeaderButtonDown, tsBandSizing, tsBandDragging, tsBandDown,
tsBandButtonDown, tsBandPanelSizing, tsHeaderPanelSizing, tsRowSizing);
property State : TdxTreeListState;
```

Description

This is a ReadOnly property. It returns the current state of a tree list control. The following table describes state indicators:

tsNormal	Indicates normal state of a tree list control. There are no columns or nodes being edited or dragged.
tsColumnSizing	User changes the width of columns.
tsEditing	The tree list is in editing mode.
tsColumnDraging	User drags one of the tree list columns.
tsColumnDown	Indicates that the mouse button has been pressed within one of the columns. This flag is included in the State property until the user releases the mouse button.
tsNodeDraging	User drags one of the tree list nodes.
tsNodeDown	Indicates that the mouse button has been pressed over a node. This flag is included in the State property until the user releases the mouse button.
tsHeaderButtonDown	Indicates that the header button is pressed. The header button is a button in the indicator column to the left of column headers.
tsBandSizing	User changes the width of a band.

tsBandDragging	User drags one of the tree list's bands.
tsBandDown	Indicates that the mouse button has been pressed over a band.
tsBandButtonDown	Indicates that the band button is pressed. Band button is a button in the indicator column to the left of bands.
tsBandPanelSizing	User changes the height of the band panel. Band panel is the region that contains bands.
tsHeaderPanelSizing	User changes the height of the header panel. The header panel is the area at the top of a tree list control that contains column headers.
tsRowSizing	Indicates that a user re-sized node height.

See Also

TCustomdxTreeList.Options; TCustomdxTreeList.OptionsEx

StateImages property — read / write

Specifies images that appear to the left of a node's text.

```
property StateImages : TImageList;
```

Description

Use this property to set or access an image list, which contains images displayed to the left of a node's text. The index of an image associated with the node is stored in the StateIndex property of TdxTreeListNode.

See Also

TCustomdxTreeList.Images; TdxTreeListNode.ImageIndex; TdxTreeListNode.SelectedIndex; TdxTreeListnode.StateIndex

TabStop property — read / write

Determines whether a tree list control can be selected via the Tab key.

```
property TabStop;
```

Description

If False, users cannot reach the tree list control using the Tab key. If True, a tree list control is in the Tab order. Despite TabStop is False, users may select it via mouse actions.

See Also

TCustomdxTreeList.TopNode; TCustomdxTreeList.TopVisibleNode

TopIndex property — read / write

Determines the index of the topmost visible node.

```
property TopIndex : Integer;
```

Description

Specifies the absolute index of the topmost visible node in the tree list control window. You can also refer to the topmost visible node via the TopVisibleNode property.

See Also

TCustomdxTreeList.TopNode; TCustomdxTreeList.TopVisibleNode

TopNode property — read only

Determines the first node within a tree list control.

```
property TopNode : TdxTreeListNode;
```

Description

Returns the first node in the node hierarchy of a tree list control.

See Also

TCustomdxTreeList.TopIndex; TCustomdxTreeList.TopVisibleNode

TopVisibleNode property — read only

Determines the topmost visible nodes within a tree list control window.

```
property TopVisibleNode : TdxTreeListNode;
```

Description

Specifies the topmost node within a tree list control. Vertical navigation through tree list nodes changes the TopVisibleNode property.

See Also

TCustomdxTreeList.TopNode; TCustomdxTreeList.TopIndex; TCustomdxTreeList.OnChangeTopVisibleNode

TreeLineColor property — read / write

Determines the color of tree lines.

```
property TreeLineColor: TColor;
```

Description

Tree lines are lines between a parent and child node. Use this property to set or access tree line color.

TreeLineStyle property — read / write

Determines tree line style.

```
type
```

```
  TdxTreeLineStyle = (tlSolid, tlDot);
property TreeLineStyle: TdxTreeLineStyle;
```

Description

Tree lines are lines between parent and child nodes within a tree list control. Use this property to specify the style in which tree lines are drawn. The available values include:

Value	Meaning
tlSolid	Solid line
tlDot	Dotted line

VisibleRowCount property — read only

Determines the number of visible rows for nodes in a tree list control.

```
property VisibleRowCount : Integer;
```

Description

Returns the total number of rows for the visible nodes in a tree list control. Unlike the RowCount property, VisibleRowCount returns the number of all visible rows. Regardless of that only a portion of a node is visible, the VisibleRowCount property counts it. If a node shifts to the right due to nodes on previous levels, VisibleRowCount also counts this a node.

See Also

TCustomdxTreeList.RowCount

WaitForExpandNodeTime property — read / write

Determines the time span before expanding a node.

```
property WaitForExpandNodeTime: Integer;
```

Description

Use this property to specify a time interval between the time a user presses the '+' button to expand a node and actual node expansion. This property is in effect if the aoDragExpand or aoDragCollapse option is active. If aoDragExpand is set, during drag & drop operations, collapsed nodes will be expanded. If aoDragCollapse is set, during drag & drop operations, expanded nodes will be collapsed. By default, WaitForExpandNodeTime is equal to 500 milliseconds.

Copyright © 2000 Developer Express Inc.

TCustomdxTreeList Methods

TCustomdxTreeList

Add	**public**	GetNodeAt	**public**
AddFirst	**public**	GetScrollVertGridRect	**protected**
AdjustColumnsWidth	**public**	GotoFirst	**public**
BeginSelection	**public**	GotoLast	**public**
BeginUpdate	**public**	GotoNext	**public**
CalcDrawInfo	**public**	GotoPrev	**public**
CancelEditor	**public**	HideEditor	**public**
CellRect	**public**	IndexOf	**public**
ClearColumnsSorted	**public**	Insert	**public**
ClearNodes	**public**	InvalidateSelection	**protected**
ClearSelection	**protected**	IsInvertSelect	**public**
CloseEditor	**public**	IsNodeVisible	**public**
ColumnsCustomizing	**public**	MakeBandVisible	**public**
CopyAllToClipboard	**public**	MakeColumnVisible	**public**
CopySelectedToClipboard	**public**	MakeNodeVisible	**public**
Create	**public**	PointInCustomizingForm	**public**
DeleteSelection	**protected**	SaveAllToStrings	**public**
Destroy	**public**	SaveAllToTextFile	**public**
EndColumnsCustomizing	**public**	SaveSelectedToStrings	**public**
EndSearch	**public**	SaveSelectedToTextFile	**public**
EndSelection	**public**	SelectNodes	**protected**
EndUpdate	**public**	SetEditingText	**protected**
FindInplaceEdit	**protected**	SetEditText	**protected**
FindNodeByText	**protected**	ShowButtonEditorMouse	**public**
FreeDrawInfo	**public**	ShowEditor	**public**
FullCollapse	**public**	ShowEditorChar	**public**
FullExpand	**public**	ShowEditorMouse	**public**
GetEditRect	**public**	StartDragBand	**protected**
GetFocusedAbsoluteIndex	**public**	StartDragHeader	**protected**
GetFocusedVisibleIndex	**public**	StartSearch	**public**
GetHitInfo	**public**	UpdateScrollBars	**protected**
GetHitTestInfoAt	**public**		
GetNextVisible	**public**		

Derived from TCustomdxContainer

BeginInitEdit	**protected**	GetReadOnly	**protected**
CanEditAcceptKey	**protected**	HideDrawFocusRect	**protected**
CanEditModify	**protected**	InplaceEditKeyDown	**protected**
CheckEditorWidth	**protected**	IsEditorMoved	**protected**
CreateEditStyle	**protected**	IsImmediateEditor	**protected**
EndInitEdit	**protected**	IsShowButtonAlways	**protected**
GetDefaultPopupBorderStyle	**public**	IsTabs	**protected**
GetEditColor	**protected**	MoveCol	**protected**
GetEditFont	**protected**	RedrawSelection	**protected**

Add method

Adds a new node to a tree list control.

```
function Add: TdxTreeListNode;
```

Description

This function creates and adds a new node to a tree list control. A new node is placed behind the last node at the topmost level within a tree list control. The Add function returns the newly created node. The Add method adds only the topmost level node. In order to create nodes at a lower level; use the TdxTreeListNode AddChild method.

The following example inserts new nodes before and after the first node of a TdxTreeList control.

```
// Delphi
procedure TForm1.Button1Click(Sender: TObject);
var
  N: TdxTreeListNode;
begin
  N := dxTreeList1.Insert(dxTreeList1.Items[0]);
  N.Values[0] := 'The first node';
  N := dxTreeList1.Add;
  N.Values[0] := 'The last node';
end;

// C++Builder
void __fastcall TForm1::Button1Click(TObject *Sender)
{
  TdxTreeListNode *N;

  N = dxTreeList1->Insert(dxTreeList1->Items[0]);
  N->Values[0] = "The first node";
  N = dxTreeList1->Add();
  N->Values[0] = "The last node";
}
```

See Also

TCustomdxTreeList.AddFirst; TCustomdxTreeList.Insert; TCustomdxTreeList.AddNode; TCusotmdxTreeList.Count

Copyright © 2000 Developer Express Inc.

AddFirst

Adds a node as the first node to a tree list control.

```
function AddFirst: TdxTreeListNode;
```

Description

Creates and adds a node, as the first node to a tree list control. A node is added before the first node at the topmost level in a tree list control. The AddFirst method returns the newly created node. The AddFirst method adds only topmost level nodes. In order to add a node to lower levels use the TdxTreeListNode AddChildFirst method.

See Also

TCustomdxTreeList.Add; TCustomdxTreeList.Insert; TCustomdxTreeList.AddNode

AdjustColumnsWidth method

Displays all visible columns within the client area of a tree list control.

```
procedure AdjustColumnsWidth;
```

Description

Some columns can be outside the client area of a tree list control. To reach these columns, users should employ the horizontal scroll bar. The AdjustColumnsWidth method changes column widths in such a manner as to display all visible columns on screen.

Note: *The AdjustColumnsWidth method deactivates the aoAutoWidth option (egoAutoWidth option for the ExpressQuantumGrid).*

See Also

TCustomdxTreeList.OptionsEx; TCustomdxDBGrid_OptionsEx

BeginSelection method

Prevents multiple generation of the OnSelectedCountChange event.

```
procedure BeginSelection;
```

Description

BeginSelection and EndSelection work together to prevent multiple generation of the OnSelectedCountChange event. Every call of BeginSelection increases the internal counter by one. Every call of EndSelection decreases this counter by one. The OnSelectedCountChange event is generated when the internal counter is 0. Be sure to call EndSelection after each call of BeginSelection to successfully work with the tree list.

See Also

TCustomdxTreeList.EndSelection; TCustomdxTreeList.OnChangeSelectedCount; TCustomdxTreeList.SelectedNodes

BeginUpdate method

Disables tree list refresh operations.

```
procedure BeginUpdate;
```

Description

Calling the BeginUpdate method disables tree list repainting. If the BeginUpdate procedure has been called, any changes to nodes (or columns) are not refreshed until calling the EndUdpate method. The BeginUpdate and EndUpdate methods work together to prevent numerous updates of a tree list control when adding nodes or columns. Be sure to call the EndUpdate method after calling BeginUpdate. Every call of the BeginUpdate method increases the internal counter by one. Every call of the EndUpdate decreases this counter by one. A tree list control can refresh itself only when the internal counter is 0. Assume you need to add several nodes to your tree list. If you do not use BeginUpdate and EndUpdate methods, the tree list redraws itself as many times as the number of added nodes. Instead of prolonged delays, you may use BeginUpdate before adding the first node and EndUpdate after adding the last node; And thus, the control redraws itself only once.

See Also

TCustomdxTreeList.EndUpdate; TCustomdxTreeList.LockUpdate

CalcDrawInfo method

Calculates drawing area size in a tree list control.

```
procedure CalcDrawInfo(var DrawInfo: TdxGridDrawInfo);
```

Description

CalcDrawInfo is called when drawing a tree list control. This method fills in the DrawInfo structure with data on size and layout of grid drawing areas: the size of bands, column headers, footers, etc. Calculation of drawing areas allocates additional memory for TdxGridDrawInfo fields; therefore, after calling CalcDrawInfo call the FreeDrawInfo method to free the allocated memory.

See Also

TCustomdxTreeList.FreeDrawInfo; TCustomdxTreeList.GetHitInfo; TdxGridDrawInfo; TdxGridBandDrawInfo; TdxGridHeaderDrawInfo; TdxGridEmptyRectDrawInfo; TdxGridRowDrawInfo

CancelEditor method

Closes an inplace editor.

```
procedure CancelEditor;
```

Description

The function closes an open inplace editor. Sets the tree list InplaceColumn property to -1, the InplaceEditor property to nil and calls the CloseEditor method.

See Also

TCustomdxTreeList.ShowEditor; TCustomdxTreeList.CloseEditor; TCustomdxTreeList.HideEditor; TCustomdxTreeList.EditingText; TCustomdxTreeList.OnEditing; TCustomdxTreeList.OnEdited

Copyright © 2000 Developer Express Inc.

CellRect method

Returns rectangle, occupied by a specific cell.

```
function CellRect(ANode:TdxTreeListNode; AColumn : Integer): TRect;
```

Description

This method returns a rectangle of a specified tree list cell. The ANode and AColumn parameters determine a tree list node to which a cell and index of a cell's column belong. CellRect returns a rectangle in a tree list control coordinates.

ClearColumnsSorted method

Clears the list of sorted columns.

```
procedure ClearColumnsSorted;
```

Description

This method removes columns from the column-sorting list. After calling this method a tree list control has no sorted columns.

ClearNodes method

Removes all nodes from a tree list.

```
procedure ClearNodes;
```

Description

Use this method to destroy all nodes in a tree list control.

See Also

TCustomdxTreeList.Items; TCustomdxTreeList.DeleteNode; TCustomdxTreeList.CreateNode; TCustomdxTreeList.OnDeletion

ClearSelection method

Clears selection in a tree list.

```
procedure ClearSelection; virtual;
```

Description

The purpose of this method is to un-select all selected nodes in a tree list. Descendants override this method as public.

Note: *Despite the fact that a tree list has no selected node after calling the ClearSelection method, it still has focus.*

See Also

TCustomdxTreeList.SelectedNodes; TCusotmdxTreeList.DeleteSelection

CloseEditor method

Closes an active inplace editor.

```
procedure CloseEditor;
```

Description

Closes an active inplace editor. The CloseEditor method is called when pressing the Esc, Enter keys on the keyboard or when a user navigates to another node while an inplace editor is active. CloseEditor validates data modifications in a column, updates a column on screen and turns the tree list component to a tsNormal state.

See Also

TCustomdxTreeList.ShowEditor; TCustomdxTreeList.CancelEditor; TCustomdxTreeList.EditingText; TCustomdxTreeList.OnEditing; TCustomdxTreeList.OnEdited

ColumnsCustomizing method

Starts column customizing.

```
procedure ColumnsCustomizing;
```

Description

The intent of this method is to create the column customizing form and display it on screen.

See Also

TCustomdxTreeList.OnEndColumnsCustomizing; sdxGrColumns; sdxGrBandsCaption; sdxGrHeadersCaption; TCustomdxTreeList.CustomizingForm; TCustomdxTreeList.CustomizingPos

CopyAllToClipboard method

Copies all nodes of a tree list control to the Clipboard.

```
procedure CopyAllToClipboard;
```

Description

Use this method to copy all nodes of a tree list control to the Windows clipboard. Columns are delimited by Tabs. Column headers are also copied.

See Also

TCustomdxTreeList.CopySelectedToClipboard; TCustomdxTreeList.SaveSelectedToStrings; TCustomdxTreeIlst.SaveAllToStrings

CopySelectedToClipboard method

Copies selected nodes to the clipboard.

```
procedure CopySelectedToClipboard;
```

Description

Use this method to copy selected tree list nodes to the Windows clipboard. This method is called when a user presses the Ctrl-C or Ctrl-Ins keys to copy selected nodes.

See Also

TCustomdxTreeList.CopyAllToClipboard; TCustomdxTreeList.SaveSelectedToStrings; TCustomdxTreeIlst.SaveAllToStrings

Create method

Creates an instance of type TCustomdxTreeList.

```
constructor Create(AOwner : TComponent);  override;
```

Description

Creates and initializes TCustomdxTreeList. After calling the inherited contstructor, the Create method sets all tree list properties to their default values.

DeleteSelection method

Removes selected nodes from a tree list.

```
procedure DeleteSelection;  virtual;
```

Description

Deletes all selected nodes from a tree list control; However, it still maintains focus. Descendants override it as public.

See Also

TCustomdxTreeList.ClearSelection

Destroy method

```
destructor Destroy;  override;
```

Description

Destroys an instance of the TCustomdxTreeList type and releases any memory allocated.

EndColumnsCustomizing method

Closes the column customizing form.

```
procedure EndColumnsCustomizing;
```

Description

Override this method to perform special operations right before or after closing the column customizing form.

See Also

TCustomdxTreeList.ColumnsCustomizing; TCustomdxTreeList.OnEndColumnsCustomizing

EndSearch method

Deactivates search mode.

```
procedure EndSearch;
```

Description

When search mode is active, a tree list control draws an extra frame around a focused cell. The EndSearch method deactivates the extra frame.

Note: *Call this method after calling StartSearch.*

See Also

TCustomdxTreeList.StartSearch; TCustomdxTreeList.AutoExpandOnSearch; TCustomdxTreeList.AutoSearchColor; TCustomdxTreeList.AutoSearchTextColor; TCustomdxTreeList.OptionsEx

EndSelection method

This method is used to prevent multiple generation of the OnSelectedCountChange event.

```
procedure EndSelection;
```

Description

BeginSelection and EndSelection work together to prevent multiple generation of the OnSelectedCountChange event. Every call of BeginSelection increases the internal counter by one. Every call of EndSelection decreases this counter by one. EndSelection generates the OnSelectedCountChange event if the internal counter is 0. Be sure to call EndSelection after each call of BeginSelection.

See Also

TCustomdxTreeList.BeginSelection

EndUpdate method

Enables tree list control to refresh operations.

```
procedure EndUpdate;
```

Description

The BeginUpdate and EndUpdate methods work together to prevent numerous updates of a tree list control when adding nodes or columns. Be sure to call the EndUpdate method after calling

BeginUpdate. Every call of the BeginUpdate method increases the internal counter by one. Every call of the EndUpdate method decreases the internal counter by one. A tree list control can refresh itself only when the internal counter is 0.

See Also

TCustomdxTreeList.BeginUpdate

FindInplaceEdit method

Returns the inplace editor type for a specific column.

```
function FindInplaceEdit(AColumn : Integer): TdxInplaceEdit;  virtual;
```

Description

Different columns in a tree list control can have different types of editors. FindInpalceEditor returns a specific inplace editor for a particular column, which is determined by the AColumn parameter.

FindNodeByText method

Searches for a node that contains text within a specified cell.

```
type
  TdxTreeListSearchDirection = (sdNone, sdUp, sdDown);
function FindNodeByText(AColumnIndex: Integer; const AText: string;
ADirection: TdxTreeListSearchDirection; var ANode: TdxTreeListNode):
Boolean; virtual;
```

Description

The FindNodeByText function searches for a node, which contains AText in the column specified by AColumnIndex. ADirection specifies search direction (relative to the current node):

sdNone	Forward. If the current node matches the search condition, searching is halted.
sdUp	Backward
sdDown	Forward, regardless of the current node.

After a successful search, FindNodeByText returns True and the ANode parameter specifies the located node. Otherwise, FindNodeByText returns False and ANode is equal to nil.

FreeDrawInfo method

Releases memory allocated when calling CalcDrawInfo.

```
procedure FreeDrawInfo(var DrawInfo: TdxGridDrawInfo);
```

Description

Call this method after calling the CalcDrawInfo method to free memory allocated when calculating the size of drawing areas in a tree list control.

See Also

TCustomdxTreeList.CalcDrawInfo

FullCollapse method

Collapses all nodes within a tree list control.

```
procedure FullCollapse;
```

Description

This method performs full collapse operations of tree list nodes. Call FullCollapse to hide all nodes in a tree list except for first level nodes.

FullExpand method

Expands all nodes within a tree list control.

```
procedure FullExpand;
```

Description

This method performs full expand operations of tree list nodes. Call FullExpand to display all nodes in a tree list. If the ShowButtons property is True, all buttons change from '+' to '-'.

GetEditRect method

Returns a rectangle occupied by a specific cell.

```
function GetEditRect(Node: TdxTreeListNode; Column: Integer): TRect;
```

Description

The GetEditRect method returns a rectangle of a cell in a tree list control. The Node and Column parameters determine the cell.

Use GetEditRect to obtain an area for a potential inplace editor. For instance, GetRectInfo can be used to highlight a potential edited cell when a user places the mouse inside of a cell.

The following code highlights a cell within the ExpressQuantumGrid when the mouse is placed above it.

```
var
  . . .
  CurNode : TdxTreeListNode;
  CurColumn : TdxDBTreeListColumn;

  . . .

// The onMouseMove event handler.
```

```
procedure TForm1.dxDBGrid1MouseMove(Sender: TObject; Shift: TShiftState; X,
Y: Integer);
var
 ARect : TRect;
 P : TPoint;
 AColumn : TdxDBTreeListColumn;
 ANode : TdxTreeListNode;
begin
  GetCursorPos(P);
  P := ScreenToClient(P);
  ANode := dxDBGrid1.GetNodeAt(P.x, P.y);
  AColumn := dxDBGrid1.GetColumnAt(P.x, P.y);
  if (ANode <> nil) and (AColumn <> nil) then
  begin
    ARect := dxdBGrid1.GetEditRect(ANode, AColumn.Index);
    if PtInRect(ARect, P) then
    begin
      CurNode := ANode;
      CurColumn := AColumn;
    end
    else
    begin
      CurNode := nil;
      CurColumn := nil;
    end;
    dxDBGrid1.Invalidate;
   end;
end;

// The OnCutomDrawCEll event handler.
procedure TForm1.dxDBGrid1CustomDrawCell(Sender: TObject; ACanvas: TCanvas;
ARect: TRect; ANode: TdxTreeListNode; AColumn: TdxTreeListColumn;
ASelected, AFocused, ANewItemRow: Boolean; var AText: String; var AColor:
TColor; AFont: TFont; var AAlignment: TAlignment; var ADone: Boolean);
begin
  if (ANode = CurNode) and (AColumn = CurColumn) then
  begin
    AColor := clSilver;
    AFont.Color := clBlue;
  end;
end;
```

GetFocusedAbsoluteIndex method

Returns the absolute index of a focused column.

```
function GetFocusedAbsoluteIndex(FocusedIndex: Integer): Integer;
```

Description

Use this method to obtain index of a focused column in the Columns array (the Columns property).

See Also

TCustomdxTreeList.GetFocusedVisibleIndex; TCustomdxTreeList.FocusedAbsoluteIndex

GetFocusedVisibleIndex method

Returns position of a focused column among visible columns of a tree list.

```
function GetFocusedVisibleIndex(AbsoluteIndex: Integer): Integer;
```

Description

Use this method to obtain index of a column in the VisibleColumns array (the VisibleColumns property). AbsoluteIndex is the index of a column in the Columns array (the Columns property).

See Also

TCustomdxTreeList.GetFocusedAbsoluteIndex; TCustomdxTreeList.FocusedAbsoluteIndex

GetHitInfo method

Returns information on an area that covers a point with specified coordinates.

```
type
  TdxTreeListHitTest = (htNowhere, htColumn, htColumnEdge, htButton,
htIcon, htStateIcon, htIndent, htLabel, htRight, htOutside, htGroupPanel,
htBandButton, htBand, htBandEdge, htHeaderButton, htIndicator,
htNewRowItem, htSummaryNodeFooter, htSummaryFooter, htBandPanelEdge,
htHeaderPanelEdge, htRowEdge, htPreview);
TdxTreeListHitInfo = record
  Band: Integer;
  Column: Integer;
  Node: TdxTreeListNode;
  Row: Integer;
  FooterRow: Integer;
  hitType: TdxTreeListHitTest;
end;
function GetHitInfo(Pos: TPoint): TdxTreeListHitInfo;
```

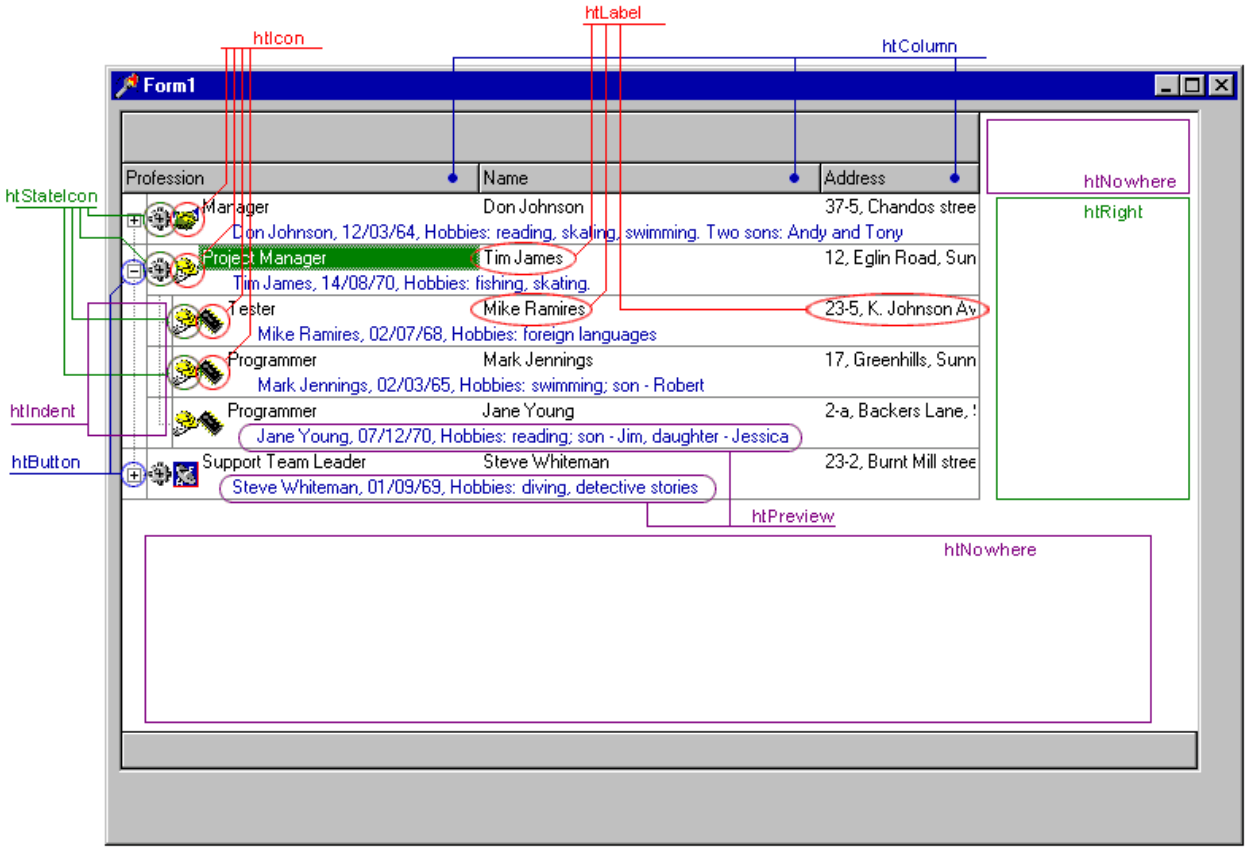
Copyright © 2000 Developer Express Inc.

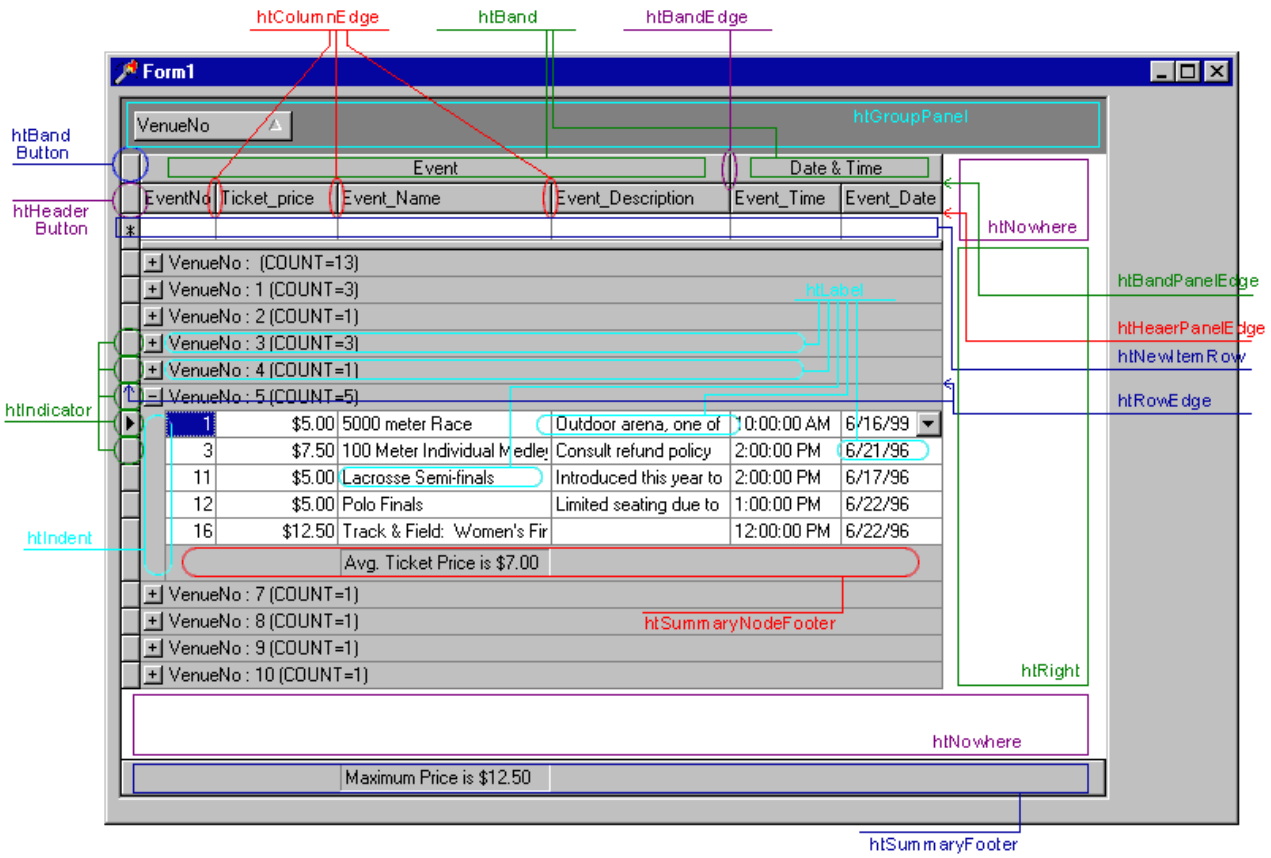
Description

Use the GetHitInfo method to determine to which region within a tree list control a specified point belongs. TdxTreeListhitInfo provides the following information:

Band	Returns the *visible* index of a band.
Column	Returns the *absolute* index of a column.
Node	Determines the node which contains the specified point.
Row	Index of the row relative to the first visible row.
FooterRow	Returns index of the footer row.
hitType	Determines which area contains the specified point.

hitType	Description
htNowhere	The point does not belong to any region within a tree list control.
htColumn	The point belongs to the column header.
htColumnEdge	The point belongs to the column edge.
htButton	The point belongs to the '+' or '-' buttons.
htIcon	The point belongs to an image to the left of a node.
htStateIcon	The point belongs to a static icon to the left of a node.
htIndent	The point belongs to the indent area.
htLabel	The point belongs to any tree list cell (except for new item row cells).
htRight	The point belongs to the empty area to the right of the last column.
htOutside	The point does not belong to the tree list control.
htGroupPanel	The point belongs to the group panel.
htBandButton	The point belongs to the band button.
htBand	The point belongs to the band caption.
htBandEdge	The point belongs to the band edge.
htHeaderButton	The point belongs to the header button.
htIndicator	The point belongs to the indicator area.
htNewRowItem	The point belongs to the new item row.
htSummaryNodeFooter	The point belongs to the row summary.
htSummaryFooter	The point belongs to the footer summary.
htBandPanelEdge	The point belongs to the bottom edge of the band caption.
htHeaderPanelEdge	The point belongs to the bottom edge of the column header.
htRowEdge	The point belongs to the line that separates two nodes.
htPreview	The point belongs to the preview region.


htIcon
htLabel
htColumn
Form1
htStateIcon
htIndent
htButton
htNowhere
htRight
htPreview
htNowhere
Profession
Name
Address
Manager
Don Johnson
37-5, Chandos stree
Don Johnson, 12/03/64, Hobbies: reading, skating, swimming. Two sons: Andy and Tony
Project Manager
Tim James
12, Eglin Road, Sun
Tim James, 14/08/70, Hobbies: fishing, skating.
Tester
Mike Ramires
23-5, K. Johnson Av
Mike Ramires, 02/07/68, Hobbies: foreign languages
Programmer
Mark Jennings
17, Greenhills, Sunn
Mark Jennings, 02/03/65, Hobbies: swimming; son - Robert
Programmer
Jane Young
2-a, Backers Lane, !
Jane Young, 07/12/70, Hobbies: reading; son - Jim, daughter - Jessica
Support Team Leader
Steve Whiteman
23-2, Burnt Mill stree
Steve Whiteman, 01/09/69, Hobbies: diving, detective stories


htColumnEdge
htBand
htBandEdge
Form1
htBand Button
htHeader Button
htIndicator
htIndent
htGroupPanel
htNowhere
htBandPanelEdge
htHeaerPanelEdge
htNewItemRow
htRowEdge
htLabel
htSummaryNodeFooter
htRight
htNowhere
htSummaryFooter
VenueNo
Event
Date & Time
EventNo
Ticket_price
Event_Name
Event_Description
Event_Time
Event_Date
VenueNo : (COUNT=13)
VenueNo : 1 (COUNT=3)
VenueNo : 2 (COUNT=1)
VenueNo : 3 (COUNT=3)
VenueNo : 4 (COUNT=1)
VenueNo : 5 (COUNT=5)
1 $5.00 5000 meter Race Outdoor arena, one of 10:00:00 AM 6/16/99
3 $7.50 100 Meter Individual Medley Consult refund policy 2:00:00 PM 6/21/96
11 $5.00 Lacrosse Semi-finals Introduced this year to 2:00:00 PM 6/17/96
12 $5.00 Polo Finals Limited seating due to 1:00:00 PM 6/22/96
16 $12.50 Track & Field: Women's Fir 12:00:00 PM 6/22/96
Avg. Ticket Price is $7.00
VenueNo : 7 (COUNT=1)
VenueNo : 8 (COUNT=1)
VenueNo : 9 (COUNT=1)
VenueNo : 10 (COUNT=1)
Maximum Price is $12.50

The following code demonstrates a way to display hints for different areas of the ExpressQuantumGrid: column headers and summary cells. Dipslaying hints for other ExpressQuantumGrid regions can be performed in a similar manner.

Initial settings for ExpressQuantumGrid properties:

```
Hint = ‘’
ShowHint = False
```

```
procedure TForm1.dxDBGrid1MouseMove(Sender: TObject; Shift: TShiftState; X,
Y: Integer);
var
  G: TdxDBGrid;
  P : TPoint;
  Col : TdxDBTreeListColumn;
  HitTest: TdxTreeListHitInfo;
begin
  G := Sender as TdxDBGrid;
  P.x := X; P.y := Y;
  HitTest := G.GetHitInfo(P);
  G.Hint := '';
  if HitTest.hitType = htColumn then
  begin
    Col := G.GetColumnAt(X, Y);
    if Col <> nil then
      G.Hint := 'Column: ' + Col.Caption;
  end;
  if HitTest.hitType = htSummaryFooter then
  begin
    Col := G.GetFooterColumnAt(X,Y);
    if Col <> nil then
      G.Hint := 'Symmary: ' + Col.Caption;
  end;
  if HitTest.hitType = htSummaryNodeFooter then
  begin
    Col := G.GetNodeFooterColumnAt(X,Y);
    if Col <> nil then
      G.Hint := 'Node Symmary: ' + Col.Caption;
  end;
  if G.Hint <> '' then
    G.Showhint := True
  else
    G.ShowHint := False;
end;
```

GetHitTestInfoAt method

Determines whether a point belongs to a specific area within a tree list control.

```
type
  TdxTreeListHitTest = (htNowhere, htColumn, htColumnEdge, htButton,
htIcon, htStateIcon, htIndent, htLabel, htRight, htOutside, htGroupPanel,
htBandButton, htBand, htBandEdge, htHeaderButton, htIndicator,
htNewRowItem, htSummaryNodeFooter, htSummaryFooter, htBandPanelEdge,
htHeaderPanelEdge, htRowEdge, htPreview);
function GetHitTestInfoAt(X, Y: Integer): TdxTreeListHitTest;
```

Description

Use GetHitTestInfoAt to obtain the area within a tree list control which contains a point specified by X and Y coordinates. This function uses the GetHitInfo method to return a proper value. For complete information on TdxTreeListHitTest, see the GetHitInfo topic.

GetNextVisible method

Returns the next visible node after the specified node.

```
function GetNextVisible(Node: TdxTreeListNode): TdxTreeListNode;
```

Description

Use this method to obtain the next visible node after the node, specified by the Node parameter. If a node is the last visible, GetNextVisible returns nil.

GetNodeAt method

Returns a node under specified coordinates.

```
function GetNodeAt(X, Y: Integer): TdxTreeListNode;
```

Description

Use this method to access a node under a point, specified by X and Y coordinates. If a node under such coordinates does not exist, GetNodeAt returns nil.

See Also

TCustomdxTreeList.GetEditRect

GetScrollVertGridRect method

Returns the scrolled rectangle of a tree list control.

```
function GetScrollVertGridRect: TRect; virtual;
```

Description

GetScrollVertGridRect returns the rectangle which is actually scrolled within a tree list control, i.e. this region does not include non-scrolled areas such as column and band headers, summary footer rows, scrollbars, etc.

GotoFirst method

Returns the first node within a tree list control.

```
function GotoFirst;
```

Description

Use the **GotoFirst** method to return the first node in the node hierarchy of a tree list control.

See Also

TCustomdxTreeList.GotoLast; TCustomdxTreeList.GotoPrev; TCustomdxTreeList.GotoNext

GotoLast method

Returns the last node of a tree list control.

```
procedure GotoLast(AGotoHidden: Boolean);
```

Description

Use this method to return the last node of the TdxTreeList or TdxDBGrid controls.

The *AGotoHidden* parameter specifies whether to return the last visible node or the last hidden one.

See Also

TCustomdxTreeList.GotoFirst; TCustomdxTreeList.GotoPrev; TCustomdxTreeList.GotoNext

GotoNext method

Specified the next node accessibility.

```
function GotoNext(AGotoHidden: Boolean): Boolean;
```

Description

Use the **GotoNext** method to specify whether the next node in the tree list (grid) hierarchy relative to a given node can be obtained.

The *AGotoHidden* parameter specifies the visibility of the obtaining node.

See Also

TCustomdxTreeList.GotoFirst; TCustomdxTreeList.GotoPrev; TCustomdxTreeList.GotoLast

GotoPrev method

Specified the previous node accessibility.

```
function GotoPrev(AGotoHidden: Boolean): Boolean;
```

Description

Use the GotoPrev method to specify whether the previous node in the tree list (grid) hierarchy relative to a given node can be obtained.

The *AGotoHidden* parameter specifies the visibility of the obtaining node.

See Also

TCustomdxTreeList.GotoFirst; TCustomdxTreeList.GotoNext; TCustomdxTreeList.GotoLast

HideEditor method

Hides an inplace editor.

```
procedure HideEditor;
```

Description

This method hides active inplace editors and changes tree list state from tsEditing to tsNormal.

See Also

TCustomdxTreeList.ShowEditor; TCustomdxTreeList.CloseEditor; TCustomdxTreeList.CancelEditor

IndexOf method

Returns the index of a node among root nodes of a tree list.

```
function IndexOf(Value: TdxTreeListNode): Integer;
```

Description

Use this method to obtain the Value index in the list of root nodes of a tree list control (the Items property). If a node is not in the list of root nodes, this function returns -1.

Insert method

Inserts a new node into a tree list before BeforeNode.

```
function Insert(BeforeNode: TdxTreeListNode): TdxTreeListNode;
```

Description

This method creates a new node with empty columns and inserts it into the list of root nodes before a node specified by the BeforeNode parameter. The node, specified by BeforeNode should be a root node; otherwise, Insert raises an exception.

See Also

TCustomdxTreeList.Add; TCustomdxTreeList.AddFirst

Copyright © 2000 Developer Express Inc.

InvalidateSelection method

Redraws all selected nodes.

```
procedure InvalidateSelection;  virtual;
```

Description

This method repaints all selected node within a tree list control.

IsInvertSelect method

```
function IsInvertSelect: Boolean;
```

Description

Returns True if the aoInvertSelect option is active. See the OptionEx property for more information on this feature.

See Also

TCustomdxTreeList.OptionEx

IsNodeVisible method

Determines whether a specified node is visible on screen.

```
function IsNodeVisible(Node: TdxTreeListNode): Boolean;
```

Description

IsNodeSelected returns True if a tree list node, specified by Node, is visible; otherwise False.

See Also

TdxTreeListNode.IsVisible; TdxTreeListNode.MakeVisible; TCustomdxTreeList.MakeNodeVisible

MakeBandVisible method

Displays a band on screen.

```
procedure MakeBandVisible(VisibleIndex: Integer);
```

Description

A tree list band becomes invisible when a user horizontally scrolls the tree list window. Use this method to show a band, specified by the Visible index, on screen. VisibleIndex is the index of the band among visible tree list bands. This method displays a band only if the ShowBands property is True.

See Also

TCustomdxTreeList.MakeNodeVisible; TCustomdxTreList.MakeColumnVisible; TdxTreeListBand.Visible

MakeColumnVisible method

```
procedure MakeColumnVisible(AbsoluteIndex: Integer);
```

Description

Scrolls the tree list control horizontally to display a specific column on screen. The AbsoluteIndex parameter determines the absolute index of a column. If a column is invisible, MakeColumnVisible has no impact. For instance, the following code displays the 'Name' column on screen:

```
var
  ColIndex , i : integer;
begin
  ColIndex := -1;
  for i := 0 to dxTreeList1.ColumnCount -1 do
    if dxTreeList1.Columns[i].Caption = 'Name' then
      ColIndex := dxTreeList1.Columns[i].Index;
  if ColIndex <> -1 then
  begin
    dxTreeList1.Columns[ColIndex].Visible := True;
    dxTreeList1.MakeColumnVisible(ColIndex);
  end;
end;
```

See Also

TCustomdxTreeList.MakeBandVisible; TCustomdxTreeList.MakeNodeVisible

MakeNodeVisible method

Makes a specified node visible.

```
procedure MakeNodeVisible(Node: TdxTreeListNode);
```

Description

If a node is invisible due to the fact that its parent is collapsed, MakeNodeVisible expands the parent node to display the specified item on screen.

See Also

TdxTreeListNode.Visible; TCustomdxTreeList.MakeColumnVisible; TCustomdxTreeList.MakeBandVisible; TdxTreeListNode.MakeVisible; TdxTreeList.IsVisible

PointInCustomizingForm method

Determines whether a point belongs to the customization form.

```
function PointInCustomizingForm(P:TPoint): Boolean;
```

Description

Returns True, if a point, specified by P, is in the column customization form; otherwise False.

See Also

TCustomdxTreeList.CustomizingForm; TCustomdxTreeList.ColumnsCustomizing

SaveAllToStrings method

Saves all tree list nodes into a list of strings.

```
procedure SaveAllToStrings(List : TStrings);  virtual;
```

Description

Use this method to store all nodes to a list of type TStrings. All columns of a node are concatenated in a string, which is added to List. Thus, every node corresponds to a separate string in List.

See Also

TCustomdxTreeList.SaveAllToTextFile; TCustomdxTreeList.SaveSelectedToTextFile; TCustomdxTreeList.SaveSelectedToStrings; TCustomdxTreeList.CopyAllToClipboard

SaveAllToTextFile method

Saves all tree list nodes to a text file, specified by the FileName parameter.

```
procedure SaveAllToTextFile(const FileName : String);
```

Description

Use this method to store tree list nodes to a text file. SaveAllToTextFile saves both visible and invisible columns to a file. The file's first line contains names of tree list columns. The FileName parameter entirely describes the name of a file to which to store nodes. No file extension is added to the file's name.

See Also

TCustomdxTreeList.SaveAllToStrings; TCustomdxTreeList.SaveSelectedToTextFile; TCustomdxTreeList.SaveSelectedToStrings; TCustomdxTreeList.CopyAllToClipboard

SaveSelectedToStrings method

Saves selected nodes of a tree list into the list of strings.

```
procedure SaveSelectedToStrings(List : TStrings);  virtual;
```

Description

Use this method to store all selected nodes to the list of TStrings type. All columns of a selected node are concatenated in a string, which is added to List. Every selected node corresponds to a separate string in List.

See Also

TCustomdxTreeList.SaveAllToStrings; TCustomdxTreeList.SaveSelectedToTextFile; TCustomdxTreeList.SaveAllToTextFile; TCustomdxTreeList.CopyAllToClipboard

SaveSelectedToTextFile method

Saves all selected nodes to a text file, specified by the FileName parameter.

```
procedure SaveSelectedToTextFile(const FileName : String);
```

Description

Use this method to store selected nodes of a tree list to a text file. SaveSelectedToTextFile saves both visible and invisible columns to a file. The first line of this file contains names of tree list columns. The FileName parameter fully describes the name of the file to which to store nodes. No file extension is added to the file name.

See Also

TCustomdxTreeList.SaveAllToStrings; TCustomdxTreeList.SaveAllToTextFile; TCustomdxTreeList.SaveSelectedToStrings; TCustomdxTreeList.CopyAllToClipboard

SelectNodes method

Selects a range of nodes.

```
procedure SelectNodes(N1, N2 : TdxTreeListNode); virtual;
```

Description

Use this method to select all nodes between N1 and N2.

See Also

TCustomxTreeList.OnSelectedCountChange

SetEditingText method

This method is used internally.

```
procedure SetEditingText(Value : String);  virtual;
```

Description

Sets the text of an active inplace editor to Value.

See Also

TCustomdxTreeList.EditingText

SetEditText method

Sets the text of a specific cell in a tree list control.

```
procedure SetEditText(ANode : TdxTreeListNode; AColumn:Integer; const
Value: string); virtual;
```

Description

Sets the text of a cell specified by the ANode and AColumn parameters to Value and calls the DoAfterEditing method in order to generate the OnEdited event.

See Also

TCustomdxTreeList.EditingText

ShowButtonEditorMouse method

Simulates mouse button clicks within an active inplace editor.

```
procedure ShowButtonEditorMouse(X, Y: Integer);
```

Description

A tree list control column can include edit buttons to modify data in cells. This method is effective only when the aoShowAlwaysButton option is active (egoShowAlwaysButton for the ExpressQuantumGrid). If the edit button includes a point with X and Y coordinates, ShowButtonEditorMouse activates the inplace editor and simulates an edit button press event. X and Y are specified in tree list coordinates.

Note: *ShowButtonEditorMouse does not work if the aoEditing option (egoEditing for ExpressQuantumGrid) is inactive.*

ShowEditor method

Displays an inplace editor.

```
procedure ShowEditor;
```

Description

Activates an inplace editor within a focused cell. If the following tree list options are set, the inplace editor is inactive:

ExpressTreeList	*ExpressQuantumGrid*
aoRowSelected = True	**egoRowSelected** = True
aoEditing = False	**egoEditing** = False
	egoCanNavigation = False

See Also

TCustomdxTreeList.HideEditor; TCustomdxTreeList.CloseEditor; TCustomdxTreeList.CancelEditor; TCustomdxTreeList.InplaceEditor; TCustomdxTreeList.InplaceColumnIndex

ShowEditorChar method

Sends a character to the inplace editor window.

```
procedure ShowEditorChar(Ch: Char);
```

Description

This method sends a WM_CHAR message to the window of an active inplace editor simulating a Ch key press.

See Also

TCustomdxTreeList.ShowEditor; TCustomdxTreeList.ShowEditorMouse; TCustomdxTreeList.IsCustomizing

ShowEditorMouse method

Shows an inplace editor by mouse click.

```
procedure ShowEditorMouse(X, Y : Integer);
```

Description

This method activates, if possible, an inplace editor within a specific column of a node. X and Y specifies mouse coordinates within the tree list control window.

See Also

TCustomdxTreeList.ShowEditor; TCustomdxTreeList.IsCustomizing

StartDragBand method

Initiates band dragging.

```
procedure StartDragBand(AbsoluteIndex: Integer); virtual;
```

Description

This method is called when a user starts dragging a band specified by AbsoluteIndex in a tree list control or in a customizing form. StartDragBand prepares and initiates dragging. Descendants override this method to generate the OnStartBandDragging event.

See Also

TCustomdxTreeList.StartDragHeader; TCustomdxTreeListControl.OnStartBandDragging

StartDragHeader method

Initiates column dragging.

```
procedure StartDragHeader(AbsoluteIndex: Integer); virtual;
```

Description

This method is called when a user starts dragging a column specified by AbsoluteIndex in a tree list control or customizing form. StartDragHeader prepares and initiates dragging. Descendants override this method to generate the OnStartHeaderDragging event.

Copyright © 2000 Developer Express Inc.

Descendants can override StartDragHeader, if required to drag & drop columns from any other location, e.g. from a custom panel that includes column headers or from another tree list control.

See Also

TCustomdxTreeList.StartDragBand; TCustomdxTreeListControl.OnStartHeaderDragging

StartSearch method

Searches for text in a specified column.

```
function StartSearch(AColumnIndex: Integer; const AText: string): Boolean;
```

Description

Activates an auto-search in one of the tree list columns. AColumnIndex specifies the visible index of a column in which the tree list control searches for AText. If AColumnIndex is -1, the grid begins searching in the currently focused column. If searching is successful, StartSearch returns True and the located text is marked with colors specified by the AutoSearchColor and AutoSearchTextColor properties. Otherwise, StartSearch returns False.

Be certain to call the EndSearch method after calling StartSearch in order to terminate search mode.

See Also

TCustomdxTreeList.OptionsEx; TCustomdxTreeList.AutoSearchColor; TCustomdxTreeList.AutoSearchTextColor; TCustomdxTreeList.EndSearch; TCustomdxTreeList.AutoExpandedOnSearch

UpdateScrollBars method

Updates scroll bars of a tree list.

```
procedure UpdateScrollBars;  virtual;
```

Description

This method is used internally to update scrollbars within a tree list. Scroolbars are updated, for instance, when adding/deleting nodes or changing size of the client area.

TCustomdxTreeList Events

TCustomdxTreeList

OnBeginDragNode	**protected**	OnEdited	**protected**
OnCanNodeSelected	**protected**	OnEditing	**protected**
OnChangeColumn	**protected**	OnEditValidate	**protected**
OnChangeLeftCoord	**protected**	OnEndColumnsCustomizing	**protected**
OnChangeNode	**protected**	OnExpanded	**protected**
OnChangeTopVisibleNode	**protected**	OnExpanding	**protected**
OnCollapsed	**protected**	OnHotTrackNode	**protected**
OnCollapsing	**protected**	OnGetEditColor	**protected**
OnCompare	**protected**	OnSelectedCountChange	**protected**
OnDeletion	**protected**		
OnEditChange	**protected**		

OnBeginDragNode event

Occurs when dragging a node.

```
type
  TTLExpandedEvent = procedure(Sender: TObject; Node: TdxTreeListNode) of
object;
property OnBeginDragNode: TTLExpandedEvent;
```

Description

Use the OnBeginDragNode event handler to implement special processing when a user starts to drag a node. The node parameter specifies the dragged node.

See Also

TCustomdxTreeList.DragNode; TCustomdxTreeListControl.OnGetNodeDragText

OnCanNodeSelected event

Occurs just before a node is selected.

```
type
  TdxTLCanNodeSelected = procedure(Sender : TObject; ANode :
TdxTreeListNode;  var Allow : Boolean ) of object;
property OnCanNodeSelected : TdxTLCanNodeSelected;
```

Description

Occurs before node selection. A node is selected when a user moves focus from another control to a tree list or moves from one node to another within a tree list. The ANode parameter specifies the checking node. The Allow parameter determines the ability to select nodes. Set the Allow parameter

to False to avoid selection of a given node. The following code disables selection of nodes on the second level:

```
procedure TForm1.dxTreeList1CanNodeSelected(Sender: TObject;  ANode:
TdxTreeListNode; var Allow: Boolean);
begin
if ANode.Level = 1 then
  Allow := False;
end;
```

See Also

TdxTreeLIstNode.Selected

OnChangeColumn event

Occurs when changing a column.

```
type
  TTLChangeColumnEvent = procedure(Sender: TObject; Node: TdxTreeListNode;
Column : Integer) of object;
property OnChangeColumn : TTLChangeColumnEvent;
```

Description

This event is generated when a column receives focus. The Column parameter specifies the visible index of the currently focused column. It is equal to the FocusedColumn property value. The Node parameter determines the node to which a column belongs.

See Also

TCustomdxTreeList.OnChangeColumn; TCustomdxDBGrid.ChangeNodeEx

OnChangeLeftCoord event

Occurs when changing the LeftCoord property.

```
type
  TNotifyEvent = procedure (Sender: TObject) of object;
property OnChangeLeftCoord: TNotifyEvent;
```

Description

The LeftCoord property changes when a user horizontally scrolls the tree list window. Write an OnChangeLeftCoord event handler to perform specific actions during horizontal scrolling of the tree list window.

See Also

TCustomdxTreeList.LeftCoord

OnChangeNode event

Occurs when moving focus from a node.

```
type
  TTLChangeNodeEvent = procedure(Sender: TObject; OldNode, Node:
TdxTreeListNode) of object;
property OnChangeNode : TTLChangeNodeEvent;
```

Description

This event is generated when moving focus from one node to another within a tree list. The OldNode parameter specifies a node that has focus. Node is the currently focused node.

See Also

TCustomdxTreeList.OnChangeColumn; TCustomdxDBGrid.ChangeNodeEx

OnChangeTopVisibleNode event

Responds to changes in the topmost visible node.

```
property OnChangeTopVisibleNode : TNotifyEvent
```

Description

This event occurs when changing the topmost visible node in a tree list. For instance, the OnChangeTopVisibleNode event is generated when a user scrolls the node list within the window of a tree list control.

See Also

TCustomdxTreeList.TopVisibleNode

OnCollapsed event

Occurs when a node has just collapsed.

```
type
  TTLExpandedEvent = procedure(Sender: TObject; Node: TdxTreeListNode) of
object;
property OnCollapsed : TTLExpandedEvent;
```

Description

The Node parameter determines the collapsed node. Write an OnCollapsed event handler to perform special actions when collapsing particular nodes. For instance, the following code prevents the collapse of a top-level node:

```
procedure TForm1.dxTreeListCollapsed(Sender: TObject;  Node:
TdxTreeListNode);
begin
if Node.Level = 0 then
```

```
  Node.Expand(False);
end;
```

See Also

TCustomdxTreeList.OnCollapsing; TCustomdxTreeList.OnExpanding; TCustomdxTreeList.OnExpanded; TCustomdxTreeList.Collapse

OnCollapsing event

Occurs when a node is about to collapse.

```
type
  TTLExpandingEvent = procedure(Sender: TObject; Node: TdxTreeListNode; var
Allow: Boolean) of object;
property OnCollapsing : TTLExpandingEvent;
```

Description

Occurs just before a node collapses. Write an OnCollapsing event handler to perform special actions when collapsing a node. The node parameter specifies a node to collapse. The Allow parameter determines whether collapsing is available. Set Allow to False to prevent node collapsing. The following code disables collapsing of any node:

```
procedure TForm1.dxTreeListCollapsing(Sender: TObject;  Node:
TdxTreeListNode; var Allow: Boolean);
begin
  Allow := False;
end;
```

See Also

TCustomdxTreeList.OnCollapsed; TCustomdxTreeList.OnExpanding; TCustomdxTreeList.OnExpanded; TCustomdxTreeList.Collapse

OnCompare event

Occurs when a user compares two nodes of a tree list control.

```
type
  TdxTLCompareEvent = procedure(Sender: TObject; Node1, Node2:
TdxTreeListNode;  var Compare: Integer) of object;
property OnCompare : TdxTLCompareEvent;
```

Description

This event occurs when two nodes of a tree list must be compared during node sorting in a treelist control. Write an OnCompare event handler when it is necessary to use your own procedure for comparison of two nodes during sort operations.

Node1 and Node2 parameters specify two nodes to compare. The Compare parameter should return a positive value if Node1 is “greater“ than Node2 and a negative value if Node1 is “less” than Node2. If Node1 is “equal” to Node2, then Compare should be set to 0.

The code below demonstrates how to sort nodes by an ExpressTreeList column containing integer values:

```
// The aoAutoSort option should be active.
// The aoMultiSort option should be inactive.

// Delphi
procedure TForm1.dxTreeList1Compare(Sender: TObject; Node1, Node2:
TdxTreeListNode; var Compare: Integer);
var
  Val1, Val2, v : Integer;
begin
  // Check whether a column contains integer values
  if dxTreeList1.SortedColumn = dxTreeList1Column1 then
  begin
    Val(Node1.Strings[dxTreeList1.SortedColumn.Index], Val1, v);
    Val(Node2.Strings[dxTreeList1.SortedColumn.Index], Val2, v);
    if Val1 > Val2 then
     Compare := -1
   else
     if Val1 < Val2 then
       Compare := 1
     else
       Compare := 0;
  end;
end;

// C++Builder
void __fastcall TForm1::dxTreeList1Compare(TObject *Sender, TdxTreeListNode
*Node1, TdxTreeListNode *Node2, int &Compare)
{
  int Val1 = 0, Val2 = 0;
  // Check whether a column contains integer values
  if ( typeid(dxTreeList1->SortedColumn).name() ==
typeid(dxTreeList1Column1).name() )
  {
      Val1 = StrToInt("0"+Node1->Strings[dxTreeList1->SortedColumn-
>Index]);
      Val2 = StrToInt("0"+Node2->Strings[dxTreeList1->SortedColumn-
>Index]);
    if (Val1 > Val2)
     Compare = -1;
    else
```

```
        if (Val1 < Val2)
          Compare = 1;
        else
          Compare = 0;
      }
    }
```

See Also

TCustomdxTreeList.Options; TCustomdxTreeListContorl.OnColumnSorting; TCustomdxTreeListControl.OnColumnSorted

OnDeletion event

Occurs when deleting a node.

```
type
  TTLExpandedEvent = procedure(Sender: TObject; Node: TdxTreeListNode) of object;
property OnDeletion : TTLExpandedEvent;
```

Description

The OnDeletion event is generated when destroying a node in a treelist control. The Node parameter determines the node being destroyed. Write an OnDeletion event handler to release memory associated with a node via the node Data property.

OnEditChange event

Occurs when modifing the contents of a cell within a TreeList.

```
type
  TNotifyEvent = procedure (Sender: TObject) of object;
property OnEditChange: TNotifyEvent;
```

Description

Write an OnEditChange event handler to perform specific actions when any change is made within a cell of a TreeList.

OnEdited event

Occurs when a user finishes editing.

```
type
  TTLExpandedEvent = procedure(Sender: TObject; Node: TdxTreeListNode) of object;
property OnEdited : TTLExpandedEvent;
```

Description

Occurs right after a user finishes editing text in a column. Node specifies the TdxTreeListNode that contains the edited column. Write an OnEdited event handler to reflect changes within a tree list according to the new value of a column.

OnEditing event

Occurs before the tree list state has changed to tsEditing.

```
type
  TTLExpandingEvent = procedure(Sender: TObject; Node: TdxTreeListNode;
var Allow: Boolean) of object;
property OnEditing : TTLExpandingEvent;
```

Description

Occurs before a user begins editing data in a column. The Node parameter specifies a node for editing. The Allow parameter determines whether editing is enabled. Set Allow to False to avoid data modification.

The following code prevents editing an active cell within the ExpressQuantumGrid if the CheckColumn is checked.

```
// Delphi
procedure TForm1.dxDBGrid1Editing(Sender: TObject; Node: TdxTreeListNode;
var Allow: Boolean);
var
  i : Integer;
begin
  i := TdxDBGrid(Sender).ColumnByFieldName('check').Index;
  if (TdxDBGrid(Sender).FocusedColumn <> i) and (Node.Values[i] <> 'True')
then
    Allow := False;
end;
```

```
// C++Builder
void __fastcall TForm1::dxDBGrid1Editing(TObject *Sender, TdxTreeListNode
*Node, bool &Allow)
{
  int i = ((TdxDBGrid*)Sender)->ColumnByFieldName("check")->Index;
  if ((((TdxDBGrid*)Sender)->FocusedColumn != i) && (VarToStr(Node-
>Values[i]) != "True"))
    Allow = false;
}
```

See Also

TCustomdxTreeList.OnEdited; TCustomdxTreeList.EditingText; TCustomdxTreeList.State

OnEditValidate event

Occurs when a user confirms data input (Enter key, for instance) within any cell of a TreeList.

```
type
  TdxEditValidateEvent = procedure (Sender: TObject; var ErrorText: string;
var Accept: Boolean) of object;
property OnEditValidate: TdxEditValidateEvent;
```

Description

Write an OnEditValidate event handler to validate data changes made within any cell of a TreeList before the data is posted. This event occurs when a user confirms data input by pressing the Enter key within a cell or the OK button within a dropdown window. The OnEditValidate event allows an application to entirely validate data. The Sender parameter determines the column whose cell is being processed. The ErrorText parameter specifies the text displayed when an exception arises. The Accept parameter specifies whether to accept or cancel the changes. If data is accepted, the Accept parameter is **True**. Otherwise; it's **False**.

OnEndColumnsCustomizing event

Occurs when closing the column customization form.

```
type
  TNotifyEvent = procedure(Sender: TObject) of object;
property OnEndColumnsCustomizing : TNotifyEvent;
```

Description

This event is generated when closing the column customization form. Write an OnEndColumnsCustomizing event handler to perform specific actions, which are dependent on the state of the customizing form.

The following code enables/disables the "Customizing…" menu item according to the state of the customizing form.

```
// Delphi
procedure TForm1.dxDBGrid1EndColumnsCustomizing(Sender: TObject);
begin
  // enables the Customising... menu item
  MenuItemCustomizing.Enabled := True;
end;

procedure TForm1.MenuItemCustomizingClick(Sender: TObject);
begin
  // Activates the customizing form
  dxDBGrid1.ColumnsCustomizing;
  // Disables the "Customizing…" menu item.
  MenuItemCustomizing.Enabled := False;
```

```
end;

// C++Builder
void __fastcall TForm1::dxDBGrid1EndColumnsCustomizing(TObject * Sender);
{
  // enables the Customising... menu item
  MenuItemCustomizing->Enabled = true;
}

void __fastcall TForm1::MenuItemCustomizingClick(TObject * Sender);
{
  // Activates the customizing form
  dxDBGrid1->ColumnsCustomizing();
  // Disables the "Customizing…" menu item.
  MeniItemCustomizing->Enabled = false;
}
```

See Also

TCustomdxTreeList.ColumnsCustomizing; TCustomdxTreeList.CustomizingForm

OnExpanded event

Occurs after a node has expanded.

```
type
  TTLExpandedEvent = procedure(Sender: TObject; Node: TdxTreeListNode) of
object;
property OnExpanded : TTLExpandedEvent;
```

Description

Write an OnExpanded event handler when a node in a treelist expands. The Node parameter specifies the expanded node.

The following code demonstrates how to change a node image when it is collapsed; i.e. To change the open folder to the closed folder icon and vice versa.

```
// Delphi
const
  ClosedFolder = 1;  // Icon indexes in an image list
  OpenFolder = 2;

procedure TForm1.dxTreeList1Collapsed(Sender: TObject; Node:
TdxTreeListNode);
begin
  Node.ImageIndex := ClosedFolder;
end;
```

Copyright © 2000 Developer Express Inc.

```
procedure TForm1.dxTreeList1Expanded(Sender: TObject; Node:
TdxTreeListNode);
begin
  Node.ImageIndex := OpenFolder;
end;

// C++Builder
const
  ClosedFolder = 1, OpenFolder = 2;  // Icon indexes in an image list

void __fastcall TForm1::dxTreeList1Collapsed(TObject *Sender,
TdxTreeListNode *Node)
{
  Node->ImageIndex = ClosedFolder;
}

void __fastcall TForm1::dxTreeList1Expanded(TObject *Sender,
TdxTreeListNode *Node)
{
  Node->ImageIndex = OpenFolder;
}
```

See Also

TCustomdxTreeList.OnExpanding; TCustomdxTreeList.OnCollapsing; TCustomdxTreeList.OnCollapse; TCustomdxTreeList.Expand

OnExpanding event

Occurs before expanding a node.

```
type
  TTLExpandingEvent = procedure(Sender: TObject; Node: TdxTreeListNode; var
Allow: Boolean) of object;
property OnExpanding : TTLExpandingEvent;
```

Description

Occurs when a node, specified by the Node parameter, is about to be expanded. The Allow parameter determines whether it is possible to expand a node. The following code disables expansion of nodes at the second level:

```
procedure TForm1.dxTreeListExpanding(Sender: TObject; Node:
TdxTreeListNode; var Allow: Boolean);
begin
if Node.Level = 1 then
   Allow := False;
```

```
end;
```

See Also

TCustomdxTreeList.OnExpanded; TCustomdxTreeList.OnCollapsing; TCustomdxTreeList.OnCollapse; TCustomdxTreeList.Expand

OnGetEditColor event

Occurs when activating an inplace editor in a specific cell of a data-aware tree list.

```
type
  TdxTLGetEditColor = procedure(Sender: TObject; var AColor: TColor) of object;
property OnGetEditColor : TdxGetEditColor;
```

Description

Write an OnGetEditColor event handler to set the specific background color for an inplace editor in a tree list cell. AColor determines the background color. For instance, the following code sets a green background for an inplace editor:

```
procedure TfmMain.dxDBGridGetEditColor(Sender: TObject; ANode:
TdxTreeListNode; AColumn: TdxDBTreeListColumn; var AColor: TColor);
begin
  AColor := clGreen;
end;
```

See Also

TCustomdxTreeList.InplaceEditor

OnHotTrackNode event

Occurs when the mouse is placed over a node.

```
type
  TdxTLHotTrackNode = procedure (Sender: TObject; AHotTrackInfo:
TdxTreeListHotTrackInfo; var ACursor: TCursor) of object;
property OnHotTrackNode: TdxTLHotTrackNode;
```

Description

Write an OnHotTrackNode event handler to specify the manner in which a node responds to mouse movements or to perform specific actions. The AHotTrackInfo parameter determines the TdxTreeListHotTrackInfo structure that stores information for the hot track feature. The ACursor parameter specifies the cursor used within a hot track node.

OnSelectedCountChange event

Occurs when changing the number of selected nodes.

Copyright © 2000 Developer Express Inc.

```
type
  TNotifyEvent = procedure(Sender : TObject) of object;
property OnSelectedCountChange : TNotifyEvent;
```

Description

If the TCustomdxTreeList Options property includes the aoMultiSelect flag, users can select several nodes. The OnSelectedCountChange event is generated when the number of selected nodes changes.

See Also

TCustomdxTreeList.SelectedCount; TCustomdxTreeList.SelectedNodes; TdxTreeListNode.Selected

TCustomdxTreeListControl

Unit

dxTL

```
TCustomdxTreeListControl = class(TCustomdxTreeList);
```

Description

TCustomdxTreeListControl is a base class for standard (non data-aware) tree list controls. TdxTreeList (ExpressTreeList) is inherited directly from TCustomdxTreeListControl.

Hierarchy

- TObject
  - TPersistent
    - TComponent
      - TControl
        - TWinControl
          - TCustomControl
            - TCustomdxContainer
              - TCustomdxTreeList

Copyright © 2000 Developer Express Inc.

TCustomdxTreeListControl Properties

TCustomdxTreeListControl

BandMaxRowCount	**published / read / write**	IniFileName	**protected / read / write**
BandRowCount	**published / read / write**	IniSectionName	**protected / read / write**
Bands	**published / read / write**	IsCustomStoring	**protected / read / write**
ColumnCount	**public / read only**	RegistryPath	**protected / read / write**
Columns	**public / read / write**	SortedColumnCount	**public / read only**
DefaultLayout	**published / read / write**	SortedColumns	**public / read only**
HeaderMinRowCount	**published / read / write**	VisibleColumnCount	**public / read only**
HeaderPanelMaxRowCount	**published / read / write**	VisibleColumns	**public / read / write**
HeaderPanelRowCount	**published / read / write**		

Derived from TCustomdxTreeList

ArrowsColor	**protected / read / write**	FooterRowHeight	**public / read only**
AutoExpandOnSearch	**protected / read / write**	FooterRowNodeHeight	**public / read only**
AutoSearchColor	**protected / read / write**	GridLineColor	**protected / read / write**
AutoSearchTextColor	**protected / read / write**	GroupNodeColor	**protected / read / write**
BandButtonPushed	**public / read only**	GroupNodeTextColor	**protected / read / write**
BandColor	**protected / read / write**	HeaderButtonPushed	**public / read only**
BandFont	**protected / read / write**	HeaderColor	**protected / read / write**
BandPanelHeight	**public / read only**	HeaderFont	**protected / read / write**
BorderStyle	**protected / read / write**	HeaderHeight	**public / read only**
Count	**public / read only**	HeaderPanelHeight	**public / read only**
CustomizingBandListBox	**public / read only**	HeaderRowHeight	**public / read only**
CustomizingForm	**public / read only**	HideFocusRect	**protected / read / write**
CustomizingHeaderListBox	**public / read only**	HideSelection	**protected / read / write**
CustomizingPos	**public / read / write**	HideSelectionColor	**protected / read / write**
CustomizingRowCount	**public / read / write**	HideSelectionTextColor	**protected / read / write**
DblClkExpanding	**protected / read / write**	HighlightColor	**protected / read / write**
DefaultRowHeight	**protected / read / write**	HighlightTextColor	**protected / read / write**
DescTextHeight	**public / read only**	Images	**protected / read / write**
DownBandIndex	**public / read only**	Indent	**protected / read only**
DownBandPushed	**public / read only**	IndentDesc	**protected / read / write**
DownColumnIndex	**public / read only**	InplaceColumnIndex	**public / read only**
DownColumnPushed	**public / read only**	InplaceEditor	**public / read only**
DragAbsoluteBandIndex	**public / read only**	IsCustomizing	**public / read only**
DragAbsoluteHeaderIndex	**public / read only**	Items	**public / read only**
DragNode	**protected read only**	LastNode	**public / read only**
EditingText	**protected / read /write**	LeftCoord	**protected / read / write**
FixedBandLineColor	**protected / read /write**	LockSelection	**public / read only**
FixedBandLineWidth	**protected / read /write**	LockUpdate	**protected / read only**
FocusedAbsoluteIndex	**public / read only**	LookAndFeel	**protected / read / write**
FocusedColumn	**protected / read / write**	MaxRowLineCount	**protected / read / write**
FocusedNode	**public / read only**	NewItemRowActive	**protected / read / write**
FocusedNumber	**protected / read / write**	NewItemRowHeight	**public / read only**
FooterPanelHeight	**public / read only**	Options	**public / read / write**

OptionsEx	**public / read / write**	ShowIndicator	**protected / read / write**
PaintStyle	**protected read / write**	ShowLines	**protected / read / write**
PreviewFont	**protected read / write**	ShowNewItemRow	**protected / read / write**
PreviewLines	**protected / read / write**	ShowPreviewGrid	**protected / read / write**
RowCount	**protected / read only**	ShowRoot	**protected / read / write**
RowFooterColor	**protected / read / write**	ShowRowFooter	**protected / read / write**
RowFooterTextColor	**protected / read / write**	SimpleCustomizeBox	**protected / read / write**
RowHeight	**public / read only**	State	**public / read only**
RowSeparatorLineWidth	**protected / read / write**	StateImages	**protected / read / write**
ScrollBars	**protected / read / write**	TabStop	**published / read / write**
SelectedCount	**protected / read only**	TopIndex	**protected / read / write**
SelectedNodes	**protected / read only**	TopNode	**public / read only**
SelectionAnchor	**protected / read / write**	TopVisibleNode	**public / read only**
ShowBands	**protected / read / write**	TreeLineColor	**protected / read / write**
ShowButtons	**protected / read / write**	TreeLineStyle	**protected / read / write**
ShowFooter	**protected / read / write**	VisibleRowCount	**protected / read only**
ShowGrid	**protected / read / write**	WaitForExpandNodeTime	**protected / read / write**
ShowHeader	**protected / read / write**		
ShowHiddenInCustomizeBox	**protected / read / write**		

Derived from TCustomdxContainer

IsInitEdit	**public / read only**
ReadOnly	**public / read only**

BandMaxRowCount property — read / write

Determines the maximum number of rows in the band area.

```
property BandMaxRowCount: Integer;
```

Description

Use this property to limit the height of the band area. The current number of rows, which the band area occupies, is specified by the BandRowCount property.

See Also

TCustomdxTreeListControl.BandRowCount; TCustomdxTreeListControl.Bands; TdxTreeListBand

BandRowCount property — read / write

Determines the height of the band area in rows.

```
property BandRowCount: Integer;
```

Description

Use this property to change the vertical size of the band area. It specifies the current height of the band area. The maximum number of rows the band area can occupy is determined by the BandMaxRowCount property. By default, a band occupies one row.

Copyright © 2000 Developer Express Inc.

See Also

TCustomdxTreeListControl.Bands; TCustomdxTreeListControl.BandMaxRowCount; TdxTreeListBand

Bands property — read / write

Represents tree list bands.

```
property Bands : TdxTreeListBands;
```

Description

Use this property to work with bands of a tree list control.

See Also

TCustomdxTreeListControl.Columns; TCustomdxTreeListControl.VisibleColumns; TCustomdxTreeListControl.OnCustomDrawBand

ColumnCount property — read only

Returns the number of columns in a tree list control.

```
property ColumnCount: Integer;
```

Description

Use this property to obtain total number of tree list columns. The total number includes both visible and invisible columns.

See Also

TCustomdxTreeListControl.Columns; TCustomdxTreeListControl.VisibleColumnCount

Columns property — read / write

Represents an indexed array of tree list columns.

```
property Columns[Index: Integer]: TdxTreeListColumn;
```

Description

Use this property to access a particular column in a tree list control by its index. Index of the first element of the Columns array is 0. Total number of columns in a tree list control is determined by the ColumnCount property of TCustomdxTreeListControl.

See Also

TCustomdxTreeListControl.ColumnCount; TCustomdxTreeListControl.VisibleColumns; TCustomdxTreeListControl.ColumnByName; TCustomdxTreeListControl.CreateColumn; TCustomdxTreeListControl.CreateColumnEx; TCustomdxTreeListControl.DestroyColumns; TCustomdxTreeListControl.Bands

DefaultLayout property — read only

Determines compatibility mode.

```
property DefaultLayout: Boolean;
```

Description

The intent of this property is to control compatibility of ExpressQuantumGrid with ExpressGrid ver 1.4x. If True, ExpressQuantumGrid behavior is the same as that of ExpressGrid 1.4x. If you use any new features, this property should be set to False.

HeaderMinRowCount property — read / write

Determines the minimal number of rows in the header panel.

```
property HeaderMinRowCount: Integer;
```

Description

Header panel is an area at the top of a tree list control. It contains column captions. Columns within a tree list control can be placed one under another. The HeaderMinRowCount property determines the minimum number of rows into which columns can be placed. By default, HeaderMinRowCount is 1. This is the lowest available value for the HeaderMinRowCount property.

See Also

TCustomdxTreeListControl.HeaderPanelMaxRowCount;
TCustomdxTreeListControl.HeaderPanelRowCount

HeaderPanelMaxRowCount property — read / write

Determines the maximum number of rows in the header area.

```
property HeaderPanelMaxRowCount: Integer;
```

Description

Header panel is an area at the top of a tree list control. It contains column captions. Columns within a tree list control can be placed one under another. The HeaderPanelMaxRowCount property specifies the maximum number of rows into which columns can be placed one under another.

See Also

TCustomdxTreeListControl.HeaderPanelRowCount; TCustomdxTreeListControl_HeaderMinRowCount

HeaderPanelRowCount property — read / write

Determines the total number of rows within the header panel.

```
property HeaderPanelRowCount: Integer;
```

Description

Header panel is an area at the top of a tree list control, which contains captions of columns. Generally, columns can be placed one under another. The HeaderPanelRowCount property specifies the number of rows which columns occupy within the header panel.

Copyright © 2000 Developer Express Inc.

See Also

TCustomdxTreeListControl.HeaderPanelMaxRowCount;
TCustomdxTreeListControl.HeaderMinRowCount

IniFileName property — read / write

Specifies the name of the INI file in which to store column settings of the ExpressTreeList/DBTreeList or ExpessQuantumGrid.

```
property IniFileName : String;
```

Description

If the aoStoreToIniFile option is active (Options property), ExpressTreeList settings are stored to an INI file. This property sets the name of an INI file in which the ExpressTreeList stores column settings. The IniFileName property determines the file name; that is, the 'ini' extension isn't added by the component to the file name specified by the IniFileName property.

See Also

TCustomdxTreeListControl.IniSectionName; TCustomdxTreeListControl.LoadFromIniFile;

TCustomdxTreeList.Options; TCustomdxTreeListControl.RegistryPath;

TCustomdxTreeListControl.SaveToIniFile

IniSectionName property — read / write

Determines the section name of an ini file in which ExpressTreeList/DBTreeList or ExpessQuantumGrid settings are stored.

```
property IniSectionName : string;
```

Description

If the aoStoreToIniFile option is active (Options property), ExpressTreeList settings are stored to an INI file. The IniSectionName property sets the name of the INI file section in which ExpressTreeList stores column settings. This property enables you to store grid settings within an existing INI file. If the aoStoreToIniFile option is active, the ExpressTreeList stores its settings in an INI file (specified by the IniFileName property) in the section, specified by the IniSectionName property.

See Also

TCustomdxTreeListControl.LoadFromIniFile; TCustomdxTreeListControl.RegistryPath;

TCustomdxTreeList.Options; TCustomdxTreeListControl.IniFileName;

TCustomdxTreeListControl.SaveToIniFile

IsCustomStoring property — read / write

```
property IsCustomStoring: Boolean;
```

Description

This property is used internally. It determines whether a tree list saves columns that do not belong to the form to a stream. For instance, it is necessary when saving TdxDBGridLayout columns to a DFM file.

RegistryPath property

Specifies the Registry key into which ExpressTreeList/DBTreeList or ExpessQuantumGrid stores the column settings.

```
property RegistryPath : String;
```

Description

If the aoStoreToRegistry option is active (Options property), the ExpressTreeList saves columns settings to the Registry. Use this property to set the Registry key in which the grid stores column settings. If RegistryPath is empty, the tree list does not save its column settings to the Registry.

See Also

TCustomdxTreeListControl.IniSectionName; TCustomdxTreeListControl.SaveToRegistry

TCustomdxTreeList.Options; TCustomdxTreeListControl.IniFileName;

SortedColumnCount property **read only**

Returns total number of sorted columns.

```
property SortedColumnCount : Integer;
```

Description

In general, nodes within ExpressTreeList can be sorted by several columns. Use this property to obtain total number of sorted columns.

SortedColumns property **read only**

Represents a list of sorted columns.

```
property SortedColumns[Index: Integer]: TdxTreeListColumn;
```

Description

Nodes of a tree list control can be sorted by several columns. This property contains an indexed array of sorted columns. Index of the first column in the array is 0. The total number of sorted columns is determined by the SortedColumnsCount property.

See Also

TCustomdxTreeListControl.SortedColumnsCount; TCustomdxTreeListControl.OnColumnSorting; TCustomdxTreeListControl.OnColumnsSorted

Copyright © 2000 Developer Express Inc.

SortedColumnsCount property read only

Returns the number of sorted columns.

```
property SortedColumnCount: Integer;
```

Description

Nodes of a tree list control can be sorted by several columns. This property specifies the total number of columns by which nodes are sorted.

See Also

TdxTreeListColumn.Sorted; TCustomdxTreeListControl.Columns; TCustomdxTreeListControl.OnColumnSorted; TCustomdxTreeListControl.OnColumnSorting

VisibleColumnCount property read only

Returns the number of visible columns within a tree list control.

```
property VisibleColumnCount: Integer;
```

Description

Use this property to obtain the total number if visible columns in a tree list control.

See Also

TCustomdxTreeListControl.ColumnCount; TCustomdxTreeListControl.Columns; TCustomdxTreeListControl.VisibleColumns; TdxTreeListColumn.Visible

VisibleColumns property read only

Represents an indexed array of visible columns in a tree list control.

```
property VisibleColumns[Index: Integer]: TdxTreeListColumn;
```

Description

Use this property to access a specific visible column. Index of the first element in the VisibleColumns array is 0 and the total number of visible columns is determined by VisibleColumnCount property.

See Also

TCustomdxTreeListControl.VisibleColumnCount; TCustomdxTreeListControl.Columns; TdxTreeListColumn.Visible; TCustomdxTreeListControl.ColumnByName; TCustomdxTreeListControl.Bands

TCustomdxTreeListControl Methods

TCustomdxTreeListControl

ApplyBestFit	**public**	GetHeaderColumnAt	**public**
AssignColumns	**public**	GetNodeFooterColumnAt	**public**
BeginSorting	**public**	GetVisibleColumnIndex	**public**
ColumnByName	**public**	LoadFromIniFile	**public**
CreateColumn	**public**	LoadFromRegistry	**public**
CreateColumnEx	**public**	RefreshDefaultColumnsWidths	**public**
DestroyColumns	**public**	RefreshSorting	**public**
EndSorting	**public**	RestoreColumnsDefaults	**public**
GetAbsoluteColumnIndex	**public**	SaveToIniFile	**public**
GetBandAt	**public**	SaveToRegistry	**public**
GetColumnAt	**public**	UpdateColumn	**public**
GetDisplayValue	**public**	UpdateHeaders	**public**
GetFooterColumnAt	**public**		

Derived from TCustomdxTreeList

Add	**public**	GetFocusedVisibleIndex	**public**
AddFirst	**public**	GetHitInfo	**public**
AdjustColumnsWidth	**public**	GetHitTestInfoAt	**public**
BeginSelection	**public**	GetNextVisible	**public**
BeginUpdate	**public**	GetNodeAt	**public**
CalcDrawInfo	**public**	GetScrollVertGridRect	**protected**
CancelEditor	**public**	GotoFirst	**public**
CellRect	**public**	GotoLast	**public**
ClearColumnsSorted	**public**	GotoNext	**public**
ClearNodes	**public**	GotoPrev	**public**
ClearSelection	**public**	HideEditor	**public**
CloseEditor	**public**	IndexOf	**public**
ColumnsCustomizing	**public**	Insert	**public**
CopyAllToClipboard	**public**	InvalidateSelection	**protected**
CopySelectedToClipboard	**public**	IsInvertSelect	**public**
Create	**public**	IsNodeVisible	**public**
DeleteSelection	**public**	MakeBandVisible	**public**
Destroy	**public**	MakeColumnVisible	**public**
EndColumnsCustomizing	**public**	MakeNodeVisible	**public**
EndSearch	**public**	PointInCustomizingForm	**public**
EndSelection	**public**	SaveAllToStrings	**public**
EndUpdate	**public**	SaveAllToTextFile	**public**
FindInplaceEdit	**protected**	SaveSelectedToStrings	**public**
FindNodeByText	**protected**	SaveSelectedToTextFile	**public**
FreeDrawInfo	**public**	SelectNodes	**protected**
FullCollapse	**public**	SetEditingText	**protected**
FullExpand	**public**	SetEditText	**protected**
GetEditRect	**public**	ShowButtonEditorMouse	**public**
GetFocusedAbsoluteIndex	**public**	ShowEditor	**public**

Copyright © 2000 Developer Express Inc.

ShowEditorChar	**public**	StartSearch	**public**
ShowEditorMouse	**public**	UpdateScrollBars	**protected**
StartDragBand	**protected**		
StartDragHeader	**protected**		

Derived from TCustomdxContainer

BeginInitEdit	**protected**	GetReadOnly	**protected**
CanEditAcceptKey	**protected**	HideDrawFocusRect	**protected**
CanEditModify	**protected**	InplaceEditKeyDown	**protected**
CheckEditorWidth	**protected**	IsEditorMoved	**protected**
CreateEditStyle	**protected**	IsImmediateEditor	**protected**
EndInitEdit	**protected**	IsShowButtonAlways	**protected**
GetDefaultPopupBorderStyle	**public**	IsTabs	**protected**
GetEditColor	**protected**	MoveCol	**protected**
GetEditFont	**protected**	RedrawSelection	**protected**

ApplyBestFit method

Applies the best fit to a column, passed as the Column parameter.

```
procedure ApplyBestFit(AColumn: TdxTreeListColumn); virtual;
```

Description

Use this method to set the proper width of a specified column. If the column width is too small, the width is enlarged. If it is quite large, it is set to a lesser value. If AColumn is nil, the fit procedure is applied to all tree list columns.

See Also

TCustomdxTreeList.AdjustColumnsWidth

AssignColumns method

Creates the same column as columns of a specified tree list.

```
procedure AssignColumns(ATreeListControl: TCustomdxTreeListControl);
virtual;
```

Description

AssignColumns destroys existing columns of a given TCustomdxDBTreeListControl and creates new columns of the same type and with the same name as columns of ATreeListControl.

See Also

TCustomdxTreeListControl.Columns

BeginSorting method

Use this method to sort nodes by several columns.

```
procedure BeginSorting;
```

Description

BeginSorting and EndSorting together provide the multi-sorting feature of ExpressQuantumGrid. Assume you need to sort data within a tree list by three columns. If you change the Sorted property of the columns one after another, nodes will be re-sorted three times and you won't obtain the desirable result. In order to create multi-sorted columns, call BeginSorting before changing the Sorted property of the first column, and then change the Sorted property of every column, by which you wish to sort, and finally call EndSorting. BeginSorting only 'freezes' column sorting, nodes are really sorted on calling the EndSorting method.

Note: *Be certain to call EndSorting after each call to BeginSorting.*

See Also

TCustomdxTreeListControl.EndSorting; TCustomdxTreeListControl.OnColumnSorting; TCustomdxTreeListControl.OnColumnSorted; TCustomdxTreeListControl.RefreshSorting

ColumnByName method

Returns a column by its name.

```
function ColumnByName(const AName: string): TdxTreeListColumn;
```

Description

Use this method to access a column, whose name equals AName.

See Also

TCustomdxTreeListControl.Columns

CreateColumn method

Creates a column of specified type.

```
type
  TdxTreeListColumnClass = class of TdxTreeListColumn;
function CreateColumn(ColumnClass: TdxTreeListColumnClass):
TdxTreeListColumn;
```

Description

Creates a new column of type ColumnClass and adds it to tree list columns (the tree list Columns property).

See Also

TCustomdxTreeListControl.Columns; TCustomdxTreeListControl.CreateColumnEx; TCustomdxTreeListControl.DestroyColumns

Copyright © 2000 Developer Express Inc.

CreateColumnEx method

Creates a column using some extra parameters.

```
type
  TdxTreeListColumnClass = class of TdxTreeListColumn;
function CreateColumnEx(ColumnClass: TdxTreeListColumnClass; AOwner:
TComponent): TdxTreeListColumn;
```

Description

This method creates a column of type, determined by ColumnClass. The AOwner parameter determines a tree list control to which created column belongs.

See Also

TCustomdxTreeListControl.Columns; TCustomdxTreeListControl.CreateColumn; TCustomdxTreeListControl.DestroyColumns

DestroyColumns method

Destroys tree list columns.

```
procedure DestroyColumns; virtual;
```

Description

Destroys all tree list columns and releases the memory they allocate. Be careful when using this method so as to avoid data loss.

See Also

TCustomdxTreeListControl.Columns; TCustomdxTreeListControl.CreateColumn; TCustomdxTreeListControl.CreateColumnEx

EndSorting method

Use this property when you require sorting by several columns.

```
procedure EndSorting;
```

Description

BeginSorting and EndSorting together provide multi-sorting feature of ExpressQuantumGrid. Call BeginSorting before sorting nodes by several nodes. Next, set the Sorted property for several columns. Finally, call the EndSorting method to perform sorting. Note that BeginSorting only 'freezes' column sorting, nodes are really sorted on calling EndSorting.

Note: *It is strongly recommended to call EndSorting right after the sorting procedure has finished (that is in the same function or procedure). Please use the following sequence: BeginSorting* – Your Sorting procedure – *EndSorting.*

See Also

TCustomdxTreeListControl.BeginSorting; TCustomdxTreeListControl.OnColumnSorting; TCustomdxTreeListControl.OnColumnSorted; TCustomdxTreeListControl.RefreshSorting

GetAbsoluteColumnIndex method

Returns the absolute index of a column by its position among visible columns.

```
function GetAbsoluteColumnIndex(VisibleIndex: Integer): Integer;
```

Description

The GetAbsoluteColumnIndex property returns the column index in the Columns property of a TCustomdxTreeListControl object. VisibleIndex determines column position in the TdxTreeListColumns VisibleItems property.

See Also

TCustomdxTreeListControl.Columns; TCustomdxTreeListControl.ColumnCount;TCustomdxTreeListControl.VisibleColumns; TCustomdxTreeListControl.GetVisibleColumnIndex; TCustomdxTreeList.FocusedColumn

GetBandAt method

Returns a band by coordinates.

```
function GetBandAt(X, Y: Integer): TdxTreeListBand;
```

Description

This function returns a band, if its rectangle contains a point with X and Y coordinates. If there is no such a band, GetBandAt returns nil.

See Also

TCustomdxTreeListControl.GetAbsoluteColumnIndex; TCustomdxTreeListControl.VisibleColumns; TCustomdxTreeListControl.VisibleColumnCount

GetColumnAt method

Returns a column by coordinates.

```
function GetColumnAt(X, Y: Integer): TdxTreeListColumn;
```

Description

This method returns column if any cell of this column contains a point, specified by X and Y. If there is no such column, GetColumnAt returns nil.

See Also

TCustomdxTreeListControl.GetBandAt; TCustomdxTreeListControl.GetHeaderColumnAt; TCustomdxTreeListControl.GetFooterColumnAt; TCustomdxTreeListControl.GetNodeFooterAt

Copyright © 2000 Developer Express Inc.

GetDisplayValue method

Returns the displayed value for a specified column.

```
function GetDisplayValue(Node: TdxTreeListNode; AbsoluteIndex: Integer):
string;
```

Description

Use this method to get the displayed value for a column. The Node parameter determines a node to which the column belongs. AbsoluteIndex is the index of the column among other columns of the tree list (see the TCustomdxTreeListControl Columns property).

See Also

TCustomdxTreeListControl.Columns; TdxTreeListColumn

GetFooterColumnAt method

Returns a column by specified point in its footer.

```
function GetFooterColumnAt(X, Y: Integer): TdxTreeListColumn;
```

Description

This method returns column, whose footer contains a point, specified by X and Y parameters. If no column footer contains this point, GetFooterColumnAt returns nil.

See Also

TCustomdxTreeListControl.GetBandAt; TCustomdxTreeListControl.GetColumnAt; TCustomdxTreeListControl.GetHeaderColumnAt

GetHeaderColumnAt method

Returns a column by specified coordinates.

```
function GetHeaderColumnAt(X, Y: Integer): TdxTreeListColumn;
```

Description

This method returns a column if the rectangle of its header contains points, specified by X and Y parameters. If no column header contains this point, GetHeaderColumnAt returns nil.

See Also

TCustomdxTreeListControl.GetBandAt; TCustomdxTreeListControl.GetColumnAt; TCustomdxTreeListControl.GetFooterColumnAt

GetNodeFooterColumnAt method

Returns columns by node footer coordinates.

```
function GetNodeFooterColumnAt(X, Y: Integer): TdxTreeListColumn;
```

Description

Returns a column corresponding to a node footer cell, if a cell contains a points with X and Y coordinates. If no cell contains such a point, GetNodeFooterColumnAt returns nil.

See Also

TCustomdxTreeListControl.GetBandAt; TCustomdxTreeListControl.GetColumnAt; TCustomdxTreeListControl.GetFooterColumnAt; TCustomdxTreeListControl.GetHeaderColumnAt

GetVisibleColumnIndex method

Returns the visible index of a column by its position among tree list columns.

```
function GetVisibleColumnIndex(AbsoluteIndex: Integer): Integer;
```

Description

The GetVisibleColumnIndex property returns the column index in the VisibleColumns property of a TCustomdxTreeListControl by column index in the Columns property.

See Also

TCustomdxTreeListControl.GetAbsoluteColumnIndex; TCustomdxTreeListControl.VisibleColumns; TCustomdxTreeListControl.VisibleColumnCount

LoadFromIniFile method

Loads ExpressTreeList/DBTreeList and ExpressQuantumGrid column settings from a specified INI file.

```
procedure LoadFromIniFile(const AFileName : String);
```

Description

Use this method to load settings of tree list columns from an INI file, specified by the AFileName parameter. If the aoStoreToIniFile option is active (Options property), the ExpressTreeList uses an INI file to store column settings. It calls the LoadFromIniFile method passing it the IniFileName property as a parameter. The AFileName property determines the file name; that is, the 'ini' extension is not added to the file name specified by the AFileName property.

See Also

TCustomdxTreeListControl.LoadFromRegistry; TCustomdxTreeListControl.SaveToIniFile;

TCustomdxTreeListControl.IniFileName; TCustomdxTreeListControl.IniSectionName;

TCustomdxTreeListControl.SaveToRegistry

LoadFromRegistry method

Loads columns setting from the Registry.

```
procedure LoadFromRegistry(const ARegPath : String);
```

Description

Loads grid column settings from the Registry, specified by the ARegPath parameter. If a grid uses the Registry to store column settings, it calls LoadFromRegistry with the ARegPath parameter equal to

RegistryPath, to obtain column settings from the Registry.

The LoadFromRegistry method allows loading different tree list settings for different users. For instance, an application can be used by different users on the same machine. If you'd like to save grid settings for each user into a separate registry key, deactivate the aoStoreToRegistry option and perform load settings based on individual login info. When a program starts, users enter their login name. The application obtains a registry path for the current user (via the GetRegPathByUserInfo function - not implemented in the sample below) and loads grid settings:

```
// Delphi
procedure TForm1.FormCreate(Sender: TObject);
begin
  . . . // Getting login info
  // Obtains the Registry key for a current user
  dxTreeList1.RegistryPath := GetRegPathByUserInfo(LogInfo);
  // Loads custom user grid settings
 dxTreeList1.LoadFromRegistry(dxTreeList1.RegistryPath);
  // Forces ExpressTreeList to save settings into Registry
  // when application terminates
 dxTreeList1.Options := dxTreeList1.Options + [aoStoreToRegistry];
  . . .
end;

// C++Builder
void __fastcall TForm1::FormCreate(TObject * Object)
{
  . . . // Getting login info
  // Obtains the Registry key for a current user
 dxTreeList1->RegistryPath = GetRegPathByUserInfo(LogInfo);
  // Loads custom user grid settings
 dxTreeList1->LoadFromRegistry(dxTreeList1->RegistryPath);
  // Forces ExpressTreeList to save settings to Registry
  // when application terminates
 dxTreeList1->Options = dxTreeList1->Options << aoStoreToRegistry;
  . . .
}
```

See Also

TCustomdxTreeListControl.RegistryPath; TCustomdxTreeListControl.LoadFromIniFile; TCustomdxTreeListControl.SaveToIniFile; TCustomdxTreeListControl.SaveToRegistry

RefreshDefaultColumnsWidths method

Refreshes widths of tree list columns according to their captions.

```
procedure RefreshDefaultColumnsWidths;
```

Description

Sets the width of every TdxTreeListColumn object in a tree list control to the length of column's Caption (if defined). If TdxTreeListColumn Caption has not been defined, RefreshDefaultColumnsWidths resets the column width to the default value.

See Also

TCustomdxTreeListControl.RestoreColumnsDefaults; TdxTreeListColumns.RefreshDefaultWidths; TdxTreeListColumn.Width

RefreshSorting method

Sorts tree list nodes.

```
procedure RefreshSorting;
```

Description

RefreshSorting sorts tree list nodes by first sorted columns. This method performs sorting only if the aoAutoSort option is active. It is applicable only to TdxTreeList. TdxDBGrid (with load all records) calls it automatically.

See Also

TCustomdxTreeListControl.OnColumnSorting; TCustomdxTreeListControl.OnColumnSorted; TdxTreeListColumn.Sorted

RestoreColumnsDefaults method

Restores default settings for all columns in a tree list.

```
procedure RestoreColumnsDefaults;
```

Description

Use this method to reset settings of TCustomdxTreeListControl columns to their default values.

See Also

TCustomdxTreeListControl.RefreshDefaultColumnsWidths; TdxTreeListColumn.RestoreDefaults

SaveToIniFile method

Saves ExpressTreeList/DBTreeList and ExpressQuantumGrid column settings to an INI file.

```
procedure SaveToIniFile(const AFileName : String);
```

Description

This method saves column settings to an INI file specified by AFileName. If the aoStoreToIniFile option is active (Options property), the ExpressTreeList calls this method to store column settings in the INI file, specified by the IniFileName property. Call this method to store column settings at any time. The AFileName property specifies the file's name; the 'ini' file extension isn't added to the file's

name specified by the AFileName property.

See Also

TCustomdxTreeListControl.LoadFromRegistry; TCustomdxTreeListControl.StoreToRegistry

TCustomdxTreeListControl.IniSectionName; TCustomdxTreeListControl.LoadFromIniFile;

TCustomdxTreeList.Options; TCustomdxTreeListControl.IniFileName;

SaveToRegistry method

Saves ExpressTreeList/DBTreeList and ExpressQuantumGrid column settings to the Registry.

```
procedure SaveToRegistry(const ARegPath : String);
```

Description

Use this method to store column settings to the Registry specified by ARegPath. If the aoStoreToRegistry option is active, the ExpressTreeList calls SaveToRegistry with the ARegPath parameter equal to the RegistryPath property.

See Also

TCustomdxTreeListControl.LoadFromRegistry; TCustomdxTreeListControl.RegistryPath;

TCustomdxTreeList.Options; TCustomdxTreeListControl.LoadFromIniFile;

TCustomdxTreeListControl.SaveToIniFile

UpdateColumn method

```
procedure UpdateColumn(Column: TdxTreeListColumn); virtual;
```

Description

This method is used internally. It is called from the RemoveColumn procedure and when changing Column properties. It forces a tree list control to update itself.

See Also

TCustomdxTreeListControl.Columns; TCustomdxTreeListControl.RemoveColumn

UpdateHeaders method

```
procedure UpdateHeaders; virtual;
```

Description

This method is used internally when necessary to update headers of tree list columns. Descendants can override it to perform special processing when changing height of a header panel.

See Also

TCustomdxTreeListControl.UpdateBands; TCustomdxTreeListControl.HeaderPanelRowCount; TCustomdxTreeListControl.Columns

TCustomdxTreeListControl Events

TCustomdxTreeListControl

OnBandButtonClick	**protected**	OnDragEndHeader	**protected**
OnBandClick	**protected**	OnDragOverBand	**protected**
OnCalcRowLineHeight	**protected**	OnDragOverHeader	**protected**
OnCanBandDragging	**protected**	OnEndDragColumn	**protected**
OnCanHeaderDragging	**protected**	OnGetImageIndex	**protected**
OnChangedColumnsWidth	**protected**	OnGetLevelColor	**protected**
OnColumnClick	**protected**	OnGetNodeDragText	**protected**
OnColumnMoved	**protected**	OnGetSelectedIndex	**protected**
OnColumnSorted	**protected**	OnGetStateIndex	**protected**
OnColumnSorting	**protected**	OnHeaderButtonClick	**protected**
OnCustomDraw	**protected**	OnHeaderMoved	**protected**
OnCustomDrawBand	**protected**	OnHideBand	**protected**
OnCustomDrawCell	**protected**	OnHideHeader	**protected**
OnCustomDrawColumnHeader	**protected**	OnIsExistFooterCell	**protected**
OnCustomDrawFooter	**protected**	OnShowBand	**protected**
OnCustomDrawFooterNode	**protected**	OnShowHeader	**protected**
OnCustomDrawPreview	**protected**	OnStartBandDragging	**protected**
OnCustomDrawPreviewCell	**protected**	OnStartHeaderDragging	**protected**
OnDragEndBand	**protected**		

Derived from TCustomdxTreeList

OnBeginDragNode	**protected**	OnEditChange	**protected**
OnCanNodeSelected	**protected**	OnEdited	**protected**
OnChangeColumn	**protected**	OnEditing	**protected**
OnChangeLeftCoord	**protecte d**	OnEditValidate	**protected**
OnChangeNode	**protected**	OnEndColumnsCustomizing	**protected**
OnChangeTopVisibleNode	**protected**	OnExpanded	**protected**
OnCollapsed	**protected**	OnExpanding	**protected**
OnCollapsing	**protected**	OnGetEditColor	**protected**
OnCompare	**protected**	OnHotTrackNode	**protected**
OnDeletion	**protected**	OnSelectedCountChange	**protected**

OnBandButtonClick event

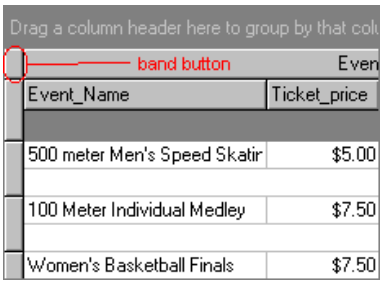
Occurs when clicking the band area indicator.

```
type
  TNotifyEvent = procedure (Sender: TObject) of object;
property OnBandButtonClick : TNotifyEvent;
```

Description

Column area is an area at the top left of a tree list control to the left of columns.

This event is generated only if the egoBandButtonClicking option (the OptionEx property) is active. Write an OnBandButtonClick event handler to perform specific actions when a user clicks the band area indicator.



The following code collapses all nodes of a tree list control when clicking the band area indicator:

```
procedure TForm1.dxDBGrid1BandButtonClick(Sender: TObject);
begin
 dxDBGrid1.FullCollapse;
end;
```

See Also

TCustomdxTreeList.BandButtonPushed; TCustomdxTreeListControl.OnHeaderButtonClick

OnBandClick event

Occurs when clicking a band.

```
type
  TdxTLStartBandDragging = procedure(Sender: TObject; ABand:
TdxTreeListBand) of object;
property OnBandClick : TdxTLStartBandDragging;
```

Description

Write an OnBandClick event handler to perform specific actions when clicking ABand.

Copyright © 2000 Developer Express Inc.

See Also

TCustomdxTreeList.DoBandClick; TCustomdxTreeListControl.OnHeaderClick

OnCalcRowLineHeight event

Occurs when the tree list control draws cells.

```
type
  TdxTLCalcRowLineHeight = procedure(Sender: TObject; Node:
TdxTreeListNode; Column: TdxTreeListColumn; const Text: string;
BoundsWidth, TextHeight, MaxLineCount: Integer; InflateFlag, GridFlag:
Boolean; var LineCount, LineHeight: Integer; var Done: Boolean) of object;
property OnCalcRowLineHeight: TdxTLCalcRowLineHeight;
```

Description

Write an OnCalcRowLineHeight event handler to specify the height for a cell. This event is called for each cell of a node. Node height is equal to the maximum height of its cell. OnCalcRowLineHeight uses the following parameters:

Node and **Column**	Identify cell location.
Text	Specifies the text a cell contains.
BoundsWidth	Specifies the width of a cell.
TextHeight	Determines the Text height in pixels.
MaxLineCount	Determines the maximum height of a cell in text lines.
InflateFlag	Determines whether a short distance exists between text and the top cell border.
GridFlag	Determines whether a short distance exists between text and the left cell border.
LineCount	Determines the height of a cell in lines.
LineHeight	Specifies the height of a line in pixels.
Done	Determines whether tree list will accept a new height.

Note: *This event is generated when the tree list control is in* 'auto row height' *mode:*

The aoPreview options is inactive (egoPreview for the ExpressQuantumGrid).

The aoAutoRowHeight option is active (egoAutoRowHeight for the ExpressQuantumGrid).

All columns are placed on the same row within the tree list control.

See Also

TCustomdxTreeList.Option; TCustomdxTreeList.OptionsEx

OnCanBandDragging event

Occurs before dragging a band.

```
type
  TdxTLCanBandDragging = procedure(Sender: TObject; ABand: TdxTreeListBand;
var Allow: Boolean) of object;
property OnCanBandDragging : TdxTLCanBandDragging;
```

Description

Write an OnCanBandDragging event handler to enable/disable band dragging at runtime. ABand specifies a dragged band. The Allow parameter determines whether it is possible to drag a specified band. Set Allow to False to prevent dragging of a particular band.

See Also

TCustomdxTreeListControl.OnStartBandDragging; TCustomdxTreeListControl.OnDragEndBand; TCustomdxTreeListControl.OnDragOverBand

OnCanHeaderDragging event

Occurs before dragging a column.

```
type
  TdxTLCanHeaderDragging = procedure(Sender: TObject; AColumn:
TdxTreeListColumn; var Allow: Boolean) of object;
property OnCanHeaderDragging : TdxTLCanHeaderDragging;
```

Description

Write an OnCanHeaderDragging event handler to enable/disable column dragging at runtime. AColumn specifies the dragged column. The Allow parameter determines whether it is possible to drag a specified column. Set Allow to False to avoid dragging of a particular column.

See Also

TCustomdxTreeListControl.OnStartHeaderDragging; TCustomdxTreeListControl.OnDragEndHeader; TCustomdxTreeListControl.OnDragOverHeader

OnChangedColumnsWidth event

Occurs when column width has changed.

```
type
  TNotifyEvent = procedure (Sender : TObject) of object;
property OnChangedColumnsWidth : TNotifyEvent;
```

Description

This event is generated after changing column width. Write an OnChangedColumnsWidth event handler to perform specific actions when column width has been changed.

See Also

TdxTreeListColumn.Width; TCustomdxTreeList.OnColumnMoved

OnColumnClick event

Occurs when clicking a column.

```
type
```

```
  TTLColumnClick = procedure(Sender : TObject; Column : TdxTreeListColumn)
of object;
property OnColumnClick : TTLColumnClick;
```

Description

Write an OnColumnClick event handler to respond to column click events. The Column parameter determines the clicked column.

See Also

TCustomdxTreeListControl.OnBandClick

OnColumnMoved event

Occurs when moving a column.

```
type
  TTLColumnMovedEvent = procedure(Sender: TObject; FromIndex, ToIndex:
Longint) of object;
property OnColumnMoved : TTLColumnMovedEvent;
```

Description

If the Options property includes the aoColumnMoving flag, users can move columns. The OnColumnMoved event is generated when a column has just moved. FromIndex determines the index of a column before moving. ToIndex specifies the new index of a column.

Note: *OnColumnMoved is obsolete with version 2 of the ExpressQuantumGrid Suite. Use OnDragEndHeader instead.*

See Also

TCustomdxTreeListControl.OnChangeColumn; TCustomdxTreeListControl.OnChangeColumnsWidth; TCustomdxTreeListControl.OnDragEndHeader

OnColumnSorted event

Occurs when drawing a column header.

```
type
  TdxTreeListColumnSort = (csNone, csDown, csUp);
  TTLColumnSorted = procedure (Sender: TObject; Column: TdxTreeListColumn;
var Sorted : TdxTreeListColumnSort) of object;
property OnColumnSorted: TTLColumnSorted;
```

Description

Write an OnColumnSorted event handler to specify an arrow, which is drawn next to the Column caption as an indicator for sort order. Sorted determines direction of an arrow:

csUp	up arrow

csDown	down arrow
csNone	no arrow

This event has been included for backward compatibility. Use the TdxTreeListColumn Sorted property to determine the arrow. You should not need to use this event in the future.

See Also

TdxTreelistColumn.Sorted; TCustomdxTreeListControl.OnColumnSorting; TCustomdxTreeListControl.SortedColumnsCount; TCustomdxTreeListControl.RefreshSorting

OnColumnSorting event

Occurs when nodes are about to be sorted.

```
type
  TdxTreeListColumnSorting = procedure(Sender: TObject; Column:
TdxTreeListColumn; var Allow: Boolean) of object;
property OnColumnSorting: TdxTreeListColumnSorting;
```

Description

Write an OnColumnSorting event handler to specify whether nodes can be sorted by a specific column. Set the Allow parameter to False within the event handler to prevent sorting by a column. For instance, the following code prevents sorting by the “Price” column:

```
procedure TForm1.dxTreeList1(Sender : TObject; Column : TdxTreeListColumn;
var Allow : Boolean);
begin
  if Column.Caption = ‘Price’ then
    Allow := False;
end;
```

See Also

TdxTreeListColumn.Sorted; TCustomdxTreeListControl.PrepareColumnSorted; TCustomdxTreeListControl.OnColumnSorted; TCustomdxTreeListControl.SortedColumnsCount; TCustomdxTreeListControl.RefreshSorting

OnCustomDraw event

Occurs when drawing a cell in a tree list control.

```
type
  TCustomDrawEvent = procedure(Sender : TObject; ACanvas : TCanvas; ARect :
TRect; ANode : TdxTreeListNode; AColumn : TdxDBTreeListColumn; const AText
: String; AFont : TFont; var AColor : TColor; ASelected, AFocused :
Boolean; var ADone : Boolean ) of object;
property OnCustomDraw: TCustomDrawEvent;
```

Description

Write an OnCustomDraw event handler to draw the contents of all or particular cells within a tree list control. Draw a cell using the TCanvas methods. The ARect parameter indicates location of a cell on the canvas:

Parameters	Meaning
ANode and **AColumn**	Node and column of a cell.
AText	Context of a cell.
AFont and **AColor**	Font and color used to draw a cell.
ASelected	Determines whether a node to which a cell belongs is selected.
AFocused	Specifies whether a node to which a cell belongs is focused.
ADone	Determines whether the OnCustomDraw event handler entirely draws a cell. If ADone is True, the default draw procedure is not called.

Note: *The OnCustomDraw event is included for backward compatibility with ExpressGrid ver. 1.4x. For ExpressQuantumGrid, you should use OnCustomDrawCell.*

See Also

TCustomdxTreeListControl.OnCustomDrawBand; TCustomdxTreeListControl.OnCustomDrawCell; TCustomdxTreeListControl.OnCustomDrawColumnHeader; TCustomdxTreeListControl.OnCustomDrawFooter; TCustomdxTreeListControl.OnCustomDrawPreview; TCustomdxTreeListControl.OnCustomDrawPreviewCell; TCustomdxTreeListControl.OnGetLevelColor

OnCustomDrawBand event

Occurs during drawing of a band.

```
type
  TdxTLCustomDrawBand = procedure(Sender: TObject; ABand: TdxTreeListBand;
ACanvas: TCanvas; ARect: TRect; var AText: string; var AColor: TColor;
AFont: TFont; var AAlignment: TAlignment; var ADone: Boolean) of object;
property OnCustomDrawBand : TdxTLCustomDrawBand;
```

Description

Write an OnCustomDrawBand event handler to draw bands based on your specific requirements. The available parameters include:

Parameter	Meaning
ABand	Specifies the drawing band.
ACanvas	Determines canvas for drawing.
ARect	Specifies rectangle for drawing.
AText	Determines text that is drawn as a band caption.
AColor	Specifies color of a band.
AFont	Specifies font for drawing band caption.

AAlignment	Specifies text alignment within a band.
ADone	Determines whether the OnCustomDrawBand event handler entirely draws a band. If ADone is True, the default drawing procedure is not called.

In order to specify the color, text, font or alignment, used to draw a particular band, set proper values for AColor, AText, AFont or AAlignment parameters and set ADone to False within the OnCustomDrawBand event handler.

See Also

TCustomdxTreeListControl.OnCustomDraw; TCustomdxTreeListControl.OnCustomDrawCell; TCustomdxTreeListControl.OnCustomDrawColumnHeader; TCustomdxTreeListControl.OnCustomDrawFooter; TCustomdxTreeListControl.OnCustomDrawPreview; TCustomdxTreeListControl.OncustomDrawPreviewCell; TCustomdxTreeListControl.OnGetLevelColor

OnCustomDrawCell event

Occurs when drawing a particular cell within a tree list control.

```
type
  TdxTLCustomDrawCell = procedure(Sender: TObject; ACanvas: TCanvas; ARect:
TRect; ANode: TdxTreeListNode; AColumn: TdxTreeListColumn; ASelected,
AFocused, ANewItemRow: Boolean; var AText: string; var AColor: TColor;
AFont: TFont; var AAlignment: TAlignment; var ADone: Boolean) of object;
property OnCustomDrawCell : TdxTLCustomDrawCell;
```

Description

Write an OnCustomDrawCell event handler to draw a particular cell of a tree list control based on your specific requirements. This event is an extended version of the OnCustomDraw event. If your define an OnCustomDrawCell event handler, the OnCustomDraw event is not generated. It is strongly recommended that you use OnCustomDrawCell instead of OnCustomDraw.

Parameter	Meaning
AColumn	Specifies the drawing column.
ACanvas	Determines canvas for drawing.
ARect	Specifies the rectangle for drawing.
ANode	Specifies the node to which a cell belongs.
AColumn	Specifies column of a drawing cell.
AText	Determines text drawn as a column caption.
ASelected	Determines whether a node, to which the drawing cell belongs, is selected.
AFocused	Determines whether a node, to which the drawing cell belongs, is focused.
ANewItemRow	Determines whether a node, to which the drawing cell belongs, is a new item row.
AColor	Specifies the color of a column.
AFont	Specifies the font for drawing column caption.

Copyright © 2000 Developer Express Inc.

AAlignment	Specifies the text alignment within a column.
ADone	Determines whether the OnCustomDrawCell event handler entirely draws a column header. If ADone is True, the default drawing procedure is not called.

In order to specify the color, font, text or alignment, used for drawing the header of a particular column, set proper values for AColor, AFont, AText or AAlignment parameters and set ADone to False within the OnCustomDrawCell event handler.

See Also

TCustomdxTreeListControl.OnCustomDrawBand; TCustomdxTreeListControl.OnCustomDraw; TCustomdxTreeListControl.OnCustomDrawColumnHeader; TCustomdxTreeListControl.OnCustomDrawFooter; TCustomdxTreeListControl.OnCustomDrawPreview; TCustomdxTreeListControl.OncustomDrawPreviewCell; TCustomdxTreeListControl.OnGetLevelColor

OnCustomDrawColumnHeader event

Occurs when drawing a column header.

```
type
  TdxTreeListColumnSort = (csNone, csDown, csUp);
  TdxTLCustomDrawHeader = procedure(Sender: TObject; AColumn:
TdxTreeListColumn; ACanvas: TCanvas; ARect: TRect; var AText: string; var
AColor: TColor; AFont: TFont; var AAlignment: TAlignment; var ASorted:
TdxTreeListColumnSort; var ADone: Boolean) of object;
property OnCustomDrawColumnHeader : TdxTLCustomDrawHeader;
```

Description

Write an OnCustomDrawColumnHeader event handler to draw a column header in a manner required by your application. This event is an extended version of the OnCustomDrawHeader event. If your define an OnCustomDrawColumnHeader event handler, the OnCustomDrawHeader event is not generated. It is strongly recommended to use OnCustomDrawColumnHeader instead of OnCustomDrawHeader.

Parameter	Meaning
AColumn	Specifies the drawing column.
ACanvas	Determines canvas for drawing.
ARect	Specifies rectangle for drawing.
AText	Determines text that is drawn as column caption.
AColor	Specifies color of a column.
AFont	Specifies the font for drawing column caption.
AAlignment	Specifies text alignment within a column.
ASorted	Determines whether nodes are sorted by a column, whose header is drawn
ADone	Determines whether the OnCustomDrawColumnHeader event handler entirely draws a column header. If ADone is True, the default drawing procedure is not called.

In order to specify the color, text, font or alignment, used for drawing the header of a particular column, set proper values for AColor, AText, AFont or AAlignment parameters and set ADone to False within the OnCustomDrawColumnHeader event handler.

See Also

TCustomdxTreeListControl.OnCustomDrawBand; TCustomdxTreeListControl.OnCustomDrawCell; TCustomdxTreeListControl.OnCustomDraw; TCustomdxTreeListControl.OnCustomDrawFooter; TCustomdxTreeListControl.OnCustomDrawPreview; TCustomdxTreeListControl.OncustomDrawPreviewCell; TCustomdxTreeListControl.OnGetLevelColor

OnCustomDrawFooter event

Occurs when drawing the footer of a tree list control.

```
type
  TdxTLCustomDrawFooter = procedure(Sender: TObject; ACanvas: TCanvas;
ARect: TRect; ANode: TdxTreeListNode; AColumn: TdxTreeListColumn; var
AText: string; var AColor: TColor; AFont: TFont; var AAlignment:
TAlignment; var ADone: Boolean) of object;
property OnCustomDrawFooter : TdxTLCustomDrawFooter;
```

Description

Write an OnCustomDrawFooter event handler to draw the footer of a tree list column as you require.

Parameter	Meaning
AColumn	Specifies the drawing column.
ACanvas	Determines canvas for drawing.
ARect	Specifies the rectangle for drawing.
ANode	Specifies the node to which a cell belongs.
AColumn	Specifies column of the drawing cell.
AText	Determines text that is drawn as a footer caption.
AColor	Specifies the color of a footer.
AFont	Specifies the font for drawing footer caption.
AAlignment	Specifies text alignment within a footer.
ADone	Determines whether the OnCustomDrawFooter event handler entirely draws a column footer. If ADone is True, the default drawing procedure is not called.

In order to specify the color, text, font or alignment, used to draw the tree list footer, specify AColor, AFont, AText or AAlignment parameters and set ADone to False within the OnCustomDrawFooter event handler.

See Also

TCustomdxTreeListControl.OnCustomDrawBand; TCustomdxTreeListControl.OnCustomDrawCell; TCustomdxTreeListControl.OnCustomDrawColumnHeader; TCustomdxTreeListControl.OnCustomDraw; TCustomdxTreeListControl.OnCustomDrawPreview; TCustomdxTreeListControl.OncustomDrawPreviewCell; TCustomdxTreeListControl.OnGetLevelColor

Copyright © 2000 Developer Express Inc.

OnCustomDrawFooterNode event

Occurs when drawing a footer node.

```
type
  TdxTLCustomDrawFooterNode = procedure(Sender: TObject; ACanvas: TCanvas;
ARect: TRect; ANode: TdxTreeListNode; AColumn: TdxTreeListColumn;
AFooterIndex: Integer; var AText: string; var AColor: TColor; AFont: TFont;
var AAlignment: TAlignment; var ADone: Boolean) of object;
property OnCustomDrawFooterNode : TdxTLCustomDrawFooterNode;
```

Description

Write an OnCustomDrawFooterNode event handler to draw the footer node as you require.

Parameter	Meaning
ACanvas	Determines canvas for drawing.
ARect	Specifies the rectangle for drawing.
ANode	Specifies the node to which a cell belongs.
AColumn	Specifies the column of a drawing cell.
AFooterIndex	Specifies index of the footer.
AText	Determines text that is drawn for a column caption
AColor	Specifies the color of a column.
AFont	Specifies the font for drawing column caption.
AAlignment	Specifies the text alignment within a column.
ADone	Determines whether the OnCustomDrawCell event handler entirely draws a column header. If ADone is True, the default drawing procedure is not called.

In order to specify the color, text, font or alignment, used for drawing the header of a particular column, set proper values for AColor, AFont, AText or AAlignment parameters and set ADone to False within the OnCustomDrawFooterNode event handler.

See Also

TCustomdxTreeListControl.OnCustomDrawFooter; TCustomdxTreeListControl.OnGetLevelColor; TdxTreeList.OnIsExistRowFooterCell

OnCustomDrawPreview event

Occurs when drawing the preview section.

```
type
  TTLCustomDrawPreviewEvent = procedure(Sender : TObject; ACanvas :
TCanvas; ARect : TRect; ANode : TdxTreeListNode; ASelected : Boolean; const
AText : String; var ADone : Boolean ) of object;
property OnCustomDrawPreview : TTLCustomDrawPreviewEvent;
```

Description

Write an OnCustomDrawPreview event handler to draw the preview section yourself. The ACanvas parameter specifies a TCanvas object to draw. ARect determines the rectangle to draw. The ANode parameter specifies a node, whose data is used by the preview section. ASelected determines whether the drawing node is selected. If ASelected is False, the node is not selected. AText determines the text string which appears in the preview section. The ADone variable determines whether to call the default procedure for preview section drawing. If ADone is True, the default drawing procedure is not called.

Note: *The OnCustomDrawPreview event is included for backward compatibility with ExpressGrid ver. 1.4x, For ExpressQuantumGrid, you should use OnCustomDrawPreviewCell.*

See Also

TCustomdxTreeListControl.OnCustomDraw; TCustomdxTreeListControl.OnCustomDrawBand; TCustomdxTreeListControl.OnCustomDrawCell; TCustomdxTreeListControl.OnCustomDrawColumnHeader; TCustomdxTreeListControl.OnCustomDrawFooter; TCustomdxTreeListControl.OncustomDrawPreviewCell; TCustomdxTreeListControl.OnGetLevelColor

OnCustomDrawPreviewCell event

Occurs when drawing a specific cell in the preview pane.

```
type
  TdxTLCustomDrawPreview = procedure(Sender : TObject; ACanvas: TCanvas;
ARect: TRect; ANode: TdxTreeListNode; ASelected: Boolean; var AText:
string; var AColor, ATextColor: TColor; AFont: TFont; var ADone: Boolean)
of object;
property OnCustomDrawPreviewCell : TdxTLCustomDrawPreview;
```

Description

Write an OnCustomDrawPreviewCell event handler to draw cells of the preview pane as you require.

Parameter	Meaning
AColumn	Specifies the drawing column.
ACanvas	Determines canvas for drawing.
ARect	Specifies the rectangle for drawing.
ANode	Specifies the node to which a cell belongs
ASelected	Specifies whether a node to which the preview pane belongs is selected.
AText	Determines text that is drawn for a column caption.
AColor	Specifies the color of a column.
AFont	Specifies the font for drawing a column caption.
AAlignment	Specifies text alignment within a column.
ADone	Determines whether the OnCustomDrawCell event handler entirely draws a column header. If ADone is True, the default drawing procedure is not called.

Copyright © 2000 Developer Express Inc.

In order to specify the color, text, font or alignment, used for drawing a preview cell, set proper values for AColor, AFont, AText or AAlignment parameters and set ADone to False within the OnCustomDrawPreviewCell event handler.

See Also

TCustomdxTreeListControl.OnCustomDrawBand; TCustomdxTreeListControl.OnCustomDrawCell; TCustomdxTreeListControl.OnCustomDrawColumnHeader; TCustomdxTreeListControl.OnCustomDrawFooter; TCustomdxTreeListControl.OnCustomDrawPreview; TCustomdxTreeListControl.OncustomDraw; TCustomdxTreeListControl.OnGetLevelColor

OnDragEndBand event

Occurs when a user has dropped a band onto a tree list control.

```
type
  TdxTLDragEndBand = procedure(Sender: TObject; ABand: TdxTreeListBand; P:
TPoint; AbsoluteIndex, VisibleIndex: Integer; var NewIndex: Integer; var
Accept: Boolean) of object;
property OnDragEndBand: TdxTLDragEndBand;
```

Description

Write an OnDragEndBand event handler to perform special processing when ending band dragging. Set the Accept parameter to False within the OnDragEndBand event handler to reject the dragged band. Accept, set to True, allows users to drop the dragged band. The P parameter determines a point at which ABand is dropped. AbsoluteIndex is the absolute index of a band among bands of a tree list control. VisibleIndex is a position of ABand among visible bands of a tree list control. NewIndex specifies new position of ABand among visible bands.

See Also

TCustomdxTreeListControl.OnCanBandDragging; TCustomdxTreeListControl.OnDragOverBand, TCustomdxTreeListControl.OnStartBandDragging

OnDragEndHeader event

Occurs when a user has dropped a band onto a tree list control.

```
type
  TdxTLDragEndHeader = procedure(Sender: TObject; AColumn:
TdxTreeListColumn; P: TPoint; var NewPosInfo: TdxHeaderPosInfo; var Accept:
Boolean) of object;
property OnDragEndHeader : TdxTLDragEndHeader;
```

Description

Write an OnDragEndHeader event handler to perform special processing when ending column drag events. Set the Accept parameter to False within the OnDragEndBand event handler to reject the dragged column. Set Accept to True in order to allow users to drop the dragged column. The P parameter determines a point at which the column is dropped. NewPosInfo specifies the new position of a column.

See Also

TCustomdxTreeListControl.OnCanHeaderDragging; TCustomdxTreeListControl.OnDragOverHeader, TCustomdxTreeListControl.OnStartHeaderDragging

OnDragOverBand event

Occurs when dragging a band.

```
type
  TdxTLDragOverBand = procedure(Sender: TObject; ABand: TdxTreeListBand; P:
TPoint; var Accept: Boolean) of object;
property OnDragOverBand : TdxTLDragOverBand;
```

Description

Use it to control drag & drop operations on bands. The ABand parameter specifies the band being dragged. The P parameter specifies a point where the dragged band is located. Set Accept to True within the OnDragOverBand event handler to prevent drawing of the “not allowed” symbol.

See Also

TCustomdxTreeListControl.OnCanBandDragging; TCustomdxTreeListControl.OnDragEndBand; TCustomdxTreeListControl.OnStartBandDragging

OnDragOverHeader event

Occurs when dragging a column header.

```
type
  TdxTLDragOverHeader = procedure(Sender: TObject; AColumn:
TdxTreeListColumn; P: TPoint; var Accept: Boolean) of object;
property OnDragOverHeader : TdxTLDragOverHeader;
```

Description

This event is generated when dragging a column header. Use it to control drag & drop operations for a specific column. The AColumn parameter determines the column being dragged. The P parameter specifies a point at which the dragged column is located. Set Accept to True within an OnDragOverHeader event handler to prevent drawing of the “not allowed” symbol.

See Also

TCustomdxTreeListControl.OnCanHeaderDragging; TCustomdxTreeListControl.OnDragEndHeader; TCustomdxTreeListControl.OnStartHeaderDragging

OnEndDragColumn event

Occurs when a dragged column is dropped, that is, when the mouse button is released.

```
type
  TdxTLEndDragColumn = procedure(Sender : TObject; X, Y: Integer;  Column:
TdxTreeListColumn; var NewPosition: Integer) of object;
property OnEndDragColumn: TdxTLEndDragColumn;
```

Description

Write an OnEndDragColumn event handler to perform an action when column dragging completes. X and Y determine the coordinates where a column is dropped. NewPosition specifies the new index of Column in the array of tree list visible columns. If a column has been dragged out of the tree list control, the NewIndex is equal to -1. The following OnEndDragColumn event handler prevents dragging of the first column:

```
procedure TForm1.dxTreeListEndDragColumn(Sender: TObject; X, Y: Integer;
Column: TdxTreeListColumn; var NewPosition: Integer);
begin
  if Column = dxTreeList.Columns[0] then
    NewPosition := 0;
end;
```

Note: *OnEndDragColumn is obsolete with version 2 of the ExpressQuantumGrid Suite. Use the OnDragEndHeader event instead.*

See Also

TCustomdxTreeListControl.OnDragEndHeader

OnGetImageIndex event

Occurs when accessing the ImageIndex property of a node.

```
type
  TTLGetImageEvent = procedure(Sender: TObject; Node: TdxTreeListNode; var
Index: Integer) of object;
property OnGetImageIndex: TTLGetImageEvent;
```

Description

This event occurs when a component accesses the image index for a particular node. The Node parameter specifies the node for which an image index is specified. The Index parameter determines the image index of a node. Use the Index parameter to associate different pictures with different nodes.

Note: *The following code within the OnGetImageIndex event handler leads to stack overflow. You should avoid it:*

```
  Node.ImageIndex :=  image_index; // <-incorrect
```

The correct method:

```
  Index :=  image_index; // <- correct
```

The following example associates a special image with nodes, whose 'Budget' column is less than 20000:

```
procedure TForm1.dxTreeListGetImageIndex(Sender: TObject;  Node:
TdxTreeListNode; var Index: Integer);
var
```

```
  i :  integer;
begin
  i:=0;
  while i<dxTreeList.Columns.Count do
  if dxTreeList.Columns.Items[i].Caption = 'Budget' then
    break
  else
    i:=i+1;

  if i < dxTreeList.Columns.Count then
    if TdxTreeListTextNode(Node).Strings[i] <> '' then
      if StrToFloat(TdxTreeListTextNode(Node).Strings[i]) < 20000 then
        Index := 2
      else
        Index := 1;
end;
```

See Also

TCustomdxTreeListControl.OnGetSelectedImage; TCustomdxTreeListControl.OnGetStateImage; TCustomdxTreeList.Images

OnGetLevelColor event

Occurs when drawing nodes of a tree list control.

```
type
  TdxTLGetLevelColor = procedure(Sender: TObject; ALevel: Integer; var
AColor: TColor) of object;
property OnGetLevelColor : TdxTLGetLevelColor;
```

Description

Write an OnGetLevelColor event handler to specify a different color for nodes on different levels. The ALevel parameter determines the level and AColor specifies the color for this level. The following example sets the node color of the first three levels:

```
procedure TForm1.dxDBGrid1GetLevelColor(Sender: TObject; ALevel: Integer;
var AColor: TColor);
begin
  case Alevel of
    0 : AColor := clBlue;
    1 : AColor := clRed;
    2 : AColor := clWhite;
  end;
end;
```

***Note**: The OnGetLevelColor event is fired only if the PaintStyle property is Outlook*

See Also

TCustomdxTreeListControl.OnCustomDrawBand; TCustomdxTreeListControl.OnCustomDrawCell; TCustomdxTreeListControl.OnCustomDrawColumnHeader; TCustomdxTreeListControl.OnCustomDrawFooter; TCustomdxTreeListControl.OnCustomDrawPreview; TCustomdxTreeListControl.OnCustomDrawPreviewCell; TCustomdxTreeListControl.OnCustomDraw

OnGetNodeDragText event

Occurs when dragging a node.

```
type
  TdxTLGetNodeDragText = procedure(Sender: TObject; Node: TdxTreeListNode;
Column: TdxTreeListColumn; var AText: string) of object;
property OnGetNodeDragText: TdxTLGetNodeDragText;
```

Description

The OnGetNodeDragText property is generated when a user drags a node within a tree list control. By default, within the dragged node the tree list control draws text stored in a focused column. Write an OnGetNodeDragText event handler to specify text for the dragged node. The OnGetNodeDragText event is most useful if a user drags a node by a check or graphics column: i.e. If a user drags a node by a graphics column, the dragged node contains the following text string: (GRAPHIC).

To specify different text for a dragged node, assign the desired string to the AText parameter within the event handler. The Node parameter determines the dragged node. Column is a focused tree list column. If the aoRowSelect option is active, the Column parameter specifies the first column in a tree list. For instance, the following code specifies the column caption as a dragged node text:

```
procedure TForm1.dxDBGrid1GetNodeDragText(Sender: TObject; Node:
TdxTreeListNode; Column: TdxTreeListColumn; var AText: String);
begin
  AText := Column.Caption;
end;
```

See Also

TCustomdxTreeList.DragNode; TCustomdxTreeList.OnBeginDragNode

OnGetSelectedIndex event

Occurs when accessing the SelectedIndex property of a node.

```
type
  TTLGetImageEvent = procedure(Sender: TObject; Node: TdxTreeListNode; var
Index: Integer) of object;
property OnGetSelectedIndex: TTLGetImageEvent;
```

Description

SelectedIndex specifies an image, which is displayed when a node is selected. Use this event to associate a particular image with the Node parameter. The Index parameter determines the index of an image in the image list; that is, Index determines the SelectedIndex property value.

Note: *The following code within the OnGetSelectedIndex event handler leads to stack overflow. You should avoid it:*

```
Node.SelectedIndex := image_index; // <- incorrect
```

The correct method:

```
Index := image_index; // <- correct
```

See Also

TCustomdxTreeListControl.OnGetImageIndex; TCustomdxTreeListControl.OnGetStateIndex; TCustomdxTreeList.Images; TdxTreeListNode.SelectedIndex

OnGetStateIndex event

Occurs when accessing the StateIndex property of a node.

```
type
  TTLGetImageEvent = procedure(Sender: TObject; Node: TdxTreeListNode; var
Index: Integer) of object;
property OnGetStateIndex: TTLGetImageEvent;
```

Description

The StateIndex property determines the picture shown to the left of a node, when a node is in its standard state. Use this event to associate a given image as a state image with a node. The Node parameter determines the node with which to associate the picture. The Index parameter determines the picture index in the image list; that is, Index is a value of the StateImage property.

Note: *The following code within the OnGetStateImage event handler leads to a stack overflow. You should avoid it:*

```
Node.StateIndex := image_index;  //<- incorrect
```

The correct method:

```
Index := image_index;  // <- correct
```

See Also

TCustomdxTreeListControl.OnGetImageIndex; TCustomdxTreeListControl.OnGetSelectedIndex; TCustomdxTreeList.StateImages

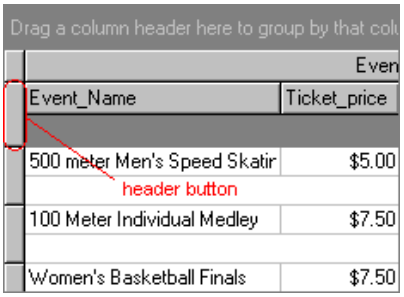
OnHeaderButtonClick event

Occurs when clicking the column area indicator (header button).

```
type
  TNotifyEvent = procedure (Sender: TObject) of object;
property OnHeaderButtonClick : TNotifyEvent;
```

Description

Column area is an area at the top left of a tree list control to the left of columns. This event is generated only if the egoHeaderButtonClicking option (the OptionEx property) is active. Write an OnHeaderButtonClick event handler to perform specific actions when a user clicks the column area indicator (header button).



The following code expands all nodes of the tree list control when clicking the header button:

```
// Delphi
procedure TForm1.dxDBGrid1HeaderButtonClick(Sender: TObject);
begin
 dxDBGrid1.FullExpand;
end;

// C++Builder
void __fastcall TForm1::dxDBGrid1HeaderButtonClick(TObject * Sender);
{
  dxDBGrid1->FullExpand();
}
```

See Also

TCustomdxTreeList.HeaderButtonPushed; TCustomdxTreeListControl.OnBandButtonClick

OnHeaderMoved event

Occurs when a user moves the column header in a tree list control.

```
type
  TdxTLHeaderMoved = procedure(Sender: TObject; Column: TdxTreeListColumn)
of object;
property OnHeaderMoved: TdxTLHeaderMoved;
```

Description

Write an OnHeaderMoved event handler to perform specific actions when a user has moved a column to another location within a tree list control. The Column parameter specifies the dragged column.

See Also

TCustomdxTreeListControl.OnColumnMoved

OnHideBand event

Occurs when a user moves a band from the tree list control to the customizing form.

```
type
  TdxTLStartBandDragging = procedure(Sender: TObject; ABand:
TdxTreeListBand) of object;
property OnHideBand: TdxTLStartBandDragging;
```

Description

Write an OnHideBand event handler to perform specific actions when dragging a band from the tree list control onto the customizing form. ABand specifies the dragged band.

See Also

TCustomdxTreeListControl.OnShowBand; TcustomdxTreeList.CustomizingForm; TCustomdxTreeList.HideBand

OnHideHeader event

Occurs when a user moves a column header from a tree list control to the customizing form.

```
type
  TdxTLStartHeaderDragging = procedure(Sender: TObject; AColumn:
TdxTreeListColumn) of object;
property OnHideHeader : TdxTLStartHeaderDragging;
```

Description

Write an OnHideHeader event handler to perform specific actions when dragging a column from the tree list control onto the customizing form. The AColumn parameter specifies the dragged column.

The following code prevents hiding the ‘Name’ column:

```
// Delphi
procedure TForm1.dxTreeList1HideHeader(Sender: TObject; AColumn:
TdxTreeListColumn);
begin
  if AColumn.Caption = 'Name' then
    AColumn.Visible := True;
end;

// C++Builder
void __fastcall TForm1::dxTreeList1HideHeader(TObject * Sender;
TdxTreeListColumn * AColumn);
{
```

```
    if (AColumn->Caption == "Name")
      AColumn->Visible = true;
  }
```

See Also

TCustomdxTreeListControl.OnShowHeader; TCustomdxTreeList.CustomizingForm

OnIsExistFooterCell event

Occurs when drawing the footer of a tree list control.

```
type
  TdxTLIsExistFooterCell = procedure(Sender: TObject; AColumn: Integer; var AExist: Boolean) of object;
property OnIsExistFooterCell : TdxTLIsExistFooterCell;
```

Description

Write an OnIsExistFooterCell event handler to specify whether a particular column has a footer cell. Set AExist to True within the OnIsExistFooterCell event handler to indicate that AColumn has a footer cell. Use OnCustomDrawFooter to draw the footer cell in a specific manner as required by your application. For instance, the following code draws footer cells for each column:

```
procedure TForm1.dxTreeList1IsExistFooterCell(Sender: TObject; AColumn: Integer; var AExist: Boolean);
begin
  AExist := True;
end;
```

See Also

TdxTreeList.OnIsLevelFooter; TdxTreeList.OnIsExistRowFooterCell; TCustomdxTreeListControl.OnCustomDrawFooter

OnShowBand event

Occurs when a user drags a band from the customizing form onto the tree list control.

```
type
  TdxTLShowBand = procedure(Sender: TObject; ABand: TdxTreeListBand; NewIndex: Integer) of object;
property OnShowBand : TdxTLShowBand;
```

Description

Write an OnShowBand event handler to perform special actions when dragging a band from the customizing form onto the tree list control. The ABand parameter specifies the dragged band. NewIndex specifies a new position of a band among visible bands of a tree list control.

See Also

TCustomdxTreeListControl.OnHideBand; TCustomdxTreeList.CustomizingForm

OnShowHeader event

Occurs when a user drags a column from the customizing form onto a tree list control.

```
type
  TdxTLShowHeader = procedure(Sender: TObject; AColumn: TdxTreeListColumn;
BandIndex, RowIndex, ColIndex: Integer) of object;
property OnShowHeader : TdxTLShowHeader;
```

Description

Write an OnShowHeader event handler to perform specific actions when a user has moved a column from the customizing form to a tree list control. The AColumn parameter specifies the dragged column header. BandIndex specifies the band wherein a column should be displayed. RowIndex specifies a vertical row within the band where a column is displayed. ColIndex determines column position within a row.

The following code prevents the display of the 'PHONE' column:

```
// Delphi
procedure TForm1.dxTreeList1ShowHeader(Sender: TObject; AColumn:
TdxTreeListColumn; BandIndex, RowIndex, ColIndex: Integer);
begin
  if AColumn.Caption = 'PHONE' then
    AColumn.Visible := False;
end;
```

```
// C++Builder
void __fastcall TForm1::dxTreeList1HideHeader(TObject * Sender;
TdxTreeListColumn * AColumn);
{
  if (AColumn->Caption == "PHONE")
    AColumn->Visible = false;
}
```

See Also

TCustomdxTreeListControl.OnHideHeader; TCustomdxTreeList.DoShowHeader; TCustomdxTreeList.CustomizingForm

OnStartBandDragging event

Occurs when a user begins dragging a band within a tree list control.

```
type
```

```
  TdxTLStartBandDragging = procedure(Sender: TObject; ABand:
TdxTreeListBand) of object;
property OnStartBandDragging : TdxTLStartBandDragging;
```

Description

Write an OnStartBandDragging event handler to implement special processing when a user starts dragging a band. Sender is a tree list control, which contains a band to be dragged. ABand is a band being dragged.

See Also

TCustomdxTreeListControl.OnCanBandDragging; TCustomdxTreeListControl.OnDragEndBand; TCustomdxTreeListControl.OnDragOverBand

OnStartHeaderDragging event

Occurs when a user begins dragging a band within a tree list control.

```
type
  TdxTLStartHeaderDragging = procedure(Sender: TObject; AColumn:
TdxTreeListColumn) of object;
property OnStartHeaderDragging : TdxTLStartHeaderDragging;
```

Description

Write an OnStartHeaderDragging event handler to implement special processing when a user starts dragging a particular column header. Sender is a tree list control, which contains a column header to be dragged. AColumn specifies a column being dragged.

See Also

TCustomdxTreeListControl.OnCanHeaderDragging; TCustomdxTreeListControl.OnDragEndHeader; TCustomdxTreeListControl.OnDragOverHeader

TdxTreeList

Unit

dxTL

```
TdxTreeList = class(TCustomdxTreeListControl);
```

Description

TdxTreeList is a hybrid of the standard tree view and tree list, but also includes unique features including full runtime column customization and the ability to edit any column within a node. TdxTreeList is inherited directly from TCustomdxTreeListControl and publishes most of its properties.

***Note**: Inplace and standalone editors of the ExpressQuantumGrid Suite require Windows RichEdit component 2.0 or higher to work properly. If the editors on your or client's machine do not work, make certain that the riched20.dll library is in the Windows system folder (Windows\System or WINNT\System32). Copy the specified DLL to fix the problem.*

Hierarchy

TObject
- TPersistent
  - TComponent
    - TControl
      - TWinControl
        - TCustomControl
          - TCustomdxContainer
            - TCustomdxTreeList
              - TCustomdxTreeListControl

TdxTreeList Properties

TdxTreeList

SortedColumn	**public / read only**

Derived from TCustomdxTreeListControl

BandMaxRowCount	**published / read / write**	IniFileName	**published / read / write**
BandRowCount	**published / read / write**	IniSectionName	**published / read / write**
Bands	**published / read / write**	IsCustomStoring	**protected / read/ write**
ColumnCount	**public / read only**	RegistryPath	**published / read / write**
Columns	**public / read / write**	SortedColumnCount	**public / read only**
DefaultLayout	**published / read / write**	SortedColumns	**public / read only**
HeaderMinRowCount	**published / read / write**	VisibleColumnCount	**public / read only**
HeaderPanelMaxRowCount	**published / read / write**	VisibleColumns	**public / read / write**
HeaderPanelRowCount	**published / read/ write**		

Derived from TCustomdxTreeList

ArrowsColor	**published / read / write**	FocusedNode	**public / read only**
AutoExpandOnSearch	**published / read / write**	FocusedNumber	**public / read / write**
AutoSearchColor	**published / read / write**	FooterPanelHeight	**public / read only**
AutoSearchTextColor	**published / read / write**	FooterRowHeight	**public / read only**
BandButtonPushed	**public / read only**	FooterRowNodeHeight	**public / read only**
BandColor	**published / read / write**	GridLineColor	**published / read / write**
BandFont	**published / read / write**	GroupNodeColor	**published / read / write**
BandPanelHeight	**public / read only**	GroupNodeTextColor	**published / read / write**
BorderStyle	**published / read / write**	HeaderButtonPushed	**public / read only**
Count	**public / read only**	HeaderColor	**published / read / write**
CustomizingBandListBox	**public / read only**	HeaderFont	**published / read / write**
CustomizingForm	**public / read only**	HeaderHeight	**public / read only**
CustomizingHeaderListBox	**public / read only**	HeaderPanelHeight	**public / read only**
CustomizingPos	**public read / write**	HeaderRowHeight	**public / read only**
CustomizingRowCount	**published / read/write**	HideFocusRect	**published / read / write**
DblClkExpanding	**published / read/write**	HideSelection	**published / read / write**
DefaultRowHeight	**published / read/write**	HideSelectionColor	**published / read / write**
DescTextHeight	**public / read only**	HideSelectionTextColor	**published / read / write**
DownBandIndex	**public / read only**	HighlightColor	**published / read / write**
DownBandPushed	**public / read only**	HighlightTextColor	**published / read / write**
DownColumnIndex	**public / read only**	Images	**published / read / write**
DownColumnPushed	**public / read only**	Indent	**protected / read only**
DragAbsoluteBandIndex	**public / read only**	IndentDesc	**published / read / write**
DragAbsoluteHeaderIndex	**public / read only**	InplaceColumnIndex	**public / read only**
DragNode	**public / read only**	InplaceEditor	**public / read only**
EditingText	**public / read / write**	IsCustomizing	**public / read only**
FixedBandLineColor	**published / read / write**	Items	**public / read only**
FixedBandLineWidth	**published / read / write**	LastNode	**public / read only**
FocusedAbsoluteIndex	**public / read only**	LeftCoord	**public / read/write**
FocusedColumn	**public / read / write**	LockSelection	**public / read only**

LockUpdate	**protected / read only**
LookAndFeel	**published / read / write**
MaxRowLineCount	**published / read / write**
NewItemRowActive	**protected / read / write**
NewItemRowHeight	**public / read only**
Options	**published / read / write**
OptionsEx	**published / read / write**
PaintStyle	**published / read / write**
PreviewFont	**published / read / write**
PreviewLines	**published / read / write**
RowCount	**public / read only**
RowFooterColor	**published / read / write**
RowFooterTextColor	**published / read / write**
RowHeight	**public / read only**
RowSeparatorLineWidth	**published / read / write**
ScrollBars	**published / read / write**
SelectedCount	**public / read only**
SelectedNodes	**public / read only**
SelectionAnchor	**protected / read / write**
ShowBands	**published / read / write**
ShowButtons	**published / read / write**
ShowFooter	**published / read / write**
ShowGrid	**published / read / write**
ShowHeader	**published / read / write**
ShowHiddenInCustomizeBox	**published / read / write**
ShowIndicator	**published / read / write**
ShowLines	**published / read / write**
ShowNewItemRow	**published / read / write**
ShowPreviewGrid	**published / read / write**
ShowRoot	**published / read / write**
ShowRowFooter	**published / read / write**
SimpleCustomizeBox	**published / read / write**
State	**public / read only**
StateImages	**published / read / write**
TabStop	**published / read / write**
TopIndex	**public / read / write**
TopNode	**public / read only**
TopVisibleNode	**public / read only**
TreeLineColor	**published / read / write**
TreeLineStyle	**published / read / write**
VisibleRowCount	**public / read only**
WaitForExpandNodeTime	**published / read / write**

Derived from TCustomdxContainer

IsInitEdit	**public / read only**
ReadOnly	**public / read only**

SortedColumn property — read only

Returns a sorted column.

```
property SortedColumn : TdxTreeListColumn;
```

Description

This property returns a column by which tree list nodes are sorted. If the aoMultisort option is active, SortedColumn returns the first sorted column.

See Also

TCustomdxTreeList.OptionsEx; TCustomdxTreeListControl.SortedColumnCount; TCustomdxTreeListControl.SortedColumns

Copyright © 2000 Developer Express Inc.

TdxTreeList Methods

TdxTreeList

AssignNodes	**public**	SaveToStream	**public**
LoadFromFile	**public**	SelectAll	**public**
LoadFromStream	**public**		
SaveToFile	**public**		

TCustomdxTreeListControl

ApplyBestFit	**public**	GetFooterColumnAt	**public**
AssignColumns	**public**	GetHeaderColumnAt	**public**
BeginSorting	**public**	GetNodeFooterColumnAt	**public**
ColumnByName	**public**	GetVisibleColumnIndex	**public**
CreateColumn	**public**	LoadFromIniFile	**public**
CreateColumnEx	**public**	LoadFromRegistry	**public**
DestroyColumns	**public**	RefreshDefaultColumnsWidths	**public**
EndSorting	**public**	RefreshSorting	**public**
GetAbsoluteColumnIndex	**public**	RestoreColumnsDefaults	**public**
GetBandAt	**public**	SaveToIniFile	**public**
GetColumnAt	**public**	SaveToRegistry	**public**
GetDisplayValue	**public**		

Derived from TCustomdxTreeList

Add	**public**	FindNodeByText	**protected**
AddFirst	**public**	FreeDrawInfo	**public**
AdjustColumnsWidth	**public**	FullCollapse	**public**
BeginSelection	**public**	FullExpand	**public**
BeginUpdate	**public**	GetEditRect	**public**
CalcDrawInfo	**public**	GetFocusedAbsoluteIndex	**public**
CancelEditor	**public**	GetFocusedVisibleIndex	**public**
CellRect	**public**	GetHitInfo	**public**
ClearColumnsSorted	**public**	GetHitTestInfoAt	**public**
ClearNodes	**public**	GetNextVisible	**public**
ClearSelection	**public**	GetNodeAt	**public**
CloseEditor	**public**	GetScrollVertGridRect	**protected**
ColumnsCustomizing	**public**	GotoFirst	**public**
CopyAllToClipboard	**public**	GotoLast	**public**
CopySelectedToClipboard	**public**	GotoNext	**public**
Create	**public**	GotoPrev	**public**
DeleteSelection	**public**	HideEditor	**public**
Destroy	**public**	IndexOf	**public**
EndColumnsCustomizing	**public**	Insert	**public**
EndSearch	**public**	InvalidateSelection	**protected**
EndSelection	**public**	IsInvertSelect	**public**
EndUpdate	**public**	IsNodeVisible	**public**
FindInplaceEdit	**protected**	MakeBandVisible	**public**

MakeColumnVisible	**public**	ShowButtonEditorMouse	**public**
MakeNodeVisible	**public**	ShowEditor	**public**
PointInCustomizingForm	**public**	ShowEditorChar	**public**
SaveAllToStrings	**public**	ShowEditorMouse	**public**
SaveAllToTextFile	**public**	StartDragBand	**protected**
SaveSelectedToStrings	**public**	StartDragHeader	**protected**
SaveSelectedToTextFile	**public**	StartSearch	**public**
SelectNodes	**protected**	UpdateScrollBars	**protected**
SetEditingText	**protected**		
SetEditText	**protected**		

Derived from TCustomdxContainer

BeginInitEdit	**protected**	GetReadOnly	**protected**
CanEditAcceptKey	**protected**	HideDrawFocusRect	**protected**
CanEditModify	**protected**	InplaceEditKeyDown	**protected**
CheckEditorWidth	**protected**	IsEditorMoved	**protected**
CreateEditStyle	**protected**	IsImmediateEditor	**protected**
EndInitEdit	**protected**	IsShowButtonAlways	**protected**
GetDefaultPopupBorderStyle	**public**	IsTabs	**protected**
GetEditColor	**protected**	MoveCol	**protected**
GetEditFont	**protected**	RedrawSelection	**protected**

Copyright © 2000 Developer Express Inc.

AssignNodes method

Loads nodes from another object.

```
procedure AssignNodes(Source: TPersistent);
```

Description

Use this procedure to retrieve nodes of another tree list and load them into the nodes of a given tree list.

LoadFromFile method

Loads nodes from a file.

```
procedure LoadFromFile(const FileName: string);
```

Description

Use the LoadFromFile method to retrieve tree list nodes from a file, specified by FileName, and loads them into a tree list. FileName entirely defines the file name. No extension is added to FileName.

See Also

TdxTreeList.LoadFromStream; TdxTreeList.SaveToFile; TdxTreeList.SaveToStream

LoadFromStream method

Reads tree list nodes from a stream and stores the contents within TdxTreeList.

```
procedure LoadFromStream(Stream: TStream);
```

Description

Use LoadFromStream to read the nodes of a tree list from the specified stream.

Note: *Stream must contain data in TdxTreeList internal format (for example, data is written to a stream by another TdxTreeList via the SaveToStream method).*

See Also

TdxTreeList.LoadFromFile; TdxTreeList.SaveToFile; TdxTreeList.SaveToStream

SaveToFile method

Saves TdxTreeList nodes to a file.

```
procedure SaveToFile(const FileName: string);
```

Description

Use this method to save all nodes of TdxTreeList to a specified file. Later these nodes can be reloaded from this file into another tree list using the LoadFromFile method. FileName entirely defines the file name. No extension is added to FileName.

See Also

TdxTreeList.LoadFromFile; TdxTreeList.LoadFromStream; TdxTreeList.SaveToStream

SaveToStream method

Writes nodes to a stream passed as the Stream parameter.

```
procedure SaveToStream(Stream: TStream);
```

Description

Use SaveToStream to stream out TdxTreeList nodes. It can be streamed back into another TdxTreeList object using the LoadFromStream method.

See Also

TdxTreeList.SaveToFile; TdxTreeList.LoadFromFile; TdxTreeList.LoadFromStream

SelectAll method

Selects all tree list nodes.

```
procedure SelectAll;
```

Description

Use this method to select all tree list nodes. Before selection, SelectAll expands all nodes. The selected nodes can be saved by the SaveSelectedToTextFile method into a text file or by the SaveSelectedToStrings method into strings.

See Also

TCustomdxTreeList.DeleteSelection; TCustomdxTreeList.ClearSelection; TCustomdxTreeList.SelectedNodes; TCustomdxTreeList.SelectedCount

Copyright © 2000 Developer Express Inc.

TdxTreeList Events

TdxTreeList

OnGetFooterCellText	**published**	OnIsExistRowFooterCell	**published**
OnGetPreviewLineCount	**publis hed**	OnIsLevelFooter	**published**
OnGetPreviewText	**published**		
OnGetRowLineCount	**published**		

Derived from TCustomdxTreeListControl

OnBandButtonClick	**published**	OnDragEndHeader	**published**
OnBandClick	**published**	OnDragOverBand	**published**
OnCalcRowLineHeight	**published**	OnDragOverHeader	**published**
OnCanBandDragging	**published**	OnEndDragColumn	**published**
OnCanHeaderDragging	**published**	OnGetImageIndex	**published**
OnChangedColumnsWidth	**published**	OnGetLevelColor	**published**
OnColumnClick	**published**	OnGetNodeDragText	**published**
OnColumnMoved	**published**	OnGetSelectedIndex	**published**
OnColumnSorted	**published**	OnGetStateIndex	**published**
OnColumnSorting	**published**	OnHeaderButtonClick	**published**
OnCustomDraw	**published**	OnHeaderMoved	**published**
OnCustomDrawBand	**published**	OnHideBand	**published**
OnCustomDrawCell	**published**	OnHideHeader	**published**
OnCustomDrawColumnHeader	**published**	OnIsExistFooterCell	**published**
OnCustomDrawFooter	**published**	OnShowBand	**published**
OnCustomDrawFooterNode	**published**	OnShowHeader	**published**
OnCustomDrawPreview	**published**	OnStartBandDragging	**published**
OnCustomDrawPreviewCell	**published**	OnStartHeaderDragging	**published**
OnDragEndBand	**published**		

Derived from TCustomdxTreeList

OnBeginDragNode	**published**	OnEditChange	**published**
OnCanNodeSelected	**published**	OnEdited	**published**
OnChangeColumn	**published**	OnEditing	**published**
OnChangeLeftCoord	**published**	OnEditValidate	**published**
OnChangeNode	**published**	OnEndColumnsCustomizing	**published**
OnChangeTopVisibleNode	**published**	OnExpanded	**published**
OnCollapsed	**published**	OnExpanding	**published**
OnCollapsing	**published**	OnGetEditColor	**published**
OnCompare	**published**	OnHotTrackNode	**published**
OnDeletion	**published**	OnSelectedCountChange	**published**

OnGetFooterCellText event

```
type
```

```
  TdxTLGetFooterCellText = procedure(Sender: TObject;
ANode: TdxTreeListNode; AColumn, AFooterIndex: Integer; var AText: string)
of object;
property OnGetFooterCellText: TdxTLGetFooterCellText;
```

Description

This event is generated when a tree list control draws footer cells (both node footer and tree list footer cells). Use the OnGetFooterCellText event to specify the text drawn within a footer cell (set the AText parameter to the desired text within the OnGetFooterCellText event handler). The AColumn parameter identifies the column under which a cell is drawn. The –1 value in AFooterIndex represents the drawing tree list footer. Otherwise, it specifies the node footer index. The ANode determines a node, for which the node footer is drawn.

The OnGetFooterCellText event is generated under the following conditions:

- The tree list control contains columns.
- The ShowFooter property is True.
- The AExist parameter is set to True within the OnIsExistFooterCell event handler.

See Also

TCustomdxTreeList.ShowFooter; TCustomdxTreeList.ShowFooter

OnGetPreviewLineCount event

Occurs when drawing the preview section.

```
type
  TdxTLGetPreviewLineCount = procedure(Sender: TObject; ANode:
TdxTreeListNode; var LCount: Integer) of object;
property OnGetPreviewLineCount : TdxTLGetPreviewLineCount;
```

Description

Write an OnGetPreviewLineCount event handler to specify the number of lines in a preview section of a node. The ANode parameter specifies the node for which the preview section is drawn. Set LCount parameter to specify the number of lines in the preview section.

See Also

TdxTreeList.OnGetPreviewText

OnGetPreviewText event

Occurs when drawing preview section text.

```
type
  TdxGetPreviewTextEvent = procedure(Sender : TObject; Node:
TdxTreeListNode; var Text: String) of object;
property OnGetPreviewText : TdxGetPreviewTextEvent;
```

Description

Write an OnGetPreviewText event handler to customize the text of the preview section. The Node parameter determines the node for which the preview section is drawn. Use the Text parameter to specify preview section text.

See Also

TdxTreeList.OnGetPreviewText

OnGetRowLineCount event

Occurs when moving through tree list nodes and before node repainting.

```
type
  TdxTLGetPreviewLineCount = procedure(Sender: TObject; ANode:
TdxTreeListNode; var LCount: Integer) of object;
property OnGetRowLineCount: TdxTLGetPreviewLineCount;
```

Description

A node within a tree list can occupy several rows. Write an OnGetRowLineCount event handler in order to specify number of rows for a particular node. The LCount parameter determines height of ANode in rows.

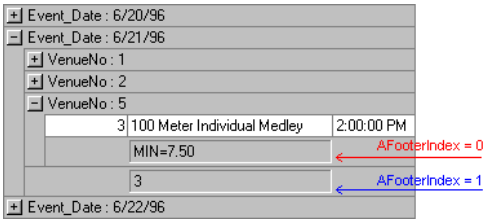
OnIsExistRowFooterCell event

This event is generated when drawing the ExpressTreeList.

```
type
  TdxTLIsExistRowFooterCell = procedure(Sender: TObject; ANode:
TdxTreeListNode; AColumn, AFooterIndex: Integer; var AExist: Boolean) of
object;
property OnIsExistRowFooterCell: TdxTLIsExistRowFooterCell;
```

Description

Write an OnIsExistRowFooterCell event handler to determine whether the ExpressTreeList draws a cell in the node footer for a column specified by the AColumn parameter. ANode is the node whose footer is drawn. AFooterIndex specifies index of the drawn footer.



To specify that a column incorporates a cell in the node footer, set the AExist parameter to True within the event handler.

See Also

TCustomdxTreeListControl.OnCustomDrawFooter; TdxTreeList.OnIsLevelFooter

OnIsLevelFooter event

Occurs when moving through tree list nodes and before node repainting.

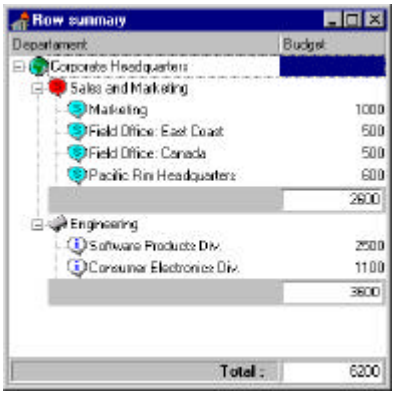
```
type
  TdxTLIsLevelFooter = procedure(Sender: TObject; ALevel: Integer; var
AExist: Boolean) of object;
property OnIsLevelFooter: TdxTLIsLevelFooter;
```

Description

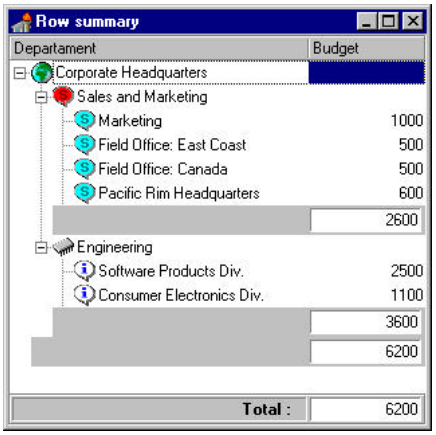
Write an OnIsLevelFooter event handler in order to draw a summary node footer. Set the AExist parameter to True to specify that nodes on level, defined by ALevel, have footers.

The following two examples demonstrates using the AExist parameter:

```
procedure TMainForm.dxTreeList1IsLevelFooter(Sender: TObject; ALevel:
Integer; var AExist: Boolean);
begin
  // the node footer is drawn for the second level only
  AExist := ALevel = 1;
end;
```




```
procedure TMainForm.dxTreeList1IsLevelFooter(Sender: TObject; ALevel:
Integer; var AExist: Boolean);
begin
  // the node footer is drawn for all levels
  AExist := True;
end;
```



See Also

TCustomdxTreeListControl.OnCustomDrawFooterNode;
TCustomdxTreeListControl.OnIsExistFooterCell

TCustomdxDBTreeListControl

Unit

dxDBCtrl

```
TCustomdxDBTreeListControl = class(TCustomdxTreeListControl);
```

Description

TCustomdxDBTreeListControl is the base control for all data-aware tree lists. It is inherited directly from TCustomdxTreeListControl. TCustomdxDBGrid is inherited directly from TCustomdxDBTreeListControl.

Hierarchy

TObject
- TPersistent
  - TComponent
    - TControl
      - TWinControl
        - TCustomControl
          - TCustomdxContainer
            - TCustomdxTreeList
              - TCustomdxTreeListControl

TCustomdxDBTreeListControl Properties

TCustomdxDBTreeListControl

Columns	**public / read / write**	PreviewFieldName	**protected / read / write**
DataLink	**public / read only**	PreviewMaxLength	**protected / read / write**
DataSource	**public / read only**	SelectedRows	**protected / read only**
FocusedField	**protected / read / write**	SortedColumn	**public / read only**
GrIndicatorWidth	**protected / read / write**	SortedColumns	**public / read only**
KeyField	**protected / read / write**	VisibleColumns	**public / read / write**

Derived from TCustomdxTreeListControl

BandMaxRowCount	**published / read / write**	IniFileName	**protected / read / write**
BandRowCount	**published / read / write**	IniSectionName	**protected / read / write**
Bands	**published / read / write**	IsCustomStoring	**protected / read / write**
ColumnCount	**public / read only**	RegistryPath	**protected / read / write**
DefaultLayout	**published / read / write**	SortedColumnCount	**public / read only**
HeaderMinRowCount	**published / read / write**	VisibleColumnCount	**public / read only**
HeaderPanelMaxRowCount	**published / read / write**		
HeaderPanelRowCount	**published / read / write**		

Derived from TCustomdxTreeList

ArrowsColor	**protected / read / write**	EditingText	**protected / read /write**
AutoExpandOnSearch	**protected / read / write**	FixedBandLineColor	**protected / read /write**
AutoSearchColor	**protected / read / write**	FixedBandLineWidth	**protected / read /write**
AutoSearchTextColor	**protected / read / write**	FocusedAbsoluteIndex	**public / read only**
BandButtonPushed	**public / read only**	FocusedColumn	**protected / read / write**
BandColor	**protected / read / write**	FocusedNode	**public / read only**
BandFont	**protected / read / write**	FocusedNumber	**protected / read / write**
BandPanelHeight	**public / read only**	FooterPanelHeight	**public / read only**
BorderStyle	**protected / read / write**	FooterRowHeight	**public / read only**
Count	**public / read only**	FooterRowNodeHeight	**public / read only**
CustomizingBandListBox	**public / read only**	GridLineColor	**protected / read / write**
CustomizingForm	**public / read only**	GroupNodeColor	**protected / read / write**
CustomizingHeaderListBox	**public / read only**	GroupNodeTextColor	**protected / read / write**
CustomizingPos	**public / read / write**	HeaderButtonPushed	**public / read only**
CustomizingRowCount	**public / read / write**	HeaderColor	**protected / read / write**
DblClkExpanding	**protected / read / write**	HeaderFont	**protected / read / write**
DefaultRowHeight	**protected / read / write**	HeaderHeight	**public / read only**
DescTextHeight	**public / read only**	HeaderPanelHeight	**public / read only**
DownBandIndex	**public / read only**	HeaderRowHeight	**public / read only**
DownBandPushed	**public / read only**	HideFocusRect	**protected / read / write**
DownColumnIndex	**public / read only**	HideSelection	**protected / read / write**
DownColumnPushed	**public / read only**	HideSelectionColor	**protected / read / write**
DragAbsoluteBandIndex	**public / read only**	HideSelectionTextColor	**protected / read / write**
DragAbsoluteHeaderIndex	**public / read only**	HighlightColor	**protected / read / write**
DragNode	**protected read only**	HighlightTextColor	**protected / read / write**

Images	**protected / read / write**	SelectedNodes	**protected / read only**
Indent	**protected / read only**	SelectionAnchor	**protected / read / write**
IndentDesc	**protected / read / write**	ShowBands	**protected / read / write**
InplaceColumnIndex	**public / read only**	ShowButtons	**protected / read / write**
InplaceEditor	**public / read only**	ShowFooter	**protected / read / write**
IsCustomizing	**public / read only**	ShowGrid	**protected / read / write**
Items	**public / read only**	ShowHeader	**protected / read / write**
LastNode	**public / read only**	ShowHiddenInCustomizeBox	**protected / read / write**
LeftCoord	**protected / read / write**	ShowIndicator	**protected / read / write**
LockSelection	**public / read only**	ShowLines	**protected / read / write**
LockUpdate	**protected / read only**	ShowNewItemRow	**protected / read / write**
LookAndFeel	**protected / read / write**	ShowPreviewGrid	**protected / read / write**
MaxRowLineCount	**protected / read / write**	ShowRoot	**protected / read / write**
NewItemRowActive	**protected / read / write**	ShowRowFooter	**protected / read / write**
NewItemRowHeight	**public / read only**	SimpleCustomizeBox	**protected / read / write**
Options	**public / read / write**	State	**public / read only**
OptionsEx	**public / read / write**	StateImages	**protected / read / write**
PaintStyle	**protected read / write**	TabStop	**published / read / write**
PreviewFont	**protected read / write**	TopIndex	**protected / read / write**
PreviewLines	**protected / read / write**	TopNode	**public / read only**
RowCount	**protected / read only**	TopVisibleNode	**public / read only**
RowFooterColor	**protected / read / write**	TreeLineColor	**protected / read / write**
RowFooterTextColor	**protected / read / write**	TreeLineStyle	**protected / read / write**
RowHeight	**public / read only**	VisibleRowCount	**protected / read only**
RowSeparatorLineWidth	**protected / read / write**	WaitForExpandNodeTime	**protected / read / write**
ScrollBars	**protected / read / write**		
SelectedCount	**protected / read only**		

Derived from TCustomdxContainer

IsInitEdit	**public / read only**
ReadOnly	**public / read only**

Columns property — read / write

Contains an array of data-aware tree list columns.

```
property Columns[Index:Integer] : TdxDBTreeListColumn;
```

Description

Represents an array of tree list columns. Use this property to add/ delete columns in a data-aware tree list control, or to edit their display properties. When setting Columns at design time, the Columns Editor dialog appears. The index of a column in the array is 0. The number of columns in TCustomdxDBTreeListControl is determined by the ColumnCount property.

See Also

TCustomdxTreeListControl.ColumnCount; TCustomdxDBTreeListControl.VisibleColumns;

TCustomdxDBTreeListControl.ColumnByName; TCustomdxDBTreeListControl.ColumnByField

Copyright © 2000 Developer Express Inc.

DataLink property read only

Determines an object of type TDataLink used by a data-aware grid to access data.

```
property DataLink : TDataLink
```

Description

Specifies a datalink used to coordinate actions of a grid and dataset and to respond to data events.

See Also

TCustomdxDBTreeListControl.Datasource

DataSource property read only

Determines a dataset from where a data-aware tree list control retrieves its data.

```
property DataSource : TDataSource;
```

Description

Set DataSource to a valid TDataSource object in order to fetch data for a data-aware tree list. DataSource allows a data-aware tree list control to read from, write to, and navigate within a dataset, while maintaining record concurrency with other data-aware objects using the same data source.

See Also

TCustomdxDBTreeListControl.DataLink

FocusedField property read / write

Determines the field corresponding to a focused column.

```
property FocusedField : TField;
```

Description

Every column of the TCustomdxDBTreeListControl is associated with a dataset field. This property specifies the field corresponding to a focused column. Set this property to move focus to a column associated with the new focused field.

See Also

TCustomdxDBTreeListControl.Columns

GrIndicatorWidth property read / write

Specifies the width of the indicator area.

```
property GrIndicatorWidth: Integer;
```

Description

Indicator area is an region at the left edge of a grid control. It contains the selected row indicator. It is displayed when the egoIndicator option is active (the Options property). Use the GrIndicatorWidth property to change indicator area width.

KeyField property — read / write

Used to coordinate operations with a dataset. It specifies the key field of a dataset whose data is displayed within the grid.

```
property KeyField : string;
```

Description

The KeyField provides proper operations in LoadAllRecords mode. If the KeyField property is set incorrectly, grouping operations are automatically disabled. If you do not have a unique field and wish to use grouping, you must leave this field empty and either deactivate the egoCanNavigation option (see the Options property) or create a unique calculated field. However, in the latter, you cannot use the egoUseLocate and egoSmartRefresh options.

Note: *Set the KeyField propert to implement an OnChangeNodeEx event handler in standard grid mode.*

The KeyField property is set incorrectly in LoadAllRecords mode if:

- Edited values do not correspond to displayed values.
- Multi-selection works incorrectly.
- There is no syncronization between the component and the dataset. The focused node does not correspond to the active dataset record.

See Also

TCustomdxDBGrid.Options; TCustomdxDBGrid.FindNodeByKeyValue

PreviewFieldName property — read / write

Determines preview pane field name.

```
property PreviewFieldName : String;
```

Description

Use this property to specify a dataset field whose value is displayed in the preview section. Setting this property to an empty string hides the preview section.

See Also

TCustomdxDBTreeListControl.OnGetPreviewText; TCustomdxDBTreeListControl.PreviewFieldName

TCustomdxDBGrid.Preview; TCustomdxDBGrid.PreviewLines;

PreviewMaxLength property — read / write

Specifies the maximum width of the preview section in characters.

```
property PreviewMaxLength: Integer;
```

Description

Use this property to decrease text displayed within the preview section. If PreviewMaxLegth is 0, then the preview section contains the entire text of the preview field.

See Also

TCustomdxDBTreeListControl.OnGetPreviewText; TCustomdxDBTreeListControl.PreviewFieldName

TCustomdxDBGrid.Preview; TCustomdxDBGrid.PreviewLines;

SelectedRows property **read only**

Specifies a set of bookmarks for all records in a dataset that correspond to nodes selected in the grid.

```
property SelectedRows[Index:Integer] : TBookmarkStr;
```

Description

Use SelectedRows to refer to dataset records associated with selected nodes. SelectedRows is used only when the egoMultiSelect option is active. The total number of selected rows is specified by the SelectedCount property.

The SelectedRows property is presented as a list of bookmarks that can be used to locate records in a dataset and then obtain field values via the FieldByName or FieldValues method of TDataset.

Note: *If the ExpressQuantumGrid is in Partial Load or LoadAllRecords mode and the UseBookmarks property is False, the SelectedRows property* ***cannot be used to refer to dataset records****. In this instance, you should use the SelectedNodes property to obtain values of selected records.*

See Also

TCustomdxDBGrid.UseBookmarks; TCustomdxDBTreeListControl.RefreshBookmarks;

TCustomdxTreeList.SelectedCount;TCustomdxTreeList.SelectedNodes

SortedColumn property **read only**

Returns a sorted column.

```
property SortedColumn : TdxDBTreeListColumn;
```

Description

Specifies a column by which nodes are sorted within a tree list. If multi-sorting is active, SortedColumn returns the first sorted column. The column Sorted property determines sorting order.

See Also

TdxTreeListColumn.Sorted; TCustomdxDBTreeListControl.SortedColumns;

TCustomdxTreeListControl.SortedColumnCount

SortedColumns property — read only

Contains an array of sorted columns.

```
property SortedColumns[Index: Integer]: TdxDBTreeListColumn;
```

Description

Nodes in a data-aware tree list control can be sorted by several columns. The SortedColumns property contains a list of sorted columns. First, nodes are sorted by the first sorted column, then by the second, etc. The Sorted property of a sorted column determines the sort order.

Note: *Multi-sorting is enabled if the egoMultiSort option (the ExpressQuantumGrid OptionsEx property) is active.*

See Also

TCustomdxDBGrid.OptionsEx; TCustomdxDBTreeListControl.SortedColumn;

TCustomdxTreeListControl.SortedColumnCount; TdxTreeListColumn.Sorted

VisibleColumns property — read / write

Represents an array of visible columns.

```
property VisibleColumns[Index:Integer] : TdxDBTreeListColumn;
```

Description

Use the VisibleColumns array to access a column in a tree list by index. The index of the first column in the array is 0. The number of columns in the array is determined by the TCustomdxDBTreeListControl VisibleColumnCount property.

See Also

TCustomdxDBTreeListControl.VisibleColumnCount; TCustomdxDBTreeListControl.Columns;

TdxDBTreeListColumn.Visible)

Copyright © 2000 Developer Express Inc.

TCustomdxDBTreeListControl Methods

TCustomdxDBTreeListControl

ClearGroupColumns	**public**	GetColumnAt	**public**
ColumnByFieldName	**public**	GetDefaultColumnClass	**public**
ColumnByName	**public**	GetDefaultFieldColumnClass	**public**
CreateColumn	**public**	GetFooterColumnAt	**public**
CreateColumnEx	**public**	GetHeaderColumnAt	**public**
FindColumnByFieldName	**public**	RefreshBookmarks	**public**
FullRefresh	**public**		

Derived from TCustomdxTreeListControl

ApplyBestFit	**public**	GetFooterColumnAt	**public**
AssignColumns	**public**	GetHeaderColumnAt	**public**
BeginSorting	**public**	GetNodeFooterColumnAt	**public**
ColumnByName	**public**	GetVisibleColumnIndex	**public**
CreateColumn	**public**	LoadFromIniFile	**public**
CreateColumnEx	**public**	LoadFromRegistry	**public**
DestroyColumns	**public**	RefreshDefaultColumnsWidths	**public**
EndSorting	**public**	RefreshSorting	**public**
GetAbsoluteColumnIndex	**public**	RestoreColumnsDefaults	**public**
GetBandAt	**public**	SaveToIniFile	**public**
GetColumnAt	**public**	SaveToRegistry	**public**
GetDisplayValue	**public**		

Derived from TCustomdxTreeList

Add	**public**	EndUpdate	**public**
AddFirst	**public**	FindInplaceEdit	**protected**
AdjustColumnsWidth	**public**	FindNodeByText	**protected**
BeginSelection	**public**	FreeDrawInfo	**public**
BeginUpdate	**public**	FullCollapse	**public**
CalcDrawInfo	**public**	FullExpand	**public**
CancelEditor	**public**	GetEditRect	**public**
CellRect	**public**	GetFocusedAbsoluteIndex	**public**
ClearColumnsSorted	**public**	GetFocusedVisibleIndex	**public**
ClearNodes	**public**	GetHitInfo	**public**
ClearSelection	**public**	GetHitTestInfoAt	**public**
CloseEditor	**public**	GetNextVisible	**public**
ColumnsCustomizing	**public**	GetNodeAt	**public**
CopyAllToClipboard	**public**	GetScrollVertGridRect	**protected**
CopySelectedToClipboard	**public**	GotoFirst	**public**
Create	**public**	GotoLast	**public**
DeleteSelection	**public**	GotoNext	**public**
Destroy	**public**	GotoPrev	**public**
EndColumnsCustomizing	**public**	HideEditor	**public**
EndSearch	**public**	IndexOf	**public**
EndSelection	**public**	Insert	**public**

InvalidateSelection	**protected**	SetEditingText	**protected**
IsInvertSelect	**public**	SetEditText	**protected**
IsNodeVisible	**public**	ShowButtonEditorMouse	**public**
MakeBandVisible	**public**	ShowEditor	**public**
MakeColumnVisible	**public**	ShowEditorChar	**public**
MakeNodeVisible	**public**	ShowEditorMouse	**public**
PointInCustomizingForm	**public**	StartDragBand	**protected**
SaveAllToStrings	**public**	StartDragHeader	**protected**
SaveAllToTextFile	**public**	StartSearch	**public**
SaveSelectedToStrings	**public**	UpdateScrollBars	**protected**
SaveSelectedToTextFile	**public**		
SelectNodes	**protected**		

Derived from TCustomdxContainer

BeginInitEdit	**protected**	GetReadOnly	**protected**
CanEditAcceptKey	**protected**	HideDrawFocusRect	**protected**
CanEditModify	**protected**	InplaceEditKeyDown	**protected**
CheckEditorWidth	**protected**	IsEditorMoved	**protected**
CreateEditStyle	**protected**	IsImmediateEditor	**protected**
EndInitEdit	**protected**	IsShowButtonAlways	**protected**
GetDefaultPopupBorderStyle	**public**	IsTabs	**protected**
GetEditColor	**protected**	MoveCol	**protected**
GetEditFont	**protected**	RedrawSelection	**protected**

ClearGroupColumns method

Clears the list of grouped columns.

```
procedure ClearGroupColumns;  virtual;
```

Description

Removes all columns from the list of grouped columns.

ColumnByFieldName method

Returns a column associated with a specified dataset field.

```
function ColumnByFieldName(const FieldName: string): TdxDBTreeListColumn;
```

Description

Columns of a data-aware tree list are associated with particular dataset fields. Use this method to obtain a column associated with a particular field. FieldName specifies the name of this field. If no column is found, ColumnByFieldName raises an exception.

The following code groups nodes by a column, associated with the ‘OrderNo’ field:

```
TdxDBGridColumn(dxDBGrid1.ColumnByFieldName('OrderNo')).GroupIndex := 0;
```

See Also

TCustomdxDBTreeListControl.Datasource; TCustomdxDBTreeListControl.FindColumnByFieldName

TCustomdxDBTreeListControl.Columns; TCustomdxDBTreeListControl.ColumnByName;

ColumnByName method

Returns a column by its name.

```
function ColumnByName(const AName : String): TdxDBTreeListColumn;
```

Description

Use this method to access a column whose name equals AName.

See Also

TCustomdxDBTreeListControl.Columns; TCustomdxDBTreeListControl.VisibleColumns;

TCustomdxDBTreeListControl.ColumnByFieldName)

CreateColumn method

Creates a column of specified type.

```
type
  TdxDBTreeListColumnClass = class of TdxDBTreeListColumn;
function CreateColumn(ColumnClass : TdxDBTreeListColumnClass):
TdxDBTreeListColumn;
```

Description

Creates a new column of type ColumnClass and adds it to tree list columns (the tree list Columns property). The data-aware tree list control is specified as an owner of a new column.

See Also

TCustomdxDBTreeListControl.Columns; TCustomdxDBTreeListControl.CreateColumnEx;

TCustomdxTreeListControl.DestroyColumns

CreateColumnEx method

Creates a data aware tree list column.

```
type
  TdxDBTreeListColumnClass = class of TdxDBTreeListColumn;
function CreateColumnEx(ColumnClass: TdxDBTreeListColumnClass; AOwner:
TComponent): TdxDBTreeListColumn;
```

Description

This method is used to create a data aware column of type ColumnClass. AOwner specifies the owner of the new column. For all columns created at design time, AOwner is a form on which the data-aware tree list control is located. CreateColumnEx returns the newly created column. The name of a new column is specified in the following manner:

ColumnName = *Name_of_the_DataAware_TreeList_Control* + 'Column' + *column_index*

See Also

TCustomdxDBTreeListControl.CreateColumn; TCustomdxDBTreeListControl.Columns;

TCustomdxTreeListControl.ColumnCount; TCustomdxTreeListControl.DestroyColumns

FindColumnByFieldName method

Returns a column, associated with a specified field.

```
function FindColumnByFieldName(const FieldName: string):
TdxDBTreeListColumn;
```

Description

Columns of a data-aware tree list are associated with dataset fields. This method returns a column, which is associated with the field specified by FieldName. If no column is found, FindColumnByFieldName returns nil.

See Also

TCustomdxDBTreeListControl.ColumnByFieldName

FullRefresh method

Refreshes data within the ExpressQuantumDBTreeList.

```
procedure FullRefresh;
```

Description

Call this method when you wish to refresh the ExpressQuantumDBTreeList on screen.

See Also

TCustomdxDBGrid.LockRefresh; TCustomdxDBGrid.OnRefreshNodeData;

TCustomdxDBGrid.DoRefreshNodeData

GetColumnAt method

Returns a column by specified coordinates.

```
function GetColumnAt(X, Y: Integer): TdxDBTreeListColumn;
```

Copyright © 2000 Developer Express Inc.

Description

Returns a TdxDBTreeListColumn, which contains a point with X and Y coordinates. If such a column is not found, GetColumnAt returns nil.

See Also

TCustomdxDBGrid.GetGroupColumnAt; TCustomdxDBGrid.GetNodeFooterColumnAt;

TCustomdxDBTreeListControl.GetFooterColumnAt; TCustomdxTreeList.GetHitInfo

TCustomdxDBGrid.PointInFooterPanel; TCustomdxDBGrid.PointInGroupPanel;

GetDefaultColumnClass method

Returns the default column class for the ExpressQuantumGrid.

```
type
  TdxDBTreeListColumnClass = class of TdxDBTreeListColumn;
class function GetDefaultColumnClass: TdxDBTreeListColumnClass; virtual;
```

Description

By default, this method returns a TdxDBTreeListColumn class. Descendants can override this method to use any other class as the default for columns.

See Also

TCustomdxDBTreeListControl.GetDefaultFieldColumnClass

TCustomdxDBTreeListControl.CreateDefaultColumns;

TCustomdxDBTreeListControl.CreateDefaultColumn;

GetDefaultFieldColumnClass method

```
type
  TdxDBTreeListColumnClass = class of TdxDBTreeListColumn;
class function GetDefaultFieldColumnClass(AField: TField):
TdxDBTreeListColumnClass; virtual;
```

Description

GetDefaultFieldColumnClass returns the default column class for a dataset field of a specific type: i.e. the TdxDBTreeListDateColumn dass for Date fields; the TdxDBTreeListCurrencyColumn class for Currency fields, etc. For unknown field types, GetDefaultFieldColumnClass returns the TdxDBTreeListColumn class.

See Also

TCustomdxDBTreeListControl.GetDefaultColumnClass

TCustomdxDBTreeListControl.CreateDefaultColumns;

TCustomdxDBTreeListControl.CreateDefaultColumn;

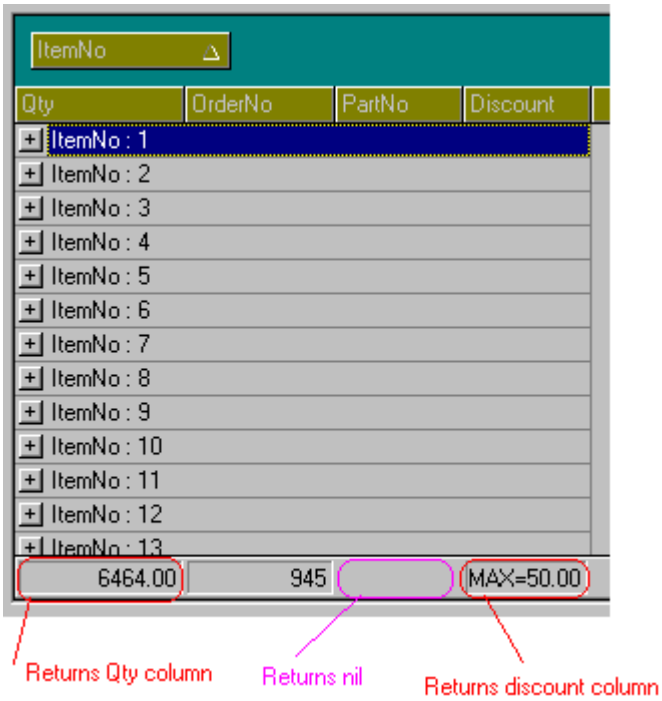
GetFooterColumnAt method

Returns columns by coordinates of the ExpressQuantumGrid footer.

```
function GetFooterColumnAt(X, Y: Integer): TdxDBTreeListColumn;
```

Description

Returns a column if a point with X and Y coordinates is in the footer rectangle. If no column exists, returns nil.



See Also

TCustomdxDBTreeListControl.GetColumnAt; TCustomdxDBTreeListControl.GetHeaderColumnAt; TCustomdxDBGrid.GetGroupColumnAt; TCustomdxDBGrid.GetNodeFooterColumnAt; TCustomdxDBGrid_PointInFooterPanel; TCustomdxDBGrid.PointInGroupPanel; TCustomdxTreeList.GetHitInfo

GetHeaderColumnAt method

Returns a column by specified coordinates.

```
function GetHeaderColumnAt(X, Y: Integer): TdxDBTreeListColumn;
```

Description

This method returns a column if the rectangle of its header contains a point, specified by X and Y parameters. If no column header contains such a point, GetHeaderColumnAt returns nil.

See Also

TCustomdxDBTreeListControl.GetColumnAt; TCustomdxDBTreeListControl.GetFooterColumnAt;

TCustomdxDBGrid.GetGroupColumnAt; TCustomdxDBGrid.GetNodeFooterColumnAt; TCustomdxDBGrid.PointInFooterPanel; TCustomdxDBGrid.PointInGroupPanel; TCustomdxTreeList_GetHitInfo

RefreshBookmarks method

Checks whether all bookmarks, specified in the SelectedRows property, are valid.

```
function RefreshBookmarks: Boolean;
```

Description

Bookmarks, corresponding to selected nodes are not automatically deleted when records of the dataset are deleted. Therefore, some bookmarks in the SelectedRows array may be invalid. Call RefreshBookmarks to ensure that every bookmark in the bookmark list is valid. RefreshBookmarks attempts to locate every record of the dataset that corresponds to a bookmark in the SelectedRows property array. All bookmarks that do not correspond to records of the dataset are removed. If any of the bookmarks, specified by SelectedRows, is invalid, RefreshBookmarks returns True and invalidates the grid control so that it repaints, reflecting removal of all invalid records. If all bookmarks in the list are still valid, RefreshBookmarks returns False.

See Also

TCustomdxDBTreeListControl.SelectedRows

TCustomdxDBTreeListControl Events

TCustomdxDBTreeListControl

OnBeforeCalcSummary	**protected**	OnGetEditColor	**public**
OnCalcSummary	**protected**	OnGetPreviewText	**protected**
OnChangeNodeEx	**protected**	OnRefreshNodeData	**protected**

Derived from TCustomdxTreeListControl

OnBandButtonClick	**protected**	OnDragEndHeader	**protected**
OnBandClick	**protected**	OnDragOverBand	**protected**
OnCalcRowLineHeight	**protected**	OnDragOverHeader	**protected**
OnCanBandDragging	**protected**	OnEndDragColumn	**protected**
OnCanHeaderDragging	**protected**	OnGetImageIndex	**protected**
OnChangedColumnsWidth	**protected**	OnGetLevelColor	**protected**
OnColumnClick	**protected**	OnGetNodeDragText	**protected**
OnColumnMoved	**protected**	OnGetSelectedIndex	**protected**
OnColumnSorted	**protected**	OnGetStateIndex	**protected**
OnColumnSorting	**protected**	OnHeaderButtonClick	**protected**
OnCustomDraw	**protected**	OnHeaderMoved	**protected**
OnCustomDrawBand	**protected**	OnHideBand	**protected**
OnCustomDrawCell	**protected**	OnHideHeader	**protected**
OnCustomDrawColumnHeader	**protected**	OnIsExistFooterCell	**protected**
OnCustomDrawFooter	**protected**	OnShowBand	**protected**
OnCustomDrawFooterNode	**protected**	OnShowHeader	**protected**
OnCustomDrawPreview	**protected**	OnStartBandDragging	**protected**
OnCustomDrawPreviewCell	**protected**	OnStartHeaderDragging	**protected**
OnDragEndBand	**protected**		

Derived from TCustomdxTreeList

OnBeginDragNode	**protected**	OnEditChange	**protected**
OnCanNodeSelected	**protected**	OnEdited	**protected**
OnChangeColumn	**protected**	OnEditing	**protected**
OnChangeLeftCoord	**protected**	OnEditValidate	**protected**
OnChangeNode	**protected**	OnEndColumnsCustomizing	**protected**
OnChangeTopVisibleNode	**protected**	OnExpanded	**protected**
OnCollapsed	**protected**	OnExpanding	**protected**
OnCollapsing	**protected**	OnGetEditColor	**protected**
OnCompare	**protected**	OnHotTrackNode	**protected**
OnDeletion	**protected**	OnSelectedCountChange	**protected**

Copyright © 2000 Developer Express Inc.

OnBeforeCalcSummary event

Occurs before calculating a summary value.

```
type
  TNotifyEvent = Procedure(Sender: TObject) of object;
property OnBeforeCalcSummary : TNotifyEvent;
```

Description

This is a notifying event. Write an OnBeforeCalcSummary event handler to perform specific actions before calculating a summary value. If the calculation of a summary consumes excessive volumes of time, the OnBeforeCalcSummary event handler displays a message that states that a summary is being calculated. When calculation is complete, close the window via the OnCalcSummary event handler.

See Also

TCustomdxDBTreeListControl.DoBeforeCalcSummary;

TCustomdxDBTreeListControl.OnCalcSummary

OnCalcSummary event

Occurs when the ExpressQuantumGrid completes summary calculations.

```
type
  TNotifyEvent = procedure(Sender: TObject) of object;
property OnCalcSummary : TNotifyEvent;
```

Description

This is a notifying event and is generated when the ExpressQuantumGrid calculates node summaries. Use it to perform specific actions upon calculation of a summary. For example, the OnBeforeCalcSummary event handler can display a window notifying a user that a summary is being calculated. In this instance, the OnCalcSummary event handler deactivates this window when a summary is calculated.

See Also

TCustomdxDBGrid.OnGetCustomSummaryText; TCustomdxDBGrid.SummarySeparator;

TCustomdxDBTreeListControl.OnBeforeCalcSummary

OnChangeNodeEx event

Occurs when focus moves across nodes.

```
type
  TNotifyEvent = procedure(Sender : TObject) of object;
property OnChangeNodeEx: TNotifyEvent;
```

Description

This event occurs when focus moves from one node to another. It happens when a user moves along dataset records via the ExpressQuantumGrid or ExpressQuantumDBTreeList or any other data-aware control or programmatically (using dataset methods).

The OnChangeNodeEx event is generated when changing the selection state of a focused node. For instance, when the focused node is not selected and a user clicks it with the Ctrl key pressed, the node is selected and the OnChangeNodeEx is generated.

Use the OnChangeNodeEx event handler to process changes for a focused node.

Note: Set a proper value for the KeyField property to use the OnChangeNodeEx event when the ExpressQuantumGrid is in standard grid mode.

OnChangeNodeEx resembles the OnChangeNode event, which is also called when replacing the current node. However, there are substantial differences between them. They are as follows:

OnChangeNodeEx works differently than OnChangeNode when the egoMultiSelect option is active. For example, if this option is active and a user selects three nodes within the ExpressQuantumGrid, the OnChangeNode event is generated three times – once for each selected node. The OnChangeNodeEx event is generated only once for the three selected nodes.

OnChangeNodeEx is generated when a user moves along dataset records using any other data-aware control or methods of the dataset.

OnChangeNodeEx is generated when changing selection state of a focused node.

Note: *OnChangeNodeEx is generated after the OnChangeNode event.*

See Also

TCustomdxDBTreeListControl.DoChangeNodeEx; TCustomdxTreeList_OnChangeNode

OnGetEditColor event

Occurs when activating an inplace editor in a particular cell of a data-aware tree list.

```
type
  TdxGetEditColor = procedure(Sender : TObject; ANode: TdxTreeListNode;
AColumn: TdxDBTreeListColumn; var AColor : TColor) of object;
property OnGetEditColor : TdxGetEditColor;
```

Description

Write an OnGetEditColor event handler to set the specific background color for an inplace editor in a tree list cell. ANode and AColumn determine the cell in a data-aware tree list. AColor determines the background color. For instance, the following code sets a green background for an inplace editor:

```
procedure TfmMain.dxDBGridGetEditColor(Sender: TObject; ANode:
TdxTreeListNode; AColumn: TdxDBTreeListColumn; var AColor: TColor);
begin
  AColor := clGreen;
end;
```

See Also

TCustomdxTreeListControl.OnEditing; TCustomdxTreeListControl.OnEdited;

TCustomdxTreeListControl.EditingText

OnGetPreviewText event

Occurs when drawing preview section text.

```
type
  TdxGetPreviewTextEvent = procedure(Sender : TObject; Node:
TdxTreeListNode; var Text: String) of object;
property OnGetPreviewText : TdxGetPreviewTextEvent;
```

Description

Write an OnGetPreviewText event handler to customize the text of the preview section. The Node parameter determines the node for which the preview section is drawn. Use the Text parameter to specify preview section text.

The following code demonstrates how to display text across two lines within the preview section:

```
// Delphi
procedure TForm1.dxDBTreeList1GetPreviewText(Sender: TObject; Node:
TdxTreeListNode; var Text: String);
begin
  Text := 'Address : ' + Node.Values[dxDBTreeList1Addr1.Index] +
    #10 + 'City : ' + Node.Values[dxDBTreeList1City.Index];
end;
```

```
// C++Builder
void __fastcall TForm1:: dxDBTreeList1GetPreviewText(TObject *Sender,
TdxTreeListNode *Node, AnsiString &Text)
{
  Text = "Address : " + Node->Values[dxDBTreeList1Addr1->Index] +
    "\n" + "City : " + Node->Values[dxDBTreeList1City->Index];
}
```

See Also

TCustomdxDBTreeListControl.PreviewFieldName; TCustomdxDBTreeListControl.PreviewMaxLength

TCustomdxDBGrid.Options; TCustomdxDBGrid.PreviewLines;

OnRefreshNodeData event

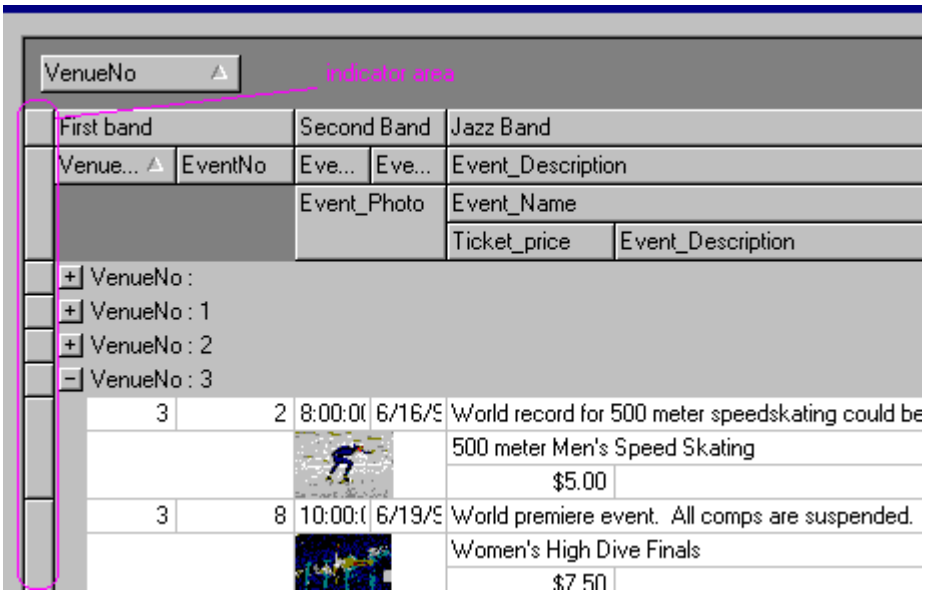
Occurs when a grid loads a dataset record into a node.

```
type
  TTLExpandedEvent = procedure(Sender: TObject; Node: TdxTreeListNode) of
object;
```

```
property OnRefreshNodeData : TTLExpandedEvent;
```

Description

Generated when the ExpressQuantumGrid retrieves data from a dataset record and loads it into a node. Write an OnRefreshNodeData event handler to associate a node with other data via the Data property of this node.



See Also

TCustomdxDBGrid.FullRefresh; TCustomdxDBTreeListControl.OnRefreshNodeData

TCustomdxDBTreeList

TCustomdxDBTreeList is the base class for TdxDBTreeList.

Unit

dxDBTL

```
TCustomdxDBTreeList = class(TCustomdxDBTreeListControl);
```

Description

TCustomdxDBTreeList is directly inherited from TCustomdxDBTreeListControl. It contains methods and properties that provide functionality for TdxDBTreeList (ExpressDBTreeList).

Hierarchy

TObject
- TPersistent
  - TComponent
    - TControl
      - TWinControl
        - TCustomControl
          - TCustomdxContainer
            - TCustomdxTreeList
              - TCustomdxTreeListControl
                - TCustomdxDBTreeListControl

TCustomdxDBTreeList Properties

TCustomdxDBTreeList

DataSetChangedBusy	**public / read only**	OptionsDB	**protected / read / write**
ImageIndexFieldName	**protected / read / write**	OptionsView	**protected / read / write**
LockLayout	**protected / read only**	ParentField	**published / read / write**
LockRefresh	**public / read / write**	RootValue	**protected / read / write**
OptionsBehavior	**protected / read / write**	SelectedIndex	**protected / read / write**
OptionsCustomize	**protected / read / write**	StateIndexFieldName	**protected / read / write**

Derived from TCustomdxDBTreeListControl

Columns	**public / read / write**	PreviewFieldName	**protected / read / write**
DataLink	**public / read only**	PreviewMaxLength	**protected / read / write**
DataSource	**public / read only**	SelectedRows	**protected / read only**
FocusedField	**protected / read / write**	SortedColumn	**public / read only**
GrIndicatorWidth	**protected / read / write**	SortedColumns	**public / read only**
KeyField	**published / read / write**	VisibleColumns	**public / read / write**

Derived from TCustomdxTreeListControl

BandMaxRowCount	**published / read / write**	HeaderPanelRowCount	**published / read / write**
BandRowCount	**published / read / write**	IniFileName	**protected / read / write**
Bands	**published / read / write**	IniSectionName	**protected / read / write**
ColumnCount	**public / read only**	IsCustomStoring	**protected / read / write**
DefaultLayout	**published / read / write**	RegistryPath	**protected / read / write**
HeaderMinRowCount	**published / read / write**	SortedColumnCount	**public / read only**
HeaderPanelMaxRowCount	**published / read / write**	VisibleColumnCount	**public / read only**

Derived from TCustomdxTreeList

ArrowsColor	**protected / read / write**	DefaultRowHeight	**protected / read / write**
AutoExpandOnSearch	**protected / read / write**	DescTextHeight	**public / read only**
AutoSearchColor	**protected / read / write**	DownBandIndex	**public / read only**
AutoSearchTextColor	**protected / read / write**	DownBandPushed	**public / read only**
BandButtonPushed	**public / read only**	DownColumnIndex	**public / read only**
BandColor	**protected / read / write**	DownColumnPushed	**public / read only**
BandFont	**protected / read / write**	DragAbsoluteBandIndex	**public / read only**
BandPanelHeight	**public / read only**	DragAbsoluteHeaderIndex	**public / read only**
BorderStyle	**protected / read / write**	DragNode	**protected read only**
Count	**public / read only**	EditingText	**protected / read /write**
CustomizingBandListBox	**public / read only**	FixedBandLineColor	**protected / read /write**
CustomizingForm	**public / read only**	FixedBandLineWidth	**protected / read /write**
CustomizingHeaderListBox	**public / read only**	FocusedAbsoluteIndex	**public / read only**
CustomizingPos	**public / read / write**	FocusedColumn	**protected / read / write**
CustomizingRowCount	**public / read / write**	FocusedNode	**public / read only**
DblClkExpanding	**protected / read / write**	FocusedNumber	**protected / read / write**

Copyright © 2000 Developer Express Inc.

FooterPanelHeight	**public / read only**	PaintStyle	**protected read / write**
FooterRowHeight	**public / read only**	PreviewFont	**protected read / write**
FooterRowNodeHeight	**public / read only**	PreviewLines	**protected / read / write**
GridLineColor	**protected / read / write**	RowCount	**protected / read only**
GroupNodeColor	**protected / read / write**	RowFooterColor	**protected / read / write**
GroupNodeTextColor	**protected / read / write**	RowFooterTextColor	**protected / read / write**
HeaderButtonPushed	**public / read only**	RowHeight	**public / read only**
HeaderColor	**protected / read / write**	RowSeparatorLineWidth	**protected / read / write**
HeaderFont	**protected / read / write**	ScrollBars	**protected / read / write**
HeaderHeight	**public / read only**	SelectedCount	**protected / read only**
HeaderPanelHeight	**public / read only**	SelectedNodes	**protected / read only**
HeaderRowHeight	**public / read only**	SelectionAnchor	**protected / read / write**
HideFocusRect	**protected / read / write**	ShowBands	**protected / read / write**
HideSelection	**protected / read / write**	ShowButtons	**protected / read / write**
HideSelectionColor	**protected / read / write**	ShowFooter	**protected / read / write**
HideSelectionTextColor	**protected / read / write**	ShowGrid	**protected / read / write**
HighlightColor	**protected / read / write**	ShowHeader	**protected / read / write**
HighlightTextColor	**protected / read / write**	ShowHiddenInCustomizeBox	**protected / read / write**
Images	**protected / read / write**	ShowIndicator	**protected / read / write**
Indent	**protected / read only**	ShowLines	**protected / read / write**
IndentDesc	**protected / read / write**	ShowNewItemRow	**protected / read / write**
InplaceColumnIndex	**public / read only**	ShowPreviewGrid	**protected / read / write**
InplaceEditor	**public / read only**	ShowRoot	**protected / read / write**
IsCustomizing	**public / read only**	ShowRowFooter	**protected / read / write**
Items	**public / read only**	SimpleCustomizeBox	**protected / read / write**
LastNode	**public / read only**	State	**public / read only**
LeftCoord	**protected / read / write**	StateImages	**protected / read / write**
LockSelection	**public / read only**	TabStop	**published / read / write**
LockUpdate	**protected / read only**	TopIndex	**protected / read / write**
LookAndFeel	**protected / read / write**	TopNode	**public / read only**
MaxRowLineCount	**protected / read / write**	TopVisibleNode	**public / read only**
NewItemRowActive	**protected / read / write**	TreeLineColor	**protected / read / write**
NewItemRowHeight	**public / read only**	TreeLineStyle	**protected / read / write**
Options	**public / read / write**	VisibleRowCount	**protected / read only**
OptionsEx	**public / read / write**	WaitForExpandNodeTime	**protected / read / write**

Derived from TCustomdxContainer

IsInitEdit	**public / read only**
ReadOnly	**public / read only**

DataSetChangedBusy property — read only

Indicates whether a data-aware tree list operates with a dataset.

```
property DataSetChangedBusy : Boolean;
```

Description

Use this value to obtain information on whether the component works with a dataset. If the DataSetChagedBusy property is True, the grid is busy with data-aware operations.

ImageIndexFieldName property — read / write

Determines the name of a field from which to acquire indexes for images.

```
property ImageIndexFieldName: string;
```

Description

Specifies the name of a field that contains indexes for the TdxTreeListNode's ImageIndex property that determines what images to display when a node is in its normal state and is currently selected.

See Also

TCustomdxDBTreeList.StateIndexFieldName; TdxTreeListNode.ImageIndex

LockLayout property — read only

Determines whether reloading of records is enabled.

```
property LockUpdate : Integer;
```

Description

Specifies how many times BeginLayout was called. If LockLayoutis is greater than 0, records are not loaded. Every call of the BeginLayout method increases the LockLayout property by one. Every call of the EndLayout method decreases the LockLayout property by one.

See Also

TCustomdxDBTreeList.BeginLayout; TCustomdxDBTreeList.EndLayout

LockRefresh property — read / write

Prevents multiple refresh events of a grid control when navigating through a dataset.

```
property LockRefresh : Boolean;
```

Description

Set this property to True to prevent multiple refresh events of a grid control when a user navigates through dataset records. This navigation calls the TDataSet DataChaged method.

See Also

TCustomdxDBTreeList.RefreshTreeList

OptionsBehavior property — read / write

Specifies behavioral properties of a data-aware tree list.

Copyright © 2000 Developer Express Inc.

```
type
 TdxDBTreeListOptionBehavior = (etoAnsiSort, etoAutoDragDrop,
etoAutoDragDropCopy, etoAutoSearch, etoAutoSort, etoBandButtonClicking,
etoCaseInsensitive, etoDblClick, etoDragCollapse, etoDragExpand,
etoDragScroll, etoEditing, etoEnterShowEditor, etoEnterThrough,
etoHeaderButtonClicking, etoHorzThrough, etoImmediateEditor,
etoMouseScroll, etoMultiSelect, etoShowHourGlass, etoStoreToIniFile,
etoStoreToRegistry, etoTabs, etoTabThrough, etoVertThrough);


TdxDBTreeListOptionsBehavior = set of TdxDBTreeListOptionBehavior;


property OptionsBehavior: TdxDBTreeListOptionsBehavior;
```

Description

The available values include:

Setting	Value
etoAnsiSort	When active, the tree list control uses Windows locale settings to sort rows. This option is useful when the tree list's cells contain native language text.
etoAutoDragDrop	When active, records are physically moved within a dataset when nodes are drag & dropped.
etoAutoDragDropCopy	When active, nodes are copied instead of moving during drag & drop.
etoAutoSearch	Activates/deactivates the auto-search feature: The ExpressDBTreeList automatically locates a record that matches the text a user enters within a focused column. To repeat searching press the CTRL-ENTER (forward) or SHIFT-CTRL-ENTER (backward) keys.
etoAutoSort	When active, multi sort is enabled. To sort a tree list by several columns, click these columns while pressing the Shift key. This option is effective when the ExpressDBTreeList is in LoadAllRecords mode.
etoBandButtonClicking	If set, the ExpressDBTreeList automatically sorts nodes. Click the header of the appropriate column to sort nodes by this column. If etoAutoSort is inactive, the ExpressDBTreeList does not perform any sort operations. The etoAutoSort option is effective when the ExpressDBTreeList is in LoadAllRecords mode.
etoBandButtonClicking	When active, the OnBandButtonClick event is generated when a user clicks the band area indicator. For more detailed information see OnBandButtonClick.
etoCaseInsensitive	Used to sort tree list nodes. If set, sorting is case insensitive.
etoDblClick	Determines whether the OnDblClick event fires when a user double-clicks somewhere within the tree list. If inactive, the OnDblClick event is generated only if a data node has been double-clicked.
etoDragCollapse	When active, an expanded node collapses when a user drags a node to it.
etoDragExpand	When active, a collapsed node expands when a user drags a node to it.
etoDragScroll	If set, focus moves towards the dragged node during drag & drop

	operations.
etoEditing	When active, end users can modify data within tree list cells. This option has no impact when the etoRowSelect option is active.
etoEnterShowEditor	When active (the default state), an inplace editor is displayed when a user presses the ENTER key or starts typing text. Otherwise, an inplace editor is activated when a user clicks the selected cell or presses F2 on the keyboard.
etoEnterThrough	Pressing the ENTER key moves focus to the next column. This option is only effective when etoEnterShowEditor is active.
etoHeaderButtonClicking	When active, the OnHeaderButtonClick event is generated when a user clicks the header area indicator. See OnHeaderButtonClick for more detailed information.
etoHorzThrough	When active, pressing Right/Left arrows within the last/first column of a node moves focus to the next/previous node. Otherwise, the last/first column remains focused.
etoImmediateEditor	Determines whether an inplace editor is displayed immediately during editing operations within a grid. If inactive, a user has to press ENTER or click a cell to edit data.
etoMouseScroll	When active, mouse movement scrolls dropdown lists in edited cells.
etoMultiSelect	Determines whether more than one row in a tree list can be selected at a time. If etoMultiSelect is not set, only nodes with the same parent group can be included in multi-select operations. This option has no impact when the etoCanNavigation option is inactive.
etoShowHourGlass	Determines whether the ExpressDBTreeList changes the cursor to crHourglass when performing long data processing events.
etoStoreToIniFile	Determines whether to store ExpressDBTreeList column settings in an INI file, specified by the IniFileName property.
etoStoreToRegistry	When active, ExpressDBTreeList columns store settings and columns layouts in a Registry key specified by the RegistryPath property.
etoTabs	Enables navigation within columns using Tab and Shift+Tab keys. If False, the TAB key press events move focus away from the ExpressDBTreeList to the next control of a given form rather than moving focus from one cell to another.
etoTabThrough	Pressing the Tab key within the last column of a row (node) moves focus to the first column of the next row (node). This option is effective only if the etoTabs is active.
etoVertThrough	Effective when nodes are drawn across several rows (multiple lines). If active, pressing the Up/Down arrow moves focus through the lines of the same node. Otherwise, it moves focus to the previous/next node.

The default values for the OptionBehavior property include:

[etoAutoDragDrop, etoAutoDragDropCopy, etoAutoSort, etoDragExpand, etoDragScroll, etoEditing, etoEnterShowEditor, etoImmediateEditor, etoTabThrough]

See Also

TCustomdxDBTreeList.OptionsCustomize; TCustomdxDBTreeList.OptionsDB;

TCustomdxDBTreeList.OptionsView

OptionsCustomize property read / write

Specifies customization properties of a data-aware tree list.

```
type
 TdxDBTreeListOptionCustomize = (etoBandMoving, etoBandPanelSizing,
etoBandSizing, etoColumnMoving, etoColumnSizing, etoExtCustomizing,
etoFullSizing, etoHeaderPanelSizing, etoKeepColumnWidth, etoRowSizing);

TdxDBTreeListOptionsCustomize = set of TdxDBTreeListOptionCustomize;

property OptionsCustomize: TdxDBTreeListOptionsCustomize;
```

Description

The available values include:

Setting	Value
etoBandMoving	When active, users can move bands. Else, band movement is disabled.
etoBandPanelSizing	When active, a user can change the height of the band area.
etoBandSizing	Activate it to allow users to change the size of a band.
etoColumnMoving	Allows a user to move a column within a tree list.
etoColumnSizing	Allows a user to change column width.
etoExtCustomizing	Determines column customization style and behavior. By default, etoExtCustomizing is inactive. When a user moves (drags) a given column, this column is displayed either within the tree list control or in the column customization window. When active, etoExtCustomizing enables users to remove active grid columns from the tree list display by simply removing it from the tree list header region. This feature allows users to remove columns without activating the column customization window.
etoFullSizing	When active, the size of the last column or band changes even if the end user drags the cursor out of the tree list window. Otherwise, the end user can resize the last column within the tree list window.
etoHeaderPanelSizing	If set, users can change header area height.
etoKeepColumnWidth	When active, the ExpressDBTreeList does not change the values of column width back to initial settings after deactivating the etoAutoWidth option.
etoRowSizing	When active, users can change row height.

The default values for the OptionCustomize property include:

[etoBandMoving, etoBandSizing, etoColumnMoving, etoColumnSizing, etoKeepColumnWidth]

See Also

TCustomdxDBTreeList.OptionsBehavior; TCustomdxDBTreeList.OptionsDB;

TCustomdxDBTreeList.OptionsView

OptionsDB property read / write

Specifies properties of a data-aware tree list to work with a dataset.

```
type
 TdxDBTreeListOptionDB = (etoAutoCalcKeyValue, etoCancelOnExit,
etoCanDelete, etoCanInsert, etoCanNavigation, etoCheckHasChildren,
etoConfirmDelete, etoLoadAllRecords, etoResetColumnFocus, etoSyncSelection,
etoUseBookmarks);

TdxDBTreeListOptionsDB = set of TdxDBTreeListOptionDB;

property OptionsDB: TdxDBTreeListOptionsDB;
```

Description

The available values include:

Setting	Value
etoAutoCalcKeyValue	When active and the tree list is in LoadAll Records mode, then the value of the key field of a newly created record is calculated automatically.
etoCancelOnExit	When a user exits the tree list after inserting a new record, to which he makes no modifications, the inserted record is not posted to the dataset and thus helps prevent accidental posting of empty records.
etoCanDelete	Determines whether a user can delete a record by pressing Ctrl+Delete keystrokes.
etoCanInsert	Determines whether a user can insert a record by pressing the Insert key or via the new item row.
etoCanNavigation	When in LoadAllRecords mode and the etoCanNavigation option is False, the ExpressDBTreeList does not go to the record associated with the selected node and data modification is disabled. In all other cases, when changing a selected node, the ExpressDBTreeList activates the linked record.
etoCheckHasChildren	When active and the tree list is not in LoadAllRecords mode, all nodes are checked whether they have children; when inactive all nodes are displayed with a '+' to the left.
etoConfirmDelete	Determines whether a message box that confirms a user's request to delete a row within the tree list after pressing the Ctrl+Delete keys appears.
etoLoadAllRecords	Determines whether the ExpressDBTreeList loads all records into memory. In order to perform sorting on the fly, the ExpressDBTreeList requires loading of all records into memory. This option requires the KeyField property to be set correctly.
etoResetColumnFocus	When new records are added, focus moves to the first column of the new node.
etoSyncSelection	If set, the ExpressDBTreeList moves a selection according to the active record of a dataset; i.e. the ExpressDBTreeList moves the selection, when a user scrolls the dataset via external data-aware controls or programmatically. When this option is inactive, the ExpressDBTreeList moves focus only when the active record is changed. The etoSyncSelection option affects multi-selection when the dataset associated with the ExpressDBTreeList is scrolled. Assume,

	etoSyncSelection is active. If a user selects several nodes and then scrolls the dataset, the ExpressDBTreeList cancels selection and selects the node corresponding to the dataset's active record. If an application scrolls a dataset via the Locate method, the selected node does not change.
etoUseBookmarks	When active, bookmarks are used to navigate through a dataset.

The default values for the OptionDB property include:

[etoAutoCalcKeyValue, etoCancelOnExit, etoCanDelete, etoCanNavigation, etoCheckHasChildren, etoConfirmDelete, etoLoadAllRecords]

See Also

TCustomdxDBTreeList.OptionsBehavior; TCustomdxDBTreeList.OptionsCustomize;

TCustomdxDBTreeList.OptionsView

OptionsView property — read / write

Specifies display properties of a data-aware tree list.

```
type
 TdxDBTreeListOptionView = (etoAutoCalcPreviewLines,
etoAutoHeaderPanelHeight, etoAutoWidth, etoBandHeaderWidth,
etoDrawEndEllipsis, etoHideFocusRect, etoIndicator, etoInvertSelect,
etoPreview, etoRowAutoHeight, etoRowSelect, etoShowButtonAlways,
etoUseBitmap, etoUseImageIndexForSelected);

TdxDBTreeListOptionsView = set of TdxDBTreeListOptionView;

property OptionsView: TdxDBTreeListOptionsView;
```

Description

The following values are available:

Setting	Value
etoAutoCalcPreviewLines	When active, the ExpressDBTreeList automatically calculates the number of lines in the preview section. Otherwise, the number of lines occupied by the preview section is specified by the PreviewLines property.
etoAutoHeaderPanelHeight	ExpressDBTreeList column headers may be arranged one under another. If this option is active, the ExpressDBTreeList sets the height of the header area equal to the number of header rows. This option is automatically disabled after a user changes the HeaderPanelRowChange property or manually resizes the header area at runtime.
etoAutoWidth	When set, the ExpressDBTreeList changes all column widths in order to display all columns on screen thus eliminating the need for a horizontal grid scroll bar.
etoBandHeaderWidth	When active, the width of a band is equal to width of columns, which belong to this band. This option is only effective when a

	band is not fixed or until a user re-sizes it.
etoDrawEndEllipsis	The width of data in a column maybe greater than a column's width. When set, ellipsis ('…') are drawn to indicate to users that data does not fit inside the boundaries of a column.
etoHideFocusRect	Specifies whether to draw a rectangle around a focused column. When set, the bound rectangle around a focused column is hidden.
etoIndicator	Determines whether to display the data-aware tree list's indicator (a line to the left of the tree list with the current node's indicators).
etoInvertSelect	When active, the ExpressDBTreeList highlights only focused columns (MS Outlook™ selection style); otherwise it highlights all columns of a focused node except for the currently selected column.
etoPreview	Specifies whether to display the ExpressDBTreeList preview section.
etoRowAutoHeight	When active, the ExpressDBTreeList automatically calculates the number of rows in a cell. It is not recommended to activate this option if the ExpressDBTreeList includes a huge number of columns (~100 or more).
etoRowSelect	Determines whether a user can select the entire row versus individual cells within a row. Once a user selects a row within the tree list, the entire row is highlighted. If False, only the selected cell is highlighted. If active, etoEditing has no effect.
etoShowButtonAlways	When active, the edit button is always displayed within a column with the dropdown window. This button appears only for columns of a focused node. If etoShowButtonAlways is inactive, the edit button is displayed only within a focused cell.
etoUseBitmap	When active, a temporary bitmap is used to draw cells within a tree list. It decreases flickers, but draws a cell a bit slowly.
etoUseImageIndexForSelected	When active, the ImageIndex property's value is used instead of the SelectedIndex property of TdxTreeListNode.

The default values for the OptionView property include:

[etoBandHeaderWidth, etoRowSelect, etoUseBitmap, etoUseImageIndexForSelected]

See Also

TCustomdxDBTreeList.OptionsBehavior; TCustomdxDBTreeList.OptionsCustomize;

TCustomdxDBTreeList.OptionsDB

ParentField property — read / write

Determines the name of a parent field necessary for the creation of a tree structure.

```
property ParentField: string;
```

Description

To present a DBTreeList control as a tree, you should specify the field's name for this property. Values of this field help to build the tree-hierarchy to determine parent nodes.

See Also

TCustomdxDBTreeListControl.KeyField

RootValue property — read / write

Determines the value that identifies root nodes.

```
property RootValue: string;
```

Description

The RootValue property is used internaly to mark nodes at the topmost level.

SelectedIndex property — read / write

Specifies the index of the image, displayed for a selected node in the TdxTreeList image list.

```
property SelectedIndex : Integer;
```

Description

Use the SelectedIndex property to specify what image to display when the TdxTreeListNode is selected. If SelectedIndex is -1, then the image is not drawn for the selected node, except for the etoUseImageIndexForSelected option is active; in this instance, the standard image is drawn.

See Also

TCustomdxDBTreeList.ImageIndexFieldName; TCustomdxDBTreeList.StateIndexFieldName

StateIndexFieldName property — read / write

Determines the name of the field from which to acquire indexes for state images.

```
property ImageIndexFieldName: string;
```

Description

Specifies the name of a field that contains indexes for the StateIndex property of the TdxTreeListNode to determine images that reflect state information.

See Also

TCustomdxDBTreeList.ImageIndexFieldName; TCustomdxDBTreeList.SelectedIndex;

TdxTreeListNode.StateIndex

TCustomdxDBTreeList Methods

TCustomdxDBTreeList

BeginLayout	**public**	FindNodeByKeyValue	**public**
EndLayout	**public**	SelectAll	**public**

Derived from TCustomdxDBTreeListControl

ClearGroupColumns	**public**	GetColumnAt	**public**
ColumnByFieldName	**public**	GetDefaultColumnClass	**public**
ColumnByName	**public**	GetDefaultFieldColumnClass	**public**
CreateColumn	**public**	GetFooterColumnAt	**public**
CreateColumnEx	**public**	GetHeaderColumnAt	**public**
FindColumnByFieldName	**public**	RefreshBookmarks	**public**
FullRefresh	**public**		

Derived from TCustomdxTreeListControl

ApplyBestFit	**public**	GetFooterColumnAt	**public**
AssignColumns	**public**	GetHeaderColumnAt	**public**
BeginSorting	**public**	GetNodeFooterColumnAt	**public**
ColumnByName	**public**	GetVisibleColumnIndex	**public**
CreateColumn	**public**	LoadFromIniFile	**public**
CreateColumnEx	**public**	LoadFromRegistry	**public**
DestroyColumns	**public**	RefreshDefaultColumnsWidths	**public**
EndSorting	**public**	RefreshSorting	**public**
GetAbsoluteColumnIndex	**public**	RestoreColumnsDefaults	**public**
GetBandAt	**public**	SaveToIniFile	**public**
GetColumnAt	**public**	SaveToRegistry	**public**
GetDisplayValue	**public**		

Derived from TCustomdxTreeList

Add	**public**	DeleteSelection	**public**
AddFirst	**public**	Destroy	**public**
AdjustColumnsWidth	**public**	EndColumnsCustomizing	**public**
BeginSelection	**public**	EndSearch	**public**
BeginUpdate	**public**	EndSelection	**public**
CalcDrawInfo	**public**	EndUpdate	**public**
CancelEditor	**public**	FindInplaceEdit	**protected**
CellRect	**public**	FindNodeByText	**protected**
ClearColumnsSorted	**public**	FreeDrawInfo	**public**
ClearNodes	**public**	FullCollapse	**public**
ClearSelection	**public**	FullExpand	**public**
CloseEditor	**public**	GetEditRect	**public**
ColumnsCustomizing	**public**	GetFocusedAbsoluteIndex	**public**
CopyAllToClipboard	**public**	GetFocusedVisibleIndex	**public**
CopySelectedToClipboard	**public**	GetHitInfo	**public**
Create	**public**	GetHitTestInfoAt	**public**

Copyright © 2000 Developer Express Inc.

GetNextVisible	**public**	SaveAllToStrings	**public**
GetNodeAt	**public**	SaveAllToTextFile	**public**
GetScrollVertGridRect	**protected**	SaveSelectedToStrings	**public**
GotoFirst	**public**	SaveSelectedToTextFile	**public**
GotoLast	**public**	SelectNodes	**protected**
GotoNext	**public**	SetEditingText	**protected**
GotoPrev	**public**	SetEditText	**protected**
HideEditor	**public**	ShowButtonEditorMouse	**public**
IndexOf	**public**	ShowEditor	**public**
Insert	**public**	ShowEditorChar	**public**
InvalidateSelection	**protected**	ShowEditorMouse	**public**
IsInvertSelect	**public**	StartDragBand	**protected**
IsNodeVisible	**public**	StartDragHeader	**protected**
MakeBandVisible	**public**	StartSearch	**public**
MakeColumnVisible	**public**	UpdateScrollBars	**protected**
MakeNodeVisible	**public**		
PointInCustomizingForm	**public**		

Derived from TCustomdxContainer

BeginInitEdit	**protected**	GetReadOnly	**protected**
CanEditAcceptKey	**protected**	HideDrawFocusRect	**protected**
CanEditModify	**protected**	InplaceEditKeyDown	**protected**
CheckEditorWidth	**protected**	IsEditorMoved	**protected**
CreateEditStyle	**protected**	IsImmediateEditor	**protected**
EndInitEdit	**protected**	IsShowButtonAlways	**protected**
GetDefaultPopupBorderStyle	**public**	IsTabs	**protected**
GetEditColor	**protected**	MoveCol	**protected**
GetEditFont	**protected**	RedrawSelection	**protected**

BeginLayout method

Disables record reloading.

```
procedure BeginLayout;
```

Description

Calling the BeginLayout method disables reloading of dataset records. If the BeginLayout method has been called, any changes are not reflected until calling the EndLayout method. The BeginLayout and EndLayout methods prevent numerous updates of the DBTreeList control when changing different properties such as RootValue, KeyField, ParentField and others. Every call of the BeginLayout method increases the internal counter by one. Every call of the EndLayout method decreases the internal counter by one.

***Note**: Make certain that you call the EndLayout method after the BeginUpdate method. Use these methods in the same procedure.*

See Also

TCustomdxDBTreeList.EndLayout

EndLayout method

Enables records reloading.

```
procedure EndLayout;
```

Description

Calling the EndLayout method enables reloading of dataset records. The BeginLayout and EndLayout methods prevent numerous updates of the DBTreeList control when changing different properties such as RootValue, KeyField, ParentField and others. Every call of the BeginLayout method increases the internal counter by one. Every call of the EndLayout method decreases the internal counter by one.

***Note**: Remember to call the EndLayout method after the BeginUpdate method in the same procedure.*

See Also

TCustomdxDBTreeList.BeginLayout

FindNodeByKeyValue method

Finds a node by the key field value.

```
function FindNodeByKeyValue(const Value: Variant):TdxDBTreeListNode;
```

Description

Every node in a data-aware tree list corresponds to a dataset record. This method finds a node that corresponds to the record with the key field value equal to Value.

See Also

TCustomdxDBTreeListControl.KeyField

SelectAll method

Selects all tree list nodes.

```
procedure SelectAll;
```

Description

Use this method to select all tree list nodes. Before selection, SelectAll expands all nodes. The selected nodes can be saved by the SaveSelectedToTextFile method into a text file or by the SaveSelectedToStrings method into strings.

See Also

TCustomdxTreeList.DeleteSelection; TCustomdxTreeList.ClearSelection;

TCustomdxTreeList.SelectedNodes; TCustomdxTreeList.SelectedCount

Copyright © 2000 Developer Express Inc.

TCustomdxDBTreeList Events

TCustomdxDBTreeList

OnGetUniqueKeyValue	**protected**
OnMoveTo	**protected**

Derived from TCustomdxDBTreeListControl

OnBeforeCalcSummary	**protected**	OnGetEditColor	**public**
OnCalcSummary	**protected**	OnGetPreviewText	**protected**
OnChangeNodeEx	**protected**	OnRefreshNodeData	**protected**

Derived from TCustomdxTreeListControl

OnBandButtonClick	**protected**	OnDragEndHeader	**protected**
OnBandClick	**protected**	OnDragOverBand	**protected**
OnCalcRowLineHeight	**protected**	OnDragOverHeader	**protected**
OnCanBandDragging	**protected**	OnEndDragColumn	**protected**
OnCanHeaderDragging	**protected**	OnGetImageIndex	**protected**
OnChangedColumnsWidth	**protected**	OnGetLevelColor	**protected**
OnColumnClick	**protected**	OnGetNodeDragText	**protected**
OnColumnMoved	**protected**	OnGetSelectedIndex	**protected**
OnColumnSorted	**protected**	OnGetStateIndex	**protected**
OnColumnSorting	**protected**	OnHeaderButtonClick	**protected**
OnCustomDraw	**protected**	OnHeaderMoved	**protected**
OnCustomDrawBand	**protected**	OnHideBand	**protected**
OnCustomDrawCell	**protected**	OnHideHeader	**protected**
OnCustomDrawColumnHeader	**protected**	OnIsExistFooterCell	**protected**
OnCustomDrawFooter	**protected**	OnShowBand	**protected**
OnCustomDrawFooterNode	**protected**	OnShowHeader	**protected**
OnCustomDrawPreview	**protected**	OnStartBandDragging	**protected**
OnCustomDrawPreviewCell	**protected**	OnStartHeaderDragging	**protected**
OnDragEndBand	**protected**		

Derived from TCustomdxTreeList

OnBeginDragNode	**protected**	OnEditChange	**protected**
OnCanNodeSelected	**protected**	OnEdited	**protected**
OnChangeColumn	**protected**	OnEditing	**protected**
OnChangeLeftCoord	**protected**	OnEditValidate	**protected**
OnChangeNode	**protected**	OnEndColumnsCustomizing	**protected**
OnChangeTopVisibleNode	**protected**	OnExpanded	**protected**
OnCollapsed	**protected**	OnExpanding	**protected**
OnCollapsing	**protected**	OnGetEditColor	**protected**
OnCompare	**protected**	OnHotTrackNode	**protected**
OnDeletion	**protected**	OnSelectedCountChange	**protected**

OnGetUniqueKeyValue event

Occurs when generating a unique value for the key field.

```
type
TdxDBTLGetUniqueKeyValueEvent = procedure(Sender: TObject; var
UniqueKeyValue: Variant) of object;
property OnGetUniqueKeyValue: TdxDBTLGetUniqueKeyValueEvent;
```

Description

Write an OnGetUniqueKeyValue event handler to perform specific actions when generating the value for the key field of a newly created record. The UniqueKeyValue parameter represents this value.

See Also

TCustomdxDBTreeList.GetUniqueKeyValue

OnMoveTo event

Occurs when dropping a node.

```
type
 TdxTreeListNodeAttachMode = (natlAdd, natlAddFirst, natlAddChild,
natlAddChildFirst, natlInsert);

 TdxDBTLMoveToEvent = procedure(Sender: TObject; AttachNode:
TdxDBTreeListNode; AttachMode: TdxTreeListNodeAttachMode; Nodes: TList; var
IsCopy, Done: Boolean) of object;

property OnMoveTo: TdxDBTLMoveToEvent;
```

Description

Write an OnMoveTo event handler to respond to the dropping of a node within a DBTreeList control. The AttachNode parameter determines the node onto which the dragged node was dropped. The AttachMode parameter specifies which attach mode is enabled. The IsCopy parameter specifies whether the dragged node is copied (True) or moved (False). The Done parameter determines whether the OnMoveTo event handler entirely processes the operation.

The following attach modes are available:

Mode	Description
natlAdd	A node is added to the end of the nodes list at the same level as a node, specified by the AttachNode parameter.
natlAddChild	A node is added to the end of the child list of a node, specified by the AttachNode parameter.
natlAddChildFirst	A node is added as a child node of AttachNode to the beginning of its child list.
natlAddFirst	A node is added to the beginning of the nodes list at the same level as a node, specified by the AttachNode parameter.
natlInsert	A node is inserted after the AttachNode node.

Copyright © 2000 Developer Express Inc.

TdxDBTreeList

Unit

dxDBTL

```
TdxDBTreeList = class(TCustomdxDBTreeListControl);
```

Description

TdxDBTreeList is a data-aware hybrid of the standard tree view and list, but also contains unique features including full runtime column customization, the ability to edit any column within a node and to create tree structures from a dataset. TdxDBTreeList is inherited directly from the TCustomdxDBTreeListControl and publishes most of its properties.

***Note**: Inplace and standalone editors of the ExpressQuantumGrid Suite require Windows RichEdit component 2.0 or higher to work properly. If the editors on your or client's machine do not work, make certain that the riched20.dll library is in the Windows system folder (Windows\System or WINNT\System32). Copy the specified DLL to fix the problem.*

Hierarchy

TObject
- TPersistent
  - TComponent
    - TControl
      - TWinControl
        - TCustomControl
          - TCustomdxContainer
            - TCustomdxTreeList
              - TCustomdxTreeListControl
                - TCustomdxDBTreeListControl

TdxDBTreeList Properties

Derived from TCustomdxDBTreeList

DataSetChangedBusy	**public / read only**	OptionsDB	**published / read / write**
ImageIndexFieldName	**published / read / write**	OptionsView	**published / read / write**
LockLayout	**protected / read only**	ParentField	**published / read / write**
LockRefresh	**public / read / write**	RootValue	**published / read / write**
OptionsBehavior	**published / read / write**	SelectedIndex	**published / read / write**
OptionsCustomize	**published / read / write**	StateIndexFieldName	**published / read / write**

Derived from TCustomdxDBTreeListControl

Columns	**public / read / write**	PreviewFieldName	**published / read / write**
DataLink	**public / read only**	PreviewMaxLength	**published / read / write**
DataSource	**published / read / write**	SelectedRows	**public / read only**
FocusedField	**public / read / write**	SortedColumn	**public / read only**
GrIndicatorWidth	**published / read / write**	SortedColumns	**public / read only**
KeyField	**published / read / write**	VisibleColumns	**public / read / write**

Derived from TCustomdxTreeListControl

BandMaxRowCount	**published / read / write**	HeaderPanelRowCount	**published / read / write**
BandRowCount	**published / read / write**	IniFileName	**published / read / write**
Bands	**published / read / write**	IniSectionName	**published / read / write**
ColumnCount	**public / read only**	IsCustomStoring	**protected / read / write**
DefaultLayout	**published / read / write**	RegistryPath	**published / read / write**
HeaderMinRowCount	**published / read / write**	SortedColumnCount	**public / read only**
HeaderPanelMaxRowCount	**published / read / write**	VisibleColumnCount	**public / read only**

Derived from TCustomdxTreeList

ArrowsColor	**published / read / write**	DefaultRowHeight	**published / read / write**
AutoExpandOnSearch	**published / read / write**	DescTextHeight	**public / read only**
AutoSearchColor	**published / read / write**	DownBandIndex	**public / read only**
AutoSearchTextColor	**published / read / write**	DownBandPushed	**public / read only**
BandButtonPushed	**public / read only**	DownColumnIndex	**public / read only**
BandColor	**published / read / write**	DownColumnPushed	**public / read only**
BandFont	**published / read / write**	DragAbsoluteBandIndex	**public / read only**
BandPanelHeight	**public / read only**	DragAbsoluteHeaderIndex	**public / read only**
BorderStyle	**published / read / write**	DragNode	**public / read only**
Count	**public / read only**	EditingText	**public / read / write**
CustomizingBandListBox	**public / read only**	FixedBandLineColor	**published / read / write**
CustomizingForm	**public / read only**	FixedBandLineWidth	**published / read / write**
CustomizingHeaderListBox	**public / read only**	FocusedAbsoluteIndex	**public / read only**
CustomizingPos	**public / read / write**	FocusedColumn	**public / read / write**
CustomizingRowCount	**published / read / write**	FocusedNode	**public / read only**
DblClkExpanding	**published / read / write**	FocusedNumber	**protected / read / write**

Copyright © 2000 Developer Express Inc.

FooterPanelHeight	**public / read only**
FooterRowHeight	**public / read only**
FooterRowNodeHeight	**public / read only**
GridLineColor	**published / read / write**
GroupNodeColor	**published / read / write**
GroupNodeTextColor	**published / read / write**
HeaderButtonPushed	**public / read only**
HeaderColor	**published / read / write**
HeaderFont	**published / read / write**
HeaderHeight	**public / read only**
HeaderPanelHeight	**public / read only**
HeaderRowHeight	**public / read only**
HideFocusRect	**published / read / write**
HideSelection	**published / read / write**
HideSelectionColor	**published / read / write**
HideSelectionTextColor	**published / read / write**
HighlightColor	**published / read / write**
HighlightTextColor	**published / read / write**
Images	**published / read / write**
Indent	**published / read only**
IndentDesc	**published / read / write**
InplaceColumnIndex	**public / read only**
InplaceEditor	**public / read only**
IsCustomizing	**public / read only**
Items	**public / read only**
LastNode	**public / read only**
LeftCoord	**public / read / write**
LockSelection	**public / read only**
LockUpdate	**protected / read only**
LookAndFeel	**published / read / write**
MaxRowLineCount	**published / read / write**
NewItemRowActive	**protected / read / write**
NewItemRowHeight	**public / read only**
Options	**published / read / write**
OptionsEx	**published / read / write**
PaintStyle	**published / read / write**
PreviewFont	**published / read / write**
PreviewLines	**published / read / write**
RowCount	**protected / read only**
RowFooterColor	**protected / read / write**
RowFooterTextColor	**protected / read / write**
RowHeight	**public / read only**
RowSeparatorLineWidth	**published / read / write**
ScrollBars	**published / read / write**
SelectedCount	**public / read only**
SelectedNodes	**public / read only**
SelectionAnchor	**protected / read / write**
ShowBands	**published / read / write**
ShowButtons	**published / read / write**
ShowFooter	**published / read / write**
ShowGrid	**published / read / write**
ShowHeader	**published / read / write**
ShowHiddenInCustomizeBox	**published / read / write**
ShowIndicator	**protected / read / write**
ShowLines	**published / read / write**
ShowNewItemRow	**published / read / write**
ShowPreviewGrid	**published / read / write**
ShowRoot	**published / read / write**
ShowRowFooter	**protected / read / write**
SimpleCustomizeBox	**published / read / write**
State	**public / read only**
StateImages	**published / read / write**
TabStop	**published / read / write**
TopIndex	**public / read / write**
TopNode	**public / read only**
TopVisibleNode	**public / read only**
TreeLineColor	**published / read / write**
TreeLineStyle	**published / read / write**
VisibleRowCount	**protected / read only**
WaitForExpandNodeTime	**published / read / write**

Derived from TCustomdxContainer

IsInitEdit	**public / read only**
ReadOnly	**public / read only**

TdxDBTreeList Methods

Derived from TCustomdxDBTreeList

BeginLayout	**public**	FindNodeByKeyValue	**public**
EndLayout	**public**	SelectAll	**public**

Derived from TCustomdxDBTreeListControl

ClearGroupColumns	**public**	GetColumnAt	**public**
ColumnByFieldName	**public**	GetDefaultColumnClass	**public**
ColumnByName	**public**	GetDefaultFieldColumnClass	**public**
CreateColumn	**public**	GetFooterColumnAt	**public**
CreateColumnEx	**public**	GetHeaderColumnAt	**public**
FindColumnByFieldName	**public**	RefreshBookmarks	**public**
FullRefresh	**public**		

Derived from TCustomdxTreeListControl

ApplyBestFit	**public**	GetFooterColumnAt	**public**
AssignColumns	**public**	GetHeaderColumnAt	**public**
BeginSorting	**public**	GetNodeFooterColumnAt	**public**
ColumnByName	**public**	GetVisibleColumnIndex	**public**
CreateColumn	**public**	LoadFromIniFile	**public**
CreateColumnEx	**public**	LoadFromRegistry	**public**
DestroyColumns	**public**	RefreshDefaultColumnsWidths	**public**
EndSorting	**public**	RefreshSorting	**public**
GetAbsoluteColumnIndex	**public**	RestoreColumnsDefaults	**public**
GetBandAt	**public**	SaveToIniFile	**public**
GetColumnAt	**public**	SaveToRegistry	**public**
GetDisplayValue	**public**		

Derived from TCustomdxTreeList

Add	**public**	Destroy	**public**
AddFirst	**public**	EndColumnsCustomizing	**public**
AdjustColumnsWidth	**public**	EndSearch	**public**
BeginSelection	**public**	EndSelection	**public**
BeginUpdate	**public**	EndUpdate	**public**
CalcDrawInfo	**public**	FindInplaceEdit	**protected**
CancelEditor	**public**	FindNodeByText	**protected**
CellRect	**public**	FreeDrawInfo	**public**
ClearColumnsSorted	**public**	FullCollapse	**public**
ClearNodes	**public**	FullExpand	**public**
ClearSelection	**public**	GetEditRect	**public**
CloseEditor	**public**	GetFocusedAbsoluteIndex	**public**
ColumnsCustomizing	**public**	GetFocusedVisibleIndex	**public**
CopyAllToClipboard	**public**	GetHitInfo	**public**
CopySelectedToClipboard	**public**	GetHitTestInfoAt	**public**
Create	**public**	GetNextVisible	**public**
DeleteSelection	**public**	GetNodeAt	**public**

Copyright © 2000 Developer Express Inc.

GetScrollVertGridRect	**protected**	SaveAllToTextFile	**public**
GotoFirst	**public**	SaveSelectedToStrings	**public**
GotoLast	**public**	SaveSelectedToTextFile	**public**
GotoNext	**public**	SelectNodes	**protected**
GotoPrev	**public**	SetEditingText	**protected**
HideEditor	**public**	SetEditText	**protected**
IndexOf	**public**	ShowButtonEditorMouse	**public**
Insert	**public**	ShowEditor	**public**
InvalidateSelection	**protected**	ShowEditorChar	**public**
IsInvertSelect	**public**	ShowEditorMouse	**public**
IsNodeVisible	**public**	StartDragBand	**protected**
MakeBandVisible	**public**	StartDragHeader	**protected**
MakeColumnVisible	**public**	StartSearch	**public**
MakeNodeVisible	**public**	UpdateScrollBars	**protected**
PointInCustomizingForm	**public**		
SaveAllToStrings	**public**		

Derived from TCustomdxContainer

BeginInitEdit	**protected**	GetReadOnly	**protected**
CanEditAcceptKey	**protected**	HideDrawFocusRect	**protected**
CanEditModify	**protected**	InplaceEditKeyDown	**protected**
CheckEditorWidth	**protected**	IsEditorMoved	**protected**
CreateEditStyle	**protected**	IsImmediateEditor	**protected**
EndInitEdit	**protected**	IsShowButtonAlways	**protected**
GetDefaultPopupBorderStyle	**public**	IsTabs	**protected**
GetEditColor	**protected**	MoveCol	**protected**
GetEditFont	**protected**	RedrawSelection	**protected**

TdxDBTreeList Events

TCustomdxDBTreeList

OnGetUniqueKeyValue	**published**
OnMoveTo	**published**

Derived from TCustomdxDBTreeListControl

OnBeforeCalcSummary	**published**	OnGetEditColor	**public**
OnCalcSummary	**published**	OnGetPreviewText	**published**
OnChangeNodeEx	**published**	OnRefreshNodeData	**published**

Derived from TCustomdxTreeListControl

OnBandButtonClick	**published**	OnDragEndHeader	**published**
OnBandClick	**published**	OnDragOverBand	**published**
OnCalcRowLineHeight	**published**	OnDragOverHeader	**published**
OnCanBandDragging	**published**	OnEndDragColumn	**protected**
OnCanHeaderDragging	**published**	OnGetImageIndex	**protected**
OnChangedColumnsWidth	**published**	OnGetLevelColor	**published**
OnColumnClick	**published**	OnGetNodeDragText	**published**
OnColumnMoved	**protected**	OnGetSelectedIndex	**protected**
OnColumnSorted	**protected**	OnGetStateIndex	**protected**
OnColumnSorting	**published**	OnHeaderButtonClick	**published**
OnCustomDraw	**protected**	OnHeaderMoved	**published**
OnCustomDrawBand	**published**	OnHideBand	**published**
OnCustomDrawCell	**published**	OnHideHeader	**published**
OnCustomDrawColumnHeader	**published**	OnIsExistFooterCell	**published**
OnCustomDrawFooter	**published**	OnShowBand	**published**
OnCustomDrawFooterNode	**published**	OnShowHeader	**published**
OnCustomDrawPreview	**published**	OnStartBandDragging	**published**
OnCustomDrawPreviewCell	**published**	OnStartHeaderDragging	**published**
OnDragEndBand	**published**		

Derived from TCustomdxTreeList

OnBeginDragNode	**published**	OnEditChange	**published**
OnCanNodeSelected	**published**	OnEdited	**published**
OnChangeColumn	**published**	OnEditing	**published**
OnChangeLeftCoord	**published**	OnEditValidate	**published**
OnChangeNode	**published**	OnEndColumnsCustomizing	**published**
OnChangeTopVisibleNode	**published**	OnExpanded	**published**
OnCollapsed	**published**	OnExpanding	**published**
OnCollapsing	**published**	OnGetEditColor	**published**
OnCompare	**published**	OnHotTrackNode	**published**
OnDeletion	**published**	OnSelectedCountChange	**published**

Copyright © 2000 Developer Express Inc.

TCustomdxDBGrid

TCustomdxDBGrid is a base class for TdxDBGrid.

Unit

dxDBGrid

```
TCustomdxDBGrid = class(TCustomdxDBTreeListControl);
```

Description

TCustomdxDBGrid is directly inherited from TCustomdxDBTreeListControl. It contains methods and properties that provide functionality of TdxDBGrid (ExpressQuantumGrid).

Hierarchy

TObject
- TPersistent
  - TComponent
    - TControl
      - TWinControl
        - TCustomControl
          - TCustomdxContainer
            - TCustomdxTreeList
              - TCustomdxTreeListControl
                - TCustomdxDBTreeListControl

TCustomdxDBGrid Properties

TCustomdxDBGrid

DataSetChangedBusy	**public / read only**	OptionsCustomize	**protected / read / write**
DefaultFields	**published / read / write**	OptionsDB	**protected / read / write**
Filter	**protected / read / write**	OptionsEx	**protected / read / write**
GroupColumnCount	**public / read only**	OptionsView	**protected / read / write**
GroupColumns	**public / read only**	PartialLoad	**published / read / write**
GroupPanelColor	**protected / read / write**	PartialLoadBufferCount	**published / read / write**
GroupPanelFontColor	**protected / read / write**	ShowGroupPanel	**published / read / write**
GroupPanelHeight	**protected / read only**	ShowSummaryFooter	**published / read / write**
LockRefresh	**public / read / write**	SummaryGroups	**published / read / write**
Options	**protected / read / write**	SummarySeparator	**published / read / write**
OptionsBehavior	**protected / read / write**	UseBookmarks	**published / read / write**

Derived from TCustomdxDBTreeListControl

Columns	**public / read / write**	PreviewFieldName	**protected / read / write**
DataLink	**public / read only**	PreviewMaxLength	**protected / read / write**
DataSource	**public / read only**	SelectedRows	**protected / read only**
FocusedField	**protected / read / write**	SortedColumn	**public / read only**
GrIndicatorWidth	**protected / read / write**	SortedColumns	**public / read only**
KeyField	**published / read / write**	VisibleColumns	**public / read / write**

Derived from TCustomdxTreeListControl

BandMaxRowCount	**published / read / write**	HeaderPanelRowCount	**published / read / write**
BandRowCount	**published / read / write**	IniFileName	**protected / read / write**
Bands	**published / read / write**	IniSectionName	**protected / read / write**
ColumnCount	**public / read only**	IsCustomStoring	**protected / read / write**
DefaultLayout	**published / read / write**	RegistryPath	**protected / read / write**
HeaderMinRowCount	**published / read / write**	SortedColumnCount	**public / read only**
HeaderPanelMaxRowCount	**published / read / write**	VisibleColumnCount	**public / read only**

Derived from TCustomdxTreeList

ArrowsColor	**protected / read / write**	CustomizingRowCount	**public / read / write**
AutoExpandOnSearch	**protected / read / write**	DblClkExpanding	**protected / read / write**
AutoSearchColor	**protected / read / write**	DefaultRowHeight	**protected / read / write**
AutoSearchTextColor	**protected / read / write**	DescTextHeight	**public / read only**
BandButtonPushed	**public / read only**	DownBandIndex	**public / read only**
BandColor	**protected / read / write**	DownBandPushed	**public / read only**
BandFont	**protected / read / write**	DownColumnIndex	**public / read only**
BandPanelHeight	**public / read only**	DownColumnPushed	**public / read only**
BorderStyle	**protected / read / write**	DragAbsoluteBandIndex	**public / read only**
Count	**public / read only**	DragAbsoluteHeaderIndex	**public / read only**
CustomizingBandListBox	**public / read only**	DragNode	**protected read only**
CustomizingForm	**public / read only**	EditingText	**protected / read /write**
CustomizingHeaderListBox	**public / read only**	FixedBandLineColor	**protected / read /write**
CustomizingPos	**public / read / write**	FixedBandLineWidth	**protected / read /write**

Copyright © 2000 Developer Express Inc.

FocusedAbsoluteIndex	**public / read only**	NewItemRowHeight	**public / read only**
FocusedColumn	**protected / read / write**	PaintStyle	**protected read / write**
FocusedNode	**public / read only**	PreviewFont	**protected read / write**
FocusedNumber	**protected / read / write**	PreviewLines	**protected / read / write**
FooterPanelHeight	**public / read only**	RowCount	**protected / read only**
FooterRowHeight	**public / read only**	RowFooterColor	**protected / read / write**
FooterRowNodeHeight	**public / read only**	RowFooterTextColor	**protected / read / write**
GridLineColor	**protected / read / write**	RowHeight	**public / read only**
GroupNodeColor	**protected / read / write**	RowSeparatorLineWidth	**protected / read / write**
GroupNodeTextColor	**protected / read / write**	ScrollBars	**protected / read / write**
HeaderButtonPushed	**public / read only**	SelectedCount	**protected / read only**
HeaderColor	**protected / read / write**	SelectedNodes	**protected / read only**
HeaderFont	**protected / read / write**	SelectionAnchor	**protected / read / write**
HeaderHeight	**public / read only**	ShowBands	**protected / read / write**
HeaderPanelHeight	**public / read only**	ShowButtons	**protected / read / write**
HeaderRowHeight	**public / read only**	ShowFooter	**protected / read / write**
HideFocusRect	**protected / read / write**	ShowGrid	**protected / read / write**
HideSelection	**protected / read / write**	ShowHeader	**protected / read / write**
HideSelectionColor	**protected / read / write**	ShowHiddenInCustomizeBox	**protected / read / write**
HideSelectionTextColor	**protected / read / write**	ShowIndicator	**protected / read / write**
HighlightColor	**protected / read / write**	ShowLines	**protected / read / write**
HighlightTextColor	**protected / read / write**	ShowNewItemRow	**protected / read / write**
Images	**protected / read / write**	ShowPreviewGrid	**protected / read / write**
Indent	**protected / read only**	ShowRoot	**protected / read / write**
IndentDesc	**protected / read / write**	ShowRowFooter	**protected / read / write**
InplaceColumnIndex	**public / read only**	SimpleCustomizeBox	**protected / read / write**
InplaceEditor	**public / read only**	State	**public / read only**
IsCustomizing	**public / read only**	StateImages	**protected / read / write**
Items	**public / read only**	TabStop	**published / read / write**
LastNode	**public / read only**	TopIndex	**protected / read / write**
LeftCoord	**protected / read / write**	TopNode	**public / read only**
LockSelection	**public / read only**	TopVisibleNode	**public / read only**
LockUpdate	**protected / read only**	TreeLineColor	**protected / read / write**
LookAndFeel	**protected / read / write**	TreeLineStyle	**protected / read / write**
MaxRowLineCount	**protected / read / write**	VisibleRowCount	**protected / read only**
NewItemRowActive	**protected / read / write**	WaitForExpandNodeTime	**protected / read / write**

Derived from TCustomdxContainer

IsInitEdit	**public / read only**
ReadOnly	**public / read only**

DataSetChangedBusy property — read only

Indicates whether the grid works with a dataset.

```
property DataSetChangedBusy : Boolean;
```

Description

Use this value to obtain information on whether the ExpressQuantumGrid works with a dataset. If the DataSetChagedBusy property is True, the grid is busy with data-aware operations.

DefaultFields property — read / write

Controls auto-creation of grid columns.

```
property DefaultFields: Boolean;
```

Description

If DefaultFields is True, then the grid automatically creates columns for dataset fields when you associate a data source with it. By default, this property is False. Thus, when you specify a data source from which your grid retrieves data, the grid does not create any columns for the dataset field.

If the DefaultFields property is True, LoadAllRecords mode is disabled: It is impossible to group nodes by any column.

To create default columns at runtime when the ExpressQuantumGrid is in LoadAllRecords mode, use the CreateDefaultColumns method instead of DefaultFields. For more information, refer to CreateDefaultColumns.

Note: *The DefaultFields property and the egoLoadAllRecords option are mutually exclusive. Setting one changes the other.*

See Also

TCustomdxDBGrid.CreateDefaultColumns; TCustomdxDBGrid.DefineFieldMap

Filter property — read / write

Specifies the manner in which the filtering feature is enabled within the ExpressQuantumGrid.

```
property Filter: TdxDBGridFilter;
```

Description

MS Excel® style filtering is now available within the QuantumGrid. Use the Filter property to access properties and methods that provide this filtering functionality.

GroupColumnCount property — read only

Returns the number of grouping columns.

```
property GroupColumnCount : Integer;
```

Description

Use this property to obtain the number of grouping columns. The GroupColumnCount value is one greater than the index of the last element of the grouping columns array (the GroupColumns property).

Copyright © 2000 Developer Express Inc.

See Also

TCustomdxDBGrid.GroupColumns; TCustomdxDBGrid.OnAddGroupColumns

GroupColumns property — read only

Represents a list of group columns.

```
property GroupColumns[Index:Integer] : TdxDBTreeListColumn;
```

Description

This property represents an array of grouping columns. The first element of this array defines a grouping column at the topmost level that groups all the nodes. The second element groups children of a parent node, etc. The first element's index in the array is 0. The total number of grouping columns is specified by the GroupColumnCount property.

There are two ways to add/remove columns from the grouping column list:

The InsertGroupColumn and DeleteGroupColumn methods of ExpressQuantumGrid

The GroupIndex property of a column. Set GroupIndex to –1 in order to remove column from the group column list. Set GroupIndex to zero or any positive value to add a column to the group column list.

See Also

TCustomdxDBGrid.GroupColumnCount; TCustomdxDBTreeListControl.InsertGroupColumn; TCustomdxDBTreeListControl.DeleteGroupColumn; TCustomdxDBGrid.RebuildGroupIndexes; TCustomdxDBGrid.OnAddGroupColumn; TCustomdxDBGrid.RefreshGroupColumns; TCustomdxDBGrid.OnEndDragGroupColumn

GroupPanelColor property — read / write

Determines the background color of a group panel.

```
property GroupPanelColor : TColor;
```

Description

Group panel is the area at the top of the ExpressQuantumGrid onto which users can drag a column in order to group grid nodes. It is displayed only if the ShowGroupPanel property is True. This property determines the background color for the group panel.

Use the OnBackgroundDrawEvent to implement advanced painting for the group panel.

See Also

TCustomdxDBGrid.GroupPanelFontColor; TCustomdxDBGrid.GroupPanelHeight; TCustomdxDBGrid.ShowGroupPanel; TCustomdxDBGrid.OnBackgroundDrawEvent; sdxPanelText

GroupPanelFontColor property — read / write

Determines the text color for a group panel.

```
property GroupPanelFontColor : TColor;
```

Description

Group panel is the area at the top of the ExpressQuantumGrid onto which users can drag a column in order to group grid nodes. It is displayed only if the ShowGroupPanel property is True. This Group panel contains text similar to 'Drag a column header here to group by that column.' GroupPanelFontColor determines the color of this text.

See Also

TCustomdxDBGrid.GroupPanelColor; TCustomdxDBGrid.GroupPanelHeight; TCustomdxDBGrid.ShowGroupPanel; TCustomdxDBGrid.OnBackgroundDrawEvent; sdxPanelText

GroupPanelHeight property — read only

Determines the group panel height.

```
property GroupPanelHeight : Integer;
```

Description

Group panel is the area at the top of the ExpressQuantumGrid onto which users can drag a column in order to group grid nodes. It is displayed only if the ShowGroupPanel property is True. GroupPanelHeight determines the height of a group panel.

See Also

TCustomdxDBGrid.ShowGroupPanel; TCustomdxDBGrid.GroupPanelFontColor; TCustomdxDBGrid.GroupPanelColor

LockRefresh property — read / write

Prevents multiple refresh events of a grid control when navigating through a dataset.

```
property LockRefresh : Boolean;
```

Description

Set this property to True to prevent multiple refresh events of a grid control when a user navigates through dataset records. This navigation calls the TDataSet DataChaged method.

See Also

TCustomdxDBGrid.FullRefresh

Options property — read / write

Specifies different properties of a data-aware grid.

```
type
  TdxDBGridOption = (egoColumnSizing, egoColumnMoving, egoEditing, egoTabs,
egoTabThrough, egoRowSelect, egoMultiSelect, egoCanDelete,
egoConfirmDelete, egoCanNavigation, egoCanAppend, egoCancelOnExit,
egoImmediateEditor, egoCanInsert, egoLoadAllRecords, egoPreview,
egoDrawEndEllipsis, egoStoreToRegistry, egoAutoWidth, egoShowHourGlass,
egoCaseInsensitive, egoDblClick, egoIndicator, egoExtMultiSelect,
egoResetColumnFocus, egoExtCustomizing, egoHideFocusRect,
```

Copyright © 2000 Developer Express Inc.

```
egoStoreToIniFile, egoUseLocate, egoSmartReload, egoSmartRefresh,
egoSeekDetail);
TdxDBGridOptions = set of TdxDBGridOption;

property Options : TdxDBGridOptions;
```

Description

Used to specify ExpressQuantumGrid display and behavioral properties. The following values are available:

Setting	Value
egoColumnSizing	Allows a user to change column width.
egoColumnMoving	Allows a user to move a column within the grid.
egoEditing	If active, end users can modify data within grid cells. This option has no impact when the egoRowSelect is active.
egoTabs	Enables navigation within columns using Tab and Shift+Tab keys. If False, rather than moving focus from one cell to another, TAB key press events move focus away from ExpressQuantumGrid to the next control on a given form.
egoTabThrough	The Tab key press in the last column of a row (node) moves focus to the first column of the next row (node). This option is effective only if the egoTabs is active.
egoRowSelect	Determines whether a user can select the entire row versus individual cells within the row. Once a user selects a row within the grid, the entire row is highlighted. If False, only the selected cell is highlighted. If active, egoEditing has no effect.
egoMultiSelect	Determines whether more than one row in the grid can be selected at a time. If egoMultiSelect is not set, only nodes with the same parent group can be included in multi-select operations. This option has no impact when the egoCanNavigation option is inactive.
egoCanDelete	Determines whether a user can delete a record by pressing Ctrl+Delete keystrokes.
egoConfirmDelete	Determines whether a message box appears confirming a user's request to delete a row within the grid after pressing the Ctrl+Delete keys. Click here for additional information
egoCanNavigation	When in LoadAllRecords mode and egoCanNavigation is False, the ExpressQuantumGrid does not go to the record associated with the selected node and data modification is disabled. In all other cases, when changing a selected node, the ExpressQuantumGrid makes the linked record active. Click here for additional information
egoCanAppend	Determines whether a user can add a new record by pressing the down arrow keyboard button, if focus is on the last grid row (node).
egoCancelOnExit	When a user exits the grid after inserting a new record, to which he makes no modifications, the inserted record is not posted to the dataset and thus helps prevent accidental posting of empty records.
egoImmediateEditor	Determines whether an inplace editor is displayed immediately during editing operations within the grid. If inactive, a user has to press ENTER or click a cell to edit data.
egoCanInsert	Determines whether a user can insert records by pressing the Insert key or via the new item row.

egoLoadAllRecords Determines whether the ExpressQuantumGrid loads all records into memory. In order to perform grouping or sorting on the fly, the ExpressQuantumGrid requires loading of all records into memory. This option requires the KeyField property to be set correctly.

See ExpressQuantumGrid Modes for more information on loading mechanisms.

The egoLoadAllRecords option and the DefaultFields property are mutually exclusive. Setting one clears the other.

egoPreview Specifies whether to display the ExpressQuantumGrid preview section.

egoDrawEndEllipsis Defines whether ellipses ('...') are drawn when text of a cell does not fit within its boundaries.

egoStoreToRegistry If active, ExpressQuantumGrid columns stores settings and columns layouts in a Registry key specified by the RegistryPath property.

egoAutoWidth When set, ExpressQuantumGrid changes all column widths in order to display all columns on screen thus eliminating the need for a horizontal grid scroll bar.

egoShowHourGlass Determines whether ExpressQuantumGrid changes the cursor to crHourglass when performing long data processing events.

egoCaseInsensitive Used to sort grid nodes. If set, sorting is case insensitive.

egoDblClick Determines whether the OnDblClick event fires when a user double-clicks somewhere within the grid. If inactive, the OnDblClick event is generated only if a data node (not a grouped one) has been double-clicked.

egoIndicator Determines whether to display grid's indicator (a line to the left of the grid with current node indicators).

egoExtMultiSelect Extends the multi-selection capability of the ExpressQuantumGrid. If set, users can include in multi-select operations child nodes of different parent nodes. This option has no effect if egoMultiSelect is not set.

egoResetColumnFocus When new records are added, focus moves to the first column of the new node.

egoExtCustomizing Determines column customization style and behavior. By default, egoExtCustomizing is inactive. When a user moves (drags) a given column, this column is displayed either within the grid control or in the column customization window. When active, egoExtCustomizing enables users to remove active grid columns from the grid display by simply removing it from the grid header region. This feature allows users to remove columns without first activating the column customization window.

egoHideFocusRect Specifies whether to draw a rectangle around a focused column. If set, the bound rectangle around a focused column is hidden.

egoStoreToIniFile Determines whether to store ExpressQuantumGrid column settings in an INI file, specified by the IniFileName property.

egoUseLocate If active, Locate method is used to navigate through dataset when ExpressQuantumGrid is not in grouped mode.

egoSmartReload When active, the ExpressQuantumGrid does not read dataset contents each time it groups nodes. This enables fast data processing to improve grouping against larger datasets. If True, summaries are calculated only for dataset fields that correspond to grid columns. This option is used when the ExpressQuantumGrid is in LoadAllRecords mode. Use this option to speed up the ExpressQuantumGrid when it does not calculate summaries by dataset fields and which are not associated with any grid column.

Copyright © 2000 Developer Express Inc.

egoSmartRefresh	When a user posts changes to a dataset, ExpressQuantumGrid reloads only modified record(s). Setting this option automatically activates the egoUseLocate option. egoSmartRefresh is used only when the ExpressQuantumGrid is in LoadAllRecords mode.
egoSeekDetail	You can move through dataset records using the Next or Prior methods. In so doing, the ExpressQuantumGrid will focus the node that corresponds to the active dataset record. If egoSeekDetail is active, ExpressQuantumGrid always displays the child node that corresponds to active dataset record. If the node is not visible, its parent is expanded. A grouped node and its first child correspond to the same dataset record. If egoSeekDetail is not active, ExpressQuantumGrid does not show the child node that corresponds to the same dataset record as it parent, that is, ExpressQuantumGrid will focus the grouped node, but will not expand this grouped node and will not move focus to its first child.

The default values for the Option property is as follows:

[egoEditing, egoColumnSizing, egoColumnMoving, egoImmediateEditor, egoTabThrough].

OptionsBehavior property read / write

Specifies behavioral properties of a data-aware grid.

type

```
  TdxDBGridOptionBehavior = (edgoAnsiSort, edgoAutoCopySelectedToClipboard,
edgoAutoSearch, edgoAutoSort, edgoBandButtonClicking, edgoCaseInsensitive,
edgoCellMultiSelect, edgoCollapsedReload, edgoDblClick, edgoDragCollapse,
edgoDragExpand, edgoDragScroll, edgoEditing, edgoEnterShowEditor,
edgoEnterThrough, edgoExtMultiSelect, edgoHeaderButtonClicking,
edgoHorzThrough, edgoImmediateEditor, edgoMouseScroll, edgoMultiSelect,
edgoMultiSort, edgoSeekDetail, edgoShowHourGlass, edgoStoreToIniFile,
edgoStoreToRegistry, edgoTabs, edgoTabThrough, edgoVertThrough);
  TdxDBGridOptionsBehavior = set of TdxDBGridOptionBehavior;
property OptionsBehavior: TdxDBGridOptionsBehavior;
```

Description

The possible values of the OptionsBehavior property include:

Value	Meaning
edgoAnsiSort	If active, the ExpressQuantumGrid uses Windows locale settings to sort rows. This option is useful when cells of a grid contain native language text.
edgoAutoCopySelectedToClipboard	If active, the selected data is copied to the Windows Clipboard when a user pressec the Ctrl-C key combination.
edgoAutoSearch	Activates/deactivates the auto-search feature: The ExpressQuantumGrid automatically locates a record that matches the text a user enters within a focused column. To repeat searching press the CTRL-ENTER (forward) or SHIFT-CTRL-ENTER (backward) keys.
edgoAutoSort	If set, the ExpressQuantumGrid automatically sorts

nodes. Click the header of the appropriate column to sort nodes by any column. If egoAutoSort is inactive, the ExpressQuantumGrid does not perform any sort operations. The egoAutoSort option is effective when the ExpressQuantumGrid is in LoadAllRecords mode.

edgoBandButtonClicking If active, the OnBandButtonClick event is generated when a user clicks the band area indicator. See OnBandButtonClick for more information.

edgoCaseInsensitive Used to sort grid nodes. If set, sorting is case insensitive.

edgoCellMultiSelect MS Excel® style multiple selection. Not Implemented.

edgoCollapsedReload If active, the ExpressQuantumGrid displays all grouped nodes collapsed when records are grouped by a column. Otherwise, the ExpressQuantumGrid groups records, but expands a grouped node to display the current record.

edgoDblClick Determines whether the OnDblClick event fires when a user double-clicks somewhere within the grid. If inactive, the OnDblClick event is generated only if a data node (not a grouped one) has been double-clicked.

edgoDragCollapse If active, an expanded node collapses when a user drags a node to it.

edgoDragExpand If active, a collapsed node expands when a user drags a node to it.

edgoDragScroll If this option is set, focus moves towards the dragged node during drag & drop operations.

edgoEditing If active, end users can modify data within grid cells. This option has no impact when the edgoRowSelect is active.

edgoEnterShowEditor If active (the default state), an inplace editor is displayed when a user presses the ENTER key or starts typing text. Otherwise, an inplace editor is activated when a user clicks the selected cell or presses F2 on the keyboard.

edgoEnterThrough ENTER press moves focus to the next column. This option is only effective when edgoEnterShowEditor is active.

edgoExtMultiSelect Extends the multi-selection capabilities of the ExpressQuantumGrid. If set, users can include in multi-select operations child nodes of different parent nodes. This option has no effect if egoMultiSelect is not set.

edgoHeaderButtonClicking If active, the OnHeaderButtonClick event is generated when a user clicks the header area indicator. See OnHeaderButtonClick for more information.

edgoHorzThrough If active, pressing Right/Left arrows within the last/first column of a node moves focus to the next/previous node. Otherwise, the last/first column stays focused.

edgoImmediateEditor Determines whether an inplace editor is displayed immediately during editing operations within the grid. If inactive, a user has to press ENTER or click a cell to edit data.

edgoMouseScroll If active, mouse movement scrolls dropdown lists in edited cells.

edgoMultiSelect Determines whether more than one row in the grid can be selected at a time. If edgoMultiSelect is not set, only nodes with the same parent group can be included in

Copyright © 2000 Developer Express Inc.

	multi-select operations. This option has no impact when the edgoCanNavigation option is inactive.
edgoMultiSort	If active, multi sort is enabled. To sort the grid by several columns, click these columns while pressing the Shift key. This option is effective when the ExpressQuantumGrid is in LoadAllRecords mode.
edgoSeekDetail	You can move through dataset records using the Next or Prior methods. In so doing, the ExpressQuantumGrid will focus the node that corresponds to the active dataset record. If edgoSeekDetail is active, ExpressQuantumGrid always displays the child node that corresponds to active dataset record. If the node is not visible, its parent is expanded. A grouped node and its first child correspond to the same dataset record. If egoSeekDetail is not active, the ExpressQuantumGrid does not show child nodes that correspond to the same dataset record as its parent. The ExpressQuantumGrid will focus the grouped node, but will not expand this grouped node and will not move focus to its first child.
edgoShowHourGlass	Determines whether ExpressQuantumGrid changes the cursor to crHourglass when performing long data processing events.
edgoStoreToIniFile	Determines whether to store ExpressQuantumGrid column settings in an INI file, specified by the IniFileName property.
edgoStoreToRegistry	If active, ExpressQuantumGrid columns stores settings and columns layouts in a Registry key specified by the RegistryPath property.
edgoTabs	Enables navigation within columns using Tab and Shift+Tab keys. If False, rather than moving focus from one cell to another, TAB key press events move focus away from ExpressQuantumGrid to the next control on a given form.
edgoTabThrough	The Tab key press in the last column of a row (node) moves focus to the first column of the next row (node). This option is effective only if the egoTabs is active.
edgoVertThrough	Meaningful if nodes are drawn across several rows (multiple lines). If active, pressing the Up/Down arrow moves focus through lines of the same node. Otherwise, it moves focus to the previous/next node.

The default values for the OptionsBehavior property include: [edgoAutoSort, edgoDragScroll, edgoEditing, edgoEnterShowEditor, edgoImmediateEditor, edgoTabThrough, edgoVertThrough].

OptionsCustomize property — read / write

Specifies customization properties of a data-aware grid.

type

```
  TdxDBGridOptionCustomize = (edgoBandMoving, edgoBandPanelSizing,
edgoBandSizing, edgoColumnMoving, edgoColumnSizing, edgoExtCustomizing,
edgoFullSizing, edgoHeaderPanelSizing, edgoKeepColumnWidth,
edgoNotHideColumn, edgoRowSizing);
```

```
  TdxDBGridOptionsCustomize = set of TdxDBGridOptionCustomize;
property OptionsCustomize: TdxDBGridOptionsCustomize;
```

Description

The possible values of the OptionsCustomize property include:

Value	Meaning
edgoBandMoving	If active, users can move bands. Else, band movement is disabled.
edgoBandPanelSizing	If active, a user can change the height of the band area.
edgoBandSizing	Activate it to allow users to change bands size.
edgoColumnMoving	Allows a user to move a column within the grid.
edgoColumnSizing	Allows a user to change column width.
edgoExtCustomizing	Determines column customization style and behavior. By default, edgoExtCustomizing is inactive. When a user moves (drags) a given column, this column is displayed either within the grid control or in the column customization window. When active, edgoExtCustomizing enables users to remove active grid columns from the grid display by simply removing it from the grid header region. This feature allows users to remove columns without first activating the column customization window.
edgoFullSizing	If active, the size of the last column or a band changes even if the end user drags the cursor out of the grid window. Otherwise, the end user can resize the last column within the grid window.
edgoHeaderPanelSizing	If set, users can change header area height.
edgoKeepColumnWidth	If active, the ExpressQuantumGrid does not change column width values back to initial settings after deactivating the edgoAutoWidths option.
edgoNotHideColumn	If set, a column remains within the grid after a user has dropped it onto the group panel.
edgoRowSizing	If active, users can change row height.

The default values for the OptionsCustomize property include: [edgoBandMoving, edgoBandSizing, edgoColumnMoving, edgoColumnSizing].

OptionsDB property — read / write

Specifies properties of a data-aware grid to work with a dataset.

```
type
  TdxDBGridOptionDB = (edgoCanAppend, edgoCancelOnExit, edgoCanDelete,
edgoCanInsert, edgoCanNavigation, edgoConfirmDelete, edgoLoadAllRecords,
edgoPartialLoad, edgoResetColumnFocus, edgoSmartRefresh, edgoSmartReload,
edgoSyncSelection, edgoUseBookmarks, edgoUseLocate);
  TdxDBGridOptionsDB = set of TdxDBGridOptionDB;
property OptionsDB: TdxDBGridOptionsDB;
```

Description

The possible values of the OptionsDB property include:

Value	Meaning
edgoCanAppend	Determines whether a user can add a new record by pressing the down arrow keyboard button, if focus is on the last grid row (node).
edgoCancelOnExit	When a user exits the grid after inserting a new record, to which he makes no modifications, the inserted record is not posted to the dataset and thus helps prevent accidental posting of empty records.
edgoCanDelete	Determines whether a user can delete a record by pressing Ctrl+Delete keystrokes.
edgoCanInsert	Determines whether a user can insert records by pressing the Insert key or via the new item row.
edgoCanNavigation	When in LoadAllRecords mode and egoCanNavigation is False, the ExpressQuantumGrid does not go to the record associated with the selected node and data modification is disabled. In all other cases, when changing a selected node, the ExpressQuantumGrid makes the linked record active. Click here for additional information
edgoConfirmDelete	Determines whether a message box appears confirming a user's request to delete a row within the grid after pressing the Ctrl+Delete keys. Click here for additional information
edgoLoadAllRecords	Determines whether the ExpressQuantumGrid loads all records into memory. In order to perform grouping or sorting on the fly, the ExpressQuantumGrid requires loading of all records into memory. This option requires the KeyField property to be set correctly. See ExpressQuantumGrid_Modes for more information on loading mechanisms. The edgoLoadAllRecords option and the DefaultFields property are mutually exclusive. Setting one clears the other.
edgoPartialLoad	When active, the ExpressQuantumGrid loads a series of records into its buffer. The buffer contains more records than the grid can display. All loaded records remain in memory until a user or an application refreshes the dataset associated with the ExpressQuantumGrid. See ExpressQuantumGrid_Modes for more information on loading mechanisms.
edgoResetColumnFocus	When new records are added, focus moves to the first column of the new node.
edgoSmartRefresh	When a user posts changes to a dataset, ExpressQuantumGrid reloads only modified record(s). Setting this option automatically activates the edgoUseLocate option. edgoSmartRefresh is used only when the ExpressQuantumGrid is in LoadAllRecords mode.
edgoSmartReload	When active, the ExpressQuantumGrid does not read dataset contents each time it groups nodes. This enables fast data processing to improve grouping against larger datasets. If True, summaries are calculated only for dataset fields that correspond to grid columns. This option is used when the ExpressQuantumGrid is in LoadAllRecords mode. Use this option to speed up the ExpressQuantumGrid when it does not calculate summaries by dataset fields and which are not associated with any grid column.
edgoSyncSelection	If set, the ExpressQuantumGrid moves selection according to the active record of a dataset. In other words, the ExpressQuantumGrid moves selection when a user scrolls the dataset via external data-aware controls or programmatically.

When this option is inactive, the ExpressQuantumGrid moves focus only when the active record is changed.

The edgoSyncSelection option affects multi-selection when the dataset associated with the ExpressQuantumGrid is scrolled. Suppose, edgoSyncSelection is active. If a user selects several nodes and then scrolls the dataset, the ExpressQuantumGrid cancels selection and selects the node corresponding to the active dataset record.

If an application scrolls a dataset via the Locate method, the selected node does not change. Below is a workaround on how to simulate the edgoSyncSelection effect when using Locate:

```
Table1.Locate(. . .);
dxDBGrid1.ClearSelection;
dxDBGrid1.FocusedNode.Selected := True;
```

edgoUseBookmarks	When active, bookmarks are used to navigate through the dataset. Bookmarks are also used when the edgoSmartRefresh or edgoPartialLoad options are activated and the edgoUseLocate option is inactive.
edgoUseLocate	If active, the Locate method is used to navigate through a dataset when the ExpressQuantumGrid is not in grouped mode.

The default values for the OptionsDB property include: [edgoCancelOnExit, edgoCanDelete, edgoCanInsert, edgoCanNavigation, edgoConfirmDelete, edgoUseBookmarks].

OptionsEx property — read / write

Specifies extended properties of ExpressQuantumGrid.

```
type
  TdxDBGridOptionEx = (egoInvertSelect, egoUseBitmap, egoBandHeaderWidth,
egoAutoCalcPreviewLines, egoBandSizing, egoBandMoving, egoHorzThrough,
egoVertThrough, egoCellMultiSelect, egoEnterThrough, egoEnterShowEditor,
egoFullSizing, egoDragScroll, egoDragExpand, egoDragCollapse,
egoRowAutoHeight, egoBandButtonClicking, egoBandPanelSizing,
egoHeaderButtonClicking, egoHeaderPanelSizing, egoRowSizing, egoAutoSort,
egoNotHideColumn, egoMultiSort, egoShowButtonAlways,
egoAutoHeaderPanelHeight, egoKeepColumnWidth, egoSyncSelection,
egoCollapsedReload, egoAnsiSort, egoMouseScroll, egoAutoSearch);
TdxDBGridOptionsEx = set of TdxDBGridOptionEx;
property OptionsEx : TdxDBGridOptionsEx;
```

Description

Use this property to customize the look and feel and behavior of the ExpressQuantumGrid.

Value	Meaning
egoInvertSelect	If active, ExpressQuantumGrid highlights only focused columns (MS Outlook™ selection style), otherwise it highlights all columns of a focused node except for the currently selected column.
egoUseBitmap	If active, a temporary bitmap is used to draw cells within a grid. It

	decreases flickers, but draws a cell a bit slowly.
egoBandHeaderWidth	If active, width of a band is equal to width of columns, which belong to this band. This option is only effective when a band is not fixed or until a user re-sizes the band.
egoAutoCalcPreviewLines	If active, ExpressQuantumGrid automatically calculates the number of lines in the preview section. Otherwise, the number of lines occupied by the preview section is defined by the PreviewLines property.
egoBandSizing	Activate it to allow users to change bands size.
egoBandMoving	If active, users can move bands. Else, band moving is disabled.
egoHorzThrough	If active, pressing Right/Left arrows within the last/first column of a node moves focus to the next/previous node. Otherwise, the last/first column stays focused.
egoVertThrough	Meaningful if nodes are drawn on several rows (multiple lines). If active, pressing the Up/Down arrow moves focus through lines of the same node. Otherwise, it moves focus to the previous/next node.
egoCellMultiSelect	MS Excel™ style multiple selection. This option is not yet implemented in ExpressQuantumGrid.
egoEnterThrough	ENTER press moves focus to the next column. This option is only effective when egoEnterShowEditor is active.
egoEnterShowEditor	If active (the default state), an inplace editor is displayed when a user presses the ENTER key or starts typing text. Otherwise, an inplace editor is activated when a user clicks the selected cell or presses F2 on the keyboard.
egoFullSizing	If active, the size of the last column or a band changes even if the end user drags the cursor out of the grid window. Otherwise, the end user can resize the last column within the grid window.
egoDragScroll	If this option is set, focus moves towards the dragged node during drag & drop operations.
egoDragExpand	If active, a collapsed node expands when a user drags a node to it.
egoDragCollapse	If active, an expanded node collapses when a user drags a node to it.
egoRowAutoHeight	If active, ExpressQuantumGrid automatically calculates the number of rows in a cell. It is not recommended to activate this option if the ExpressQuantumGrid includes a huge number of columns (~100 or more).
egoBandButtonClicking	If active, the OnBandButtonClick event is generated when a user clicks the band area indicator. See OnBandButtonClick for more information.
egoBandPanelSizing	If active, a user can change the height of the band area.
egoHeaderButtonClicking	If active, the OnHeaderButtonClick event is generated when a user clicks the header area indicator. See OnHeaderButtonClick for more information.
egoHeaderPanelSizing	If set, users can change header area height.
egoRowSizing	If active, users can change row height.
egoMultiSort	If active, multi sort is enabled. To sort the grid by several columns, click these columns while pressing the Shift key. This option is effective when the ExpressQuantumGrid is in LoadAllRecords mode.

egoAutoSort

If set, the ExpressQuantumGrid automatically sorts nodes. Click the header of the appropriate column to sort nodes by any column. If egoAutoSort is inactive, the ExpressQuantumGrid does not perform any sort operations. The egoAutoSort option is effective when the ExpressQuantumGrid is in LoadAllRecords mode.

egoShowButtonAlways

If active, the edit button is always displayed within a column with a dropdown window. This button appears only for columns of a focused node. If egoShowButtonAlways is inactive, the edit button is displayed only within a focused cell.

egoNotHideColumn

If set, a column remains within the grid after a user has dropped it onto the group panel.

egoAutoHeaderPanelHeight

ExpressQuantumGrid column headers can be placed one under the other. If this option is active, the ExpressQuantumGrid sets the height of the header area equal to the number of header rows. This option is automatically disabled after you change the HeaderPanelRowChange property or manually resize the header area at runtime.

egoKeepColumnWidth

If active, the ExpressQuantumGrid does not change column width values back to initial settings after deactivating the egoAutoWidths option.

egoSyncSelection

If set, the ExpressQuantumGrid moves selection according to the active record of a dataset. E.g. ExpressQuantumGrid moves the selection, when a user scrolls the dataset via external data-aware controls or programmatically. When this option is inactive, the ExpressQuantumGrid moves focus only when the active record is changed.

The egoSyncSelection option affects multi-selection when the dataset associated with the ExpressQuantumGrid is scrolled. Suppose, egoSyncSelection is active. If a user selects several nodes and then scrolls the dataset, the ExpressQuantumGrid cancels selection and selects the node corresponding to the active dataset record.

If an application scrolls a dataset via the Locate method, the selected node does not change. Below is a workaround on how to simulate the egoSyncSelection effect when using Locate:

```
Table1.Locate(. . .);
dxDBGrid1.ClearSelection;
dxDBGrid1.FocusedNode.Selected := True;
```

egoCollapsedReload

If active, the ExpressQuantumGrid displays all grouped nodes collapsed when records are grouped by a column. Otherwise, the ExpressQuantumGrid groups records, but expands a grouped node to display the current record.

egoAnsiSort

If active, the ExpressQuantumGrid uses Windows locale settings to sort rows. This option is useful when cells of a grid contain native language text.

egoMouseScroll

If active, mouse movement scrolls dropdown lists in edited cells.

egoAutoSearch

Activates/deactivates the auto-search feature: The ExpressQuantumGrid automatically locates a record that matches the text a user enters within a focused column. To repeat searching press the CTRL-ENTER (forward) or SHIFT-CTRL-ENTER (backward) keys.

Copyright © 2000 Developer Express Inc.

The default values for OptionsEx include [egoUseBitmap, egoBandHeaderWidth, egoBandSizing, egoBandMoving, egoAutoSort, egoEnterShowEditor, egoVertThrough].

OptionsView property read / write

Specifies display properties of a data-aware grid.

```
type
  TdxDBGridOptionView = (edgoAutoCalcPreviewLines,
edgoAutoHeaderPanelHeight, edgoAutoWidth, edgoBandHeaderWidth,
edgoDrawEndEllipsis, edgoHideFocusRect, edgoHotTrack, edgoIndicator,
edgoInvertSelect, edgoPreview, edgoRowAutoHeight, edgoRowSelect,
edgoShowButtonAlways, edgoUseBitmap);
  TdxDBGridOptionsView = set of TdxDBGridOptionView;
property OptionsView: TdxDBGridOptionsView;
```

Description

The possible values of the OptionsView property include:

Value	Meaning
edgoAutoCalcPreviewLines	If active, the ExpressQuantumGrid automatically calculates the number of lines in the preview section. Otherwise, the number of lines occupied by the preview section is defined by the PreviewLines property
edgoAutoHeaderPanelHeight	ExpressQuantumGrid column headers can be placed one under the other. If this option is active, the ExpressQuantumGrid sets the height of the header area equal to the number of header rows. This option is automatically disabled after you change the HeaderPanelRowCount property or manually resize the header area at runtime.
edgoAutoWidth	When set, ExpressQuantumGrid changes all column widths in order to display all columns on screen thus eliminating the need for a horizontal grid scroll bar.
edgoBandHeaderWidth	If active, width of a band is equal to width of columns, which belong to this band. This option is only effective when a band is not fixed or until a user re-sizes the band.
edgoDrawEndEllipsis	Defines whether ellipses ('...') are drawn when text of a cell does not fit within its boundaries.
edgoHideFocusRect	Specifies whether to draw a rectangle around a focused column. If set, the bound rectangle around a focused column is hidden.
edgoHotTrack	If active, the hot track feature is enabled. You can use the TdxTreeListHotTrackInfo structure to determine the cell which is currently under the mouse pointer. The OnHotTrackNode event is designed to specify the manner in which a cell responds to mouse movements (different cursors, fonts, colors, etc.). You can also implement this feature via custom draw events.
edgoIndicator	Determines whether to display the grid's indicator (a line to the left of the grid with current node indicators). See

	Terms for more information.
edgoInvertSelect	If active, ExpressQuantumGrid highlights only focused columns (MS Outlook® selection style), otherwise it highlights all columns of a focused node except for the currently selected column.
edgoPreview	Specifies whether to display the ExpressQuantumGrid preview section.
edgoRowAutoHeight	If active, ExpressQuantumGrid automatically calculates the number of rows in a cell. You should not activate this option if the ExpressQuantumGrid includes a huge number of columns (~100 or more).
edgoRowSelect	Determines whether a user can select the entire row versus individual cells within the row. Once a user selects a row within the grid, the entire row is highlighted. If False, only the selected cell is highlighted. If active, edgoEditing has no effect.
edgoShowButtonAlways	If active, the edit button is always displayed within a column with a dropdown window. This button appears only for columns of a focused node. If edgoShowButtonAlways is inactive, the edit button is displayed only within a focused cell.
edgoUseBitmap	If active, a temporary bitmap is used to draw cells within a grid. It decreases flickers, but draws a cell a bit more slowly.

The default values for the OptionsView property include: [edgoBandHeaderWidth, edgoUseBitmap].

PartialLoad property — read / write

Determines whether the ExpressQuantumGrid operates in Partial Load mode.

```
property PartialLoad: Boolean;
```

Description

Partial Load mode is similar to standard grid mode because the ExpressQunatumGrid does not load all records into memory. The grid works with a select number of loaded records (but not with all records), so it does not automatically sort nodes and calculate summaries. When a user scrolls the ExpressQuantumGrid, the component loads necessary records to display them on screen.

When in Partial Load mode, the ExpressQuantumGrid loads a series of records into its buffer. The buffer contains more records than the grid can display. All loaded records remain in memory until a user or an application refreshes the dataset associated with the ExpressQuantumGrid.

Partial Load mode has some advantages in comparison with standard grid mode:

? Scrolling is faster.

? Multi-selection operations perform correctly.

? The vertical scrollbar is more intelligent.

Note: *The PartialLoad property has no impact if the egoLoadAllRecords option is active.*

See Also

TCustomdxDBGrid.IsPartialLoad; TCustomdxDBGrid.PartialLoadBfferCount; TCustomdxDBGrid.IsGridmode; TCustomdxDBGrid.Options

PartialLoadBufferCount property — read / write

Specifies the number of records loaded by the ExpressQuantumGrid in Partial Load mode.

```
property PartialLoadBufferCount: Integer;
```

Description

When in the Partial Load mode, the ExpressQuantumGrid loads a number of dataset records in memory to better perform scrolling and multi-select operations. The PartialLoadBufferCount property specifies the total number of loaded records. The ExpressQuantumGrid considers this value as a desirable number of loaded records. If this number is too small, the ExpressQuantumGrid will load more records. The ExpressQuantumGrid loads records relative to the current record.

Note: *This property is used only when the ExpressQuantumGrid is in Partial Load mode.*

See Also

TCustomdxDBGrid.PartialLoad; TCustomdxDBGrid.Options; TCustomdxDBGrid.IsGridMode; TCustomdxDBGrid.IsPartialLoad

ShowGroupPanel property — read / write

Determines whether to display the grouping panel.

```
property ShowGroupPanel : Boolean;
```

Description

The grouping panel is an area at the top of the ExpressQuantumGrid onto which users place columns for the purpose of grouping data within the grid. Use ShowGroupPanel to show/hide the grouping panel.

The following code displays/hides the group panel on activating/deactivating a check box.

```
// Delphi
procedure TForm1.CheckBox1Click(Sender: TObject);
begin
  dxGrid1.ShowGroupPanel := CheckBox1.Checked;
end;

// C++Builder
void __fastcall TForm1::CheckBox1Click(TObject * Sender)
{
  dxGrid1->ShowGroupPanel = CheckBox1->Checked;
}
```

See Also

TCustomdxDBGrid.GroupPanelColor; TCustomdxDBGrid.GroupPanelFontColor; TCustomdxDBGrid.GroupPanelHeight; sdxPanelText

ShowSummaryFooter property — read / write

Determines whether to display a summary footer.

```
property ShowSummaryFooter : Boolean;
```

Description

Use this property to show/hide the summary footer at the bottom of the ExpressQuantumGrid. If ShowSummaryFooter is True, the summary footer is displayed.

See Also

TCustomdxTreeList.OnCustomDrawFooter

SummaryGroups property — read / write

Specifies summaries calculated for a column.

```
property SummaryGroups: TdxDBGridSummaryGroups;
```

Description

Use this property to specify summaries displayed when nodes are grouped by columns.

See Also

TCustomdxDBGrid.GetSummaryGroupByName

SummarySeparator property — read / write

Specifies the separator between summary values, displayed within a grouped node.

```
property SummarySeparator: string;
```

Description

Use this property to determine a string, which separates summary values for a grouped node. By default, SummarySeparator is a comma (‘, ‘).

See Also

TCustomdxDBGrid.OnCalcSummary

UseBookmarks property — read / write

Determines whether it is possible to use bookmarks to refer to dataset records corresponding to selected nodes.

Copyright © 2000 Developer Express Inc.

```
property UseBookmarks : Boolean;
```

Description

If UseBookmarks is True, an application can use the SelectedRows property to refer to dataset records that correspond to selected nodes.

When in standard grid mode, the UseBookmarks property has no impact, since the ExpressQuantumGrid always uses bookmarks to refer to records corresponding to selected dataset records.

See Also

TCustomdxDBGrid.SelectedRows

TCustomdxDBGrid Methods

TCustomdxDBGrid

AddGroupColumn	**public**	IsGridMode	**public**
BeginGrouping	**public**	IsLoadAllRecords	**public**
CanAddGroupColumn	**public**	IsPartialLoad	**public**
DeleteGroupColumn	**public**	PointInFooterPanel	**public**
EndGrouping	**public**	PointInGroupPanel	**public**
FindNodeByKeyValue	**public**	RebuildGroupIndexes	**public**
GetGroupColumnAt	**public**	RefreshGroupColumns	**public**
GetNodeFooterColumnAt	**public**	ResetFullRefresh	**public**
GetSummaryGroupByName	**public**	SetFlagSmartReload	**public**
GetSummaryItemAt	**public**		
InsertGroupColumn	**public**		

Derived from TCustomdxDBTreeListControl

ClearGroupColumns	**public**	GetColumnAt	**public**
ColumnByFieldName	**public**	GetDefaultColumnClass	**public**
ColumnByName	**public**	GetDefaultFieldColumnClass	**public**
CreateColumn	**public**	GetFooterColumnAt	**public**
CreateColumnEx	**public**	GetHeaderColumnAt	**public**
FindColumnByFieldName	**public**	RefreshBookmarks	**public**
FullRefresh	**public**		

Derived from TCustomdxTreeListControl

ApplyBestFit	**public**	GetFooterColumnAt	**public**
AssignColumns	**public**	GetHeaderColumnAt	**public**
BeginSorting	**public**	GetNodeFooterColumnAt	**public**
ColumnByName	**public**	GetVisibleColumnIndex	**public**
CreateColumn	**public**	LoadFromIniFile	**public**
CreateColumnEx	**public**	LoadFromRegistry	**public**
DestroyColumns	**public**	RefreshDefaultColumnsWidths	**public**
EndSorting	**public**	RefreshSorting	**public**
GetAbsoluteColumnIndex	**public**	RestoreColumnsDefaults	**public**
GetBandAt	**public**	SaveToIniFile	**public**
GetColumnAt	**public**	SaveToRegistry	**public**
GetDisplayValue	**public**		

Derived from TCustomdxTreeList

Add	**public**	ClearColumnsSorted	**public**
AddFirst	**public**	ClearNodes	**public**
AdjustColumnsWidth	**public**	ClearSelection	**public**
BeginSelection	**public**	CloseEditor	**public**
BeginUpdate	**public**	ColumnsCustomizing	**public**
CalcDrawInfo	**public**	CopyAllToClipboard	**public**
CancelEditor	**public**	CopySelectedToClipboard	**public**
CellRect	**public**	Create	**public**

Copyright © 2000 Developer Express Inc.

DeleteSelection	**public**	Insert	**public**
Destroy	**public**	InvalidateSelection	**protected**
EndColumnsCustomizing	**public**	IsInvertSelect	**public**
EndSearch	**public**	IsNodeVisible	**public**
EndSelection	**public**	MakeBandVisible	**public**
EndUpdate	**public**	MakeColumnVisible	**public**
FindInplaceEdit	**protected**	MakeNodeVisible	**public**
FindNodeByText	**protected**	PointInCustomizingForm	**public**
FreeDrawInfo	**public**	SaveAllToStrings	**public**
FullCollapse	**public**	SaveAllToTextFile	**public**
FullExpand	**public**	SaveSelectedToStrings	**public**
GetEditRect	**public**	SaveSelectedToTextFile	**public**
GetFocusedAbsoluteIndex	**public**	SelectNodes	**protected**
GetFocusedVisibleIndex	**public**	SetEditingText	**protected**
GetHitInfo	**public**	SetEditText	**protected**
GetHitTestInfoAt	**public**	ShowButtonEditorMouse	**public**
GetNextVisible	**public**	ShowEditor	**public**
GetNodeAt	**public**	ShowEditorChar	**public**
GetScrollVertGridRect	**protected**	ShowEditorMouse	**public**
GotoFirst	**public**	StartDragBand	**protected**
GotoLast	**public**	StartDragHeader	**protected**
GotoNext	**public**	StartSearch	**public**
GotoPrev	**public**	UpdateScrollBars	**protected**
HideEditor	**public**		
IndexOf	**public**		

Derived from TCustomdxContainer

BeginInitEdit	**protected**	GetReadOnly	**protected**
CanEditAcceptKey	**protected**	HideDrawFocusRect	**protected**
CanEditModify	**protected**	InplaceEditKeyDown	**protected**
CheckEditorWidth	**protected**	IsEditorMoved	**protected**
CreateEditStyle	**protected**	IsImmediateEditor	**protected**
EndInitEdit	**protected**	IsShowButtonAlways	**protected**
GetDefaultPopupBorderStyle	**public**	IsTabs	**protected**
GetEditColor	**protected**	MoveCol	**protected**
GetEditFont	**protected**	RedrawSelection	**protected**

AddGroupColumn method

Adds a new grouping column.

```
function AddGroupColumn(Column: TdxDBTreeListColumn): Boolean;
```

Description

Use this method to add a column, passed as Column, to the list of grouped columns. The following code groups grid nodes (rows) by the first column:

```
procedure TfmMain.Button1Click(Sender: TObject);
begin
 with dxDBGrid do
  AddGroupColumn(Columns[0]);
end;
```

See Also

TCustomdxDBGrid.InsertGroupColumn; TCustomdxDBGrid.DeleteGroupColumn; TCustomdxDBGrid.RebuildGroupIndexes; TCustomdxDBGrid.GroupColumns; TCustomdxDBGrid.GroupColumnCount; TCustomdxDBGrid.BeginGrouping; TCustomdxDBGrid.CanAddGroupColumn; TCustomdxDBGrid.OnReloadGroupList; TCustomdxDBGrid.OnReloadGroupList

BeginGrouping method

Prevents multiple reading of a dataset when grouping nodes.

```
procedure BeginGrouping;
```

Description

BeginGrouping and EndGrouping work together to prevent multiple reading of dataset records when a grid groups its nodes. Every call of BeginGrouping increases the internal counter by one. Every call of EndGrouping decreases it by one. If the internal counter is greater than zero, results of grouping are not displayed.

See Also

TCustomdxDBGrid.EndGrouping; TCustomdxDBGrid.AddGroupColumn; TCustomdxDBGrid.InsertGroupColumn

CanAddGroupColumn method

Determines whether a specified column can be added to the list of grouped columns.

```
function CanAddGroupColumn(Column : TdxDBTreeListColumn ): Boolean;
virtual;
```

Description

Returns True, if it is possible to group nodes by a column, passed as Column. Called before a column is added to the list of grouped columns.

See Also

TCustomdxDBGrid.AddGroupColumn; TCustomdxDBGrid.InsertGroupColumn

DeleteGroupColumn method

Deletes a column from the list of grouped columns.

```
procedure DeleteGroupColumn(Index : Integer);
```

Description

Use this procedure to remove a column from the list of grouped columns. The Index parameter specifies the index of a column in the list of grouped columns. The following code removes the last column from the list of grouped columns:

```
procedure TForm1.Button1Click(Sender: TObject);
begin
  with dxDBGrid do
    DeleteGroupColumn(GroupColumnCount-1);
end;
```

See Also

TCustomdxDBGrid.InsertGroupColumn; TCustomdxDBGrid.AddGroupColumn; TCustomdxDBGrid.RebuildGroupIndexes; TCustomdxDBGrid.OnReloadGroupList

EndGrouping method

Called when a grid finishes grouping nodes.

```
procedure EndGrouping;
```

Description

BeginGrouping and EndGrouping methods work together to prevent multiple reading of dataset records when a grid groups nodes. Every call of BeginGrouping increases the internal counter by one. Every call of EndGrouping decreases it by one. If the internal counter is greater than zero, the results of grouping are displayed. Be sure to call EndGrouping after each call to BeginGrouping.

See Also

TCustomdxDBGrid.BeginGrouping; TCustomdxDBGrid.GroupColumns

FindNodeByKeyValue method

Finds a node by the key field value.

```
function FindNodeByKeyValue(const Value: Variant): TdxDBGridNode;
```

Description

Every node in a data-aware grid corresponds to a record in a dataset. This method finds a node, which corresponds to a record with the key field value equal to Value.

See Also

TCustomdxDBGrid.KeyField

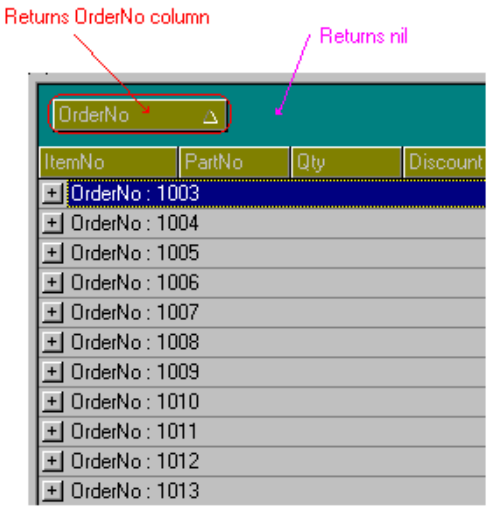
GetGroupColumnAt method

Returns a column from the group panel if it contains a point with specified coordinates.

```
function GetGroupColumnAt(X, Y: Integer): TdxDBTreeListColumn;
```

Description

Returns a grouped column if its rectangle in the group panel contains a point with X and Y coordinates. If such a column does not exist, returns nil.



See Also

TCustomdxDBGrid.GetFooterColumnAt; TCustomdxDBGrid.GetColumnAt; TCustomdxDBGrid.GetnodeFooterColumnAt; TCustomdxDBGrid.GetHeaderColumnAt; TCustomdxDBGrid.PointInFooterPanel; TCustomdxDBGrid.PointInGroupPanel; TCustomdxTreeList.GetHitInfo;

GetNodeFooterColumnAt method

Returns a column by node footer coordinates.

```
function GetNodeFooterColumnAt(X, Y: Integer): TdxDBTreeListColumn;
```

Description

Returns a column corresponding to a node footer cell, if the cell contains a point with X and Y coordinates. If no cell contains such a point, GetNodeFooterColumnAt returns nil.

See Also

TCustomdxDBGrid.GetFooterColumnAt; TCustomdxDBGrid.GetGroupColumnAt; TCustomdxDBGrid.GetColumnAt; TCustomdxTreeList.GetHitInfo; TCustomdxDBGrid.PointInFooterPanel; TCustomdxDBGrid.PointInGroupPanel

GetSummaryGroupByName method

```
function GetSummaryGroupByName(const AName: string): TdxDBGridSummaryGroup;
```

Description

The ExpressQuantumGrid can be associated with a number of TdxDBGridSummaryGroup objects used to calculate summaries within a grid. This method returns a summary group by its name. If there is no summary group with the specified name, GetSummaryNodeByName returns nil.

See Also

TCustomdxDBGrid.SummaryGroups

GetSummaryItemAt method

Returns a summary item by its coordinates.

```
function GetSummaryItemAt(X, Y: Integer; var ASummaryGroup:
TdxDBGridSummaryGroup; var AColumn: TdxDBTreeListColumn; AutoCreate:
Boolean): TdxDBGridSummaryItem;
```

Description

The GetSummaryItemAt method returns a summary item if its rectangle contains a point with X and Y coordinates. If there is no such item, this method returns **nil**.

For instance, you can use this method within the MouseUp event handler to display a PopupMenu with 'Count', 'Sum', 'Average', etc. items.

The AutoCreate parameter specifies whether to create a new summary item. A new summary item is created if the AutoCreate parameter is set to **True** and specified coordinates point to a summary row (hitType is htSummaryNodeFooter).

The ASummaryGroup and AColumn parameters represent a summary group and a column accordingly which contain the summary item specified by the X and Y coordinates.

InsertGroupColumn method

Adds a column to the list of grouped columns.

```
function InsertGroupColumn(AIndex : Integer; Column:TdxDBTreeListColumn):
Boolean;
```

Description

Use this method to insert a column into the list of grouped columns. AIndex determines position within the list of grouped columns, wherein to insert the column which is in turn specified by the Column parameter.

See Also

TCustomdxDBGrid.AddGroupColumn; TCustomdxDBGrid.DeleteGroupColumn; TCustomdxDBGrid.RebuildGroupIndexes; TCustomdxDBGrid.OnAddGroupColumn; TCustomdxDBGrid.BeginGrouping; TCustomdxDBGrid.EndGrouping; TCustomdxDBGrid.OnReloadGroupList

IsGridMode method

Determines whether the ExpressQuantumGrid is in standard grid mode.

```
function IsGridMode: Boolean;
```

Description

IsGridMode returns True, if the ExpressQuantumGrid is in standard grid mode; Otherwise, it returns False. Use the IsPartialMode method to determine whether the ExpressQuantumGrid is in Partial Load mode.

See ExpressQuantumGrid Modes for complete information on loading mechanisms available to you.

See Also

TCustomdxDBGrid.Options; TCustomdxDBGrid.OptionsEx; TCustomdxDBGrid.IsPartialMode; TCustomdxDBGrid.IsLoadAllRecords; TCustomdxDBGrid.PartialMode

IsLoadAllRecords method

Determines whether the ExpressQuantumGrid is in LoadAllRecords mode.

```
function IsLoadAllRecords: Boolean;
```

Description

The ExpressQuantumGrid is in LoadAllRecords mode when the egoLoadAllRecords option is active or nodes are grouped by one or more columns. This function returns True, if LoadAllRecords is active; Otherwise, it returns False.

See Also

TCustomdxDBGrid.IsGridMode; TCustomdxDBGrid.IsPartialLoad; TCustomdxDBGrid.Options

IsPartialLoad method

```
function IsPartialLoad: Boolean;
```

Description

Returns True if ExpressQuantumGrid is in Partial Load mode. The ExpressQuantumGrid is considered to be in partial load mode when:

? The PartialLoad property is True.

? The egoLoadAllRecords option is inactive.

? There are no grouped columns in the grid.

See Also

TCustomdxDBGrid.PartialLoad; TCustomdxDBGrid.PartialLoadBufferCount; TCustomdxDBGrid.Options; TCustomdxDBGrid.GroupColumns; TCustomdxDBGrid.IsGridMode; TCusotmdxDBgrid.IsLoadallRecords

PointInFooterPanel method

Determines whether a specified point belongs to the footer panel.

```
function PointInFooterPanel(p:TPoint): Boolean;
```

Copyright © 2000 Developer Express Inc.

Description

Footer panel is an area at the bottom of the ExpressQuantumGrid control containing summary footer text. It returns True if a point, passed as p, belongs to the footer panel; Else, returns False.

Note: *PointInFooterPanel is now obsolete. Use the GetHitInfo instead.*

See Also

TCustomdxDBGrid.PointInGroupPanel; TCustomdxDBGrid.GetColumnAt; TCustomdxDBGrid.GetHeaderColumnAt; TCustomdxDBGrid.GetNodeFooterColumnAt; TCustomdxDBGrid.GetGroupColumnAt; TCustomdxDBGrid.GetFooterColumnAt; TCustomdxTreeList.GetHitInfo

PointInGroupPanel method

Determines whether a specified point belongs to the group panel.

```
function PointInGroupPanel(p:TPoint): Boolean;
```

Description

GroupPanel is an area at the top of a control containing grouped columns. Returns True, if a point, passed as p, belongs to the group panel rectangle; Else, returns False.

Note: *PointInGroupPanel is now obsolete. Use the GetHitInfo instead.*

See Also

TCustomdxDBGrid.PointInFooterPanel; TCustomdxDBGrid.GetColumnAt; TCustomdxDBGrid.GetHeaderColumnAt; TCustomdxDBGrid.GetNodeFooterColumnAt; TCustomdxDBGrid.GetGroupColumnAt; TCustomdxDBGrid.GetFooterColumnAt; TCustomdxTreeList.GetHitInfo

RebuildGroupIndexes method

Refreshes the index of grouped columns.

```
procedure RebuildGroupIndexes;
```

Description

AddGroupColumn, InsertGroupColumn or DeleteGroupColumn change the order of columns within the list of grouped columns. Helps to correct the group index of grouped columns.

See Also

TCustomdxDBGrid.AddGroupColumn; TCustomdxDBGrid.InsertGroupColumn; TCustomdxDBGrid.DeleteGroupColumn; TCustomdxDBGrid.GroupColumns; TCustomdxDBGrid.OnAddGroupColumn; TCustomdxDBGrid.OnEndDragGroupColumn

RefreshGroupColumns method

Recalculates summaries for grid nodes.

```
procedure RefreshGroupColumns;
```

Description

Recalculates summaries when grid nodes are grouped or all records are loaded into memory (ExpressQuantumGrid is in LoadAllRecords mode).

See Also

TCustomdxDBGrid.Options; TCustomdxDBGrid.GroupColumns; TCustomdxDBGrid.RebuildGroupIndexes

ResetFullRefresh method

Sets an internal indicator to reload all dataset records.

```
procedure ResetFullRefresh;
```

Description

If the egoSmartRefresh option is active (Options property), the ExpressQuantumGrid reloads only modified records from a given dataset. Calling ResetFullRefresh indicates that it is necessary to reload all dataset records. For instance, a grid has several selected nodes and a user deletes records corresponding to these nodes from a dataset. If egoSmartRefresh is active, the grid deletes only the current node. Other nodes have no associated record. ResetFullRefresh is used to prevent such situations.

See Also

TCustomdxDBGrid.FullRefresh; TCustomdxDBGrid.SetFlagSmartReload; TCustomdxDBGrid.Options

SetFlagSmartReload method

Sets the smart reloading option.

```
procedure SetFlagSmartReload;
```

Description

Use this method to specify whether to use smart reloading. If smart reloading is used, the ExpressQuantumGrid does not read the dataset each time it groups nodes.

Note: *The SmartReload option is effective only when the ExpressQuantumGrid is in LoadAllRecords mode.*

See Also

TCustomdxDBGrid.Options; TCustomdxDBGrid.ResetFullRefresh

Copyright © 2000 Developer Express Inc.

TCustomdxDBGrid Events

TCustomdxDBGrid

OnAddGroupColumn	**protected**	OnEndDragGroupColumn	**protected**
OnBackgroundDrawEvent	**public**	OnGetCustomSummaryText	**public**
OnClearNodeData	**public**	OnHideColumn	**public**
OnColumnClick	**public**	OnReloadGroupList	**protected**
OnColumnSorting	**protected**	OnShowColumn	**public**
OnCustomDraw	**public**	OnSummaryNode	**public**

Derived from TCustomdxDBTreeListControl

OnBeforeCalcSummary	**protected**	OnGetEditColor	**public**
OnCalcSummary	**protected**	OnGetPreviewText	**protected**
OnChangeNodeEx	**protected**	OnRefreshNodeData	**protected**

Derived from TCustomdxTreeListControl

OnBandButtonClick	**protected**	OnDragEndHeader	**protected**
OnBandClick	**protected**	OnDragOverBand	**protected**
OnCalcRowLineHeight	**protected**	OnDragOverHeader	**protected**
OnCanBandDragging	**protected**	OnEndDragColumn	**protected**
OnCanHeaderDragging	**protected**	OnGetImageIndex	**protected**
OnChangedColumnsWidth	**protected**	OnGetLevelColor	**protected**
OnColumnClick	**protected**	OnGetNodeDragText	**protected**
OnColumnMoved	**protected**	OnGetSelectedIndex	**protected**
OnColumnSorted	**protected**	OnGetStateIndex	**protected**
OnColumnSorting	**protected**	OnHeaderButtonClick	**protected**
OnCustomDraw	**protected**	OnHeaderMoved	**protected**
OnCustomDrawBand	**protected**	OnHideBand	**protected**
OnCustomDrawCell	**protected**	OnHideHeader	**protected**
OnCustomDrawColumnHeader	**protected**	OnIsExistFooterCell	**protected**
OnCustomDrawFooter	**protected**	OnShowBand	**protected**
OnCustomDrawFooterNode	**protected**	OnShowHeader	**protected**
OnCustomDrawPreview	**protected**	OnStartBandDragging	**protected**
OnCustomDrawPreviewCell	**protected**	OnStartHeaderDragging	**protected**
OnDragEndBand	**protected**		

Derived from TCustomdxTreeList

OnBeginDragNode	**protected**	OnEditChange	**protected**
OnCanNodeSelected	**protected**	OnEdited	**protected**
OnChangeColumn	**protected**	OnEditing	**protected**
OnChangeLeftCoord	**protected**	OnEditValidate	**protected**
OnChangeNode	**protected**	OnEndColumnsCustomizing	**protected**
OnChangeTopVisibleNode	**protected**	OnExpanded	**protected**
OnCollapsed	**protected**	OnExpanding	**protected**
OnCollapsing	**protected**	OnGetEditColor	**protected**
OnCompare	**protected**	OnHotTrackNode	**protected**
OnDeletion	**protected**	OnSelectedCountChange	**protected**

OnAddGroupColumn event

Occurs when adding a new column to the grouping columns list.

```
type
  TAddGroupColumnEvent = procedure(Sender: TObject; Column :
TdxDBTreeListColumn; var Allow: Boolean) of object;
property OnAddGroupColumn : TAddGroupColumnEvent;
```

Description

Write an OnAddGroupColumn event handler to control addition of a new grouping column, passed as the Column parameter. The Allow parameter determines whether it is possible to add a column to the grouping columns list. Set Allow to False within an OnAddGroupColumn event handler in order to prevent column addition to the grouping columns list.

See Also

TCustomdxDBGrid.AddGroupColumn; TCustomdxDBGrid.InsertGroupColumn; TCustomdxDBGrid.GroupColumns; TCustomdxDBGrid.OnEndDragGroupColumn

OnBackgroundDrawEvent event

Occurs when necessary to draw a group panel's background.

```
type
  TdxDBGridBackgroundDrawEvent = procedure(Sender : TObject; ACanvas :
TCanvas; ARect : TRect ) of object;
property OnBackgroundDrawEvent : TdxDBGridBackgroundDrawEvent;
```

Description

Write an OnBackgroundDrawEvent event handler to draw the group panel in a specific manner as required by your application. ACanvas determines the surface for drawing. ARect specifies the group panel rectangle in grid coordinates. ARect is a clipping rectangle. Everything outside of it will be clipped.

Note: *This event is generated if the ShowGroupPanel is True.*

See Also

TCustomdxTreeListControl.OnCustomDrawCell; TCustomdxTreeListControl.OnCustomDrawBand; TCustomdxTreeListControl.OnCustomDrawCell; TCustomdxTreeListControl.OnCustomDrawFooter; TCustomdxTreeListControl.OnCustomDrawColumnHeader; TCustomdxTreeListControl.OnCustomDrawFooterNode; TCustomdxTreeListControl.OnCustomDrawPreview; TCustomdxTreeListControl.OnCustomDrawPreviewCell.

OnClearNodeData event

Occurs when a grid loads dataset records into nodes (rows).

```
type
  TNotifyEvent = procedure(Sender: TObject) of object;
property OnClearNodeData: TNotifyEvent;
```

Description

Write an OnClearNodeData event handler to release any data, associated with a node via the node Data property.

See Also

TdxTreeListNode.Data

OnColumnClick event

Occurs when clicking a particular cell.

```
type
  TdxDBTreeListColumnClick = procedure(Sender : TObject; Column :
TdxDBTreeListColumn) of object;
property OnColumnClick : TdxDBTreeListColumnClick;
```

Description

Write an OnColumnClick event handler to respond to mouse click events within a column. The Column parameter determines the clicked column.

The following example demonstrates how to perform data sorting using the index of a dataset associated with the ExpressQuantumGrid when it is in standard grid mode. The code below is an OnColumnClick event handler.

```
// Delphi
procedure TForm1.dxDBGrid1ColumnClick(Sender: TObject; Column:
TdxDBTreeListColumn);
begin
  if Column.Sorted <> csDown then
    try
      Table1.IndexFieldNames := Column.FieldName;
      Column.Sorted := csDown;
    except
    end
  else
  begin
    Table1.IndexFieldNames := '';
    Column.Sorted := csNone;
  end;
end;
```

```
// C++Builder
void __fastcall TMainForm::GridColumnClick(TObject *Sender,
TdxDBTreeListColumn *Column)
{
  if (Column->Sorted != csDown)
    try {
      Table->IndexFieldNames = Column->FieldName;
      Column->Sorted = csDown;
        }
     catch (...) {}
}
```

OnColumnSorting event

Occurs when nodes are about to be sorted.

```
type
  TdxDBTreeListColumnSorting = procedure(Sender : TObject; Column :
TdxDBTreeListColumn; var Allow: Boolean) of object;
property OnColumnSorting : TdxDBTreeListColumnSorting;
```

Description

Write an OnColumnSorting event handler to specify whether nodes can be sorted by a particular column. Set the Allow parameter to False to prevent auto sorting. For instance, the following code allows sorting only by the column associated with the 'PHONE' field:

```
// Delphi
procedure TForm1.dxDBGrid1ColumnSorting(Sender: TObject; Column:
TdxDBTreeListColumn; var Allow: Boolean);
begin
  Allow := Column.FieldName = 'PHONE';
end;
```

```
// C++Builder
void __fastcall TForm1::dxDBGrid1ColumnSorting(TObject *Sender,
TdxDBTreeListColumn *Column, bool &Allow)
{
  Allow = (Column->FieldName == "PHONE");
}
```

See Also

TdxDBTreeListColumn.Sorted; TCustomdxTreeListControl.OnColumnSorted

OnCustomDraw event

Occurs when drawing a cell in the ExpressQuantumGrid.

```
type
  TCustomDrawEvent = procedure(Sender : TObject; ACanvas : TCanvas; ARect :
TRect; ANode : TdxTreeListNode; AColumn : TdxDBTreeListColumn; const AText
: String; AFont : TFont; var AColor : TColor; ASelected, AFocused :
Boolean; var ADone : Boolean ) of object;
property OnCustomDraw: TCustomDrawEvent;
```

Description

Write an OnCustomDraw event handler to draw the contents of all or specific cells in the ExpressQuantumGrid. Draw a cell using the TCanvas method. The ARect parameter indicates location of a cell on the canvas:

ANode and **AColumn**	Node and column of a cell.
AText	Context of a cell.
AFont and **AColor**	Font and color used to draw a cell.
ASelected	Determines whether a node to which a cell belongs is selected.
AFocused	Specifies whether a node to which a cell belongs is focused.
ADone	Determines whether the OnCustomDraw event handler completely draws a cell. If ADone is True, the default draw procedure is not called.

If you wish to change the color and font properties and do not want to perform drawing manually, set the ADone parameter to False. If you wish to draw a cell manually, set the ADone parameter to True.

Note: *The OnCustomDraw event is now obsolete. Use the OnCustomDrawCell instead. OnCustomDraw is generated before the OnCustomDrawCell event.*

See Also

TCustomdxDBGrid.OnBackgroundDrawEvent; TCustomdxTreeListControl.OnCustomDrawCell; TCustomdxTreeListControl.OnCustomDrawBand; TCustomdxTreeListControl.OnCustomDrawCell; TCustomdxTreeListControl.OnCustomDrawFooter; TCustomdxTreeListControl.OnCustomDrawColumnHeader; TCustomdxTreeListControl.OnCustomDrawFooterNode; TCustomdxTreeListControl.OnCustomDrawPreviewCell.

OnEndDragGroupColumn event

Occurs when a user places a column onto the group panel.

```
type
  TEndDragGroupColumn = procedure(Sender: TObject; Column:
TdxDBTreeListColumn; NewGroupIndex: Integer; var Allow: Boolean) of object;
property OnEndDragGroupColumn : TEndDragGroupColumn;
```

Description

Write an OnEndDragGroupColumn event handler to control column drag & drop operations within a group panel. The Column parameter determines a column dropped onto the group panel. NewGroupIndex specifies the index of the column among grouping columns. The Allow parameter

determines, whether Column can be dropped onto the group panel. Setting Allow to False within the OnEndDragGroupColumn event handler prevents node grouping by Column.

For instance, the following code prevents node grouping by the 'Phone' column:

```
procedure TForm1.dxDBGrid1EndDragGroupColumn(Sender: TObject;
  Column: TdxDBTreeListColumn; NewGroupIndex: Integer; var Allow: Boolean);
begin
 if Column.Caption = 'Phone' then
   Allow := False;
end;
```

See Also

TCustomdxDBGrid.GroupColumns; TCustomdxDBGrid.GroupColumnCount; TCustomdxDBGrid.AddGroupColumn; TCustomdxDBGrid.InsertGroupColumn; TCustomdxDBGrid.OnReloadGrouplist

OnGetCustomSummaryText event

Occurs when drawing summary text within a group node.

```
type
  TdxCustomSummaryTextEvent = procedure(Sender : TObject; Node:
TdxTreeListNode; var Text: String) of object;
property OnGetCustomSummaryText : TdxCustomSummaryTextEvent;
```

Description

Write an OnGetCustomSummaryText event handler to place additional text next to the summary text of a node. The Node parameter determines the grouping node. Use the Text parameter to specify additional text for the summary.

See Also

TCustomdxDBGrid.OnSummaryNode

OnHideColumn event

Occurs when a user moves a column from the ExpressQuantumGrid to a customizing form.

```
type
  THideColumnEvent = procedure(Sender: TObject; Column :
TdxDBTreeListColumn) of object;
property OnHideColumn : THideColumnEvent;
```

Description

Use the OnHideColumn event handler to perform specific actions when a user drags a column, passed as Column, to the customizing form.

Note: *The OnHideColumn event is included for backward compatibility with ExpressGrid ver. 1.4x. For the ExpressQuantumGrid, you should use OnHideHeader.*

See Also

TCustomdxDBGrid.OnShowColumn; TCustomdxTreeListControl.OnHideHeader

OnReloadGroupList event

Occurs when changing the list of grouped columns.

```
type
  TNotifyEvent = Procedure(Sender: TObject) of object;
property OnReloadGroupList : TNotifyEvent;
```

Description

Generated to any changes in the list of grouping columns, e.g. when a new grouping column is added or an existing grouping column is deleted from the list of grouping colums.

The following code expands all nodes when a user groups by a column:

```
// Delphi
procedure TForm1.dxDBGrid1ReloadGroupList(Sender: TObject);
begin
  (Sender as TdxDBGrid).FullExpand;
end;

// C++Builder
void __fastcall TForm1::dxDBGrid1ReloadGroupList(TObject *Sender)
{
  ((TdxDBGrid*)Sender)->FullExpand();
}
```

See Also

TCustomdxDBGrid.AddGroupColumn; TCustomdxDBGrid.InsertGroupColumn; TCustomdxDBGrid.DeleteGroupColumn; TCustomdxDBGrid.RebuilGroupIndexes; TCustomdxDBGrid.OnEndDragGroupColumn

OnShowColumn event

Occurs when a user moves a column from the customizing form onto the ExpressQuantumGrid.

```
type
  TShowColumnEvent = procedure(Sender: TObject; Column :
TdxDBTreeListColumn) of object;
property OnShowColumn : TShowColumnEvent;
```

Description

Use the OnShowColumn event to perform specific events when a user drags a column, specified by the Column parameter to a grid control.

Note: *The OnShowColumn event is included for backward compatibility with ExpressGrid ver. 1.4x. For the ExpressQuantumGrid, you should use OnShowHeader.*

See Also

TCustomdxDBGrid.OnHideColumn; TCustomdxTreeList.OnShowHeader

OnSummaryNode event

Occurs when calculating summaries.

```
type
  TdxSummaryNodeEvent = procedure(Sender : TObject; Node: TdxTreeListNode;
DataSet : TDataSet; var Value : Extended) of object;
property OnSummaryNode : TdxSummaryNodeEvent;
```

Description

Generated for every dataset record when calculating a summary field. Write an OnSummaryNode event handler to perform calculations based on your specific requirements. Set the Value parameter to a new value, which will be used during summary calculations. The OnSummaryNode event handler is a suitable place for summary calculations against several fields.

Example: If your ExpressQuantumGrid includes a Payment Type and Payments column, and when grouping by Payment Type you wish to calculate the number of payments exceeding $100, you must set SummaryType of PaymentColumn to cstSum and write the following event handler:

```
procedure TForm1.dxDBGrid1Summary (Sender : TObject; DataSet : TDataSet;
var Value : Extended);
begin
  if DataSet.FindField('Payments').AsFloat > 100 then
    Value := 1
  else
    Value := 0;
end;
```

Copyright © 2000 Developer Express Inc.

TdxDBGrid

TdxDBGrid is a data-aware object that manages the display and manipulation of data in an intuitive and straightforward manner.

Unit

dxDBGrid

```
TdxDBGrid = class(TCustomdxDBGrid);
```

Description

TdxDBGrid is derived directly from TCustomdxDBGrid. This is a 'published' variant of TCustomdxDBGrid. Additionally, TdxDBGrid contains two powerful functions that allow you to export grid nodes directly to a HTML and Microsoft Excel™ file.

***Note**: Inplace and standalone editors of the ExpressQuantumGrid Suite require Windows RichEdit component 2.0 or higher to work properly. If the editors on your or client's machine do not work, make certain that the riched20.dll library is in the Windows system folder (Windows\System or WINNT\System32). Copy the specified DLL to fix the problem.*

Hierarchy

TObject
- TPersistent
  - TComponent
    - TControl
      - TWinControl
        - TCustomControl
          - TCustomdxContainer
            - TCustomdxTreeList
              - TCustomdxTreeListControl
                - TCustomdxDBTreeListControl
                - TCustomdxDBGrid

TdxDBGrid Properties

Derived from TCustomdxDBGrid

DataSetChangedBusy	**public / read only**	OptionsCustomize	**published / read / write**
DefaultFields	**published / read / write**	OptionsDB	**published / read / write**
Filter	**published / read / write**	OptionsEx	**published / read / write**
GroupColumnCount	**public / read only**	OptionsView	**published / read / write**
GroupColumns	**public / read only**	PartialLoad	**published / read / write**
GroupPanelColor	**published / read / write**	PartialLoadBufferCount	**published / read / write**
GroupPanelFontColor	**published / read / write**	ShowGroupPanel	**published / read / write**
GroupPanelHeight	**public / read only**	ShowSummaryFooter	**published / read / write**
LockRefresh	**public / read / write**	SummaryGroups	**published / read / write**
Options	**published / read / write**	SummarySeparator	**published / read / write**
OptionsBehavior	**published / read / write**	UseBookmarks	**published / read / write**

Derived from TCustomdxDBTreeListControl

Columns	**public / read / write**	PreviewFieldName	**published / read / write**
DataLink	**public / read only**	PreviewMaxLength	**published / read / write**
DataSource	**published / read / write**	SelectedRows	**public / read only**
FocusedField	**published / read / write**	SortedColumn	**public / read only**
GrIndicatorWidth	**published / read / write**	SortedColumns	**public / read only**
KeyField	**published / read / write**	VisibleColumns	**public / read / write**

Derived from TCustomdxTreeListControl

BandMaxRowCount	**published / read / write**	HeaderPanelRowCount	**published / read / write**
BandRowCount	**published / read / write**	IniFileName	**published / read / write**
Bands	**published / read / write**	IniSectionName	**published / read / write**
ColumnCount	**public / read only**	IsCustomStoring	**protected / read / write**
DefaultLayout	**published / read / write**	RegistryPath	**published / read / write**
HeaderMinRowCount	**published / read / write**	SortedColumnCount	**public / read only**
HeaderPanelMaxRowCount	**published / read / write**	VisibleColumnCount	**public / read only**

Derived from TCustomdxTreeList

ArrowsColor	**published / read / write**	CustomizingForm	**public / read only**
AutoExpandOnSearch	**published / read / write**	CustomizingHeaderListBox	**public / read only**
AutoSearchColor	**published / read / write**	CustomizingPos	**published / read / write**
AutoSearchTextColor	**published / read / write**	CustomizingRowCount	**published / read / write**
BandButtonPushed	**public / read only**	DblClkExpanding	**published / read / write**
BandColor	**published / read / write**	DefaultRowHeight	**published / read / write**
BandFont	**published / read / write**	DescTextHeight	**public / read only**
BandPanelHeight	**public / read only**	DownBandIndex	**public / read only**
BorderStyle	**published / read / write**	DownBandPushed	**public / read only**
Count	**public / read only**	DownColumnIndex	**public / read only**
CustomizingBandListBox	**public / read only**	DownColumnPushed	**public / read only**

Copyright © 2000 Developer Express Inc.

DragAbsoluteBandIndex	**public / read only**
DragAbsoluteHeaderIndex	**public / read only**
DragNode	**protected read only**
EditingText	**published / read / write**
FixedBandLineColor	**published / read / write**
FixedBandLineWidth	**published / read / write**
FocusedAbsoluteIndex	**public / read only**
FocusedColumn	**published / read / write**
FocusedNode	**public / read only**
FocusedNumber	**protected / read / write**
FooterPanelHeight	**public / read only**
FooterRowHeight	**public / read only**
FooterRowNodeHeight	**public / read only**
GridLineColor	**published / read / write**
GroupNodeColor	**published / read / write**
GroupNodeTextColor	**published / read / write**
HeaderButtonPushed	**public / read only**
HeaderColor	**published / read / write**
HeaderFont	**published / read / write**
HeaderHeight	**public / read only**
HeaderPanelHeight	**public / read only**
HeaderRowHeight	**public / read only**
HideFocusRect	**published / read / write**
HideSelection	**published / read / write**
HideSelectionColor	**published / read / write**
HideSelectionTextColor	**published / read / write**
HighlightColor	**published / read / write**
HighlightTextColor	**published / read / write**
Images	**protected / read / write**
Indent	**protected / read only**
IndentDesc	**published / read / write**
InplaceColumnIndex	**public / read only**
InplaceEditor	**public / read only**
IsCustomizing	**public / read only**
Items	**public / read only**
LastNode	**public / read only**
LeftCoord	**published / read / write**
LockSelection	**public / read only**
LockUpdate	**protected / read only**
LookAndFeel	**published / read / write**
MaxRowLineCount	**published / read / write**
NewItemRowActive	**protected / read / write**
NewItemRowHeight	**public / read only**
PaintStyle	**protected / read / write**
PreviewFont	**published / read / write**
PreviewLines	**published / read / write**
RowCount	**protected / read only**
RowFooterColor	**published / read / write**
RowFooterTextColor	**published / read / write**
RowHeight	**public / read only**
RowSeparatorLineWidth	**published / read / write**
ScrollBars	**published / read / write**
SelectedCount	**public / read only**
SelectedNodes	**public / read only**
SelectionAnchor	**protected / read / write**
ShowBands	**published / read / write**
ShowButtons	**protected / read / write**
ShowFooter	**protected / read / write**
ShowGrid	**published / read / write**
ShowHeader	**published / read / write**
ShowHiddenInCustomizeBox	**published / read / write**
ShowIndicator	**protected / read / write**
ShowLines	**protected / read / write**
ShowNewItemRow	**published / read / write**
ShowPreviewGrid	**published / read / write**
ShowRoot	**protected / read / write**
ShowRowFooter	**published / read / write**
SimpleCustomizeBox	**published / read / write**
State	**public / read only**
StateImages	**protected / read / write**
TabStop	**published / read / write**
TopIndex	**public / read / write**
TopNode	**public / read only**
TopVisibleNode	**public / read only**
TreeLineColor	**protected / read / write**
TreeLineStyle	**protected / read / write**
VisibleRowCount	**protected / read only**
WaitForExpandNodeTime	**published / read / write**

Derived from TCustomdxContainer

IsInitEdit	**public / read only**
ReadOnly	**public / read only**

TdxDBGrid Methods

TdxDBGrid

SaveToHTML	**public**	SaveToXLS	**public**
SaveToText	**public**	SaveToXML	**public**

Derived from TCustomdxDBGrid

AddGroupColumn	**public**	IsGridMode	**public**
BeginGrouping	**public**	IsLoadAllRecords	**public**
CanAddGroupColumn	**public**	IsPartialLoad	**public**
DeleteGroupColumn	**public**	PointInFooterPanel	**public**
EndGrouping	**public**	PointInGroupPanel	**public**
FindNodeByKeyValue	**public**	RebuildGroupIndexes	**public**
GetGroupColumnAt	**public**	RefreshGroupColumns	**public**
GetNodeFooterColumnAt	**public**	ResetFullRefresh	**public**
GetSummaryGroupByName	**public**	SetFlagSmartReload	**public**
GetSummaryItemAt	**public**		
InsertGroupColumn	**public**		

Derived from TCustomdxDBTreeListControl

ClearGroupColumns	**public**	GetColumnAt	**public**
ColumnByFieldName	**public**	GetDefaultColumnClass	**public**
ColumnByName	**public**	GetDefaultFieldColumnClass	**public**
CreateColumn	**public**	GetFooterColumnAt	**public**
CreateColumnEx	**public**	GetHeaderColumnAt	**public**
FindColumnByFieldName	**public**	RefreshBookmarks	**public**
FullRefresh	**public**		

Derived from TCustomdxTreeListControl

ApplyBestFit	**public**	GetFooterColumnAt	**public**
AssignColumns	**public**	GetHeaderColumnAt	**public**
BeginSorting	**public**	GetNodeFooterColumnAt	**public**
ColumnByName	**public**	GetVisibleColumnIndex	**public**
CreateColumn	**public**	LoadFromIniFile	**public**
CreateColumnEx	**public**	LoadFromRegistry	**public**
DestroyColumns	**public**	RefreshDefaultColumnsWidths	**public**
EndSorting	**public**	RefreshSorting	**public**
GetAbsoluteColumnIndex	**public**	RestoreColumnsDefaults	**public**
GetBandAt	**public**	SaveToIniFile	**public**
GetColumnAt	**public**	SaveToRegistry	**public**
GetDisplayValue	**public**		

Derived from TCustomdxTreeList

Add	**public**	BeginUpdate	**public**
AddFirst	**public**	CalcDrawInfo	**public**
AdjustColumnsWidth	**public**	CancelEditor	**public**
BeginSelection	**public**	CellRect	**public**

Copyright © 2000 Developer Express Inc.

ClearColumnsSorted	**public**	GotoNext	**public**
ClearNodes	**public**	GotoPrev	**public**
ClearSelection	**public**	HideEditor	**public**
CloseEditor	**public**	IndexOf	**public**
ColumnsCustomizing	**public**	Insert	**public**
CopyAllToClipboard	**public**	InvalidateSelection	**protected**
CopySelectedToClipboard	**public**	IsInvertSelect	**public**
Create	**public**	IsNodeVisible	**public**
DeleteSelection	**public**	MakeBandVisible	**public**
Destroy	**public**	MakeColumnVisible	**public**
EndColumnsCustomizing	**public**	MakeNodeVisible	**public**
EndSearch	**public**	PointInCustomizingForm	**public**
EndSelection	**public**	SaveAllToStrings	**public**
EndUpdate	**public**	SaveAllToTextFile	**public**
FindInplaceEdit	**protected**	SaveSelectedToStrings	**public**
FindNodeByText	**protected**	SaveSelectedToTextFile	**public**
FreeDrawInfo	**public**	SelectNodes	**protected**
FullCollapse	**public**	SetEditingText	**protected**
FullExpand	**public**	SetEditText	**protected**
GetEditRect	**public**	ShowButtonEditorMouse	**public**
GetFocusedAbsoluteIndex	**public**	ShowEditor	**public**
GetFocusedVisibleIndex	**public**	ShowEditorChar	**public**
GetHitInfo	**public**	ShowEditorMouse	**public**
GetHitTestInfoAt	**public**	StartDragBand	**protected**
GetNextVisible	**public**	StartDragHeader	**protected**
GetNodeAt	**public**	StartSearch	**public**
GetScrollVertGridRect	**protected**	UpdateScrollBars	**protected**
GotoFirst	**public**		
GotoLast	**public**		

Derived from TCustomdxContainer

BeginInitEdit	**protected**	GetReadOnly	**protected**
CanEditAcceptKey	**protected**	HideDrawFocusRect	**protected**
CanEditModify	**protected**	InplaceEditKeyDown	**protected**
CheckEditorWidth	**protected**	IsEditorMoved	**protected**
CreateEditStyle	**protected**	IsImmediateEditor	**protected**
EndInitEdit	**protected**	IsShowButtonAlways	**protected**
GetDefaultPopupBorderStyle	**public**	IsTabs	**protected**
GetEditColor	**protected**	MoveCol	**protected**
GetEditFont	**protected**	RedrawSelection	**protected**

SaveToHTML method

Exports grid nodes to a HTML file.

```
procedure SaveToHTML(const AFileName : String; ASaveAll: Boolean);
```

Description

Saves grid nodes to a HTML file, specified by AFileName. If ASaveAll is True, SaveToHTML saves all nodes to a HTML file; Else, it saves only the selected nodes. If a grid does not include selected nodes, SaveToHTML saves all nodes.

```
// Example
procedure TfmMain.Button1Click(Sender: TObject);
begin
  dxDBGrid.SaveToHTML('htmlfile.htm', False);
end;
```

SaveToText method

Exports grid nodes (rows) to a text file.

```
procedure SaveToText(const AFileName: String; ASaveAll: Boolean; const
ASeparator, ABeginString, AEndString: String);
```

Description

Call the SaveToText method to saves grid nodes to a text file, specified by the AFileName parameter. If the ASaveAll parameter is **True**, the SaveToText method saves all nodes to a file; Else, it saves only the selected nodes. If a grid does not include selected nodes, the SaveToText method saves all nodes. The ASeparator parameter specifies the separator between columns. The ABeginString and AEndString parameters specify the beginning and ending symbol strings for cell values accordingly.

```
// Delphi
procedure TfmMain.Button1Click(Sender: TObject);
begin
  dxDBGrid.SaveToText('c:\dbgrid.txt', True, #9, '|', '|');
end;

// C++Builder
void __fastcall TForm1::Button1Click(TObject *Sender)
{
  dxDBGrid->SaveToText("c:\dbgrid.txt", True, #9, "|", "|");
}
```

SaveToXLS method

Exports grid nodes (rows) to a Microsoft Excel(r) file.

```
procedure SaveToXLS(const AFileName : String; ASaveAll: Boolean);
```

Description

Exports grid nodes to a Microsoft Excel(r) file, specified by AFileName. If ASaveAll is True, SaveToXLS saves all nodes to an xls file. Otherwise, it saves only selected nodes. If a grid does not include selected nodes, SaveToXLS saves all nodes.

```
// Delphi
procedure TfmMain.Button1Click(Sender: TObject);
begin
  dxDBGrid.SaveToXLS('xlsfile.xls', False);
end;

// C++Builder
void __fastcall TForm1::Button1Click(TObject *Sender)
{
  dxDBGrid1->SaveToXLS("xlsfile.xls",true);
}
```

SaveToXML method

Exports grid nodes (rows) to a XML file.

```
procedure SaveToXML(const AFileName: String; ASaveAll: Boolean);
```

Description

Call the SaveToXML method to saves grid nodes to a XML file, specified by the AFileName parameter. If the ASaveAll parameter is **True**, the SaveToXML method saves all nodes to a XML file; Else, it saves only the selected nodes. If a grid does not include selected nodes, the SaveToXML method saves all nodes.

```
// Delphi
procedure TfmMain.Button1Click(Sender: TObject);
begin
  dxDBGrid.SaveToXML('xmlfile.xml', True);
end;

// C++Builder
void __fastcall TForm1::Button1Click(TObject *Sender)
{
  dxDBGrid->SaveToHTML("xmlfile.xml", True);
}
```

TdxDBGrid Events

TCustomdxDBGrid

OnAddGroupColumn	**published**	OnEndDragGroupColumn	**published**
OnBackgroundDrawEvent	**published**	OnGetCustomSummaryText	**published**
OnClearNodeData	**published**	OnHideColumn	**published**
OnColumnClick	**published**	OnReloadGroupList	**published**
OnColumnSorting	**published**	OnShowColumn	**published**
OnCustomDraw	**published**	OnSummaryNode	**public**

Derived from TCustomdxDBTreeListControl

OnBeforeCalcSummary	**published**	OnGetEditColor	**published**
OnCalcSummary	**published**	OnGetPreviewText	**published**
OnChangeNodeEx	**published**	OnRefreshNodeData	**published**

Derived from TCustomdxTreeListControl

OnBandButtonClick	**published**	OnDragEndHeader	**published**
OnBandClick	**published**	OnDragOverBand	**published**
OnCalcRowLineHeight	**published**	OnDragOverHeader	**published**
OnCanBandDragging	**published**	OnEndDragColumn	**published**
OnCanHeaderDragging	**published**	OnGetImageIndex	**protected**
OnChangedColumnsWidth	**published**	OnGetLevelColor	**published**
OnColumnClick	**published**	OnGetNodeDragText	**published**
OnColumnMoved	**published**	OnGetSelectedIndex	**protected**
OnColumnSorted	**protected**	OnGetStateIndex	**protected**
OnColumnSorting	**published**	OnHeaderButtonClick	**published**
OnCustomDraw	**published**	OnHeaderMoved	**published**
OnCustomDrawBand	**published**	OnHideBand	**published**
OnCustomDrawCell	**published**	OnHideHeader	**published**
OnCustomDrawColumnHeader	**published**	OnIsExistFooterCell	**published**
OnCustomDrawFooter	**published**	OnShowBand	**published**
OnCustomDrawFooterNode	**published**	OnShowHeader	**published**
OnCustomDrawPreview	**published**	OnStartBandDragging	**published**
OnCustomDrawPreviewCell	**published**	OnStartHeaderDragging	**published**
OnDragEndBand	**published**		

Derived from TCustomdxTreeList

OnBeginDragNode	**published**	OnEditChange	**published**
OnCanNodeSelected	**published**	OnEdited	**published**
OnChangeColumn	**published**	OnEditing	**published**
OnChangeLeftCoord	**published**	OnEditValidate	**published**
OnChangeNode	**published**	OnEndColumnsCustomizing	**published**
OnChangeTopVisibleNode	**published**	OnExpanded	**published**
OnCollapsed	**published**	OnExpanding	**published**
OnCollapsing	**published**	OnGetEditColor	**published**
OnCompare	**published**	OnHotTrackNode	**published**
OnDeletion	**published**	OnSelectedCountChange	**published**

Copyright © 2000 Developer Express Inc.

TdxDBGridFilter

The TdxDBGrid class is designed to implement the filtering functionality introduced in ExpressQuantumGrid v3.

Unit

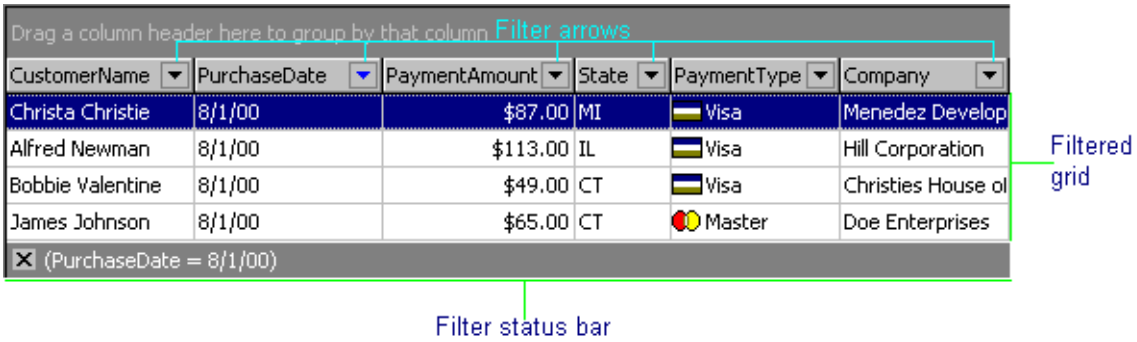
dxDBGrid

```
TdxDBGridFilter = class(TPersistent);
```

Description

MS Excel® style filtering is available within the ExpressQuantumGrid. Filtering is a quick way to find a subset of data within a dataset. When using **LoadAllRecords** mode, the QuantumGrid automatically manages all the filtering functionality for you. When working in **Grid** mode, the OnFilterChanged event allows you to filter your dataset by modifying its Filter property when required.

When using the filtering capabilities of the ExpressQuantumGrid, arrows are drawn to the right of column captions. To initiate a filter, click the appropriate arrow and select the necessary value to filter the contents of your grid. Filter arrows are displayed in blue in the columns that contain selected values. A filtered grid displays only the nodes that contain a specified value. The grid pictured below is filtered to display records with a purchase date of 8/11/00 only:

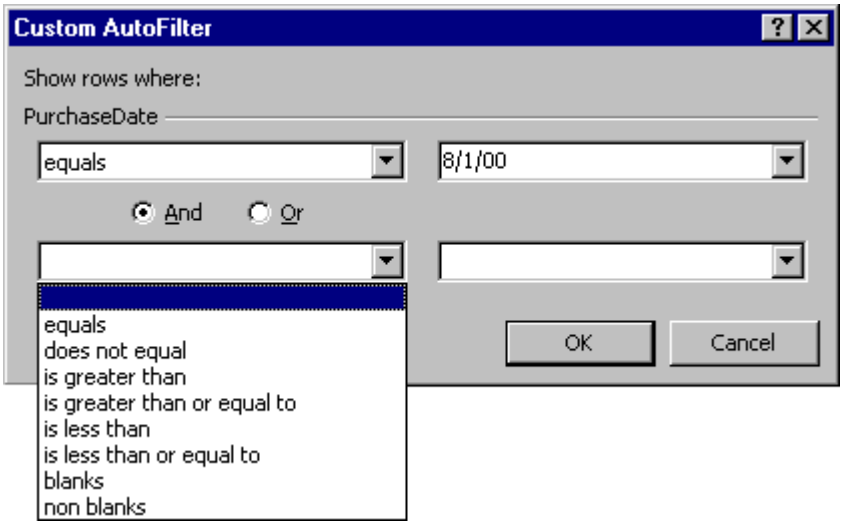


You can filter a value by more than one column simultaneously. The ExpressQuantumGrid supports the following filter criteria:

Value	Description
All	Displays all nodes.
Custom	Applies two criteria values within the current column using the comparison operators AND or OR.
Blanks	Displays only the nodes that contain a **False** value in a column.
NonBlanks	Displays only the nodes that contain a **True** value in a column.
Value	Displays only the nodes that contain the selected value.

***Note**: The Blanks and NonBlanks options are available only if the column you want to filter contains a blank cell.*

The Custom AutoFilter window allows you to determine two conditions for a column. The editor type for a value is determined by column type (for date columns, it is a date editor, for pick columns, it is a pick editor, etc.):



Hierarchy

TObject

TPersistent

TdxDBGridFilter Properties

TdxDBGridFilter

Active	**published**	FilterCaption	**public / read only**
AutoDataSetFilter	**published**	FilterStatus	**published**
CaseInsensitive	**published**	FilterText	**public / read only**
DropDownCount	**published**	MaxDropDownCount	**published**

Active property — read / write

Enables the ExpressQuantumGrid filtering feature.

```
property Active: Boolean;
```

Description

Use the Active property to include filtering within a given QuantumGrid. Set this property to **True** to display filter arrows to the right of column captions.

AutoDataSetFilter property — read / write

Sets the filter for a grid's dataset.

```
property AutoDataSetFilter: Boolean;
```

Description

If the AutoDataSetFilter property is set to **True**, the FilterText property value is assigned to the Filter property of the appropriate dataset. If this property is set to **False** and the ExpressQuantumGrid is in **LoadAllRecords**, the FilterText property is applied directly to the grid and can be used within a SQL statement.

CaseInsensitive property — read / write

Specifies whether filtering is case sensitive.

```
property CaseInsensitive: Boolean;
```

Description

If the CaseInsensitive property is set to **True**, filtering does not distinguish between capital and small letters.

***Note**: If the **AutoDataSetFilter** is **True**, the **CaseInsensitive** property setting is ignored.*

DropDownCount property — read / write

Determines the number of visible rows within a dropdown filter window.

```
property DropDownCount: Integer;
```

Description

Use the DropDownCount property to set the number of visible rows within a dropdown filter window. If the DropDowCount property is less than the total number of items to drop down, a vertical scrollbar appears to the right of the dropdown filter window. The default value for the DropDownCount property is 12.

FilterCaption property — read only

Determines the caption of the filter status bar.

```
property FilterCaption: string;
```

Description

The FilterCaption property is used to display the current (applied) filter conditions within the filter status bar. Unlike the FilterText property, the FilterCaption property can only be used for display purposes.

FilterCaption	`(State = CA) AND (PurchaseDate = 01-Mar-1976)`
FilterText	`(State = 'CA') AND (PurchaseDate = '01-Mar-1976')`

FilterStatus property — read / write

Specifies whether a filter status bar is displayed.

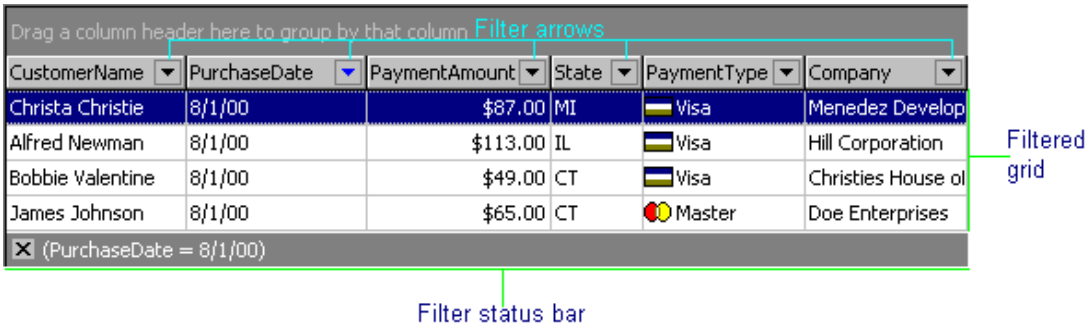
```
type
  TdxDBGridFilterStatus = (fsAuto, fsNone, fsAlways);
property FilterStatus: TdxDBGridFilterStatus;
```

Description

Possible values include:

Value	Description
fsAuto	The filter status bar is displayed only when the FilterCaption property value is not empty.
fsNone	The filter status bar is never displayed.
fsAlways	The filter status bar is always displayed regardless of whether a filter is active.

Copyright © 2000 Developer Express Inc.



FilterText property — read only

Determines filter conditions.

```
property FilterText: string;
```

Description

When filtering is applied to a grid, only nodes that meet filter conditions are displayed. The FilterText property contains a string that describes filter conditions. Filter condition values are obtained from the appropriate data fields. Unlike the FilterCaption property, the FilterText property can be applied to a dataset or a SQL statement.

MaxDropDownCount property — read / write

Determines the total number of rows within a dropdown filter window.

```
property MaxDropDownCount: Integer;
```

Description

Use the MaxDropDownCount property to set the number of rows within a dropdown filter window. This property is useful when the total number of possible values is large and only a subset of the values are needed by the end user. The default value for the MaxDropDownCount property is 1000.

FilterCaption	`(State = CA) AND (PurchaseDate = 01-Mar-1976)`
FilterText	`(State = 'CA') AND (PurchaseDate = '01-Mar-1976')`

TdxDBGridFilter Methods

TdxDBGridFilter

Add	**public**	Remove	**public**
AddNull	**public**	ShowCustomDialog	**public**
Clear	**public**		

Add method

Adds a filter condition for a specific column.

```
procedure Add(AColumn: TdxDBTreeListColumn; const AValue: Variant; const
ADisplayValue: string);
```

Description

Call the Add method to add filtering for the column specified by the AColumn parameter. The AValue parameter determines the exact value filtered nodes should contain. The ADisplayValue parameter determines the filter value as it is displayed within the filter status bar.

AddNull method

Adds a filter condition for a specific column with a blank cell(s).

```
procedure AddNull(AColumn: TdxDBTreeListColumn; IsNot: Boolean);
```

Description

Call the AddNull method to add filtering for the column specified by the AColumn parameter. This method is used to set a filter for a column with a blank cell(s). The IsNot parameter determines whether to display nodes with blank cells. If this parameter is **True**, non-blank cells are displayed.

Clear method

Removes all filter conditions applied to the current grid.

```
procedure Clear;
```

Description

Call the Clear method to remove the filters for all columns at once.

Remove method

Removes the filter conditions applied to a specific column.

```
procedure Remove(AColumn: TdxDBTreeListColumn);
```

Description

Call the Remove method to remove the filter for the column specified by the AColumn parameter.

Copyright © 2000 Developer Express Inc.

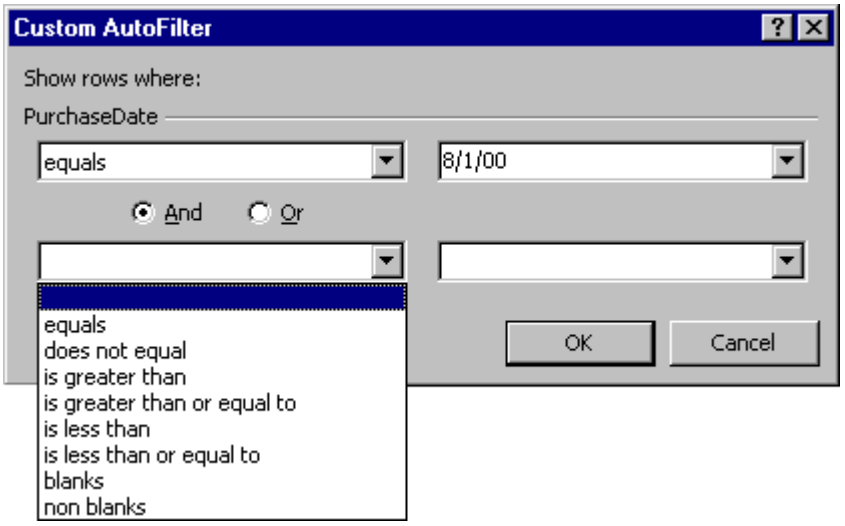
ShowCustomDialog method

Activates the Custom AutoFilter window.

```
procedure ShowCustomDialog(AColumn: TdxDBTreeListColumn);
```

Description

Call the ShowCustomDialog method to set a custom filter for the column specified by the AColumn parameter. The Custom AutoFilter window allows you to determine two conditions for a column. The editor type for a value is determined by column type (for date columns, it is a date editor; for pick columns, it is a pick editor, etc.):



Index

A

Active....231
Add.... 52, 234
AddFirst.... 53
AddGroupColumn....205, 211
AddNull....234
AdjustColumnsWidth.... 53
aoAnsiSort....36, 37
aoAutoCalcPreviewLines....36, 37
aoAutoHeaderPanelHeight....36, 37
aoAutoSearch.... 37
aoAutoSort....34, 35
aoAutoSortRefresh....36, 37
aoAutoWidth....34, 35
aoBandButtonClicking....36, 37
aoBandHeaderWidth....36, 37
aoBandMoving....36, 37
aoBandPanelSizing....36, 37
aoBandSizing....36, 37
aoCaseInsensitive.... 35
aoCellMultiSelect.... 36
aoColumnMoving....34, 35
aoColumnSizing.... 35
aoDragCollapse....36, 37
aoDragExpand....36, 37
aoDragScroll....36, 37
aoDrawEndEllipsis....34, 35
aoEditing....34, 35
aoEnterShowEditor.... 36
aoEnterThrough.... 36
aoExtCustomizing....34, 35
aoExtMultiSelect....34, 35
aoFullSizing.... 36
aoHeaderButtonClicking.... 36
aoHeaderPanelSizing....36, 37
aoHideFocusRect....34, 35
aoHorzThrough.... 36
aoImmediateEditor....34, 35
aoInvertSelect.... 36
aoKeepColumnWidth....36, 37
aoMouseScroll....36, 37
aoMultiSelect....34, 35
aoMultiSort....36, 37
aoPreview....34, 35
aoRowAutoHeight....36, 37
aoRowSelect....34, 35
aoRowSizing....36, 37
aoShowButtonAlways....36, 37
aoShowHourGlass....36, 37
aoTabs....34, 35
aoTabThrough....34, 35
aoUseBitmap.... 36
aoVertThrough.... 36
ApplyBestFit.... 97
ArrowsColor.... 15
AssignColumns.... 97
AssignNodes....135
AutoDataSetFilter....231
AutoExpandOnSearch.... 15
AutoSearchColor.... 15
AutoSearchTextColor.... 16

B

BandButtonPushed.... 16
BandColor.... 16
BandFont.... 16
BandMaxRowCount.... 90
BandPanelHeight.... 17
BandRowCount.... 90
Bands.... 91
BeginGrouping....206
BeginInitEdit.... 8
BeginLayout....173
BeginSelection.... 53
BeginSorting.... 98
BeginUpdate.... 54
BorderStyle.... 17

C

CalcDrawInfo.... 54
CanAddGroupColumn....206
CancelEditor.... 54
CanEditAcceptKey.... 8
CanEditModify.... 8
CaseInsensitive....231
CellRect.... 55
CheckEditorWidth.... 9
Clear....234
ClearColumnsSorted.... 55
ClearGroupColumns....150
ClearNodes.... 55
ClearSelection.... 55
CloseEditor.... 56
ColumnByFieldName....150
ColumnByName.... 98
ColumnCount.... 91
Columns.... 91, 144
ColumnsCustomizing.... 56
CopyAllToClipboard.... 56
CopySelectedToClipboard....56, 57
Count.... 17
Create.... 57
CreateColumn.... 98
CreateColumnEx.... 99, 152
CreateEditStyle.... 9
CustomizingBandListBox.... 17
CustomizingForm.... 18
CustomizingHeaderListBox.... 18
CustomizingPos.... 18
CustomizingRowCount.... 19

D

Data....215
DataLink....145
DataSetChangedBusy....185
DataSource....145
DblClkExpanding.... 19
DefaultFields....186
DefaultLayout.... 91
DefaultRowHeight.... 19
DeleteGroupColumn....206, 211
DeleteSelection.... 57
DescTextHeight....19, 20
Destroy.... 57

DestroyColumns....99
DownBandIndex....20
DownBandPushed....20
DownColumnIndex....20
DownColumnPushed....21
DragAbsoluteBandIndex....21
DragAbsoluteHeaderIndex....21
DragNode....22
DropDownCount....231, 232

E

EditingText....22
egoAnsiSort....196, 198
egoAutoCalcPreviewLines....196, 197
egoAutoHeaderPanelHeight....196, 198
egoAutoSearch....198
egoAutoSort....196, 198, 199
egoAutoWidth....188, 190
egoAutoWidths....198
egoBandButtonClicking....196, 197
egoBandHeaderWidth....196, 197, 199
egoBandMoving....196, 197, 199
egoBandPanelSizing....196, 197
egoBandSizing....196, 197, 199
egoCanAppend....188, 189
egoCancelOnExit....188, 189
egoCanDelete....188, 189
egoCanInsert....188, 189
egoCanNavigation....188, 189
egoCaseInsensitive....188, 190
egoCellMultiSelect....196, 197
egoCollapsedReload....196, 198
egoColumnMoving....188, 189, 191
egoColumnSizing....189, 191
egoConfirmDelete....188, 189
egoDblClick....188, 190
egoDragCollapse....196, 197
egoDragExpand....196, 197
egoDragScroll....196, 197
egoDrawEndEllipsis....188, 190
egoEditing....188, 189
egoEnterShowEditor....196, 197, 199
egoEnterThrough....196, 197
egoExtCustomizing....188, 190
egoExtMultiSelect....188, 190
egoFullSizing....196, 197
egoHeaderButtonClicking....196, 197
egoHeaderPanelSizing....196, 197
egoHideFocusRect....188, 190
egoHorzThrough....196, 197
egoImmediateEditor....188, 189, 191
egoIndicator....188, 190
egoInvertSelect....196
egoKeepColumnWidth....196, 198
egoLoadAllRecords....186, 188, 190, 200, 210
egoMouseScroll....196, 198
egoMultiSelect....188, 189, 190
egoMultiSort....196, 197
egoNotHideColumn....196, 198
egoPreview....188, 190
egoResetColumnFocus....188, 190
egoRowAutoHeight....196, 197
egoRowSelect....188, 189
egoRowSizing....196, 197
egoSeekDetail....191
egoShowButtonAlways....196, 198
egoShowHourGlass....188, 190
egoSmartRefresh....189, 191, 212
egoSmartReload....189, 190
egoStoreToIniFile....102, 189, 190
egoStoreToRegistry....94, 188, 190
egoSyncSelection....196, 198
egoTabs....188, 189
egoTabThrough....188, 189
egoUseBitmap....196
egoUseLocate....189, 190, 191
egoVertThrough....196, 197
EndColumnsCustomizing....57
EndGrouping....206, 207
EndInitEdit....9
EndSearch....58
EndSelection....58
EndSorting....99
EndUpdate....58, 59
ExpressSupportForum....5

F

Filter....186, 229, 231
FilterCaption....232, 233
FilterStatus....232
FilterText....231, 232, 233
FindColumnByFieldName....152
FindInplaceEdit....59
FindNodeByKeyValue....207
FindNodeByText....59
FixedBandLineColor....22
FixedBandLineWidth....23
FocusedAbsoluteIndex....23
FocusedColumn....23
FocusedField....145
FocusedNode....23
FocusedNumber....24
FooterPanelHeight....24
FooterRowHeight....24
FooterRowNodeHeight....25
FreeDrawInfo....59
FullCollapse....60
FullExpand....60

G

GetAbsoluteColumnIndex....100
GetBandAt....100
GetColumnAt....100
GetDefaultPopupBorderStyle....9
GetDisplayValue....101
GetEditColor....9, 10
GetEditFont....10
GetEditRect....60
GetFocusedAbsoluteIndex....62
GetFocusedVisibleIndex....62
GetFooterColumnAt....101
GetGroupColumnAt....207
GetHeaderColumnAt....101, 154
GetHitInfo....62, 63
GetHitTestInfoAt....66
GetNextVisible....66
GetNodeAt....66
GetNodeFooterColumnAt....101, 102, 208
GetReadOnly....10
GetScrollVertGridRect....66, 67
GetSummaryGroupByName....208
GetSummaryItemAt....209

Getting Support

....5
GetVisibleColumnIndex....102
GotoFirst....67
GotoLast....67
GotoNext....67
GotoPrev....67
grid....203, 215
grid mode....209
GridLineColor....25
GrIndicatorWidth....146
GroupColumnCount....186
GroupColumns....187
GroupNodeColor....25
GroupNodeTextColor....26
GroupPanelColor....187

GroupPanelFontColor....187
GroupPanelHeight....188

H

HeaderButtonPushed.... 26
HeaderColor.... 26
HeaderFont.... 27
HeaderHeight.... 27
HeaderMinRowCount.... 92
HeaderPanelHeight.... 27
HeaderPanelMaxRowCount.... 92
HeaderPanelRowChange....198
HeaderPanelRowCount.... 92
HeaderRowHeight.... 27
HideDrawFocusRect.... 10
HideEditor.... 68
HideFocusRect.... 28
HideSelection.... 28
HideSelectionColor.... 28
HideSelectionTextColor.... 29
HighlightColor.... 29
HighlightTextColor.... 29

I

Images.... 29
Indent.... 30
IndentDesc.... 30
IndexOf.... 68
IniFileName.... 93, 190
IniSectionName.... 93
InplaceColumnIndex.... 30
InplaceEditKeyDown.... 10
InplaceEditor.... 30
Insert.... 68
InsertGroupColumn....209, 211
InvalidateSelection.... 69
IsCustomizing.... 31
IsCustomStoring.... 93
IsEditorMoved.... 11
IsGridMode....209, 210
IsImmediateEditor.... 11
IsInitEdit....7, 8, 9
IsInvertSelect.... 69
IsLoadAllRecords....210
IsNodeVisible.... 69
IsPartialLoad....210
IsShowButtonAlways.... 11
IsTabs....11, 12
Items.... 31

K

KeyField....146

L

LastNode.... 31
LeftCoord.... 32
LoadAllRecords....186, 189, 190, 191, 197, 198, 210, 212
LoadFromFile....135
LoadFromIniFile....102
LoadFromRegistry....102
LoadFromStream....135
LockRefresh....188
LockSelection.... 32
LockUpdate....32, 33
LookAndFeel.... 33

M

MakeBandVisible.... 69
MakeColumnVisible.... 70
MakeNodeVisible.... 70
MaxDropDownCount....233
MaxRowLineCount.... 33
MoveCol.... 12
MS Outlook....196

N

NewItemRowActive.... 34
NewItemRowHeight.... 34

O

OnAddGroupColumn....214
OnBackgroundDrawEvent....214
OnBandButtonClick....108, 197
OnBandClick....108
OnBeginDragNode.... 76
OnCalcRowLineHeight....109
OnCalcSummary....157
OnCanBandDragging....109, 110
OnCanHeaderDragging....110
OnCanNodeSelected.... 76
OnChangeColumn.... 77
OnChangedColumnsWidth....110
OnChangeLeftCoord.... 77
OnChangeNode.... 78
OnChangeNodeEx....158
OnChangeTopVisibleNode.... 78
OnClearNodeData....214, 215
OnCollapsed.... 78
OnCollapsing.... 79
OnColumnClick.... 110, 111, 215
OnColumnMoved....111
OnColumnSorted....111
OnColumnSorting....112, 216
OnCompare.... 79
OnCustomDraw.... 112, 113, 216, 217
OnCustomDrawBand....113, 114
OnCustomDrawCell....114, 115
OnCustomDrawColumnHeader....115, 116
OnCustomDrawFooter....116
OnCustomDrawFooterNode....117
OnCustomDrawPreview....117, 118
OnCustomDrawPreviewCell....118, 119
OnDeletion.... 81
OnDragEndBand....119
OnDragEndHeader....119
OnDragOverBand....120
OnDragOverHeader....120
OnEditChange.... 81
OnEdited....81, 82
OnEditing.... 82
OnEditValidate.... 83
OnEndColumnsCustomizing.... 83
OnEndDragColumn....120, 121
OnEndDragGroupColumn....217, 218
OnExpanded.... 84
OnExpanding.... 85
OnGetCustomSummaryText....218
OnGetEditColor.... 86, 158
OnGetFooterCellText....137, 138
OnGetImageIndex....121
OnGetLevelColor....122
OnGetNodeDragText....123
OnGetPreviewLineCount....138
OnGetPreviewText.... 138, 139, 159
OnGetRowLineCount....139
OnGetSelectedIndex....123, 124
OnGetStateIndex....124
OnHeaderButtonClick.... 124, 125, 197
OnHeaderMoved....125
OnHideBand....126
OnHideColumn....218, 219
OnHideHeader....126
OnHotTrackNode.... 86
OnIsExistFooterCell....127
OnIsExistRowFooterCell....139
OnIsLevelFooter....140

OnMoveTo....176
OnRefreshNodeData....160
OnReloadGroupList....219
OnSelectedCountChange....86, 87
OnShowBand....127
OnShowColumn....219, 220
OnShowHeader....128
OnStartBandDragging....128, 129
OnStartHeaderDragging....129
OnSummaryNode....220
Options....34, 35, 188, 189
OptionsBehavior....164, 191
OptionsCustomize....167, 193
OptionsDB....168, 194
OptionsEx....35, 36, 196, 199
OptionsView....169, 199

P

PaintStyle....37, 38
Partial Load....200, 201, 210
PartialLoad....200, 210
PartialLoadBufferCount....201
PointInCustomizingForm....70
PointInFooterPanel....210, 211
PointInGroupPanel....211
preview....190
PreviewFont....38
PreviewLines....38, 197

R

ReadOnly....7
RebuildGroupIndexes....211
RedrawSelection....12
RefreshBookmarks....155
RefreshDefaultColumnsWidths....103, 104
RefreshGroupColumns....211, 212
RefreshSorting....104
Registry....94
RegistryPath....94
Remove....234
ResetFullRefresh....212
RestoreColumnsDefaults....104
RootValue....171
RowCount....38
RowFooterColor....39
RowFooterTextColor....39
RowHeight....39
RowSeparatorLineWidth....40

S

SaveAllToStrings....71
SaveAllToTextFile....71
SaveSelectedToStrings....71
SaveSelectedToTextFile....72
SaveToFile....135
SaveToHTML....225, 226
SaveToIniFile....104
SaveToRegistry....105
SaveToStream....136
SaveToText....226
SaveToXLS....226, 227
SaveToXML....227
ScrollBars....40
See alsoveSelectedToTextFile....72
SelectAll....136, 174
SelectedCount....40
SelectedIndex....171
SelectedNodes....41
SelectedRows....203
SelectionAnchor....42
SelectNodes....72
SetEditingText....72
SetEditText....72
SetFlagSmartReload....212
ShowBands....43
ShowButtonEditorMouse....73
ShowButtons....43
ShowCustomDialog....235
ShowEditor....73
ShowEditorChar....73
ShowEditorMouse....74
ShowFooter....43
ShowGrid....43
ShowGroupPanel....201
ShowHeader....44
ShowHiddenInCustomizeBox....44
ShowIndicator....44
ShowLines....44, 45
ShowNewItemRow....45
ShowPreviewGrid....45
ShowRoot....45, 46
ShowRowFooter....46
ShowSummaryFooter....202
SimpleCustomizeBox....46
SortedColumn....132, 147
SortedColumnCount....94
SortedColumns....94, 148
SortedColumnsCount....95
StartDragBand....74
StartDragHeader....74, 75
StartSearch....75
State....47
StateImages....48
SummaryGroups....202
SummarySeparator....202

Support....5

T

TabStop....48
TcustomdxContainer
  IsShowButtonAlways....11
TCustomdxContainer....6
  BeginInitEdit....8
  CanEditAcceptKey....8
  CanEditModify....8
  CheckEditorWidth....9
  CreateEditStyle....9
  EndInitEdit....9
  GetDefaultPopupBorderStyle....9
  GetEditColor....9
  GetEditFont....10
  GetReadOnly....10
  HideDrawFocusRect....10
  InplaceEditKeyDown....10
  IsEditorMoved....11
  IsImmediateEditor....11
  IsInitEdit....7, 8, 9
  IsTabs....11
  MoveCol....12
  ReadOnly....7
  RedrawSelection....12
TCustomdxContainer Methods....8
TCustomdxContainer Properties....7
TCustomdxDBGrid....183
  AddGroupColumn....205
  BeginGrouping....206
  CanAddGroupColumn....206
  DataSetChangedBusy....185
  DefaultFields....186
  DeleteGroupColumn....206
  EndGrouping....207
  Filter....186, 229, 231
  FindNodeByKeyValue....207
  GetGroupColumnAt....207
  GetNodeFooterColumnAt....208
  GetSummaryGroupByName....208

GetSummaryItemAt....209
GroupColumnCount....186
GroupColumns....187
GroupPanelColor....187
GroupPanelFontColor....187
GroupPanelHeight....188
IniFileName....93
IniSectionName....93
InsertGroupColumn....209
IsGridMode....209
IsLoadAllRecords....210
IsPartialLoad....210
LoadFromIniFile....102
LoadFromRegistry....102
LockRefresh....188
OnAddGroupColumn....214
OnBackgroundDrawEvent....214
OnClearNodeData....214
OnColumnClick....215
OnColumnSorting....216
OnCustomDraw....216
OnEndDragGroupColumn....217
OnGetCustomSummaryText....218
OnHideColumn....218
OnReloadGroupList....219
OnShowColumn....219
OnSummaryNode....220
Options....188
OptionsBehavior....164, 191
OptionsCustomize....167, 193
OptionsDB....168, 194
OptionsEx....196
OptionsView....169, 199
PartialLoad....200
PartialLoadBufferCount....201
PointInFooterPanel....210
PointInGroupPanel....211
RebuildGroupIndexes....211
RefreshGroupColumns....211
RegistryPath....94
ResetFullRefresh....212
SaveToIniFile....104
SaveToRegistry....105
SetFlagSmartReload....212
ShowGroupPanel....201
ShowSummaryFooter....202
SummaryGroups....202
SummarySeparator....202
UseBookmarks....202
TCustomdxDBTreeList
DataSetChangedBusy....163
EndLayout....174
LockRefresh....164
TCustomdxDBTreeListControl....142, 145
FullRefresh....152
GetColumnAt....153
GetDefaultFieldColumnClass....153
OnBeforeCalcSummary....157
PreviewFieldName....146
SelectedRows....147
TCustomdxTreeList....13
Add....52
AddFirst....53
AdjustColumnsWidth....53
ArrowsColor....15
AutoExpandOnSearch....15
AutoSearchColor....15
AutoSearchTextColor....16
BandButtonPushed....16
BandColor....16
BandFont....16
BandPanelHeight....17
BeginSelection....53
BeginUpdate....54
BorderStyle....17
CalcDrawInfo....54
CancelEditor....54
CellRect....55
ClearColumnsSorted....55
ClearNodes....55
ClearSelection....55
CloseEditor....56
ColumnsCustomizing....56
CopyAllToClipboard....56
CopySelectedToClipboard....56
Count....17
Create....57
CustomizingBandListBox....17
CustomizingForm....18
CustomizingHeaderListBox....18
CustomizingPos....18
CustomizingRowCount....19
DblClkExpanding....19
DefaultRowHeight....19
DeleteSelection....57
DescTextHeight....19
Destroy....57
DownBandIndex....20
DownBandPushed....20
DownColumnIndex....20
DownColumnPushed....21
DragAbsoluteBandIndex....21
DragAbsoluteHeaderIndex....21
DragNode....22
EditingText....22
EndColumnsCustomizing....57
EndSearch....58
EndSelection....58
EndUpdate....58
FindInplaceEdit....59
FindNodeByText....59
FixedBandLineColor....22
FixedBandLineWidth....23
FocusedAbsoluteIndex....23
FocusedColumn....23
FocusedNode....23
FocusedNumber....24
FooterPanelHeight....24
FooterRowHeight....24
FooterRowNodeHeight....25
FreeDrawInfo....59
FullCollapse....60
FullExpand....60
GetEditRect....60
GetFocusedAbsoluteIndex....62
GetFocusedVisibleIndex....62
GetHitInfo....62
GetHitTestInfoAt....66
GetNextVisible....66
GetNodeAt....66
GetScrollVertGridRect....66
GotoFirst....67
GotoLast....67
GotoNext....67
GotoPrev....67
GridLineColor....25
GroupNodeColor....25
GroupNodeTextColor....26
HeaderButtonPushed....26
HeaderColor....26
HeaderFont....27
HeaderHeight....27
HeaderPanelHeight....27
HeaderRowHeight....27
HideEditor....68
HideFocusRect....28
HideSelection....28
HideSelectionColor....28
HideSelectionTextColor....29
HighlightColor....29

HighlightTextColor....29
Images....29
Indent....30
IndentDesc....30
IndexOf....68
InplaceColumnIndex....30
InplaceEditor....30
Insert....68
InvalidateSelection....69
IsCustomizing....31
IsInvertSelect....69
IsNodeVisible....69
Items....31
LastNode....31
LeftCoord....32
LockSelection....32
LockUpdate....32
LookAndFeel....33
MakeBandVisible....69
MakeColumnVisible....70
MakeNodeVisible....70
MaxRowLineCount....33
NewItemRowActive....34
NewItemRowHeight....34
OnBeginDragNode....76
OnCanNodeSelected....76
OnChangeColumn....77
OnChangeLeftCoord....77
OnChangeNode....78
OnChangeTopVisibleNode....78
OnCollapsed....78
OnCollapsing....79
OnCompare....79
OnDeletion....81
OnEditChange....81
OnEdited....81
OnEdit ing....82
OnEditValidate....83
OnEndColumnsCustomizing....83
OnExpanded....84
OnExpanding....85
OnGetEditColor....86
OnHotTrackNode....86
OnSelectedCountChange....86
Options....34
OptionsEx....35
PaintStyle....37
PointInCustomizingForm....70
PreviewFont....38
PreviewLines....38
RowCount....38
RowFooterColor....39
RowFooterTextColor....39
RowHeight....39
RowSeparatorLineWidth....40
SaveAllToStrings....71
SaveAllToTextFile....71
SaveSelectedToStrings....71
SaveSelectedToTextFile....72
ScrollBars....40
SelectedCount....40
SelectedNodes....41
SelectionAnchor....42
SelectNodes....72
SetEditingT ext....72
SetEditText....72
ShowBands....43
ShowButtonEditorMouse....73
ShowButtons....43
ShowEditor....73
ShowEditorChar....73
ShowEditorMouse....74
ShowFooter....43
ShowGrid....43
ShowHeader....44
ShowHiddenInCustomizeBox....44
ShowIndicator....44
ShowLines....44
ShowNewItemRow....45
ShowPreviewGrid....45
ShowRoot....45
ShowRowFooter....46
SimpleCustomizeBox....46
StartDragBand....74
StartDragHeader....74
StartSearch....75
State....47
StateImages....48
TabStop....48
TopIndex....48
TopNode....49
TopVisibleNode....49
TreeLineColor....49
TreeLineStyle....49
UpdateScrollBars....75
VisibleRowCount....50
WaitForExpandNodeTime....50
TCustomdxTreeListControl....88
ApplyBestFit....97
AssignColumns....97
BandMaxRowCount....90
BandRowCount....90
Bands....91
BeginSorting....98
ColumnByName....98
ColumnCount....91
Columns....91
CreateColumn....98
CreateColumnEx....99
DefaultLayout....91
DestroyColumns....99
EndSorting....99
GetAbsoluteColumnIndex....100
GetBandAt....100
GetColumnAt....100
GetDisplayValue....101
GetFooterColumnAt....101
GetHeaderColumnAt....101
GetNodeFooterColumnAt....101
GetVisibleColumnIndex....102
HeaderMinRowCount....92
HeaderPanelMaxRowCount....92
HeaderPanelRowCount....92
IsCustomStoring....93
OnBandButtonClick....108
OnBandClick....108
OnCalcRowLineHeight....109
OnCanBandDragging....109
OnCanHeaderDragging....110
OnChangedColumnsWidth....110
OnColumnClick....110
OnColumnMoved....111
OnColumnSorted....111
OnColumnSorting....112
OnCustomDraw....112
OnCustomDrawBand....113
OnCustomDrawCell....114
OnCustomDrawColumnHeader....115
OnCustomDrawFooter....116
OnCustomDrawFooterNode....117
OnCustomDrawPreview....117
OnCustomDrawPreviewCell....118
OnDragEndBand....119
OnDragEndHeader....119
OnDragOverBand....120
OnDragOverHeader....120
OnEndDragColumn....120
OnGetImageIndex....121
OnGetLevelColor....122
OnGetNodeDragText....123

OnGetSelectedIndex....123
OnGetStateIndex....124
OnHeaderButtonClick....124
OnHeaderMoved....125
OnHideBand....126
OnHideHeader....126
OnIsExistFooterCell....127
OnShowBand....127
OnShowHeader....128
OnStartBandDragging....128
OnStartHeaderDragging....129
RefreshDefaultColumnsWidths....103
RefreshSorting....104
RestoreColumnsDefaults....104
SortedColumnCount.... 94
SortedColumns.... 94
SortedColumnsCount.... 95
UpdateColumn....105
UpdateHeaders....105
VisibleColumnCount.... 95
VisibleColumns.... 95
TdxDBGrid....221
SaveToHTML....225
SaveToText....226
SaveToXLS....226
SaveToXML....227
TdxDBGridFilter....229
Active....231
Add.... 52, 234
AddNull....234
AutoDataSetFilter....231
CaseInsensitive....231
Clear....234
DropDownCount....231, 232
FilterCaption....232, 233
FilterStatus....232
FilterText.... 231, 232, 233
MaxDropDownCount....233
Remove....234
ShowCustomDialog....235
TdxDBGridFilter Methods....234
TdxDBGridFilter Properties....231
TdxDBTreeList....177
TdxTreeList....130
AssignNodes....135
LoadFromFile....135
LoadFromStream....135
OnGetFooterCellText....137
OnGetPreviewLineCount....138
OnGetPreviewText....138
OnGetRowLineCount....139
OnIsExistRowFooterCell....139
OnIsLevelFooter....140
SaveToFile....135
SaveToStream....136
SelectAll....136
SortedColumn....132
TdxTreeList Events....137
TdxTreeList Methods....133
TdxTreeList Properties....131
TdxTreeListHitTest.... 66
TopIndex.... 48
TopNode.... 49
TopVisibleNode.... 49
TreeLineColor.... 49
TreeLineStyle.... 49

U

UpdateColumn....105
UpdateHeaders....105
UpdateScrollBars.... 75
UseBookmarks....202, 203

V

VisibleColumnCount.... 95
VisibleColumns.... 95, 148
VisibleRowCount.... 50

W

WaitForExpandNodeTime.... 50